

CONTINUATION

CONTINUATION.md — vasic umbrella monorepo

Last-Updated: 2026-09-09T00:00:00Z

Synced-Commit: 3922e35c12c3

Authority-Root: submodules/constitution

This file is the single canonical handoff document mandated by **Constitution §12.10** (“Continuation document — sacred invariant”). Its binding requirements, quoted from submodules/constitution/Constitution.md:

1. **docs/CONTINUATION.md** **MUST exist** at the project root. Its absence is a release blocker.
2. **Every non-trivial state change** **MUST** update this document in the same commit as the work itself.
3. **Top-of-file timestamp** is updated on every edit. Stale timestamps trigger gate failure.
4. **Section §3 “Active work”** lists every IN PROGRESS / BLOCKED item with enough detail that any agent can resume without conversation context — concrete commands, file paths, monitor IDs.
5. **Section §0 “How to use this document”** contains the verbatim resumption prompt — a single block any operator can paste into any CLI agent.
6. **Document MUST be self-contained.** No hyperlinks to ephemeral external systems as the only source of truth.

Path note (honest boundary, §11.4.6). §12.10’s protection 1 is internally inconsistent: it names docs/CONTINUATION.md but requires it “at the project root”. This repository resolves that in favour of the **repository root** — CONTINUATION.md — because that is where the file already existed (tracked since commit 18c84ff) and where all four

governance carriers restate the rule (“CONTINUATION.md kept in sync”). No docs/CONTINUATION.md exists here; if one is ever added, one of the two must become a pointer, not a rival.

§0 How to use this document

Read this file **first**, before any other file in the repository. It is self-contained: every fact below is reproduced here, not linked to.

Verify it is not stale before trusting it:

```
bash scripts/continuation-check.sh
# 0 = in sync · 1 = drift · 2 = undetermined
```

If that exits 1, it names the drift. **Fix the drift before doing anything else** — a stale continuation document is a §12.10 violation, and §12.10 has no override flag.

Verbatim resumption prompt (§12.10 protection 5)

Paste this block, unedited, into any CLI agent (Claude Code, Codex, Cursor, Aider, Gemini CLI, Qwen Code, OpenCode, Crush, Kimi CLI):

You are resuming work on the `vasic` umbrella monorepo.

1. Read CONTINUATION.md at the repository root, in full, before anything else.
2. Run `bash scripts/continuation-check.sh`. If it exits 1, that document is
 stale — reconcile it against the four governance carriers (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md) and docs/constitution-adoption/INVENTORY.md
 before starting new work. If it exits 2, report what could not be determined
 and why; do not treat 2 as a pass.
3. Read CLAUDE.md (or the carrier for your agent: AGENTS.md / QWEN.md / GEMINI.md — they are byte-identical below their opening section). The
 authority it points at is the git submodule at `submodules/constitution/`,
 NOT the top-level `constitution/` used as an example in the constitution's
 own prose. Read anchors on demand, never eagerly:
 awk '/^### §12.10 /{f=1} f&&/^### §12.11 /{exit} f{print}' \submodules/constitution/Constitution.md

4. Binding rules you inherit unconditionally: no bluffing (every PASS carries positive evidence, §11.4); no guessing language – "likely", "probably", "seems", "appears" are forbidden when reporting causes (§11.4.6); every new gate ships with a paired mutation proving it catches regressions (§1.1); never force-push; hardlinked backup before any destructive operation (§9); keep CONTINUATION.md in sync in the SAME commit as the work (§12.10).
5. Two PRODUCTION websites are published from this tree. Do not disable ``milosvasic.ru/.github/workflows/pages.yml`` – it is the sole publish path for the live site. See §5 of CONTINUATION.md before touching any CI or deploy file.
6. Do not commit or push unless the operator asks. If asked, use the ``commit`` wrapper on PATH, never bare ``git add`` / ``git commit`` / ``git push``. Note the wrapper runs ``git add .`` – check ``.gitignore`` first.
7. SpecKit feature 001 (``specs/001-workshop-curriculum-platform/``) has moved from planning into building, and none of it is committed yet – an untracked ``workshop/pipeline/`` tree and an untracked ``analysis.md`` exist. Claim no completion you have not measured. Host capability changed on 2026-09-01:
 - a dual ASR stack (faster-whisper 1.2.1 + whisper.cpp v1.9.1, CPU-only) and a generative model (``qwen2.5:3b-instruct-q4_K_M``) ARE now installed, but the ASR engines live in ``workshop/pipeline/venv``, NOT on the system ``python3``.
 - ``/usr/bin/whisper`` is still an unrelated GTK desktop app, not a transcriber – ``command -v whisper`` still proves nothing. Verify a capability before you plan around it. See §3 A1 and §5 H9.
8. Report the current §3 "Active work" items and ask which to take, unless the operator already told you.

Short one-line variant:

Read CONTINUATION.md at the repo root, run ``bash scripts/continuation-check.sh``, then continue the §3 "Active work" items under the constitution

mounted at
`submodules/constitution/`.

§1 What this repository is

vasic is the umbrella monorepo for two personal/portfolio websites and the shared tooling that builds, translates and validates them. It is not a framework; the sites are git submodules and everything that generates or checks them lives at the top level.

Path	What it is
vasic.digital/	Site submodule — committed static HTML , no build step.
milosvasic.ru/	Site submodule — Jekyll source; rendered <code>_site/</code> is git-ignored.
_tools/gen/	Go generator (Go 1.26) that renders the localized pages for both sites.
_tools/	Translation (<code>translate/</code>), PDF (<code>pdf/</code>), portfolio (<code>portfolio/</code>) tooling + <code>deploy-langs.sh</code> .
design-system/	Shared per-brand tokens, fonts, icons, motion, component CSS.
_tests/	Playwright + self-validating harness (visual oracle, PDF/OCR export checks, link/sitemap integrity).
_content/	English source content. Per-language siblings are <code>_content_<lang>/</code> .
scripts/	Repo-level operational scripts, including the local gate runner and the governance verifiers.
docs/constitution-adoption/	The governance audit: <code>INVENTORY.md</code> (gap register G1–G12) and the §11.4.156 decision records.
specs/001-workshop-curriculum-platform/	SpecKit feature 001 — planning artifacts, plus an untracked <code>analysis.md</code> . Building has started,

Path	What it is
	in an untracked workshop/ pipeline/ tree. Nothing is committed. See §3 item A1.
submodules/containers/	Owned submodule (vasic- digital/containers), added per §11.4.76 for the containerised workload feature 001 plans. Nothing in this repository has been built or run in a container yet.

Languages. 14 translation trees exist: ar be de es fa fr hi ja kk ko ru sr tr zh. `_tools/deploy-langs.sh` computes which are *complete* at run time from its LANGS list and the document count — it does not trust a static list of “shipped” languages.

Submodules. `.gitmodules` declares **13** gitlinks (re-measured 2026-09-01). **11** are owned consumers of the governance cascade: milosvasic.ru, vasic.digital, design-toolkit, ai_interviewing, monetization, workshop, submodules/containers, submodules/LLMProvider, submodules/RAG, submodules/verdict and submodules/passage. submodules/constitution is the governance **source** (the head of the cascade, not a consumer). submodules/superspec is **third-party** (upstream WangX0111/superspec) and is outside the owned-submodule set for tagging and propagation purposes. `scripts/verify-governance-cascade.sh` derives all of this from `.gitmodules` plus `helix-deps.yaml` — no roster is hardcoded anywhere, and C6 guards the two files against each other in both directions.

The earlier “9 gitlinks / seven owned” figure is WITHDRAWN, not restated. Four submodules joined on 2026-09-01: submodules/LLMProvider and submodules/RAG were **adopted** under the §11.4.74 catalogue-check (reuse before reimplement), and submodules/verdict and submodules/passage are **newly created PUBLIC reusables** extracted from workshop/. **All four have since LANDED** — the “STAGED, not committed” caveat this paragraph carried is discharged, verified 2026-09-02 by `git diff --cached --name-only`, which names submodules/constitution and nothing else. `scripts/verify-manifest-pins.sh` still compares against the INDEX (`git ls-files -s`) rather than HEAD, which is what keeps the one remaining staged gitlink gated instead of unverifiable.

Gitlinks as they stand in the INDEX (`git ls-files -s`), each re-checked against `git ls-remote <url> HEAD` on **2026-09-02**. `scripts/verify-manifest-pins.sh` independently reports **12 MATCH / 0 DRIFT / 0 UNDETERMINED of 12 declared deps**, exit **0** — but note what that measures: it compares `helix-deps.yaml`’s recorded ref against the **local gitlink**, not against the remote. The two questions are different, and **both are now gated** — the remote question by `scripts/verify-submodule-remote-sync.sh`, which exits 1 on this tree. “Only one of them is gated” was true of every earlier revision and is **withdrawn**; a green `verify-manifest-pins.sh` still says nothing about any remote, which is why the two must be read together and never substituted. (12 deps, 13 gitlinks: `submodules/superspec` is carried as a third-party comment in the manifest, not a `deps[]` entry, and C6 checks that both ways.)

Submodule	Gitlink	Notes
submodules/ constitution	3be10826f3d236edcdb62b7cba81cda4e6d0d47a	EQUAL TO ITS REMOTE 2026-09-02 — no longer still records <code>f16ea779b8</code> carries a STAGED, unco fast-forward <code>f16ea779 -</code> <code>3be10826</code> , each leg verifi TRUE with 0 divergent. moved with it. See the p
submodules/ passage	729cd96a39fefff5675570adff2eeba4faed0d26	match · remote CURRENT
submodules/ verdict	477dc35afe60f5f2f94d0c902a5a7a7ce0e4ec6b	match · remote CURRENT
submodules/ LLMProvider	3c1cef79eb95039ed9a414e1c568a815df6dcde9	match · remote CURRENT
submodules/RAG	8aee628e473160c76b9eca99404978c02dd992eb	match · remote CURRENT The d940b51fc247c285c8 c75b9 this row carried — it was true when wr today . Re-measured 202 <code>files -s submodules/co</code> <code>-C submodules/containe</code> both return <code>7f592256</code> . Th commits are <code>6d13ad0</code> (20 ephemeral-run primitiv reader fix) and <code>7f592256</code> NOT probed this sessio CURRENT” is WITHDR
submodules/ containers	7f5922563d8bec866b1a25eac483590c9a212817	

Submodule	Gitlink	Notes
		unmeasured rather than bash scripts/verify-submodules/ sync.sh. No other row is re-measured; treat the
submodules/ superspec	c20ac6c1ba069cc9a72dacb8044b7b193d3dde81	match — third-party, not reported as a NOTE, new
milosvasic.ru	1823d62c314af3b26ae41d8a7c9e0bc699189d26	match · remote CURRENT
vasic.digital	31928364a43ed7984fe6999ae958dde87bde3fed	match · remote CURRENT
		match · remote CURRENT tree sits at 5467a888...” carrying to carry is withdrawn — the GitHub origin now a mirror asymmetry below and still unverified from
design-toolkit	e7f3815ec35c0940515296fffb3481cd0fab4bfa6	
ai_interviewing	cde474fa3e167bfd5c8e63d4ba6d4c184d4c12b6	match · remote CURRENT
monetization	54ed7b0f5add52821d18866facb5ee8c75adef69	match · remote CURRENT
		match · pushed 2026-09-01 (b232789..692a27a) carrying the redaction mechanism verifier and the nomic i
workshop	7b7a8c9048871f47a6303c9e59c7338c959672df	Earlier in the session it times (6af5816, 50a1591, 86f2a22) — those are the flagged as drift.

Every SHA in the table above except monetization and submodules/superspec moved after the previous Synced-Commit, and the values this table used to carry are WITHDRAWN, not restated. Eleven of the thirteen rows were stale when this revision opened — every row except monetization and submodules/superspec, counted mechanically against the index rather than by eye. (A “ten of thirteen” figure was written earlier in this same revision and is wrong; it was caught by a second agent re-deriving the count instead of accepting it.) Re-derive rather than trusting the column:

```
git ls-files -s -- $(git config -f .gitmodules --get-regexp
    'submodule\..*\path' \
    | awk '{print $2}') | awk '{printf "%-28s %s\n", $4, $2}'
```

design-toolkit went PUBLIC on 2026-09-01, and its two mirrors do not match. The GitHub origin vasic-digital/design-toolkit was flipped public after a clean full-history privacy audit; it ships 0 tracked

workflow files (`git ls-files '.github/workflows/*'`), so it creates no §11.4.156 CI surface. Its **GitLab mirror is reported to remain private**, and the lag between the two mirrors is now **UNMEASURED — not zero, and no longer the “5 commits” this paragraph used to assert**. That figure was measured on 2026-09-01 against GitHub HEAD 5467a888... (GitLab HEAD 520c436c... a strict ancestor, `rev-list --left-right --count = 0 / 5` — lag, not divergence). **The GitHub side has since moved on: the gitlink and origin HEAD are both e7f3815ec35c as of 2026-09-02**, so the old count describes a comparison whose right-hand side no longer exists, and the true lag can only be larger or equal, never smaller. Honest boundary (§11.4.6): this checkout wires up **no** GitLab remote for design-toolkit (`git -C design-toolkit remote -v` shows origin only, GitHub), so **neither** half can be re-measured from this tree — `scripts/verify-submodule-remote-sync.sh` probes the declared origin and reports the submodule CURRENT, which is a true statement about GitHub and says nothing at all about GitLab. Do not write as though the mirrors agree, and do not quote “5 commits” as a live figure.

Only THREE submodules are private now, re-measured 2026-09-01 with `gh api repos/<owner>/<name> --jq '.visibility': milos85vasic/workshop_curriculum, milos85vasic/ai_interviewing, milos85vasic/monetization`. Everything else — including design-toolkit, containers, LLMPProvider, RAG, verdict, passage, both sites — is **public**. The “four private submodules” figure this document and the carriers used to carry is withdrawn.

The governance source has now been fast-forwarded TWICE, and the claim this block used to carry — that the pin is “equal to git ls-remote ... HEAD” — is WITHDRAWN, not restated. It was true when written and did not survive the week. The sequence, each step measured rather than inferred:

```
902979027a90 -> f16ea779b82a    (2026-09-01, operator-authorized,
0 divergent / 3 behind)
f16ea779b82a -> f5876a3b700e    (0 divergent / 4 behind)
f5876a3b700e -> 3be10826f3d2    (2026-09-02, operator-authorized,
0 divergent / 2 behind)
remote HEAD today                = 3be10826f3d2    (EQUAL — no
drift)
```

Re-measured 2026-09-02 **after the third fast-forward**: the index and the submodule working tree both sit at 3be10826f3d2 on branch main, which **EQUALS** `git ls-remote git@github.com:HelixDevelopment/HelixConstitution.git HEAD`. The umbrella’s HEAD still records

f16ea779b82a, so the whole two-step move is **staged and not yet committed**, and helix-deps.yaml records 3be10826f3d2 staged in the same change, exactly as C9 requires.

The third move was operator-authorized and CLASSIFIED BEFORE it was made. git fetch origin inside the submodule — the mutating command the previous revision correctly refused to run unasked — resolved the UNDETERMINED direction: merge-base --is-ancestor f5876a3b 3be10826 returned TRUE and rev-list --left-right --count returned 0 / 2 — **2 behind, 0 divergent**. git merge --ff-only performed it and would have refused anything else. diff --stat touches **two files, both inside the constitution's own scripts/gates/** — no governance document, no carrier, not Constitution.md.

C9 went FAIL for exactly as long as the manifest was ahead of the index, and that is the gate working, not a defect. Moving helix-deps.yaml first left the recorded ref at 3be10826 while git ls-files -s still reported f5876a3b; verify-manifest-pins.sh reported 11 MATCH, 1 DRIFT until the gitlink itself was staged, then returned to **12 MATCH / 0 DRIFT**.

scripts/verify-submodule-remote-sync.sh now exits 0 — 12 CURRENT / 0 DRIFT / 0 UNDETERMINED of 12 owned gitlinks. That gate has been RED since the day it was written; this is the first green it has ever produced. Heed its own printed caveat rather than banking it: *“Dated observation, not a standing fact: remotes move.”* This pin has now gone stale within a day, twice.

Nothing that this repository records about the corpus went stale in any of the three moves. Constitution.md is the SAME git blob 34eff9d86cadb325721c958d35a411feaad27681 at 902979027a90, f16ea779b82a, f5876a3b700e and 3be10826f3d2 alike — re-measured ON DISK at the current pin as **11,700 lines, 252 ### § anchors, 1,779,401 bytes**. The second fast-forward's diff --stat touches exactly two paths, neither a governance document: design-toolkit (1 -) and docs/codegraph/Status.md (18 +).

The four findings below were measured at the FIRST fast-forward and are kept because each one still describes how a pin move must be done here:

- 1. A true fast-forward, not a rebase.** merge-base --is-ancestor pin remote returned TRUE; rev-list --left-right --count pin...remote returned 0 / 3 — 3 behind, 0 divergent. The move used git merge --ff-only, which would have refused anything else. 902979027a90 was a local commit made in this checkout and is an

ancestor of the remote head, so no local work was lost. **Nothing was pushed to the constitution repository.**

2. No recorded anchor or line count in this repository went stale.

Constitution.md is the same git blob at both ends —

34eff9d86cadb325721c958d35a411feaad27681 — re-measured ON DISK AFTER the checkout at 11,700 lines and 252 ### § anchors, so the “11,700 lines / 252 anchors” figure in §2 describes the new pin unchanged. `diff --stat` touches only `design-toolkit` (1 +), `docs/codegraph/Status.md` (12 +) and the submodules/`design-toolkit` gitlink.

3. helix-deps.yaml moved in the SAME change, gitlink and manifest staged together — a bumped gitlink with a stale `ref:` is exactly what C9 catches. Re-measured after: `verify-manifest-pins.sh` exit **0**, 12 MATCH / 0 DRIFT / 0 UNDETERMINED; `verify-governance-cascade.sh` exit **0**, 12 PASS / 0 FAIL.

4. No gate catches local-vs-remote drift, and the bump did not change that. C9 and `verify-manifest-pins.sh` compare the manifest to the **local gitlink**. Both exited **0** while the pin sat 3 behind, and both exit **0** now — they cannot tell the two states apart, so neither exit code is evidence about the remote.

Upstream defect carried in by the first fast-forward: FIXED

UPSTREAM by the second, and the fix is measured, not assumed.

f16ea779b82a carried a second `design-toolkit` gitlink at the constitution’s own ROOT alongside `submodules/design-toolkit`, while its `.gitmodules` mapped the entry named `design-toolkit` to `submodules/design-toolkit` only — leaving the root gitlink unregistered. This repository correctly declined to fix it, because fixing it means committing to the constitution submodule, which this repository does not do. **Waiting was the right call: upstream removed it.** Re-measured 2026-09-02:

```
git -C submodules/constitution ls-tree f16ea779b82a --format='%
(objectmode) %(path)' | grep design-toolkit
# 160000 design-toolkit <- the unregistered
    root gitlink
git -C submodules/constitution ls-tree HEAD --format='%
(objectmode) %(path)' | grep design-toolkit
# (none) <- gone at
    f5876a3b700e
```

That deletion is the `design-toolkit | 1 -` line in the fast-forward’s `diff --stat`. Its `.gitmodules` still maps `design-toolkit` → `submodules/design-toolkit`, so the entry is now registered and singular.

The design-toolkit ref CONFLICT recorded in helix-deps.yaml is a different thing and is UNCHANGED. The constitution's own manifest still says ref: 16e4e76 — verified identical at all three pins (902979027a90, f16ea779b82a, f5876a3b700e). Do not read the gitlink fix as closing this.

Residual risk, exposed but not closed: any submodule here can sit arbitrarily far behind its upstream — corpus changes included — with every gate green, because C9 compares the manifest to the local gitlink and nothing compares the gitlink to the remote. The bump closed one INSTANCE, not the CLASS.

Re-derive without touching the submodule:

```
git -C submodules/constitution rev-parse HEAD
git ls-remote git@github.com:HelixDevelopment/
    HelixConstitution.git HEAD
P="$(mktemp -d)/probe.git"
git clone --bare git@github.com:HelixDevelopment/
    HelixConstitution.git "$P"
PIN="$(git -C submodules/constitution rev-parse HEAD)"
REM="$(git -C "$P" rev-parse HEAD)"
git -C "$P" merge-base --is-ancestor "$PIN" "$REM" && echo 'fast-
    forward safe'
git -C "$P" rev-list --left-right --count "$PIN...$REM"
git -C "$P" diff --stat "$PIN" "$REM"
for R in "$PIN" "$REM"; do
    git -C "$P" rev-parse "$R:Constitution.md"
    git -C "$P" show "$R:Constitution.md" | grep -c '^### §'
done
```

The umbrella itself is pushed: HEAD = refs/heads/main on all three configured remotes (github, origin, upstream — all the same URL, git@github.com:milos85vasic/vasic.git). That tip is the result of **two** authorized history rewrites and force-pushes — 2026-09-01 and 2026-09-02 (A9 below). 13a13a312fd9... was the tip after the first push and is **no longer the tip**: four further commits have landed since. Re-derive with git rev-parse HEAD and git ls-remote origin refs/heads/main rather than trusting a SHA quoted here. The two values this line carried before — ee3933d46211... and then 562ecf9cca2d... — both name commits that no longer exist in this repository.

Clone caveat. Init the site submodules **non-recursively**:

```
git submodule update --init vasic.digital milosvasic.ru.
milosvasic.ru embeds a nested submodule (Upstreamable) whose
.gitmodules is a broken 0-byte gitlink, so a recursive checkout of the
umbrella fails.
```

§2 Where the authority lives

All rules come from the constitution submodule. **This repository mounts it at submodules/constitution/, not the top-level constitution / that the constitution’s own prose and templates use as their example.** Wherever a quoted snippet says constitution/<file>, the file here is submodules/constitution/<file>. submodules/constitution/find_constitution.sh resolves either layout from any nested depth.

What	Path
The universal constitution	submodules/constitution/ Constitution.md
Claude Code carrier (upstream)	submodules/constitution/ CLAUDE.md
Codex / Cursor / Aider / OpenCode / Crush / Kimi carrier	submodules/constitution/ AGENTS.md
Qwen Code carrier	submodules/constitution/QWEN.md
Gemini CLI carrier	submodules/constitution/ GEMINI.md
Parent-walk resolver	submodules/constitution/ find_constitution.sh
Forbidden-command PreToolUse guard (present, not wired — see G12)	submodules/constitution/ scripts/hooks/guard-forbidden- commands.sh

Measured on the currently checked-out gitlink (3be10826f3d2): Constitution.md is **11,700 lines / 1,779,401 bytes** and carries **252 ###** §... anchor headings (wc -l, grep -c '^### §'). Both superseded pins — 902979027a90 and f16ea779b82a — carried the SAME Constitution.md blob 34eff9d86cadb325721c958d35a411feaad27681, and so does this one, so **these figures have not moved across any of the three pins.** That is a measured coincidence of what upstream happened to change, not a guarantee; re-measure after any future bump rather than assuming a pin move is corpus-neutral.

Inheritance form used here: the pointer block, not @import. Anchor §11.4.35 invariant 6 declares the two forms equivalent. This repository’s four root carriers (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md) each open with **## INHERITED FROM submodules/**

constitution/CLAUDE.md. The reason is cost: the upstream carrier is 784 KB and Constitution.md is 1.7 MB, so a native import would load roughly 165k tokens into every session before any work begins.

Lockstep (§11.4.157). The four root carriers must be byte-identical below their opening section; only the opening section differs per agent. The mechanical form of that rule used here is **line 19 to EOF**, which is what scripts/verify-governance-cascade.sh C5 ROOT-LOCKSTEP measures. Measured 2026-09-02, **after this revision's own carrier edits**: all four are **1171 lines** and share the digest

```
sha256(tail -n +19) =  
542e1c38867a14a518838db62c49808b615e05c5193935c762a1ca7c81b44d9f
```

The lockstep itself is INTACT — bisecting `tail -n +N | sha256sum` across the four gives **4** distinct digests at $N=18$ and **1** at $N=19$, exactly as the rule requires.

(prior digests, superseded and listed so a stale one is recognisable rather than trusted: `dd14cc3a0073c75a...` at 937 lines — the carriers as this session FOUND them, before the pin block was corrected — and `bef157d5a8992aae...` at 831 lines, the value this section carried before this revision; and, earlier on 2026-09-01, `bc945d409dfdea5f...`, `722d31ebbd30f863...` and `a1f3a936e0ff6817...`, the last of those taken under the old line-24 window rather than the tightened line-19 one; and `792d878a3907c8491be99762005931c08ecf77b9ae3ee66ab139b7a1fb963939` at an earlier Synced-Commit. **A digest is only comparable to another taken with the same window**, and the carriers have grown 831 → 937 → 1171 lines across three revisions, so the line count is not a check either — only the digest is.)

(re-derive: `for f in CLAUDE.md AGENTS.md QWEN.md GEMINI.md; do tail -n +19 "$f" | sha256sum; done` — four identical lines, or the lockstep is broken). **Any edit to one carrier must be applied to all four in the same commit.**

C5 was TIGHTENED on 2026-09-01 and no longer splits at a hardcoded 24. It now measures where the four carriers actually converge and enforces from there, and prints a NOTE stating the two agree:

```
C5 · declared split 19 equals the measured convergence line  
19 – the gate enforces exactly the range it claims, with no  
ungated identical remainder (ceiling 24)
```

That closed a real blind spot, and it is worth naming because this document asserted the old form as recently as this revision's first draft: a line-24 window is a strict **subset** of the shared region, so the old C5 could pass on carriers that differed at lines 19–23. Lines 1–18 are the per-agent opening block. Measured by bisecting `tail -n +N | sha256sum` across the four: N=18 gives 4 distinct digests, N=19 gives 1. **Edit the shared region as ONE artifact** — split the head off, edit the tail once, recompose — rather than editing four files by hand and trusting a gate to catch the slip:

```
for N in 18 19; do
  printf 'from %s: ' "$N"
  for f in CLAUDE.md AGENTS.md QWEN.md GEMINI.md; do tail -n +$N
    "$f" | sha256sum; done \
    | sort -u | wc -l
done # expect 4, then 1
```

The same lockstep rule applies *inside* every owned submodule, and is measured separately by cascade check **C8** — which normalises the per-agent header rather than splitting at a fixed line, because submodule carriers do not share this root's header geometry. C8 passed for all **11** owned submodules on 2026-09-01 (the earlier “7” is withdrawn; the fleet grew the same day).

Project overrides of universal rules: NONE. In particular, do not propose an `Override §11.4.156` — that rule names and refuses the exemption vocabulary (“No escape hatch — no `--allow-ci ... --ci-exempt` flag”), and a consumer carrier may only extend inherited rules, never weaken them. A **documented deviation is not an override** and must never be written up as one.

§3 Active work

EIGHT TRACKED FILES ASSERTED A GAP THAT UPSTREAM HAD ALREADY CLOSED, 2026-09-09

The finding is not “the documents were stale”. It is that this tree CONSUMED the fix and went on asserting the gap for five days. The ephemeral-run primitive landed in submodules/containers at 6d13ad03528c on **2026-09-04, two commits before the gitlink this repository is pinned at today.** Nothing in this tree noticed, because

nothing here reads the consumed submodule's interface — the gate family compares gitlinks and manifests to each other, never a recorded CLAIM to the code it describes.

Re-measured 2026-09-09, and every figure below was taken this session:

```
git ls-files -s submodules/containers ->
7f5922563d8bec866b1a25eac483590c9a212817
git -C submodules/containers rev-parse HEAD ->
7f5922563d8bec866b1a25eac483590c9a212817
(the pin the documents were written against:
d940b51fc247c285c805799452992da8d09c75b9)
```

pkg/runtime/runtime.go declares, ON the ContainerRuntime interface itself:

```
Run(
    ctx context.Context, image string, cmd []string,
    opts ...RunOption,
) (*ExecResult, error)
```

pkg/runtime/run.go and run_test.go exist. WithRunStdin(io.Reader) supplies the document and is what puts -i on the argv (run.go:119, run.go:204). StdinExecutor / ExecuteWithStdin is implemented for real, by defaultExecutor on the os/exec path (run.go:53).

So the recorded claim — “pkg/runtime has NO Run, NO Create, Exec accepts no stdin, and the only WithStdin sits in an interfaces-only package nothing implements” — is FALSE at the current pin FOR THE LOCAL RUN PATH. It was TRUE at d940b51. It is withdrawn by name in all eight files, never silently rewritten.

WHAT REMAINS TRUE, and the correction deliberately does not flatten it:

- Exec(ctx, id, cmd []string) **still accepts no stdin** — signature unchanged.
- **The REMOTE-stdin gap is OPEN.** remote.RemoteRuntime.Run (pkg/remote/runtime.go, ~line 240) explicitly REFUSES WithRunStdin, returning an error wrapping runtime.ErrStdinUnsupported, because RemoteExecutor exposes no io.Reader seam. pkg/remote/connection is still four files of interfaces and option builders with nothing implementing Connection. **Refusing rather than silently running with an empty stdin is good practice and is recorded as such.**

The consequence is a SPLIT, and reporting only the closed half would have been the bluff:

Script	§11.4.76 exception	Why
<code>_tools/ helixtranslate- container/run.sh</code>	REASON WITHDRAWN	Runs ON the remote host, so its run --rm -i is a LOCAL run there. Convertible in principle via runtime.Run + WithRunStdin / WithRunVolumes / WithRunEntrypoint / WithRunExtraArgs.
<code>_tools/ helixtranslate- container.sh</code>	STANDS, reason NARROWED	The local half, raw ssh ... < "\$IN". Still not convertible — the remote-stdin gap above is its whole and only reason now.

NEITHER SCRIPT WAS CONVERTED, and that is the decision, not an omission. Re-measured 2026-09-09 on this host: podman images | grep -i helixtranslate matches **zero** rows; getent hosts fails to resolve thinker.local and amber.local; ssh -o BatchMode=yes returns **rc 255** for both. Absent image, unreachable hosts — rc 2, and **a 2 is never a pass**. A working script must not be replaced by a rewrite that cannot be exercised end to end. *Convertible in principle* is not *verified in practice*, and no file in this tree now spends one as the other.

One upstream observation, recorded as an observation and NOT a defect (§11.4.6). defaultExecutor.ExecuteWithStdin builds its child with exec.CommandContext and folds an *exec.ExitError into exitCode while setting err = nil (run.go:53-73). A CANCELLED run is killed by the context and returns as a signal death: **ExitCode = -1 with a nil error**, so a caller must consult ctx.Err() to tell cancellation from a container that exited -1. **Cancellation genuinely works**. Whether reporting it this way is deliberate is **UNCONFIRMED** — nothing in the module states it, and it was not asked upstream.

Files corrected (nine, not eight — the ninth is named because the brief did not). The four carriers CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md (edited as ONE artifact from the measured convergence line 19 down, then recomposed, so C5 stays green), CONTINUATION.md,

`_tools/containers/README.md`, `_tools/helixtranslate-local.sh`, `_tools/helixtranslate-container/run.sh`, and `_tools/helixtranslate-container.sh` — whose “Every executed path in `digital.vasic.containers` is `stdin-less`” was equally false as a general statement and had to be narrowed rather than left standing.

NOTHING INSIDE `submodules/containers` WAS TOUCHED. It is consumed, not owned. **Nothing was committed or pushed.**

The hole this leaves open, stated rather than closed: no instrument in this tree compares a RECORDED CLAIM about a consumed submodule against that submodule’s actual code. The gitlink moved, every gate stayed green, and eight documents kept asserting a gap that had been closed. **This correction is an instance, not the class.**

TWO PROVIDER HOSTS HAD NEVER BEEN ASKED, AND THIS RECORD HAD FALLEN BEHIND ITS OWN HISTORY, 2026-09-08

Read the shape of this entry before its numbers. Two of the reds in `specs/005-clone-and-clear-red/spec.md` were closed, and they were closed in opposite ways: one by *asking a question nobody had ever put*, the other by *writing down what had already happened*. Neither was closed by moving a threshold, and one of them deliberately does not go green.

R5 — `scripts/verify-provider-ci.sh` grew two read-only adapters

`bash scripts/verify-provider-ci.sh` was, and still is, **rc 1** — and the 1 is the same real finding it has always been: one CONFIRMED standing provider-side trigger on the fleet’s Pages-in-legacy-mode repository, which has **no file-level remedy**. That row is untouched and must not be “fixed” here.

What changed is the other half of that run. Six rows on `gitflic.ru` and `gitverse.ru` had been UNVERIFIED for a reason that was not a measurement: **no adapter was registered for either host, so nothing had ever been asked**. Both now have one, and the mechanism that produced the red is named rather than merely removed — the red was *absence of an instrument*, not a failed probe.

before rows: 1 CONFIRMED · 6 UNVERIFIED · 0 HISTORICAL · 36 no-trigger · 1 out-of-scope (rc=1)

after rows: 1 CONFIRMED · 6 UNVERIFIED · 0 HISTORICAL · 36 no-trigger · 1 out-of-scope (rc=1)

The counts are identical and that is the honest result, not a failure.

What moved is what each of those six rows now SAYS. Every one of them used to read “*no read-only API adapter is registered for this host*”. They now read, per host and measured on 2026-09-08:

gitflic.ru host reachable – https://api.gitflic.ru answered HTTP 403 – but its

API requires a credential for every route; none is configured

gitverse.ru host reachable – https://api.gitverse.ru answered HTTP 401 – but its

API requires a credential for every route; none is configured

That is **FR-011c working**: “never asked” became “asked, and could not determine”. No row became a pass, and rc 2 remains reachable and is never recorded as compliance.

The two adapters have DIFFERENT ceilings, and the difference is the finding.

- **gitverse.ru CAN reach a clean verdict.** Its route set was established empirically against the live host rather than guessed: with the vendor media type application/vnd.gitverse.object+json;version=1, a route that exists answers **401** unauthenticated while a route that does not answers **400**. On that evidence the adapter asks only routes observed to exist — the repository object, the Actions workflow list, the Actions run history and the server-side webhook list.
- **gitflic.ru CANNOT, by construction, and says so on every row.** Its pipeline **scheduler** — the one standing, server-side, push-independent trigger it has — is configured in the project web UI, and its published REST API documents no route that reads it. So no read-only probe can establish that no schedule is armed, with or without a credential. The adapter still reads the project object, the mirror state and the pipeline history and reports them, then returns UNVERIFIED naming exactly the one question it could not put, and emits an operator-only MANUAL remedy pointing at the UI screen. **Reporting NONE here would be inventing an answer.**

Credentials: GITFLIC_TOKEN and GITVERSE_TOKEN, neither set on this host, so neither adapter has ever authenticated here. Every call is a GET; nothing mutates. The credential is handed to curl on **stdin** (curl -K -), never in argv, because an argv secret is readable by any process on the host through /proc.

§1.1 pairing — six new mutation cases, every one driven by DATA.

The adapters were not edited to make any of them pass; the lever is an environment variable naming a different API base (GITFLIC_API_BASE / GITVERSE_API_BASE, which self-hosted installations need anyway) plus a stub server serving different canned bodies. Each runs against a one-repository fixture tree, so it is hermetic — no case can pass or fail because of what some unrelated submodule's provider answered today.

```
bash scripts/verify-provider-ci.sh --selftest      # 18 passed, 0
failed (rc=0)

M8  gitverse unreachable          -> rc=2, row UNVERIFIED
M9  gitverse control GOES GREEN   -> rc=0, row NONE
M10 same fixture, one field moved -> rc=1, row CONFIRMED, cites
STANDING TRIGGER
M11 credential REJECTED          -> rc=2, adapter
state=broken, 0 clean rows
M12 gitflic ceiling holds even when every route answers 200 ->
rc=2, UNVERIFIED
M13 gitflic unreachable          -> rc=2, adapter
state=unreachable
M14 no credential appears as a curl ARGUMENT anywhere in the
file
M14b M14's own paired proof: the same rule finds a seeded leak,
ignores a comment
```

M9 and M10 are the pair that matters. A battery made only of “it refuses to pass” cases would be satisfied by an adapter hardwired to return UNVERIFIED. M9 is the control that must go GREEN and M10 is the identical fixture with one field changed that must go RED, so the verdict is proved to track the data.

No new file was added under scripts/, so the anti-drift half of the check registry is unaffected: bash scripts/verify-check-registry.sh is **rc 0**, and provider-ci's registered proof is still --selftest on the same entry point.

R2 — this document had fallen behind its own history

bash scripts/continuation-check.sh was **rc 1** at 7 PASS · 1 DRIFT · 0 UNDET · 15 NOTE. C3 named two commits that changed a watched governance file without updating this document in the same commit — bfe29319 and 738fbf15, both touching helix-deps.yaml. **The mechanism was a \$12.10 protection-2 miss, not a stale timestamp:** the work landed, the record did not move with it. This entry and the Synced-Commit field at the top of the file are that correction.

What this session did, verified in this tree rather than restated

Each line below was re-measured here on 2026-09-08 unless it is marked otherwise.

- **vasic-digital/curriculum-kit is published and mounted.** .gitmodules now declares **14** gitlinks (git config -f .gitmodules --get-regexp 'submodule\.*\path\$' | wc -l), including submodules/curriculum-kit. The standalone-clone gates live **inside the modules**, not at this root: workshop/scripts/verify-standalone-clone.sh and ai_interviewing/scripts/verify-standalone-clone.sh.
- **The export harness no longer has an unrunnable check.** bash _tests/run-harness-selfvalidation.sh is **rc 0** — golden-good verdict=PASS (rc=0, full coverage) and golden-bad DETECTED (rc=1). The OCR check now reports FULL-VISUAL/visual.ocr – OCR (container) ... recovered 79 legible words, so the family that used to SKIP now runs. **tesseract is still absent from this host's PATH** — the toolchain moved into a container through submodules/containers, it was not installed. The previous state was rc 2 / verdict=UNDETERMINED, and rc 2 was never a pass.
- **milosvasic.ru/_site is untracked.** Inside that submodule, git ls-files _site | wc -l is **0** against **807** files present on disk; the commit is e786ccd “*untrack the Jekyll build output — .gitignore:70 had been inert since it was added*”. Production is unaffected: that site publishes through its own server-side pages.yml workflow, which nothing here touched.
- **_tools/deploy-langs.sh no longer exits 0 on a failed build.** Its own header now declares build_fail <name> RAN and FAILED -> exit 5 and build_undet <name> COULD NOT RUN at all -> exit 2, with 5 outranking 2 so a missing toolchain can never launder a real failure, and an explicit override DEPLOY_TOLERATE_BUILD_FAILURE=1 that announces itself. The old behaviour was to increment a counter, print “tolerated”, and still exit 0.

- ****design-toolkit's two mirrors are IN SYNC for the first time in this record.**** `git -C design-toolkit ls-remote origin HEAD,`
`ls-remote gitlab HEAD and rev-parse HEAD` all return `e721b3c7eba8`,
so the lag is **0 / 0**. Every earlier figure in the carriers — 5, 6, 7, 9
commits behind — is **superseded**. A gitlab remote IS declared in
this checkout today, which the carriers' own withdrawal note said
was not the case; that note is now itself out of date and is left for
the carrier edit, not patched here.
- **The served platform answers.** `curl -s http://127.0.0.1:8087/`
`api/areas` returns **39** areas, measured directly. Three further areas
are held back.

Workshop-side figures — ATTRIBUTED, and the corpus was MOVING

The curriculum work (published areas, listed-but-unopenable areas, the learning catalogue, video anchors, question banks) happened inside the **private** workshop submodule and is recorded here **by path and count only**.

Do not read those figures as re-measured by this entry. They were produced by the agent that did that work, and when this record was written the corpus was demonstrably still moving under it: `git -C workshop ls-files curriculum/questions | wc -l` reported **17** tracked files while `ls curriculum/questions/*.json` reported **30** present — thirteen files written and not yet committed, by another agent working concurrently in that submodule. Under §11.4.6 a count taken over a moving corpus is not a finding, so no workshop count is asserted here as this session's measurement. The one figure that IS this session's own is the served `/api/areas = 39` above, taken from the running container.

Open and DELIBERATELY red — declared, not defects to chase

Three states below are red on purpose. Each names what would lift it.

1. **25 areas carry no assessment.** Structural, not an evidence gap — those area documents were never sectioned into the passage registry, so no citation can resolve and no question can be authored. Lifted by sectioning them (`specs/005-clone-and-clear-red/spec.md` FR-015, SC-011).
2. **Three documents fail publication review**, on **62 / 54 / 58** uncited claim blocks respectively. They remain unpublished, which is the correct state for a document whose claims do not carry citations. Lifted by citing the claims, never by relaxing the reviewer.

3. **The served interface paints 7 hue families against a floor of 6 — CLOSED.** The “2 families” reading is SUPERSEDED and was true when written. Re-measured twice on 2026-09-08 against build c6d039954ce2 with a stability bracket: `verify-served-palette.sh --base http://127.0.0.1:8087 → rc 0, 7 distinct hue families`, 208 surfaces graded, 0 unusable; `verify-served-contrast.sh → rc 0, 200 pairings` clear their floor in each scheme independently. Most of the gain came from **FIXING THE SAMPLER**, not from adding colour: a paint probe read `borderTopColor` without checking `border WIDTH`, and `border-color` initialises to `currentColor`, so every element reported the body **TEXT** colour as painted. Two taxonomy families were revealed, not invented. The count went *down* from 4 when the measuring instrument’s own three defects were fixed first — an honest instrument reporting a worse number is the instrument working. Lifted by FR-016..FR-019, and every hue added must carry a stated classification job.

Specifications in flight

- `specs/004-authored-ai-areas/` — full planning set (`spec.md`, `plan.md`, `research.md`, `data-model.md`, `contracts/`, `quickstart.md`, `tasks.md`, `checklists/`).
- `specs/005-clone-and-clear-red/` — `spec.md` plus `checklists/`. It is the register the reds above are numbered against (R1..R11), and its Clarifications section carries the four operator decisions this session worked under: Git LFS for archive parts going forward; the 387 baselined hardcoded paths are in scope as a per-repository campaign; **read-only adapters for both unprobed hosts** (done, above); and the governance-pin fetch is authorised with a fixed fast-forward-only sequence.

Reproduce all of it

```
bash scripts/continuation-check.sh           # 0 = in sync
bash scripts/verify-provider-ci.sh           # 1 = the ONE
confirmed finding stands
bash scripts/verify-provider-ci.sh --selftest # 0 = 18
passed, 0 failed
bash scripts/verify-check-registry.sh         # 0 = registry
closed both ways
bash _tests/run-harness-selfvalidation.sh     # 0 = good
PASS, bad DETECTED
git config -f .gitmodules --get-regexp 'submodule\.*\.path$' |
wc -l    # 14
```

```
git -C design-toolkit rev-parse HEAD # == both mirrors
curl -s http://127.0.0.1:8087/api/areas | jq length # 39
```

workshop COULD NOT BE BUILT OR RUN OUTSIDE THE UMBRELLA, AND THE REMEDY IT PRINTED WAS IMPOSSIBLE, 2026-09-07

Both modules were cloned from their real remotes and driven end to end — not inspected. Inspection says what a repo declares; only a clone says what it delivers.

ai_interviewing was already standalone and is the control that makes the workshop result meaningful: `build.sh 0`, `start.sh 0`, `health 200`, its own challenge bank **24 / 0 / 0** aimed at its own port and database, and **no replace of any kind** in `go.mod`.

workshop did not build at all.

```
go: replacement directory ../../../../submodules/LLMProvider does not exist
```

Five dependencies — containers, passage, verdict, RAG, LLMProvider — resolved through relative replace paths walking **three levels up, out of the repository. Nothing in the module declared them:** no `.gitmodules`, no manifest. Inside the umbrella it works and nobody notices.

And setup.sh told the operator to run a command that cannot work — `git submodule update --init submodules/containers`, which is `rc 1`, `pathspec` did not match because the path is not a submodule — then misreported the failure as *“network or credentials”*. The remedy is now derived from whether `.gitmodules` exists.

`helix-deps.yaml` (new) declares all five with pinned refs. `workshop/scripts/bootstrap-standalone.sh` (new) clones them and writes a `go.work` overlay — **no `go.mod` is edited**, so the umbrella layout stays the single source of truth and standalone is the overlay.

A FRESH CLONE SERVED AN EMPTY CORPUS. `main.go` derives the registry path from `-index = /var/lib/workshop`, a named volume **nothing in the repository ever populates**, while the tracked `curriculum/passages.jsonl` sat mounted read-only and unread. `compose.yml` now passes `-registry` explicitly.

standalone api-challenges.sh before 96 passed / 9 FAILED / 8
undetermined
after 112 passed / 1 FAILED / 0
undetermined

End to end after: bootstrap 0 → build 0 → vet 0 → setup.sh --yes **0**
READY → build.sh 0 → start.sh 0 → health **200**.

workshop/scripts/verify-standalone-clone.sh and ai_interviewing/
scripts/verify-standalone-clone.sh — one in each module, NOT at
this root — are the repeatable proof (8 passed each, mutations as DATA
in throwaway git repos); workshop/scripts/bootstrap-standalone.sh --
prove-failure is 6 passed. The workshop gate found the last two
impossible-remedy instances itself.

A SKIP THAT EXITED 0, in ai_interviewing/scripts/validate-
exports.sh: a standalone clone reported the \$11.4.168 export check as
PASSING while nothing had been validated. Its caller then collapsed
the third state with || exit 1. Both halves moved; fixing either alone
leaves the contract broken the other way.

STILL BLOCKING A STANDALONE workshop, unclosed: curriculum/
taxonomy.jsonl is git-ignored and needs a ~700 MB venv, so /api/
areas/... answers 503 while **setup.sh still exits 0 READY** — a real
gap. TestGateKG1 grades this module against a contract the UMBRELLA
owns; the spec was deliberately NOT copied in, because that creates a
second source of truth — **an ownership decision for the operator**.
And with WORKSHOP_PROJECT_NAME set, api-challenges.sh fell back to the
literal 127.0.0.1:8087 and reported 110 passed **against the umbrella's**
server — the cross-contamination class flagged, not fixed.

UNDETERMINED: the compose -registry change is **verified by**
equivalence, not in place — the same flag, mount and file produced
the same pid_count in the standalone clone, and the two files are byte-
identical (sha256 40e7cd00...), but proving it on the live container
requires the restart that follows this entry.

AREAS WERE UNREADABLE, AND FIXING THAT
REVEALED THEY ARE NOT AI-TECHNOLOGY AREAS AT
ALL, 2026-09-07

Reported from manual testing: knowledge areas rendered as raw
ULIDs. Root cause was **upstream of the UI** — /api/areas returned 819
areas of which 5 carried a title, so the UI rendered the only string it
had.

workshop 4fb9f99 derives titles, tags and summaries at the publish boundary from data the row already carried (member_terms, each an evidenced row with a canonical_form and significance_score). **819/819 titled, summarised and tagged**, verified on the live endpoint; determinism proved by two independent HTTP runs producing sha256 a65f46f5...65ecde over the whole projection. A NAME gate now joins the evidence gate, reporting into the existing held_back structure rather than dropping silently.

AND THE FIX MADE THE REAL PROBLEM VISIBLE, WHICH IS THE POINT. The user's requirement is that areas be *AI-technology areas actually taught in the videos*. Measured against a list of AI/ML terms:

evidence floor	extracted kept	of those, AI-termed
1 (implemented)	814	32 (3.9%)
2	36	0
3	8	0

778 of 814 rest on exactly one passage; 813 of 814 have a single member term. derive.py:propose_areas emits every unlinked term as its own area by explicit design (E1: propose, don't discard) — correct for a MINER; what was missing is any selection between the miner and the API. **These are transcript vocabulary, not AI topics, and no evidence threshold fixes that** — the 0% column is the proof. Selection needs topic-relevance or authored areas, and that is an operator decision. MinPublishedEvidenceCount is implemented at **1 so nothing is deleted**, with a lockstep assertion and a Go test that both fail if it moves.

A LESSON LINK TO “CHAPTER 01 AT 10:00” THREW THE TIME AWAY ON ONE OF TWO ROUTES

Measured before changing anything: /chapters/01/transcript?t=600 seeked to **602.5 s** and played; /chapters/01?t=600 left currentTime at **0**. ChapterDetailComponent subscribed to paramMap and never queryParamMap.

Much of what was assumed missing already existed and was already proved — #p-<pid> scrolls, seeks and plays, and auto-scroll-to-playing-position is real with a WCAG 2.2 SC 2.2.2 pause control. One contract now, on both surfaces, documented in platform/frontend/docs/time-links.md because the new curriculum-kit targets it:

/chapters/<slug>[/transcript][?t=<seconds>][&end=<seconds>][#p-<pid>]

Malformed input is IGNORED, never coerced — a mistyped ?t= must not seek to 0 of a 1h55m recording. playback.ts now clears an unconsumed seek on detach(), because it followed the reader into the NEXT recording. **155 passed / 1 failed → 162 passed / 0 failed**; the 1 was a pre-existing scrolly exact-equality race.

submodules/curriculum-kit — AND THE MECHANISM WAS NOT PORTABLE

The brief said the lesson/test mechanism “can be picked up from the ai_interviewing”. **It cannot: it does not exist there.** Grep-confirmed across that backend and frontend — no gating of a test on lesson completion, no completion state, no pass threshold, no server-side grading, no video, no time ranges, no typed material kinds. Scoring is entirely client-side; a “test” is questions filtered by kind. **This is new work, and the port framing would have produced something far thinner than asked for.**

Built as a Go module with an **empty require set**, so project-not-awareness is compiler-enforced. Catalog → Area → Lesson → Material + Area.Assessment; materials cover illustration, diagram, scheme, graph, video and document;

VideoAnchor{chapterId, startMillis, endMillis, transcriptAnchor} is a RANGE. go test 94.3% coverage, --prove-failure **32/32** with mutations as DATA fixtures, four carriers in lockstep (1 distinct normalised digest), leak scan clean with a live positive control.

Three design decisions came from defects found in ai_interviewing: answers are named **by id, never index** (its omitempty index had made 40 of 312 items unanswerable); Submit on a locked assessment returns no populated result, so an ignored error cannot read as “scored 0”; a gate over an empty required set does **not** open.

design-toolkit ddb9b65 — TWO “DISTINCT” BRANDS SHIPPED THE SAME COLOUR STRUCTURE

Reported as monotone/sepia. Measured: **neutral 36% / warm 37% / cool 27%**, mean saturation 0.34. Worse than the complaint — **both brands occupied the identical 3 hue bins over 120°, in both themes.** vector.harmonyRule was derived per seed, printed into two comments,

and consumed by nothing: a free axis spent on a comment. Now **4–5 bins / 150–180° / cool 18.4%**, with a 36-seed sweep minimum of 4 bins / 150°.

#d97706 is gone — it sits beside the warm-clay accent the frontend-design skill names as the commonest AI-generated signature, hand-painted inside a module whose premise is derivation.

The agent rejected its own first plan, which added ramps no stylesheet references — moving the metric while changing zero pixels. It re-aimed the second hue at tokens already painted (the dashed diagram plate, every focus ring). A 36-seed sweep also found all Monochrome primaries emitting the identical #5e5e5e, so every such brand got the same counterpoint. T1–T5 all still PASS, T4 at 32 pairs min 3.24:1. New anti-monotone gate whose proof's load-bearing row takes the **actual committed pre-fix candidates from git HEAD** and asserts rejection.

**THE LAST TWO CONTRACT VIOLATIONS ARE CLOSED,
AND THE FIX I ALMOST SHIPPED WAS WORSE THAN
THE DEFECT, 2026-09-07**

workshop f5a3ab1. The QA bank is **111 passed / 0 FAILED / 0 could-not-determine** against a server rebuilt from this tree. B5 and C5 read `http=400 code=unknown_parameter`.

The green was audited rather than accepted. The total moved 113 → 111, so two checks that PASSED before are absent. `g_ok/I4b` fires only when `status == unavailable`, `g_ok/I8` only when `status != ok`; before the restart `g_ok` returned `unavailable / suggest_timeout / leg=lexical`, after it returns `ok`. Both conditionals correctly stopped applying — **no coverage was lost, and that improvement is the RESTART, not the fix**, so it is not claimed as one.

The §1.1 pairing that shipped missing. Six mutations, every one DATA — a different allowed `...string` argument — never an edit to `rejectUnknownParams`. **6/6 pass on the real code; 4 of 6 FAIL against a fail-open mutation**, sha256 confirming the restore. The 2 that still pass are the acceptance controls, which is correct: without them a rejector that refused *everything* would satisfy the other four. M3 proves an EMPTY allow-list refuses rather than fails open; M5 fires 200 identical requests to prove the offending field is deterministic, because the function scans a map and Go randomises map iteration.

THE FIX I ALMOST SHIPPED WOULD HAVE BEEN WORSE THAN THE DEFECT. The term card rendered href="" — a link back to the current page — because h.deep_link ?? fallback never fires on an empty string. My first correction kept the fallback and guarded on chapter_slug. Measuring 160 rows over eight live queries stopped it:

kind	rows	destination
transcript_segment	44	deep_link supplied
doc_section	20	chapter is real
doc_section	17	chapter docs — not a chapter
code	14	chapter docs — not a chapter
term / kg_term	65	no chapter at all

/api/chapters reports 01, 02, 02.01. That fix would have pointed **31 rows** at /chapters/docs/transcript, and the SPA answers 200 for any path, so they would have LOOKED like working links and failed only on arrival.

Then the fact that settled it: **the deep_link key was absent 0 times, empty 116 times, non-empty 44.** The fallback has NEVER EXECUTED. So it is deleted rather than repaired — the obvious one-character fix, ?? → |, would have woken dead code and shipped the 31.

Proved in both directions: fix in place **11 passed**; deepLink mutated back to ?? and rebuilt → **1 FAILED, the new test and only it**; restored byte-identical by sha256; restored bundle rebuilt → **11 passed**. The other ten stayed green under the mutation, so the assertion isolates this behaviour.

The contract clause that made the original mistake look safe is corrected in specs/001-.../contracts/http-api.md. It declared kinds a “*Subset of transcript, doc_section, code, diagram*”; spec 002 added catalogue kinds and never amended it, so a QA challenge trusted the stale clause, composed "/api/passages/" + pid itself, and reported the resulting 400 malformed_pid as a SERVER fault across four checks. transcript and diagram do not appear in the live index at all.

api-challenges.sh	111 / 0 / 0	go build · vet · test	0 · 0 · 0 (19 pkgs)
ng build	0	platform health	200

GATE 5 IS rc 2, AND IT IS THE GATE WORKING — tesseract IS NOT INSTALLED, 2026-09-06

bash _tests/run-harness-selfvalidation.sh exits 2, and the reason is an absent toolchain rather than a defect:

```
[export-validator] tools: pdftotext=ok pdfimages=ok pdftoppm=ok  
tesseract=MISSING  
[SKIP] FULL-VISUAL/visual.ocr – tesseract missing – cannot OCR  
rendered pages  
[export-validator] checks: 5 PASS / 0 FAIL / 1 SKIP of 6  
[export-validator] verdict=UNDETERMINED
```

poppler-utils IS present — pdftotext, pdfinfo and pdftoppm all resolve — so 5 of 6 checks run. **The detector is demonstrably live:** the golden-bad arm still returns rc 1 and names two CONTENT faults. The gate simply refuses to call the golden-good arm a pass while a check family did not run, which is exactly the three-valued contract. **rc 2 is never a pass**, and this one is not being recorded as one.

Remedy is a package install and therefore an operator action: sudo apt-get install -y tesseract-ocr. It was NOT performed.

This is the second documented toolchain in ## Build and test entry points that is absent on this host — the other being bundle / bundler / jekyll, which puts gate 6's precondition into CANNOT DETERMINE. Both are now recorded in the carriers beside the toolchain list rather than left for the next reader to rediscover by running a gate and being surprised.

verdict e4b2f6e AND passage ad5504d — THREE DOCUMENTS CLAIMED GATES THAT DID NOT EXIST

Both were extracted from the private tree on 2026-09-01 under §11.4.74 and had never been audited. All three defects are the same shape — a document asserting enforcement by a gate that is not there, which is worse than an undocumented gap because a reader who trusts it stops looking.

verdict/README.md named TestNoConsumerShapedDependencies as what “fails if a consumer-shaped symbol is introduced”; grep -rn NoConsumerShaped over the whole module matched **only that sentence** — the test belongs to passage. verdict claimed “each gate is accompanied by” a paired mutation while its two STRUCTURAL gates shipped none. passage's two decoupling gates shipped none while both

the file header and README said each does, and CHANGELOG.md named “eight gates, four of which ship a paired mutation” for a file holding seven with two.

Every one was closed by RAISING THE SUITE, not by softening the claim — scanners extracted so mutations drive the real gate code, seeded go.mod content, seeded paths, and a schema mutation that ALTERS a real built database to re-add a retired v0.1.x column. All proven live: neutering the shared scanner makes the mutation report MUTATION SURVIVED. **verdict 20/3 → 22/5; passage 61/4 → 63/6.** Both green on build, vet, test, -race and -shuffle=on, with 0 skipped tests.

Unprompted evidence one of these gates genuinely works: a dictionary word was accidentally written into README prose while drafting, and TestNoConsumerShapedVocabularyInSource caught it on the next run at README.md:106. The prose was fixed rather than an exemption added.

Clean findings with their evidence: three-valued exits DRIVEN (missing binary → 2, dead port → 2, empty tally → **2 not 0**, problem + undetermined → 1); the empty-set family already defended by t.Fatal with a “would pass by looking at nothing” message; zero SIGPIPE instances and structurally so; and no private content carried out of the extraction — the single occurrence of workshop is an entry in passage’s own BLOCKLIST dictionary, a word being forbidden.

Reported and NOT fixed: passage/helix-deps.yaml records ref: main for verdict while go.mod pins the immutable tag v0.1.1. passage has no .gitmodules, so no gitlink exists for C9 to compare.

THE SWEEP’S 96 FAILs ARE NOT FABRICATED BY SIGPIPE — HYPOTHESIS TESTED AND REFUTED BY MEASUREMENT, 2026-09-06

A fleet sweep found **224 real x | grep -q PAT verdict-position sites under pipefail**, of which 204 sit in repositories the sweeping agent could not edit. The strongest hypothesis it raised was scripts/verify-all-constitution-rules.sh — set -uo pipefail at line 102 and 36 such sites — as the possible source of some of the sweep’s **96 FAILs**.

That hypothesis is REFUTED for today’s tree, and refuted by measurement rather than by reading. Two fail-open classifier populations were measured against the real corpus:

- **Header tests** (SOURCED / SUBCOMMAND / WRITES-BARE). 286 gates scanned; 34 headers exceed the 4096 B threshold, the largest

32,304 B. The number where one of those patterns matches with **more than 4096 B still to write** is **ZERO**.

- **Helper test** (`_names_this_file_as_helper`). Across all **666** (gate, sibling) pairs where the basename appears, the worst case is **641 B** remaining after the first non-comment line — against a 4096 B threshold.

So no current FAIL is manufactured by this defect. That is a real negative result and it closes the fleet’s strongest open lead.

The mechanism itself is narrower and sharper than first briefed, and the earlier “~64 KB pipe buffer” figure is WITHDRAWN. Two conditions must BOTH hold: more than **PIPE_BUF (4096 B)** still to write after the match, AND a **line terminator** after it — GNU grep is line-oriented, so a body with no newline is read to EOF and the writer is never cut off. Reproduced independently here:

bytes	newline after match	FAIL branch taken
4096	yes	0/100
16384	yes	100/100

262144 yes 100/100 262144 no 0/100

The four sites were converted anyway, and the two claims are kept apart. Every one is FAIL-OPEN: a spurious 141 makes the `if` false, the file is not classified as a non-gate, and it is swept as though it were one — manufacturing a FAIL about a helper that was never a gate. The comment on `_names_this_file_as_helper` already records that an earlier defect did exactly that to `lib/covenant_propagation_mutation_engine.sh` by a different route. All four are live code paths, not dead ones: the classifiers fire **HELPER 6, FIXTURE 6, SOURCED 2, WRITES-BARE 1, SUBCOMMAND 1** on the real corpus.

A here-string is not a pipeline, so `pipefail` has nothing to promote and the status is grep’s alone — same regex, same per-line anchoring. **Behaviour preservation is proved, not asserted: --list output is BYTE-IDENTICAL across all 270 discovered gates, both rc 0.**

A CHALLENGE CERTIFIED A PANICKING BINARY AS PASSED

`submodules/RAG 2dffc73, submodules/LLMProvider 4c73c8b, submodules/containers 7f59225`. Driven with a fake binary printing a Go panic followed by 200 KiB of multi-line padding, `ui_terminal_interaction_challenge.sh` **caught 0 of 20** and printed ===

RAG UI Challenge: PASSED === at rc 0. Fixed: **20 of 20**, FAIL: --help leaked: panic:, rc 1. An anti-bluff challenge certifying a panicking binary as clean.

20 verdict sites fixed across the three repos, with behavioural equivalence proved at 13/13 cases. A second, independent bug at the same site in containers: `assert_no_panic`'s loop ended on an `&&` list, so it returned the last iteration's status — 1 exactly when NO hostile pattern was found — and every caller is `... || exit 1`, so it fired for every WELL-BEHAVED binary.

A guard now exists at `submodules/containers/scripts/anti-bluff/`: three-valued, keyed on `path:ID` so a line move does not churn its baseline, **11/11 mutations all DATA**, and it caught two of its own fixtures passing for the wrong reason while being written. `make sigpipe-verdict-scan` is clean over 688 paths; a seeded tripwire returns rc 1 and naming it, rc 0 once removed.

Routed but NOT fixed — 204 findings in repos that sweep could not edit: workshop 95, umbrella root 84, submodules/constitution 24, superspec 1. Whether any of them has ever fired is **UNDETERMINED**; none was tested. The umbrella's 84 are now 80, the four above having been closed.

FOUR CARRIER CLAIMS WITHDRAWN AS MEASURED FALSE, AND A DESIGN SYSTEM WITH NO COLOURS PASSED A CONTRAST AUDIT, 2026-09-06

The four root carriers carried claims that were false, not merely stale, and they are corrected in lockstep — the shared region edited ONCE and recomposed, which is what the carriers themselves prescribe. C5 ROOT-LOCKSTEP passes on the new shared digest, so the four remain byte-identical from line 19.

1. *“milosvasic.ru's rendered `_site/` is git-ignored.”* It is **TRACKED**. `.gitignore:70` does say `_site/`, but the rule was added AFTER the files were tracked, so it does nothing. The config claims it and git disagrees.
2. *“gate 6 validates a STALE artifact ... a six-day-old build.”* The defect is **completeness, not age** — source and `_site` are 89 ms apart.
3. *“ruby and bundle are present; the jekyll gem executable is not.”* **bundle, bundler AND jekyll are all absent from PATH**, so the remedy the same paragraph names cannot even be started.
4. *“A gitlab remote **IS** declared for design-toolkit.”* **It is not.** `git remote -v` there returns `github`, `origin` and `upstream`, all three

pointing at GitHub. The commit that entry credits, 7d9240e, is real and titled “*record the GitLab mirror*” — but what it added is the inert recipe `upstreams/gitlab.sh.disabled`. **A recipe recording a mirror is not a remote, and the carrier conflated the two.** What survives: the lag is still re-derivable here without any remote, because both commits are in the object store. It is now 7, was 6 earlier the same day and 5 on 2026-09-01 — it moves whenever the GitHub side advances, which this session made it do.

`scripts/verify-check-registry.sh` is likewise corrected from “43 then 45 PASS” to **57 PASS / 0 FAIL / 1 DEBT**, and `audit-hardcoded-paths.sh` from RED to green — **by a REAL FIX, not a re-baseline**: the absolute developer path is gone from that JSON (0 occurrences of `/home/`), the file is not allow-listed, and the audit script is byte-unmodified from HEAD.

design-toolkit a135aa8 — the empty-set family of vacuous PASS

Feed `qa/run-checks.mjs` a structurally valid DTCG document with the color group deleted and it printed D2 contrast: PASS (0 pairs; min text Infinity:1) and OVERALL: PASS, exit 0. A design system with no colours passed a contrast audit, because the audit iterated an empty set of pairs and an empty conjunction is true. Every C-PLAT contrast floor did the same. The same shape one level down: T1 coverage is an ABSENCE assertion with no positive control, so an empty brand file gave PASS (brand defines 0 ... 0 missing).

Two infrastructure faults were reported as content verdicts: with `generators/node_modules` absent, Node’s own exit 1 from `ERR_MODULE_NOT_FOUND` reached the caller as “your tokens FAIL” on a machine that had simply never run `npm install`; and an unreadable `--tokens` file did the same. Both are 2 now, verified by driving them.

No paired proof existed for any of its three gates — `golden-bad-tokens.css` was referenced only by prose and nothing executed it, and the captured “negative controls” file records OVERALL: FAIL followed by exit: 0, which is the defect sitting in the evidence. `qa/prove-three-valued-exits.sh` is now 14/14 with every mutation DATA, and **6 of the 14 fail against the pre-fix gates restored via `git show HEAD`:**

Open and deliberately not closed: `check-tokens.mjs` exits 1 on both real candidates — T1 coverage: FAIL (brand defines 86, candidate defines 75; 11 missing). The gate is right and the candidates are stale;

closing it needs a generator change (dtcg-to-od.mjs emits no diagram or status family at all) plus the brand CSS in this umbrella, so it is not an audit edit.

THE BROWSER SUITE'S "302 PASSED / 0 FAILED" IS WITHDRAWN — IT IS 69 FAILED, AND ALL 69 ARE ONE MISSING BUILD, 2026-09-06

Gate 6's own command was re-run verbatim — `cd _tests && npx playwright test --project=chromium --grep-invert 'all-language':`

69 failed · 163 passed · 1 flaky · 6 skipped (4.7m)

69 of the 70 listed failure lines name milosvasic.ru. Not one is a vasic.digital defect. `_tests/playwright.config.js:17` serves `milosvasic.ru/_site`, and that directory holds **8 tracked files and exactly one rendered page**, against ~180 front-matter source files. It is a hand-staged fragment, not a build.

Three claims this document and the carriers carried are WITHDRAWN as measured false, not merely stale:

1. *"_site/ is git-ignored."* It is **TRACKED** — 8 files. `.gitignore:70` does say `_site/`, but the rule was added AFTER the files were tracked, so it does nothing. The config claims it; git disagrees. That inert rule is exactly why the carriers describe it as ignored.
2. *"gate 6 validates a six-day-old build."* Not age — **completeness**. Source and `_site` are timestamped 89 ms apart. Served alone, `_site` answers **1 of 525** sitemap URLs; the other 524 are 404, as are all 180 article fragments and all 45 download PDFs. It ships no `assets/od/` and no `assets/js/`, so even its one working page renders without stylesheets.
3. *"the remedy is bundle install inside milosvasic.ru."* That cannot even be STARTED here: ruby and gem exist, but **bundle, bundler and jekyll are all absent from PATH**. The gap is one step wider than recorded.

Production is NOT affected and this was checked, not assumed. `.github/workflows/pages.yml` runs

`bundle exec jekyll build --destination _site` and uploads the REGENERATED tree, so the committed `_site` is overwritten by that build and never read. Its whole cost is local, and the cost is that a green gate 6 on the `milosvasic.ru` half was evidence about eight files.

What was fixed is the REPORTING, because 69 red tests for one missing build is an infrastructure fault wearing content-failure red. New `_tests/preflight.js` answers one question — can this suite honestly run? — with two verdicts and deliberately no third: **0 READY** or **2 CANNOT DETERMINE**. It never exits 1, because a preflight has read no site content and may make no claim about it. The route roster is DERIVED from the specs (each spec's single `require('../env.js')` binding line, then its `goto()` calls and its table-driven `{base:, path:}` rows), so a spec added tomorrow is covered without editing the file. It reports **6 missing PAGES**, and says so in those units: `seo-meta.spec.js` alone reconciles exactly, $2 \text{ pages} \times 6 \text{ assertions} = 12 \text{ failures}$.

It also closed two holes in `scripts/pre-push-gates.sh`'s own precondition. That precondition tested for `milosvasic.ru/_site/index.html` — *the one file a fragment is guaranteed to have*, so the fragment passed it. And nothing asked whether a browser could START: `@playwright/test` is installed here and **webkit cannot launch at all** (Host system is missing dependencies to run browsers), because the failure is in system libraries rather than npm, so a package-presence check was checking the wrong thing. Verified both ways:

run	result
PREPUSH_ONLY=6	● SKIP gate 6 with the full stated reason, rc 0
PREPUSH_ONLY=6 PREPUSH_STRICT=1	PUSH BLOCKED – 1 gate(s) failed, rc 1

so it cannot become a hiding place. \$1.1 paired `proof_tests/prove-preflight.sh` is **8 passed / 0 failed**, mutations as DATA only, including an M0 vacuity control — without it, “every mutation was caught” is satisfied by a program that returns 2 unconditionally — and M6 asserting the never-exits-1 contract. Two of the eight failed on first run; both were **wrong assertions in the proof, not defects in the preflight**, and the proof was fixed rather than the code.

Left for the operator, NOT acted on: `_site` is dead weight that the production build ignores and the local harness mistakes for a site. `git rm -r --cached _site` removes nothing from disk and has no production effect, but deleting tracked files from a live public site is an operator decision. Until then gate 6's `milosvasic.ru` half honestly SKIPS.

FOUR SUBMODULES: GATES THAT REPORTED PASS FOR WHAT THEY NEVER RAN, 2026-09-06

One defect class, found independently in four repositories: **a check reporting a CONTENT verdict for an INFRASTRUCTURE fault**. Each is committed and pushed.

- **submodules/RAG 13f4bac**. The unit gate ran `go test -short`, under which four packages self-skip entirely. It tested **116 of 143** test functions and printed Results: 4/4 passed — because a package that skips everything exits 0 and the gate read the runner's exit code. All 27 skipped tests pass in full mode and need no external infrastructure, so the hole was not a dependency to document. Now **143 declared / 143 executed / 143 passed / 0 skipped**, plus 143 under `-race`, with a new assertion comparing DECLARED (counted from source) against EXECUTED (counted from the runner) — two independent sources, so a runner that silently runs nothing is caught. Six scripts that printed PASSED (SKIP-OK) now exit 2.
- **submodules/LLMProvider e2c6b7b** (on master — see the branch note below). Eight scripts, not the two briefed, returned 0 on absent preconditions. Seven timing tests asserted a **wall-clock ceiling** on a jittered retry — a claim about how busy the machine is, not about the code; five of seven broke under deliberate load. Each now keeps its *lower* bound (load can only delay a timer, never fire it early) and checks the real property over 100,000 samples. Making `IsOpenCodeInstalled()` truthful — it was return `false`, keeping three tests dark since 2026-03-19 — revealed that the unit suite would then make **real model calls** and that `HealthCheck()` would **spawn a daemon** as a side effect; both are now gated.
- **ai_interviewing 712a9c9**. A stopped server produced **19 content FAILs** rather than undetermined. An impostor on the recorded port was **certified** (`[PASS] C1 health ok=true (v9.9.9)`) and then used as the reference that accused the REAL TLS endpoint of being “a DIFFERENT server”. Driven at a dead port it is now **rc 2, 0 content FAILs, 21 UNDETERMINED**, each naming its reason. A fifth defect was found by measurement rather than from the brief:
`X | grep -q PAT under set -o pipefail fails because the pattern was found — grep exits on match, the writer dies of SIGPIPE, pipefail promotes it. Measured 1 spurious FAIL in 10 runs, and 214 non-zero in a 3000-iteration harness.`
- **_tools/ (umbrella)**. `reproducibility-selftest.sh` printed a green PASS and exited 0 for a population it no longer had — driven with all eight Python subjects deleted. Registry scanroots widened
`scripts tests → scripts tests _tools _tools/translate _tools/`

portfolio; verify-check-registry.sh is **57 PASS / 0 FAIL / 1 DEBT**, the debt loud on every run rather than papered over as an exemption.

Branch note, measured because “work on main only” has one real exception. submodules/LLMProvider lives on **master**: its main is 16 commits behind and a strict ancestor, last touched 2026-07-12. submodules/containers is the opposite — main live, master genuinely diverged (207 / 41, not fast-forwardable either way). Both remote HEADs point at the live branch, both gitlinks match exactly, and helix-deps.yaml already records which is which.

SEARCH SERVED A WINDOW OF PURE VOCABULARY — FIXED AND MEASURED, 2026-09-06

workshop at 24e14b8. A reader searching a common word got a results page with **nothing to click through to**: every row was a knowledge-catalogue entry and none linked to a chapter or passage. Six of twelve reader-shaped queries returned **zero** passages.

The mechanism was bm25 length normalisation, not a ranking opinion. bm25 divides term frequency by document length, so a catalogue row of a few tokens scores structurally higher than a passage of a few hundred for the SAME query term. The two populations never interleaved — they formed disjoint bands, and the catalogue’s worst row outscored the best passage.

Fixed with a declared presentation quota reusing the design that module already applies to its type-ahead endpoint (§11.4.74). Measured on an identical index — same root hash before and after, only the binary changed:

	before	after
twelve reader queries	220 catalogue / 20 passages	96 / 144
queries serving ZERO passages	6 of 12	0 of 12
the frontend suite for that surface	14 passed / 6 failed	20 passed / 0 failed

The gate whose targets are ALL catalogue rows — the thing a cap on catalogue was most likely to break — is **unchanged** at top-5 19/22, top-1 18/22. Workshop gates: **PASS 32 / FAIL 2 / COULD-NOT-RUN 2**, identical to before the change.

THE LESSON TO KEEP IS THAT THREE INSTRUMENTS EACH CAUGHT A DIFFERENT MISTAKE BEFORE ANY OF IT SHIPPED, and none of the three was a human review:

- go vet rejected a compile error in the new test file before it ever ran.
- The first mixed-population test **FAILED** on its first run — the fixture was wrong, not the code, and the withholding rule was working exactly as designed.
- The first attempt at the second-layer fix was rejected by two pre-existing contract tests in ninety seconds: it reordered results even when it was not truncating anything. **A selection function that permutes a list it was not asked to shorten is doing something it was not asked to do.** That property is now asserted at five widths.

A GREEN TEST RUN WAS WEAKER EVIDENCE THAN IT LOOKED.

The whole backend passed, but every pre-existing fixture on that code path carries single-population data and takes the new function's identity return, so the mixed path was never exercised. *"Nothing broke"* and *"the new behaviour is covered"* are different claims and only the first was supported. Recorded in the module's own docs/limits.md §10.17 rather than banked.

AN OPERATOR DECISION IS OPEN AND NOT TAKEN. The quota now composes every common single-word query as exactly 8 catalogue / 12 passages. That is it working — and one query composed itself sensibly (11/9) with no quota at all and is now served 8/12. A presentation quota overrides composition even where composition was fine. The share is ONE constant, argued rather than measured, deliberately in one place so changing it is visible. Making it a ceiling that engages only when a population would otherwise be starved is a different and reasonable design; it was not taken here.

DETERMINISM SWEEP — 2026-09-06 evening

Operator directive: **all results, validation and verification must be FULLY DETERMINISTIC, always.** A check that passes five times in six is not a passing check; it trains everyone to re-run until green.

A RETRIEVAL LIBRARY RETURNED DIFFERENT DOCUMENTS FOR IDENTICAL INPUT. submodules/RAG at 42a1428. Seven sites collected results with for `k := range someMap` — Go randomises map iteration — into an UNSTABLE sort.Slice with no tiebreak. Measured over 2000 identical calls per site:

site	distinct orderings
MMRReranker.Rerank	113
RRFStrategy.Fuse / LinearStrategy.Fuse / KeywordRetriever / MultiRetriever	5 each
HybridRetriever.Retrieve TopK=1	2 — which document is served changed

MMRReranker had **no sort at all**; the winner of every greedy tie was whichever map key came out first. Two of the seven were not in the brief. Fixed with sorted-key collection **plus** a total comparator — a stable sort over a randomised range is not determinism, because stability faithfully preserves the randomness. After: **1 ordering per 2000** everywhere, clean under `-race` and `-shuffle=on`.

Blast radius: pkg/pipeline was NOT insulated. pkg/grounding was not either, and instructively — *its own sort is stable, which is exactly why it could not save itself*. challenges/runner was “insulated” only by weakened assertions: it checked sortedness order-insensitively and asserted the top id was in an expected SET. **The gate had adapted to the bug rather than catching it.**

A MUTATION TEST THAT COULD NOT FAIL, FOUND INSIDE THE MUTATION PROCESS. The first tiebreak mutation SURVIVED: a 5-document all-tied fixture cannot detect a missing tiebreak, because Go’s pdqsort recognises all-equal input and returns it untouched — 0 deviations in 200. Replaced with a 24-document fixture whose ties are interleaved with strict differences and whose ids are scrambled; it deviates 200/200 and the mutation then died.

THREE GATES THAT COULD REPORT SATISFIED WHILE PROVING NOTHING. - `_tools/portfolio/self-validate.sh` accepted ANY non-zero as “mutation caught”, and the validator returned 1 for both a parse error and a real finding — so a **corrupt or absent fixture read as a caught mutation**. Fixed in the required order: validator now exits 2 when zero assertions were evaluated, *then* the harness tightened. - `_tests/visual/visual-oracle.js` had its `require` outside the rc-2 guard,

so a module-load crash exited 1 — and the harness asserted exactly 1 for its “bad” arm. **Crash and detection were indistinguishable.** Observed: PASS: golden-bad verdict=FAIL (rc=1) printed by an oracle that never opened the fixture. - _tests/export/validate-pdf.js absorbed a SKIP into SATISFIED. Now three-valued with FAIL(1) > UNDET(2) > PASS(0). Its page scan also readdirSync'd whatever was on disk and sorted lexicographically — with two stale decoys present it would have reported “*OCR of 2 rendered pages*”, **a count matching the true page count by coincidence**, while reading none of that run's output.

A CLOSED FINDING WAS NOT CLOSED. CLAUDE.md records F13/F14 as CLOSED, but one spec of 22 still hardcoded ports. At non-default ports a connection refusal reached the assertion as Received: 0 and was **reported as a defect in the production website.** Fixed; the failure changed identity to Received: 404, which is a REAL finding — milosvasic.ru/_site has no sitemap because Jekyll cannot build on this host, the defect already recorded here.

COMMITTED EVIDENCE CARRIED AN ABSOLUTE PATH FROM ANOTHER MACHINE — /run/media/.../DATA4TB/... — beside "generatedAt_omitted_for_determinism": true. The clock was made reproducible and the path was not. All three writers now record repo-relative paths, normalised at the single serialisation point so a field added later is covered automatically. **0 absolute paths remain, and the verdict JSONs are byte-identical across two consecutive runs.**

ALSO: the ai_interviewing port-fallback test asserted a property of the whole machine (it assumed base+1 was free after asking the kernel for base). Invisible serially — 25/25 pass — and 4 failures in 40 concurrent process-runs. Now the test OWNS its ports; 7 production mutations all caught; 40 concurrent runs clean. Three challenge checks used SELECT ... LIMIT 1 with **no ORDER BY**, letting SQLite choose which row to judge; and the suite runner omitted -count=1, so Go's cache could replay a stale verdict for tests that bind real ports.

STRONGEST POSITIVE RESULT: workshop/platform/backend — 132 test files, 14 full-suite runs (baseline, 8× -shuffle=on, -count=3, -race, three under load average 90–113) — **no ordering, concurrency or state-leakage finding.**

A REFUTED HYPOTHESIS, recorded because a near-miss is a result. The constitution sweep looked locale-broken: the same 270 gate names sorted under C vs en_US.UTF-8 and compared yields 130 spurious drops

and 130 spurious adds. But the real code re-sorts BOTH sides at runtime under one locale; faithful emulation gives `missing=0 added=0` under both. The gate is locale-safe.

SEMGREP is disabled in **36 of 36** profiles (was 6 — thirty had the key `ABSENT`, and `absent` is not `false`), its hook dispatcher is neutered and verified inert, and the respawn path is closed. **Two guardian MCP processes remain alive** (1351720, 2238335); they exec'd before the edit, so nothing short of ending them stops them — `guardian.yml` was rewritten at 20:23 today, after the plugin was set `false`. Killing them is blocked by the safety classifier and is an operator action.

FIVE DEFECTS FOUND BY USING THE PRODUCT AND BY ATTEMPTING CLOSURES — 2026-09-06

Workshop gates **PASS 33 / FAIL 2 / COULD-NOT-RUN 1 → PASS 33 / FAIL 1 / COULD-NOT-RUN 2**. Playwright **296 passed / 6 failed → 301 / 1**, identical across two consecutive full runs on a provably static corpus. `ai_interviewing` challenge suite **24 / 0 / 0** throughout.

None of the five was found by a gate. Four came from walking the product as a reader, one from attempting a closure that turned out to be unavailable.

1 — A FILTERED SEARCH CLAIMED THERE WAS NOTHING. `no_match` is a POSITIVE CLAIM here, so a false one makes the three-valued contract lie. `limit=8` with a kind filter returned `no_match` while ≥ 100 rows matched. The filter ran on the wrong side of each LEG's own truncation — and in **three** legs, not the two I briefed: `Catalog.Query` did not filter in SQL as I asserted, it over-fetched `limit*8` and filtered in a Go loop, **the same defect with a bigger constant**. The agent measured instead of trusting me. Now pushed into retrieval as bound predicates; `?area=` is genuinely unpushable and takes an exhaustive path *because no multiplier is provably enough*. The old figures were themselves artefacts: “27 matching” was what survived the unfiltered cut, not the total.

2 — A NONSENSE QUERY CLEARED THE RELEVANCE FLOOR. 17,874 `kg_term` rows averaging **9.7 characters** shared one cosine space with prose averaging 1,167; 19,221 of 24,357 vectors sat outside the calibrated population. A recalibration was RUN FIRST and reported the populations now **overlap** ($+0.039883 \rightarrow -0.108342$), so there was no number to move to and the floor was not moved. The semantic leg is scoped instead. What a reader loses is the *unadvertised second copy* of an entity whose advertised copy is at rank 1 by name for 11 of 12

sampled. **A3 now PASS with A0 non-vacuity also passing at 52 hits** — the green is not bought by returning nothing. The gate moved 1 → 2: the confirmed finding is closed and UNDET A1 remains, unfixable, because its calibration corpus no longer exists.

3 — A BANK REPORTED PASS OVER 17 SILENTLY SKIPPED TESTS.

go test exits 0 for a skip and the only runner grading the backend suite read the exit code — while its own header says “*a skipped bank is reported UNDETERMINED, never passed*”. The rule stopped at the bank boundary. Now graded against the JSON stream, which also catches a package that fails to compile (it emits no test event, so an exit-code grader reads silence). **11 of those tests now genuinely run and pass**, because the checkpoint fetched today satisfied their precondition.

4 — A BAD ID AND AN EMPTY COLLECTION WERE THE SAME

ANSWER on three ai_interviewing sub-resources. Three states are now three answers. Also: wrong verbs returned text/plain, a Go struct path leaked into a client error, and 8/8 plan-download pairs collided on one filename.

5 — THE RETRIEVAL BENCHMARK CANNOT BE CLOSED BY

AUTHORING, established by attempting it. All 175 violating claim blocks were measured against the eligible index and then the ENTIRE corpus: **0 of 175 have their best match in an eligible source**, and 161 best-match the area document itself, which the authoring code forbids as circular. No citation exists that would both resolve and support, so none was written. The attempt found two defects worth more than the closure: **47 blocks are permanently stuck by construction** (a citation to a later-redacted passage is never reconsidered yet can never resolve), and **the two areas that DO pass do so substantially on a formatting artifact** — 51 and 52 editorial blocks versus 2, 2 and 8.

CARRIED FORWARD, diagnosed and unfixed: - Two e2e test defects: axe scans mid-entrance-animation (positional, fails 2/2 at suite position 161, passes 17/17 isolated — the same elements report *different colours* between runs, which a static colour cannot do); and a cancelled 200 read as a parse failure, proven flaky at ~1 in 6. - search-degradation.spec.ts:51 skips its whole block because search now answers 200 — **the skip most likely to hide a regression in exactly the area just changed**. - Graph traversal has no time bound: 56 s at depth 3, 70–80 s at depth 4. Depth is clamped and cycle-safe; the time is not. - /api/ask is unreachable on this corpus — a knowledge-graph node with no publication decision can outrank publishable evidence and withhold the answer.

THE CONTENT-BOUNDARY GATE PRINTED A FABRICATED ZERO FOR FOUR DAYS — FIXED 2026-09-06

In the gate whose entire job is to stop private content reaching a PUBLIC repository before an irreversible push. Its default branch emitted a HARDCODED LITERAL — untracked (public) 0 file(s) — NOT SCANNED — and never counted them. A reader sees a zero and concludes there were none. The words “NOT SCANNED” were there, but “0 file(s)” contradicted them and **the reassuring half was the false one.** Introduced with the untracked branch itself in 402a8c7 (2026-09-02T23:30:57), carried unchanged through a75c898 — **four days, two commits.**

Not hypothetical: the gate’s own header records a 2026-09-02 incident where an untracked file in this public umbrella carried verbatim private source, and *“this gate ran green over that file every time, because it never opened it.”*

THIS IS THE SECOND INSTANCE OF ONE CLASS IN A SINGLE DAY. The other is recorded below — an environment audit reported “1 → 0” for a file that was still untracked and therefore never scanned. `git ls-files` does not list what has not been added, and the `commit` wrapper runs `git add .`, so the window between writing a file and publishing it permanently is ONE COMMAND WIDE.

The fix, and the property that had to be preserved. `cb_ls_untracked` now runs on the public side in BOTH modes; only the flag decides whether the result joins the scanned set. Default-mode paths go to a separate counter and list that no analysis pass reads — because the header makes an explicit promise that a plain run’s counts are not silently moved underneath a recorded baseline. **Verified byte-identical: 15558 (prose 14740, short 653, name 165) before and after.** Unread-but-pushable files now emit an `undet` row through the gate’s OWN existing mechanism, because a gate cannot call a tree clean when files that could leak sit in it unread, and **2 is never a pass.** Precedence is untouched and was diff-verified: a real leak still outranks undetermined.

Proof: **74 passed / 0 failed over 29 mutations** (battery 28 → 29). The new M29 family proves the true count is reported and named, that the counter is NOT constant (a tree with no untracked files still reports 0), that `rc` is 2 with no leak and 0 once the same files are read, that a leak

outranks undetermined on a tree carrying both, that .gitignore is still respected, and — the load-bearing one — that the leak counts are identical before and after an untracked file appears.

HONEST BOUNDARY (§11.4.6). This tree currently holds **ZERO** untracked files across the umbrella and all 13 declared submodules, measured twice. So the live line reads 0 file(s) as a MEASUREMENT that coincides with the old literal, and no undet row fires here today. The fix is demonstrated on the proof's fixtures, which is real (M29a reports 3, M29d4 reports 1) — but a non-zero count against the actual fleet has NOT been observed and is not claimed.

TWO GATES THAT COULD NOT ANSWER NOW DO — AND ONE ANSWER WAS A FAILURE THE MISSING TOOLCHAIN HAD BEEN HIDING

Workshop gate suite **PASS 32 / FAIL 2 / COULD-NOT-RUN 2 → PASS 33 / FAIL 2 / COULD-NOT-RUN 1.**

SC-024 — the 2 was masking a 1, which is the entire purpose of a three-valued gate. verify-sc024-export-matrix exited 2 because pandoc and weasyprint were unusable, so six formats read could_not_determine and *“their absence proves nothing”*. With a usable toolchain the same six read **ABSENT** — a determined negative. 819 areas advertised, 2 published, **0 of the 2 carried all four formats**; html, docx and pdf had never been produced. Now produced through the module's OWN canonical producers rather than a second hand-rolled pandoc command line, each output verified structurally (real <h1>, OOXML zip with both required parts, %PDF- ... %%EOF). **No image change was needed and none was made** — export.go stats the file BEFORE consulting the toolchain, so the container reports present without pandoc. Gate 2 → 0.

The floor — recalibration was RUN, and it says there is no floor to move to. Separation went from +0.039883 (**no overlap**) to -0.108342 (**OVERLAP**) against the current generation. Nothing was moved. The FAIL is a *confirmed* measurement and stays 1; downgrading it because a neighbouring assertion is UNDETERMINED was considered and rejected as the actual defect. **And an assertion had silently stopped being able to catch anything:** A1 claimed to catch a hand-edit widening the Go set, but re-derives from a calibration corpus that no longer exists. **A1b LOCKSTEP** replaces it, needing no database; mutations **12 → 15.**

Suggest — a budget that stopped describing what it was sized against. Every 1- and 2-character prefix returned 503 at ~53 ms; 3+ characters answered in 15–37 ms. Measured: counting a* (7,364 rows) costs 9.7 ms, ranking and snippeting it costs **59.4 ms**, because ORDER BY bm25 scores every matched row before LIMIT keeps thirty. The 50 ms budget was set in this platform's FIRST COMMIT and never revisited while the corpus grew to 24,357 passages. Now 250 ms. Boundaries recorded: that figure is the PASSAGE leg alone; the catalogue leg runs **serially under the same context** in a different database, and running the two concurrently is the better fix, recorded rather than attempted.

Entailment — the checkpoint had no fetch path in this tree at all. Now present in the operator's cache OUTSIDE the repository; the gate returns a decided verdict when pointed at it. **It still exits 2 in a bare shell BY DESIGN** — it refuses to discover its own inputs and its paired proof asserts that. Making the runner guess a host path would relocate the defect, and audit-hardcoded-paths would be right to flag it.

THE RETRIEVAL BENCHMARK IS NOT MEASURING WHAT ITS NUMBER LOOKS LIKE

docs/limits.md §10.18. **All three A-row misses return 404 area_not_published.** The fixture SELECTS on /api/areas (5 titled areas) and GRADES search (2 published), so it asks search for records the publication contract forbids it to return — **no retriever could ever have scored them.** Q02 sits at rank 3, inside the top five, and contributes nothing to the failure.

And its number is not reproducible from the repository. The two files deciding publication are git-ignored, so a fresh clone reads **17/22, not 19/22**, with nothing saying why. Two operator closures are written out; the bar was not widened and the fixture was not touched.

CONSTITUTION SWEEP — 176 PASS / 93 FAIL / 2 ERROR of 271 (was 96 FAIL)

Three FAILs cleared (README badge row, zero-findings sweep, ratchet ledger — all now present). **Read the shape, not the count: 86 distinct gates fail, but 8,294 of ~8,365 findings are TWO fleet-wide annotation conventions** — CM-ORACLE-STRATEGY-NAMED-AND-INDEPENDENT (7,601 test functions with no §11.4.245 annotation) and CM-DANGEROUS-COMBINATION-FAIL-CLOSED (765). These are an unadopted

convention, not defects, and annotating 7,601 tests mechanically would be exactly the bluffing §11.4.245 exists to prevent. Adopting it is a deliberate programme and an operator decision.

A GATE CANNOT SEE AN UNTRACKED FILE — AND I PUBLISHED A “1 -> 0” THAT PROVED NOTHING

Recorded because the claim was already in a pushed commit message before it was caught, and because this tree documents the identical trap elsewhere.

Yesterday’s commit reported `audit-environment-assumptions: 1 -> 0` for a new `proxy.conf.js`. The 0 was REAL and MEANINGLESS: that `audit` enumerates with `git ls-files`, the file was still UNTRACKED when it ran, and **a file git does not list is a file the gate does not scan**. Committing it is what made it visible; the next run flagged it. `CLAUDE.md` already carries the same finding for `_tests/env.js` — *“UNTRACKED, so `git ls-files` does not yet show it to the gate”* — and it did not prevent a repeat.

THE RULE: a gate run over a change that ADDS a file proves nothing about that file until the file is tracked. Stage first, then measure.

The fix was a real one rather than an allow-list entry, and the accepted form was already in the tree to copy: `_tests/env.js` puts the last-resort literal `INLINE` at the point of use, never as a bare named constant, because a reader cannot tell a documented fallback from a frozen assumption when the two are written identically. Four behaviours executed: default, `WORKSHOP_PORT` override, full `WORKSHOP_BASE_URL` override, and a loud throw on an unparseable value rather than a silent fallback. `audit-environment-assumptions: 2 -> 0`, re-run with both files tracked.

FOUR FALSE CLAIMS REMOVED IN THE SAME CHANGE

Each an instrument or a document asserting something it had not measured:

1. **A gate-pairing check was FALSELY ACCUSING a gate** of shipping no paired mutation. It recognised only the sibling-file form; the gate it accused ships five mutations and a control behind a flag — the form the check registry declares for it. Two instruments in one tree disagreed about what §1.1 accepts. Both forms are now recognised

by evidence rather than by mention, with six fixtures proving the rule was not weakened to get there.

2. **A benchmark runner printed a model name it never read from the server.** It cannot be measured on that build, so the label is marked DECLARED wherever it appears and the artifact now records where the name came from. Same shape as the HTTP/3 false-pass fixed the day before.
3. **Seven end-to-end waits used a network-idle condition on pages streaming a multi-gigabyte recording**, which can never settle while tracing is on. Four tests were unpassable for a harness reason and read as product failures.
4. **A route manifest claimed an endpoint 404s for every id.** Re-measured by probing all 819: two serve 200, the rest correctly 404. The declared response codes were already right — only the prose had rotted, which is the failure mode to watch for in a file whose gate reads the codes and not the explanation.

STILL OPEN — OPERATOR DECISIONS, NOT DEFECTS

- `verify-floor-domain` — the calibration corpus it was sized against no longer exists; needs re-calibration or an explicit withdrawal.
- `verify-retrieval-benchmark B2` — 19/22 against a 20/22 bar. **Do not widen the bar**; the gate says so itself.
- `verify-entailment-loads` — NLI checkpoint (~83 MiB) not downloaded. rc 2.
- `verify-sc024-export-matrix` — `pandoc / weasyprint` not installed. rc 2.
- A **session restart** is needed for the Semgrep plugin disable in `~/.claude-claude5/settings.json` to take effect; its hook intermittently blocked Go tooling throughout this work.

Session 2026-09-05 (evening) — sub-chapter 02.01 shipped, 14 defects fixed

Read docs/directive-register.md FIRST — 16 operator directives with dispositions and evidence, GATED by `scripts/verify-directive-register.sh`.

Workshop is at 9af0876, pushed. Sub-chapter 02.01 is transcribed, ingested, knowledge-extracted and served; `GET /api/chapters` reports the hierarchy (02 -> child 02.01, depth 2, ordinal_path [2,1]). Both sites are live: workshop `http://127.0.0.1:8087`, ai_interviewing `http://localhost:8099`.

OPEN, and each is a real thing rather than a nicety

1. **RESOLVED — crossrefs are live.** 487,140 edges over 24,357/24,357 passages, 0 unscored, generation 5. The blind spot is now WATCHED by `platform/gates/verify-crossref-currency.sh` (registered, 5/5 mutations). **The reachability defect is FIXED:** the derivation is now always ENTERED and declines on MEASURED coverage rather than on the previous pass's error, logging the counts. Two facts that made it worse than “no retry”: a partial run row is STICKY (nothing re-derives that generation, ever) and a partial derivation CORRUPTS the passages it did score (Neighbours ranks against the vectors that exist). **Checkpointing was proposed and DECLINED on measurement:** the 35 min is the $O(n^2)$ scoring walk, not the transaction — 487,140 edges insert in 2.1 s. The real remedy is ANN candidates. Still open.
2. **No gate measures PASSAGE retrieval rank.** `verify-retrieval-benchmark`'s 22 queries are 5 `area_title` + 12 `term_name_spaced` + 5 `question_stem` — all catalogue kinds, not one passage query. For generic single-word queries the top 20 is 100% catalogue. This blindness is what let a fusion attempt that gave catalogue rows a ~250x unconditional edge score 17/22 and look correct. Adding passage-answerable queries is a judgement about what the product should rank — operator's call.
3. **RESOLVED — entity twins de-duplicated.** `dedupeEntityTwins` drops the registry passage when its catalogue row is in the same result set; the survivor is chosen by CONTRACT (`corpusBlock` advertises `term`, not `kg_term`), never by score. Duplication 15.2% -> 0.0% over 240 fused rows; top-1 5/22 -> 18/22 with **top-5 unchanged at 19/22**, which is what says it is not a scoring trick. 104 orphan registry rows survive untouched. **RECORDED, NOT FIXED: the freed slot refills with the catalogue's own MORPHOLOGICAL tail, not content** — `q=index` serves 20 catalogue rows of which 12 are inflections of one stem. Entity twins are gone; entity near-duplicates are not, and collapsing the twins made that wall taller. The catalogue monopoly on single-word prefix queries is PRE-EXISTING and was deepened, not created. Also open: a `?area=` asymmetry (`q.Area` is not passed to `Catalog.Query`), unexercisable on this deployment.
4. **SC-015 is 19/22 and that IS the ceiling on this index.** 3 of 5 areas are unpublished by `run_author_stage`'s own bar (54-63 uncited claims each) — design, not defect. 20/22 needs a third area published.
5. **`verify-floor-domain.sh rc 1`** — the floor was calibrated on a 2478-passage corpus that no longer exists; `calibrated=false` already, so

- the claim is already withdrawn. Needs re-calibration or an explicit withdrawal. Operator decision; do not flip a flag to green it.
6. **verify-entailment-loads.sh rc 2** — NLI checkpoint (~83 MiB) absent. Operator download; no downloader is checked in deliberately.
 7. **verify-sc024-export-matrix.sh rc 2** — pandoc/weasyprint absent from this host. Toolchain gap, not a corpus gap.
 8. **RESOLVED — Chapter 02 is ingested and knowledge-extracted.** 1,206 segments, 74 areas, 50 notes, 45 open questions, 35 todos. Registry at 24,519 rows. verify-identifier-vocabulary stayed rc 0 across the new terms.
 9. **specs/003-chapter-hierarchy/ T001-T037** — Phases 2 and 5 landed; the ?under/?depth/filters work needs GET /api/chapters moved onto the three-state envelope first.

BOTH SITES VERIFIED END TO END, 2026-09-05 late

- workshop <http://127.0.0.1:8087> — generation 5, 24,357 passages, healthy.
- ai_interviewing <http://localhost:8099> (HTTPS **8444**, not the documented 8443 — that port is owned by an unrelated helixllm process). HTTP/3 verified with a purpose-built h3 client, not merely from an Alt-Svc header.

THE FINDING TO REMEMBER FROM THAT VERIFICATION.

ai_interviewing's evidence-gated challenge suite reported PASS for HTTP/3 on an Alt-Svc header emitted by a DIFFERENT PROGRAM, because it defaulted to a frozen :8443. An anti-bluff instrument certified a protocol on a process it was not testing. The fix was not just to aim it correctly: a resolved URL is now not TRUSTED until the responder proves it is the server under test (health ok:true plus matching version and stats). Proven against a deliberately built impostor that served valid health AND advertised h3 — port-correctness alone would still have passed it. **Any probe that names a port without attributing the responder is making an unverified identity claim.**

STILL OPEN — the four that are OPERATOR DECISIONS, not defects

1. verify-entailment-loads rc 2 — NLI checkpoint (~83 MiB) not downloaded. No downloader is checked in deliberately.
2. verify-sc024-export-matrix rc 2 — pandoc/weasyprint absent from this host.
3. verify-floor-domain rc 1 — the floor was calibrated on a 2,478-passage corpus that no longer exists; calibrated=false already, so

the claim is already withdrawn. `floor_population.go` records a proof by exhaustion that NO floor value separates sense from nonsense on this corpus. Needs re-calibration or an explicit withdrawal — do not flip a flag to green it.

4. `verify-retrieval-benchmark` 19/22 — the three failures are EXACTLY the three areas `run_author_stage` held back for 54-63 uncited claims each. It closes when a third area is authored to citation standard. That is writing, not code.

PROCESS FINDINGS worth keeping

- **Run `verify-content-boundary.sh --include-untracked` before any push that adds files.** The default run prints untracked (public) 0 file(s) – NOT SCANNED and is blind to exactly the highest-risk category.
- **The shell `grep` is a `ugrep` wrapper with `--ignore-files` and SKIPS gitignored files.** Use command `grep` when a gitignored file could matter — `taxonomy.jsonl`, `pipeline/models/`, `pipeline/transcripts/` are all ignored.
- **A `Semgrep PostToolUse` hook intermittently refuses go invocations.** It is not a code problem; it blocks test execution and must never be reported as a pass.
- **Write a commit message to a FILE and pass it as `"$(cat file)"`.** Never inline a multi-paragraph message. Commit `b3968ad`'s message is partly corrupted because it was inlined: unescaped backticks were evaluated as command substitution and every quoted token in one paragraph was replaced by the (empty) output of running it. The content committed was correct; only the message prose was damaged, and it was NOT repaired by force-push because rewriting pushed public history needs explicit per-session authorization. The near-miss is the real lesson: the same evaluation that deleted those tokens would have EXECUTED any backticked text that happened to be a valid command. The workshop commit in the same session used heredoc-to-file and was unaffected.

Session 2026-09-05 — Chapter 02, sub-chapter 02.01, and the directive register

Where to resume. Read `docs/directive-register.md` FIRST — it is the authoritative index of operator intent and it is now GATED (`bash scripts/verify-directive-register.sh`, registered as `directive-register`). Rows `D001..D014` carry this session's directives with dispositions and evidence.

DONE and verified this session

- **Chapter 02 + sub-chapter 02.01 archived**; Chapter 01's recording restored from 36 parts. All three round-trip (skip (intact)). Workshop commit 266f443, pushed; umbrella gitlink and helix-deps.yaml moved together.
- **workshop/scripts/setup.sh now restores every archived recording** (do_extract_chapters, --no-extract). Proven by removing 02.01's recording and getting it back byte-identical.
- **workshop/pipeline/venv-setup.sh**: uv venv --seed fallback (stdlib path still first), absolute-interpreter fix + assertion, and the previously missing pipeline/extract/requirements.txt install. Fresh venv: **278 tests, OK** (was FAILED (errors=32)).
- **Four defects fixed**: workshop-redact/main.go:596 HasSuffix→equality (disclosure-control severity); meeting_notes.py:98 and author.py:150 chapter-\d+ widened for sub-chapters; the venv gap above.
- **docs/directive-register.md + scripts/verify-directive-register.sh** — the operator has asked three times (work-register R15/R22/R24) that directives be archived AND regularly checked; registers existed, **nothing gated them** (grep -c in check-registry.tsv was 0). Now gated. Registry: 51 PASS.

IN PROGRESS / NEXT — concrete

1. **specs/003-chapter-hierarchy/ — 37 tasks drafted, NOT yet written to disk**. Identity scheme: 02.01 dotted, $^{[0-9]\{2,\}(\backslash.[0-9]\{2,\})^*\$}$, byte-lexicographic sort (already correct at cmd/workshop-server/main.go:2126 — do NOT change it). Spec-level blocker first: specs/001-.../data-model.md:45 types ordinal as int; every ordinal collision follows from that line.
2. **ordinal0f("02.01") == 2** — collides with chapter 02. internal/api/chapters.go:736.
3. **pkg/search/suggest-sources.json** hardcodes chapter-01 in six rows and is //go:embed-ed — no new chapter reaches /api/suggest until it is derived.
4. **Corpus re-derivation owed**: verify-identifier-vocabulary.sh is RED with **145 ULID-fragment terms** in tracked curriculum/passages.jsonl. The tokenizer fix landed; the corpus was never re-derived. The venv is now restored, so pipeline/venv/bin/python pipeline/extract/run_pipeline.py can run. **Take a backup first — passages.jsonl is TRACKED**.
5. **Chapter 02 / 02.01 are archived but NOT ingested** — curriculum/ still holds only chapter-01. Fix item 1's Phase 1 (the redaction scope defect) BEFORE ingesting a second scope.

6. 02.01 has no PDF notes — a notes-less chapter's tolerance is unverified.

KNOWN RED / BLOCKED

- scripts/verify-content-boundary.sh — **RED BY DESIGN**, leave it red.
- audit-environment-assumptions.sh — 5 frozen ENDPOINT assumptions in two workshop/platform/gates/ files. Fix inside the submodule.
- submodules/LLMProvider is on **master**, not main. Its upstream default IS master and origin/main there is 16 commits BEHIND. **Operator decision; do not “fix” by checking out origin/main.**
- A gate's rc-2 probe writes into **TRACKED evidence**: running verify-check-registry.sh --run-proofs overwrites workshop/evidence/p-u1/result.json with "real_tree": "/nonexistent". Observed twice this session and reverted twice. The probe should write to a temp path. NOT yet fixed.
- Local llama.cpp split: /usr/bin/llama-server is **CPU-only**; ~/opt/llamacpp_gpu/b10786_vulkan/ has Vulkan. libggml0-backend-cuda has **no candidate** in this archive. Ollama IS GPU-accelerated (37/37 layers). The ASR pipeline REFUSES a visible GPU across six layers — reversing that is a deliberate design change, recorded in the register as D012-CONFLICT.

Operator decisions taken 2026-09-02 — 16 blockers cleared in one pass

These are ANSWERED. Do not re-ask them. Every one was put to the operator with its measured trade-off and its cost; the answer is recorded here with the consequence that follows, so a later reader can see what was traded away.

#	Blocker	Decision	Consequence
1	T040 Chapter 1 redaction review unrecorded; T104 (push) blocked on it	Run the review now	Must complete BEFORE any push moving the workshop gitlink. Not waived.
2	Content boundary 11,158 vs recorded 285	Re-baseline, keep it loud	11,158 becomes the recorded reference with its three measured causes. No exemptions

#	Blocker	Decision	Consequence
3	Nothing committed	Commit and push everything	added. Every row stays visible. Sequenced AFTER decision 1. review.md + tasks.md must land together. git fetch classified it as a clean fast-forward (--is-ancestor TRUE, 0/2); git merge --ff-only moved the pin to 3be10826f3d2; helix-deps.yaml and the gitlink were staged TOGETHER. Corpus unchanged — same blob 34eff9d8..., 11,700 lines, 252 anchors, 1,779,401 bytes; the diff touched only two of the constitution's own scripts/gates/ files.
4	Constitution pin f5876a3b staged vs remote 3be10826, direction UNDETERMINED	Fetch and fast-forward — EXECUTED 2026-09-02	verify-submodule-remote-sync.sh now exits 0 at 12/12 CURRENT — the first time that gate has ever been green.
5	spec 002 “answering over the new material” (blocks T115/ T116)	Build the claim-to-question verifier — BUILT AND LIVE 2026-09-02. SC-010 still NOT met.	L5 = two required floors. (1) A deterministic DEMAND check classifying what the question demands back (quantity/version/ time/cause/procedure/ identity/polarity) and requiring a filler the question did not itself contain — load-bearing, because the recorded fabrication “<i>what is the annual cost in euro of the FOUR subscriptions</i>” would otherwise be

#	Blocker	Decision	Consequence
			satisfied by an answer echoing its own “four”. 0 of 57 benchmark questions unclassifiable, after two misclassifications were found and fixed. (2) A model ANSWERHOOD judge in a new pkg/answerhood/, deliberately not pkg/entail — entailment relates passage→claim, answerhood relates question→claim, and conflating them IS the defect. Live: question_verifier_kind: question-focus+llm. Both gates exit 0; prove-answer-question.sh catches 3 of 3 seeded defects. A second ingestion pipeline: new passage kind, new timings, new accuracy obligation that must be MEASURED before it is trusted. OCR resolves on this host, so this was a scope choice, not a capability one. No publication evidence floor. Extraction and E1/E5 untouched. Accepted cost: the grid is dominated by single-passage areas.
6	spec 002 what “in videos” covers	Spoken PLUS on-screen OCR	
7	497 areas from one chapter	Publish all 497 — no floor	
8	SC-015 retrieval 13/26 top-5 vs ≥90%	Run the full-scale benchmark —	Every leg FAILS . Best is nomic WITH task prefixes: 8/26 top-1 (30.8%), 17/26 top-5

#	Blocker	Decision	Consequence
9	spec 001: 32 NOT DONE + 29 PARTIAL of 120	EXECUTED 2026-09-02, SC-015 SETTLED: NO Fix the stale paths FIRST, then build — PATH REPAIR EXECUTED 2026-09-02; the build is still queued	(65.4%) against a 24/26 bar. The small-pool trap was real — nomic went 80.8% on 152 passages → 30.8% on 2,478. 36 min 19 s; live index never opened for writing (every leg read a chmod 444 copy); container never restarted. But the decisive finding is the FLOOR, not the accuracy — see below. 23 task lines marked [PATH CORRECTED] (requirement met, location differs — real path written in, superseded one named beside it) and 39 [PATH NOT BUILT] (named path absent AND no equivalent anywhere, so the path is still the destination). Ticks 54 → 59: T049 T063 T065 T101 T102 added, nothing unticked, ids still T001–T120 contiguous. Three stale claims withdrawn — questions.tsv “8 A / 10 U” → 24 A / 33 U; Phase 6’s “no generative model exists”; T068’s “LLMProvider unresolvable”. Honest boundary written into the file itself: no Go, Karma or Playwright suite was executed, so T049/T063 rest on

#	Blocker	Decision	Consequence
10	Boundary gate blind to UNTRACKED files	Add an opt-in untracked scan	reading assertions, not a green run. New flag over <code>git ls-files --others --exclude-standard</code> , with a <code>--prove-failure</code> paired proof, registered in <code>check-registry.tsv</code> per R5. Default contract unchanged.
11	2026-09-01 incident: content in PUBLIC history	Leave history, keep the redaction	No force-push, no repo rotation. Redaction stands as containment, explicitly NOT as remedy.
12	design-toolkit GitLab mirror: lag unmeasurable, visibility unverified	Add the GitLab remote and measure	Wires the mirror as a named remote so <code>verify-submodule-remote-sync.sh</code> can see it. Turns a footnote into a measured fact.
13	spec 002 checkboxes 0/122 vs 94 DONE; §3 claimed “all ten phases IMPLEMENTED and reviewed”	Tick the 94, correct the claim — EXECUTED 2026-09-02	<code>tasks.md</code> now reads 94 ticked / 28 unticked / 122 ids , the 28 exactly the 22 PARTIAL + 6 NOT DONE. 37 DONE ids were independently spot-checked across all ten phases with ZERO verdict disagreements. Each PARTIAL/NOT DONE carries a dated note naming what is missing. T115/T116 markers corrected [BLOCKED: answering clarification / U4] → [UNBUILT: decision taken 2026-09-02], and the Markers legend gained [UNBUILT] so a settled decision can never keep wearing a BLOCKED

#	Blocker	Decision	Consequence
			marker. The false §3 claim is withdrawn above. workshop/ provably untouched (git status --porcelain \\\ sha256sum unchanged).
14	SC-010 (3 fabrications) and SC-006 (p95 2094.8 ms vs 2000) breached	Fix BOTH	SC-010 closes via the decision-5 verifier, which is its acceptance test. SC-006 to be profiled and brought under budget (4.7% over).
15	883 R3 contradictions, no recorded decision on any	Sample, classify, then decide	Classify a meaningful sample into contradiction TYPES so a handful of rules replaces 883 individual judgements.
16	User-level git commit hook affecting ALL repositories	Keep it user-wide	Deliberate. ~/.claude/hooks/commit-attribution-fixed.sh stays global; revert by removing the hooks.PreToolUse entry in ~/.claude/settings.json.

A12 — the 883 R3 contradictions are 98.7% ONE ARTIFACT. Four rule decisions + 14 judgements replace 1,152 adjudications. OPERATOR DECISION NEEDED.

Two corrections to how this was framed, both measured.

- 1. The count is 1,152 today, not 883.** The 883 IS reproducible — exactly — as the **B1 promoted-branch SUBSET**. It never covered the 269 established-area rows. Branches today: B1-promoted-partial-overlap **883**, B3-established-ambiguous-multiplicity **258**, B2-established-no-clean-relation **11**. Whether the original run counted the same set **COULD NOT BE DETERMINED** — no evidence file for it exists.
- 2. R3 never compares strings — it compares PASSAGE SETS.** reconcile() fires R1 on exact set containment (overlap_size == proposal_size, no ratio, no threshold) and R3 when a proposal

shares a passage with an area and is contained in none. So the classes anyone would expect — case, whitespace, singular/plural, abbreviation-vs-expansion — have **ZERO members**, because none of those is measured anywhere on this path. The expectation that the population collapses mechanically held completely, along a **different axis**.

It is a census, not a sample: $n = N = 1,152$, sampling error zero. The discriminator (the passage KIND of each row's outside set) partitions the population exactly with no residual bucket, so classifying one row and all of them cost the same — 10.6 s.

Type	Count	Share	Rule	Cost of that rule
T1 graph-row self-reference	1137	98.70%	R1 exclude kg_* rows from what derive.extract() reads; R1b discard the 1137 already recorded, by PREDICATE not by label	2 candidate terms lost (8553→8551). R1_attach moves 11 → 1927 — ~1,900 held decisions become silent auto-attachments. R1 is upstream of R3, so it resolves nothing already recorded. R1b spends \$2.3's own "never discard" protection.
T2 real corpus boundary disagreement	14	1.22%	No rule — one human judgement each	Any rule here is exactly what \$2.3 refuses. Not having one costs 14 judgements. A deferral nobody picks up is
T4 over-merged mega-cluster	1	0.09%	Route to the area-overgeneration work	indistinguishable from a drop — needs an owner named at routing time.
T3 zero-outside		—		2a blurs boundaries

Type	Count	Share	Rule	Cost of that rule
established ambiguity	0 today, 353 projected		2a attach to all, or 2b discard as duplicate	invisibly, sometimes across 20 areas at once; 2b throws evidence away and forces an A1/A2 unattached classification. Not equivalent — the operator must pick.

T1's cause, verified independently 2026-09-02: neither `derive.py` nor `corpus.py` filters by kind — `derive.py`'s only match is prose, and `corpus.py:46`'s only comparison is a `transcript_segment` helper. Meanwhile **9,144 of 11,622 registry rows (78.7%) are `kg_* graph nodes`** (`kg_term` 8553, `kg_area` 516, plus `todo/next-point/open-question`). So `derive.extract()` reads the graph's own nodes as `corpus`: a term extracted from a `doc_section` now also "occurs" in its own `kg_term` node, which is evidence for no area, containment breaks, and R3 fires. Verified exactly rather than sampled — **1137/1137** rows have precisely one `kg_term` outside passage carrying their own member term, and all **236/236** rows with a `kg_area` outside passage have that pid **among the areas they are contradicting**.

The honest counterfactual, measured rather than promised: RULE 1 does NOT take R3 to 14. `--exclude-graph-kinds` yields **R3 = 366** (T3 353 / T2 12 / T4 1) — it exposes a different mechanical type that T1 currently MASKS. The 13 remaining B1 rows match the first recorded run's R3=13 exactly.

Analysis: `workshop/docs/session-evidence/phase3e-contradiction-typology.md`; re-derivable by the `read-only` `workshop/pipeline/extract/analyze_r3_contradictions.py`. **Nothing was resolved, merged, discarded or applied; T041 remains [].**

A61 — the constitution pin's FOURTH drift closed, and class A finally has direction evidence: 79.7% outward, 19.7% INWARD. The reassuring story is true for four rows in five and false for the fifth.

Two operator decisions, both authorized, both executed. Nothing was committed or pushed; nothing was judged, allow-listed or re-baselined.

1 — The constitution pin, fourth drift in three days. The gate opened at exit 1, 11 CURRENT / 1 DRIFT, with the direction **UNDETERMINED** because the remote object was absent from this checkout's store. The authorized --fetch resolved it and the classification was made BEFORE the move, not after: git merge-base --is-ancestor 3be10826f3d2 2887b42e9349 **TRUE**, and git rev-list --left-right --count **0 / 1 — 1 behind, 0 divergent**. A true fast-forward, so git merge --ff-only performed it; it would have refused anything else. helix-deps.yaml and the gitlink were staged **together**, which is what C9 requires — verify-manifest-pins.sh **0** at 12 MATCH / 0 DRIFT, and verify-submodule-remote-sync.sh back to **0** at 12 CURRENT / 0 DRIFT.

The move touches no governance document, and that was checked by name rather than eyeballed. Five files, 142 insertions / 20 deletions, from one commit fixing a service-status script — submodules/constitution/scripts/helix_code/ plus its four rendered exports under the submodule's own docs/scripts/. All five paths are **inside the constitution submodule**, not at this root. Filtering the name list against every governance path that submodule carries — Constitution.md, the four carriers, CLAUDE_ANCHORS_FULL.md, find_constitution.sh, helix-deps.yaml, templates/, its gates directory, its hooks directory and its post-update hook — returns **NONE**.

(Those last three are described rather than spelled as paths on purpose: continuation-check.sh resolves any bare scripts/... string against THIS root, and it correctly flagged two earlier drafts that named the submodule's scripts unqualified. The gate was right and the prose was wrong — the exact root-vs-submodule ambiguity the carriers warn about, caught by an instrument rather than by a reader.)

Every corpus figure was re-measured on BOTH sides of the move and every one reproduced: blob 34eff9d8..., 11,700 lines, 252 ### § anchors, 1,779,401 bytes, sha256 fe1de96abc84c2fc..., du 784K / 1.7M — identical at 3be10826f3d2 and at 2887b42e9349. **No figure needed**

correcting. That is a measurement, not a rule — four corpus-neutral moves in a row is what upstream happened to change, and the next one must be measured the same way.

2 — Class A direction, the larger half, and the method is stronger than the one it is compared against.

FIRST, A DEFECT IN THE INSTRUMENT ITSELF, found by controlling for my own edits: this gate's total is NOT REPRODUCIBLE run-to-run. Three full runs: **12745 / 12846 / 12867**. **Runs 1 and 3 were on a BYTE-IDENTICAL tree** — run 3 was produced by restoring the four carriers and this file to HEAD precisely to control for the session's edits — and they disagree by **122 rows**. Ruled out by measurement: the gate and .content-boundary-allow are unmodified; nothing in workshop/ or the root changed in the window (find -newermt '3 hours ago' → 0 files); the gate allocates a fresh mktemp -d per run, so concurrent instances cannot share state. **The instability is localised to a DERIVED filter, not to row emission** — already_public itself differs on identical input (2252 vs 2261). **The mechanism is UNDETERMINED. That is a 2 and never a pass.** Run 1 ran alongside two other instances that were killed mid-flight; runs 2 and 3 ran alone — a difference between the runs, **not a demonstrated cause**. The name class held at **85** in all three. **Quote the shares, not the totals.**

Class A is **~7870 rows, 61.3–61.5%**. Every one was dated on both sides at **TEXT level** — for each matched string, the earliest commit whose normalised blob actually contains it, validated against the gate's own normalisation 24-of-24 — then **widened to corpus level**, comparing earliest appearance ANYWHERE public against earliest ANYWHERE private. 55 public and 489 private path-histories for the pairwise dating; the widening ranged over the **9,414 file-revisions** the two histories hold (umbrella 8,213, workshop 1,201). **Nothing sampled** — the walk is chronological and skips a blob only when no still-undated string could be affected, an exact pruning, not a shortcut. Classes B and C were settled by a *file-level, sampled* probe; this is neither.

The probe was run independently against ALL THREE gate runs, and that is the answer to the instrument's instability: the totals move, the direction does not.

run	class A rows	OUTWARD	INWARD	UNDETERMINED
1	7835	6241 (79.7%)	1544 (19.7%)	50 (0.6%)
2	7872	6240 (79.3%)	1538 (19.5%)	94 (1.2%)

run	class A rows	OUTWARD	INWARD	UNDETERMINED
3	7890	6240 (79.1%)	1538 (19.5%)	112 (1.4%)

OUTWARD varies by ONE row and INWARD by SIX across three runs whose totals differ by 122. Every extra row a noisier run emits lands in UNDETERMINED — strings not committed on one side, i.e. working-tree-only text. **The direction finding is robust to the count instability.**

The widening moved rows BOTH ways — 113 inward→outward and 178 outward→inward — which is why it is a probe and not a rationalisation. Lead times: outward median 6.1 h, inward median 0.5 h with **92% of inward rows inside 24 h**. Spec and implementation are written the same day, in both orders.

Where the inward rows are, because that is the reading assignment (run 1's 1544): 1084 (70.2%) private source code, 153 session-evidence briefs, 117 training docs, 56 other docs, 40 data/config, 26 other — and **68 in workshop/chapters/01/, the private teaching-session material**, all from one JSON artefact, landing in specs/001-.../tasks.md (45) and specs/002-.../tasks.md (23). Three public files hold 1120 of run 1's 1544.

HONEST BOUNDARY — the first limit is the one that matters. “*Private committed first*” establishes an **ORDER, not a disclosure**. A symbol name, a commit message, a decision made in code and written up afterwards all produce INWARD rows with nothing having crossed the boundary. **~1540 rows are not ~1540 leaks and must never be reported as such.** What the number does kill is the comfortable assumption that the spec trees are simply outward propagation.

The 50 UNDETERMINED rows are the entire class A name population — one public file, one withheld identifier — and they are undetermined *because the gate redacts the matched text*: the protection and the blind spot are the same mechanism. A weaker **file-level** fallback splits them 30 outward / 20 inward, and the 20 include **15 whose private side is an ai_interviewing document first committed 2026-08-12, twenty days before the public spec file**, and 5 whose private side is the private recording-notes PDF (2026-08-31). **File-level only, weaker than the text-level result, NOT a determination** — recorded because it names the rows to open first.

The binding constraint is unchanged and was re-measured, not assumed: 85 name rows carrying 9 distinct withheld identifiers, spread across EVERY class — A 50, B 19, remainder 14, C 2. The “89

rows / 8 names” figure is **superseded by measurement, not by any clearance**; the class went 57 → 89 → 85 with the detector never touched, and **a falling name count is not progress**. No class can be pardoned wholesale without pardoning name rows with it.

What this changes in the packet: option 4 is no longer “re-judge 7550 rows” but “**judge the ~1540 inward rows plus the 50 undetermined name rows**” — an 80% cut to the reading assignment, bought with evidence rather than with exemptions. Option 3, “teach the gate direction”, now has a working prototype: this probe ran offline over 9,414 file-revisions in under a minute.

Also corrected, and both were ACTIVE MISINFORMATION rather than staleness. The carriers claimed “*nothing in this repository has yet been built or run in a container*” — a fleet container is **running right now** (podman ps:workshop-curriculum_platform_1, alpine:3.20, up 2 h, healthy) — and that submodules/containers was “*present for the workload specs/001-... plans*”, when _tools/containers/ was committed today (460266c, 7 tracked files) as its first real consumer. **A third fact neither claim mentioned:** the umbrella root has shipped **hand-rolled Containerfiles** at _tools/helixtranslate-container/ since 2026-06-26 (32dfdbd), referenced by 10 tracked files, with **no** reference to submodules/containers anywhere under it — a §11.4.76 violation by this project’s own reading. **Whether those images were ever built or run is a 2:** podman images matches helixtranslate zero times here and run.sh names two remote hosts this tree cannot probe.

Carriers edited as ONE artifact — head 1–18 split off, shared tail edited once, recomposed — verified **4 then 1**, with verify-governance-cascade.sh exit **0** at 12 PASS / 0 FAIL / 0 ENV / 8 NOTE and C5 reporting declared split 19 == measured convergence 19.

Still open and NOT done here: no row judged, no allow-list entry, no re-baseline, and **the content-boundary gate still exits 1 by design**. Classes B, C and the 349-row remainder still rest on file-level sampled evidence or none at all — the same text-level probe should be run against them.

A62 — THE LAST UNGATED DISCLOSURE SURFACE IS CLOSED. The obvious rule was implemented, measured, and rejected — “the blunt gate wearing a derivation.”

Every enumeration claim was verified before anything changed, and all three held: the lexical leg filtered on redaction **and nothing else** while returning the **full row text**; the catalogue leg had **no**

redaction clause at all on the search path — its only re-check was reachable from suggest; and the publication gate was constructed *after* the handlers were built, so wiring it needed a **boot-order change, not an argument**.

One correction that cuts against the urgency, stated plainly: on today's volume the *passage-leg* exit is **latent** — those pids still 404 because the served registry holds no knowledge rows. **The catalogue exit was LIVE:** a plain query returned four term hits. So the thing actually leaking was the one nobody had flagged.

THE RULE WAS CHOSEN BY MEASUREMENT, AND THE OBVIOUS ONE WAS REJECTED TWICE OVER. The candidate I proposed — a term inherits publication from the areas that declare it — was implemented as a measurement first and then thrown out on two independent grounds. It is **unsound**: it publishes a term from a decision taken about a **different row**, since membership and evidence are unrelated lists — *publication by derivation*, the exact failure it was told to test for. And it **does not even work**: only **483 of 8,508** terms are declared by any area, and **0** by a published one. **“It is the blunt gate wearing a derivation.”**

What shipped instead: a term is disclosable exactly when it has at least one evidence passage **and every one of them resolves and is itself disclosable under the area gate**. Soundness is not asserted but **structural** — a term cannot publish while a supporting passage is withheld, *because the withheld passage's own verdict fails the term*. A mutation deletes that check and goes RED.

What it does NOT protect is written into the code, not just the report: names already reachable through the ordinary rows they were extracted from; **aggregation** — 8,508 names are a vocabulary and this gate sees one row; the evidence *linkage*, served elsewhere; and any defect it inherits from the area gate. **“It is not a review.”**

Ten mutations, all observed RED — and every mutation is BUILD-CHECKED before its test runs, so a non-compiling mutation is UNDETERMINED and never counted as caught. That closes the trap that bit an earlier agent today.

Two findings the battery produced about ITSELF, which is the strongest kind: the first run came back all-undetermined because the sandbox copy broke module replace paths; and one mutation came back **NOT CAUGHT** because its literal matched the wrong exit first

— **it was damaging a leg it wasn't measuring.** Both fixed and re-anchored. *A mutation that silently hits the wrong target is a proof that proves nothing.*

SC-015: 15/22 → 12/22, and the loss is exactly attributed. All three lost targets are **areas that carry authored material and have no publication review** — the same three whose own endpoint already 404s. **Terms unchanged at 5/12; questions unchanged at 5/5.** *The term rule is what preserved the other twelve: the blunt gate would have taken 8,507 terms and all five areas dark.* **Nothing was tuned toward the benchmark,** and the operator's instruction — security first, report the honest number — is what was followed.

Verified live after rebuild and restart: Up (healthy), health 200, web UI 200, search 200, catalogue registered at **area 5 / term 8507 / question 44,** and answering ready over 2,478 passages. go test **18 ok / 0 FAIL** (baseline 17), latency p95 **628.8 ms** against a 2,000 ms budget, all provers green.

Four things left open and named, the first being a genuine limit: the manifest's own file-derived area and term rows **carry no minted identifier,** so **no decision can be read for them** — closing that needs minted identity, a pipeline change. A first attempt keyed the gate on catalogue *kind* and dropped them wholesale; **an existing gate's own vacuity check caught it.** Also: 16 registry rows with no taxonomy record, and 75 rows across four smaller knowledge kinds that keep the blunt withhold — *a population small enough to review individually, which is exactly why the term case needed a rule and this one does not.*

And it verified attribution rather than asserting it: another agent's change transiently broke two unrelated tests, and it confirmed the attribution **in an isolated copy** before saying so.

A84 — Ask and Search are ONE input. Intent is resolved by four LITERAL signals, not a classifier — because when a model is wrong there is nothing to print in the “because” line.

features/inquiry/inquiry.component.ts. /ask and /search both route to it and **neither redirects:** a path is an expressed intent, so /ask seeds “*answer this*” and /search seeds “*read what I type*”. Nav goes 6 items → 5.

Intent resolution, and why there is no model in it

core/intent.ts is pure, no Angular, and uses **four literal signals a reader can check by eye**: a question mark; an interrogative/imperative opener (apostrophe-stemmed, so Why's ≠whys); ≤4 words with no asking signal → lookup; otherwise ambiguous. **The reason for refusing a classifier is stated in the file: when a model is wrong there is nothing to print in the “because” line.** Recall cost is stated too — the opener list is English-only, so a non-English question lands in ambiguous, **which ASKS rather than answers.** That is the safe direction to fail.

How a reader tells which mode answered — and overrides it

- A verdict banner prints the reading **and the signal that produced it, verbatim**, with data-code="question|lookup|ambiguous".
- Two <section>s carry data-mode="answer" / data-mode="search", each with its own heading and state machine. **Nothing crosses.**
- **Every Intent member carries a non-empty override label.** Lookup → “*Answer this as a question*”, one click, no retyping. A question in flight → “*I only wanted the passages*”, which **unsubscribes and actually cancels the request** (asserted via askReq.cancelled) — **not hiding a pane while the model burns a minute.**
- A standing mode control; when fixed, the verdict says “**You chose...**” rather than dressing the reader’s own instruction as something read out of their words. Changing the mode does **not** silently re-run the last query.
- **Ambiguity is resolved by asymmetry, not by a guess:** search runs (it costs milliseconds and claims nothing) and the answer is offered visibly. Guessing wrong toward lookup costs a click; guessing wrong toward answering costs tens of seconds of a loaded host **plus a written claim nobody asked for.**

Declined / unavailable / withheld never collapse into “no results”

The answer pane has **six** branches: not_requested (a fourth state carrying the offer — **never a blank**), working, answered, declined (named §5.5 reason + four retrieval figures + closest passages), unavailable, and a new **withdrawn** — a fifth state that is **neither a refusal nor a fault.**

When the two halves disagree, pairNote() reconciles them in words: a decline over N found passages says matching words is a lower bar than supporting a claim; a decline over no_match says these are two separate findings; an answering fault over good passages says the answer is missing because the machinery did not run.

Leg failure is finally surfaced — the old search component **never rendered legs at all**. Now a chip per leg in vocabulary words (keyword match: ran, code search: not run) plus a banner on failed: *“a shorter list than the corpus would have given, not a smaller corpus.”*

The gate derives from THREE sources

verify-ui-intent-resolution.sh reads the readings from intent.ts, AnswerPhase from the component and EnvelopeStatus from contract.ts — and types none of them. Proof mutations **M3/M12/M13 each add a member to a DIFFERENT one** of the three and require the gate to redden **byte-identical**. prove-... is **24 of 24**: 13 mutations each required to *name* its assertion, 7 rc-2 levers, 2 controls, 2 non-vacuity.

ng build --configuration production	0	353.44 kB initial
ng test	0	TOTAL: 193 SUCCESS (was 162)
verify-ui-labels.sh	0	579 graded, 590 found,
11 exempt		
verify-ui-vocabulary.sh	0	76 declared members
verify-check-registry-001/002.sh	0	checks=76 debt=3
missing=0		
verify-ui-intent-resolution.sh	0	
prove-ui-intent-resolution.sh	0	24 of 24

Backend changes needed: NONE. It touched no pkg/answer, cmd/workshop-server, pkg/search OR pkg/index.

Six honest limits, and two are the kind that get hidden

1. **The Playwright e2e suite was NOT executed** — loaded host. Every /ask and /search expectation was inventoried and all 13 testids preserved, **but that is reasoning, not measurement** — a 2, not a pass.
2. One nav-label detail reasoned but unmeasured.
3. **A PRE-EXISTING e2e inconsistency was found and deliberately NOT fixed:** search-degradation.spec.ts:58 expects /suggestions unavailable/i while the component announces different wording that the unit suite pins. **Both cannot be true.** It kept the existing

wording rather than **changing copy to make an e2e pass** — which is the right call and the tempting wrong one.

4. **The running service still serves the PRE-MERGE bundle.**

platform/web/ was not rebuilt and the container was not restarted.

5. Retrieval quality untouched and unclaimed: **SC-008 at 0/11** means a reader may get poor passages here **for reasons that are not this merge**, and the component header says a leg reporting ok means the leg *ran*.

6. **A new build warning, recorded rather than silenced:** the merged stylesheet is 7.92 kB, over the 4 kB warning threshold and under the 8 kB error one. **The budget in angular.json was NOT raised;** the honest fixes are named in the component header.

workshop HEAD **1adf0f6**, pushed.

A89 — THE STALE-BUNDLE GAP IS CLOSED. The merged UI and the new design layer are SERVED, proved by fetching them from the running service.

A87 recorded the one confirmed gap: the browser was being handed a bundle from **13:52** while the merged component landed at **17:32**. It is closed.

Before, measured: `grep -rl inquiry platform/web/` → **0 files**.

Rebuilt with the repository's **own** path — `bash workshop/scripts/build.sh`, **exit 0** — not an invented `ng` invocation.

After, and this is fetched FROM THE SERVICE, not read off disk:

served asset	status	what it proves
GET /chunk-QBZGRITJ.js	200 , 82,628 B	intent-verdict ×1, data-mode ×2, withdrawn ×8, "I only wanted the passages" ×1
GET /styles-PACSXHNN.css	200 , 65,676 B	88 distinct --wk-tokens
GET /	200 , 12,431 B	main chunk carries Ask or search — the merged nav label

No container restart was needed, and the reason is worth recording: `podman inspect` shows `platform/web` is a **BIND MOUNT** of this checkout (`.../workshop/platform/web -> /opt/workshop/web`), alongside `chapters`, `curriculum`, `docs` and `run`. Only the index volume is a real volume. So a rebuild is live the moment it is written. **The production container was never stopped, restarted or reconfigured.**

`platform/web/` is untracked build output (git-ignored), so **there is nothing to commit** — which is why the deployment gap could exist at all while every gate was green.

A90 — The design system is APPLIED — and the agent applying it found NINE of the specification's own colours short.

The spec verified every edge against `--od-bg` **alone** and concluded *“Every kind edge clears 3.0:1. No value falls short.”* Re-measured against **all three brand grounds AND each edge's own wash** — the adjacent colours WCAG 1.4.11 actually names, and where a chip on a card really sits — **12 pairs across 9 tokens fell under 3:1, worst 2.64:1.**

Repairs move HSL lightness ONLY; hue and saturation are byte-identical, so every hue argument in the spec survives its own correction.

`docs/design-system/contrast.py` reads the hexes **out of the shipped stylesheet** and the grounds out of `brand-tokens.css`, so **it cannot disagree with what renders: rc 0 over 104 pairs**, targets text ≥ 4.50 / edge ≥ 3.00 / wash ≥ 1.20 . **No figure is rounded up** — light `--wk-rule-strong` on `surface-2` is exactly 3.00.

withdrawn and not_requested were rendering the BYTE-IDENTICAL class="idle" rectangle. Seven answer branches now have seven treatments; `verify-ui-intent-resolution.sh` prints branches: answer covers 7 of 7 AnswerPhase member(s) — positive evidence it saw the new member rather than a vacuous pass.

withheld vs error, five channels, three non-chromatic: hue 218° vs 0° (142° apart); chroma ~14% vs 70–75%; a repeating 45° stripe unique in the system; ▣ vs ▴; `role="status"/polite` vs `alert/assertive`. **vs empty:** a filled ground means content exists behind it.

Three live surfaces were painting custody in the FAULT colour and no longer do — `transcript.component.ts .redaction`, `passage-knowledge.component.ts .redacted`, and the area page, which drew a held area as “request failed”. `area_not_published` now renders as withheld **with no Retry**.

A mutation proves it bites: forcing `isCustodyCode()` to false gives TOTAL: 4 FAILED, 215 SUCCESS — *“an answer that exists and is held must not borrow another state.”*

Bundle discipline: component stylesheet 7.92 kB → 7.75 kB. The `angular.json` budget was **NOT raised**; three globally-declared classes, a third private copy of `.diag` and an ad-hoc shadow were removed instead.

Two defects nobody had named. `.diag` existed as **three private copies that disagreed**, and the inquiry one had **neither max-height nor overflow** — a large upstream blob would have run the page. And `inquiry.component.ts` **contained two raw control bytes (U+0000, U+0001)** inside a template literal: `grep -c 'lk-radius' <file>` returned **rc 1 with no output** while `grep -a` returned 6. **The file was opaque to every grep-based reader.** The gates were unaffected — all three read via `python3` — so this blinded humans and tooling, not the instruments.

Test-count honesty, unprompted: 193 → 219, and **17 of the 26 are its own**; the other 9 arrived from a different agent’s specs in the same tree and it declined to claim them.

Could not determine: no browser was opened, so **the palette has not been seen**; no colour-vision-deficiency simulation was run — the three kind families sit 1.03–1.19:1 apart in luminance, so they are *certainly not* greyscale- distinguishable, which is why every chip carries a text label.

A95 — Three constitution gates FAIL → PASS, and FOUR SIBLING GATES WERE PASSING VACUOUSLY. The badge row is mostly RED, and that is the compliant outcome.

The anchors were READ, and the name is not the requirement

§11.4.259 is not “add badges”. Its operative clauses:

“(C) MACHINE-DERIVED SOURCE DATA. Every badge’s color + value is COMPUTED from a live source of truth, NEVER hand-typed... A badge whose provenance is hand-typed fails §11.4.259.”
“(F) ... a badge-computer that PASSes its golden-RED fixture is the §11.4 bluff itself.” “a project whose badge row is HONESTLY red is compliant... A badge class that GENUINELY does not apply is GRAY-labeled with a reason, never omitted (silence-as-badge is a §11.4.201(6) FALSE-NULL — the reader assumes green because they see nothing red).”

So the requirement is a **computer**: twelve enumerated classes, none omissible, each derived from a live command, plus fixtures proving it can output RED.

§11.4.261’s ratchet is not a file — it must REFUSE. *“every subsequent audit run’s count MUST be $\leq$ the ratchet snapshot... a run that increases any class’s count REFUSES the seam... never an invented ratchet.”* And *“(E) HONEST ‘zero’ — a project narrowing the vocabulary to declare ‘zero’ is a §11.4.6 fabrication and a release-blocker.”*

FOUR SIBLING GATES WERE PASSING VACUOUSLY

cm_badge_closed_color_vocabulary, cm_badge_machine_derived_source, cm_badge_self_validated and cm_every_finding_closed_or_tracked were green with *“0 badge(s) present, vacuously compliant”* — **and would have gone RED the moment anyone added a badge.** All four now pass non-vacuously (12/12 tokens, 12/12 provenance entries, 26 rows all tracked). **A gate that is green because its subject does not exist is not evidence of anything.**

gate	before	after
cm_readme_badge_row_at_top	1 — no badge row found	0 — 12 badges
cm_zero_findings_audit_sweep	1 — SWEEP_SCRIPT_MISSING	0 — names all 10 vocabulary + 3 fixture classes
cm_zero_findings_monotone_ratchet	1 — RATCHET_SNAPSHOT_MISSING	0 — every class within ceiling

(All three re-run independently from this session: exit 0 each.)

The ratchet's first row is MEASURED, not seeded

shortcomings 0 · gaps 7 · weak-spots 1 · danger-zones 9 · todo-fixme 0
skipped-tests 9 · bluffs 0 · unresolved 0 · divergent-stale-orphan 0
uncatalogued 0 · TOTAL 26

gaps 7 is the seven non-CLOSED G<n> rows in INVENTORY.md; skipped-tests 9 includes ci.yml.disabled and the testIgnore deferral; danger-zones 9 includes three eval in vendored .specify/scripts/. Each has a real tracker entry with an honest mitigation. **No allow-list, no redaction, and --selftest proves every detector fires on a planted finding — so the zeros are measured, not vacuous.**

And it refuses: a new finding above ceiling → 1; --write-ratchet declining to RAISE a ceiling → 1; a missing ratchet → 2, never 0; a blinded detector → 2. 15/15 caught including a control.

The badges are truthful, and mostly RED

1 green, 2 amber, 9 red, gauge production ready – blocked, 8 red. **That is \$11.4.259's own prescribed output, not a failure to polish.** Drift detection fired for real: after two checks were registered, --check caught the evidence badge reading 20/20 while the live registry said 22/22 — rc 1.

A judgement call worth reviewing: the sweep **declares a self-exclusion** — it skips its own source in the 4 *pattern* detectors, because it carries the patterns it hunts as literals (12 self-matches). Proved to matter: in a temp tree where the new files are tracked, the ratchet **refused** at shortcomings 3 > 0 before the exclusion and holds at exactly 26 after. Recall cost stated in the script; the other 6 detectors scan it normally.

It also corrected TWO of my stated baselines, measured: verify-docs-chain.sh was **not** 0 when it arrived (C4 stale on CONTINUATION's exports — my drift, since cleared), and audit-environment-assumptions.sh was **1**, from another agent's in-flight files.

And it caught a wrong instruction of mine: I told it the proof convention is prove-<name>.sh. **At the umbrella root that convention does not exist** — zero prove-*.sh files, and the real pairing is verify-check-registry.sh R3 driven by the registry's {flag,sibling} kind, with

19 of 20 rows using flag `--prove-failure`. It followed the **enforced** convention rather than my description of it. `prove-<name>.sh` is the WORKSHOP's convention, not the umbrella's, and I had conflated them.

A96 — Five loose ends closed, and the flashcard label was thrown away at the LAST expression before the reader.

1. `practice.component.ts:158` was `q.kind === 'mcq' ? 'multiple choice' : 'short answer'` — **a binary ternary over a three-member enum**. The model fix was real and the server serves the kinds correctly typed; **the value survived all the way to the last expression and was discarded there**. `vocabulary.ts` declared no question-kind table, so one was added — **and a rule was added to `verify-ui-vocabulary.sh` that DERIVES that domain from `pkg/assessment/question.go`**. “A ternary cannot be gated; a table can.” Coverage 17 sources / 76 members → **18 / 79**. RED-before asserted **at the rendered DOM**: ternary restored → 2 FAILED, 1 SUCCESS, “*Expected ‘short answer’ to be ‘flashcard’*”; **the MCQ test passed in both states, which is the point — the fix must not be a rename**. Fixed → 3 SUCCESS. A third test walks `codesOf('questionKind')`, so a fourth kind is exercised with no edit.

2. **The `omitEmpty` sweep: 84 real tags, 7 meaningful-zero, 0 defects, 2 latent**. `CitationsVerified` was made **empirical**: the tag is real (marshalling false drops the key — measured), but the shape is **unreachable** — `Validate` rejects `answered + false` at **both** exits, the single answered constructor hardcodes `true`, and the poll path hand-builds a map. **Paired mutation both directions**: removing the tag fails one test; deleting the guard fails the other. Two LATENT rows on the SSE path where nothing reads them — **recorded as an executed test rather than a comment**. Context worth keeping: **eight numeric fields are already `*T` specifically to survive `omitEmpty`**, including the `CorrectIndex *int` that `git log -S` shows was never a plain `int`.

3. `verify_bundle.py` **1 → 0** using only the project's own tools; **no verifier was edited**. 4. `DOWNLOADS.md` gained a measured \$5 — and **corrected my figure**: I said 16 mutations, the real number is **23**. It also found `docs/the-platform/README.md` recording “*Measured 2026-09-04: rc 0, 10/10*” **while the gate was exiting 1** — a live PASS-bluff.

5. `workshop/CLAUDE.md`: every stale figure withdrawn by name — gates 18/20 → **23/27**, commands nine → **ten** (“*a WRONG LIST, not merely a stale count*” — `answer-providers` was omitted), tracked files 766 → **1,211**, passages `.jsonl` 11,622 → **13,141** rows, spec 001 66 → **69**, spec 002 107 → **110**. **Two claims were FALSE, not stale**: there are **three** show-toplevel call sites (not two — `_common.sh:703` was named nowhere and

is correct for a different reason), and “*every script anchors on BASH_SOURCE*” is false for `_portable.sh`, correctly. **The gate counts moved 22/26 → 23/27 inside the session**, so the document now tells the reader to run the `ls`.

A101 — The extractor mined vocabulary out of ULIDs because THIS PROJECT’S OWN EVIDENCE DISCIPLINE cites pids in prose. It is 145 rows, not 33 — and removing all of them does NOT fix the gibberish query.

The exact code path

pipeline/extract/derive.py:58 (pre-change):

```
_TOKEN = re.compile(r"[a-z][a-z'-]*[a-z]|[a-z]")
```

It finds maximal **ASCII letter runs**. A ULID is Crockford base-32 — letters and digits interleaved — so lowercasing one **produces words**:

```
_TOKEN.findall("01M1ET0MFJ4NDK0ZECKNTQFVWS".lower())  
-> ['m', 'et', 'mfj', 'ndk', 'zeckntqfvws']
```

Every one of those strings is a live `kg_term` row today.

Why RULE 1 did not stop it — and the irony is the finding

RULE 1 excludes `kg_*` rows from the extraction corpus. **These ULIDs are not in `kg_*` rows. They are in ORDINARY PROSE — because this project’s own documentation cites pids as evidence, and that documentation is ingested as `doc_section` and code passages.**

Measured: 352 `doc_section` + 29 code passages carry a ULID-shaped token, 174 of them from `chapter-01/knowledge/TAXONOMY.md` alone, the rest from `docs/training/**`.

The evidence-citing discipline that makes this project trustworthy is what manufactured the junk vocabulary. From there the path is ordinary: `_term_occurrences` → `propose_terms` → `taxonomy.mint_terms` → `minting.sync_areas` → `cmd/knowledge-mint` → a real registry row. **Nothing on that path ever asked where a token came from.**

The population is 145, not 33 — my figure was a sample

My 33 was a 45-item consonant-heavy sample and I reported it as a total. Derived exactly, and reproduced by two independent routes (the gate's own derivation, and a real `derive.propose_terms` run over the real corpus): **193 distinct `kg_term` texts are substrings of some `pid`; 145 have no provenance anywhere except inside an identifier.** The other **48 are real words that also occur in prose** — `key`, `pay`, `web`, `tree`, `spa`, `stt`, `ssh`, `gpt`, `www` — and must not be touched.

The rule is DERIVED, in two conjuncts, and the second one earns its keep

(a) an English word contains no digit, so an alphanumeric run mixing letters and digits is a machine token; **(b)** this corpus's identifiers are ULIDs (Crockford base-32) and its digests are hex, so require the whole run to lie in that alphabet.

(b) is what saves `sha256sum` → `sha,sum`, `utf8` → `utf`, `amazons3` → `amazons` — each carries a `u` or `o`, outside Crockford. Without it those are destroyed. **A blocklist was rejected explicitly:** the strings are a function of whichever `pids` the corpus happens to cite, so the next ingest mints a fresh crop under new names.

Recall cost measured, written into the source — AND A SELF-CORRECTION. Vocabulary 17,883 → 17,526, 375 tokens removed (dominated by hex digest fragments, also identifiers). **But 18 tokens were INTRODUCED** by hyphen truncation (`bge-small-en-v1` → `bge-small-en-v`). Its own first-draft docstring claimed masking can only remove tokens; **that claim is withdrawn as false in the committed source.**

Verified against the real production entry point: a fresh `propose_terms` run re-proposes **NONE** of the 145 and **all 48** survivors.
`EXTRACTION_POLICY_ID` bumped so pre-RULE-4 checkpoints cannot resume.

The gate grades the PRODUCER, and refuses to call the function it guards

V1 grades the artifact; V2 refuses a vacuous corpus; **V3 grades the extractor itself by feeding every pid to the real shipped derive._tokenize. And it re-derives the identifier shape rather than importing mask_identifiers — “a gate that calls the function it guards goes blind exactly when that function breaks.”**

12 assertions, 12 passed. M5 is the one that matters: the real `is_identifier_run` copied to scratch and inverted to return `False` → **rc 1 with V3 named, and V1 STAYED GREEN**, so the red is provably V3's alone.

The gate is RED today (rc 1, 145 findings) and that is designed: V3 is green — the extractor is fixed — while V1 is red, because the 145 rows are still in the registry.

REMOVING ALL 145 DOES NOT FIX THE GIBBERISH QUERY — measured, not assumed

`limit=40/60/200` all return the same **40** rows, so 40 is the complete above-floor set. Classified against the derived finding set:

- **37 of 40 are identifier-derived.** They would go.
- **3 remain, all still above the 0.6550 floor:** `zzz-doc-sweep-probe` (0.677192), `zzz-degraded-probe` (0.657942), `zzz-definitely-nonexistent` (0.657165).

Those three trace to **real prose** in `docs/limits.md`, `docs/quickstart.md` and `docs/faq.md` — worked “*search for something that doesn't exist*” examples. **They are legitimate corpus vocabulary and the gate correctly does NOT flag them.**

So the top score would fall 0.7189878 → 0.677192 and hits 40 → 3, and the query would still clear the floor. The 0.719 has a SECOND CAUSE and it is not closed here.

One more measurement worth carrying: ?

`q=zzqxxvbnmqzz&kinds=transcript_segment,doc_section,code` returns **zero** hits. **Every result this query has ever produced comes from `kg_*` rows.**

Removal: five options costed, NONE executed

Established: **no Registry.Delete/Remove/Prune exists anywhere** in submodules/passages or the backend; workshop-redact never deletes, it sets redacted:true; R1c does **not** fire for kg_term (their anchor is registry_only); and the served registry is /var/lib/workshop/passages.jsonl **inside the volume**, not curriculum/passages.jsonl.

Option A (workshop-redact, no -purge) preserves anchors and is **precedented — 41 kg_term pids are already redacted here under operator-decision-28. A2** (-purge) gives up R6 irreversibly: **recommend against. B** (hand-delete) carries an irreversible ordering trap. **C** (rebuild) is **not viable** — ingest-transcript mints no kg_* at all, so it would drop all 9,144 graph rows. **D** needs a new public API.

Every option pays a full re-embed of ~12,979 rows — and the module's own record says that at 2,478 passages, generations 55–63 each attempted a re-embed and NOT ONE FINISHED, all nine stopping on SQLITE_BUSY. That specific failure is now fixed (A92, WAL), but the corpus is 5.3× **larger** than when it failed nine times.

Executed none, for three separately disqualifying reasons: it needs an operator identity in an append-only log; it needs a container restart the brief forbade; and the re-embed risk is an operator call. **This is an operator decision and it is now costed rather than vague.**

Verified

go build / vet / test ./... -count=1	0 / 0 / 0	18 packages
ok		
verify-identifier-vocabulary.sh	1	145 findings (V1);
V3 GREEN — red by design		
prove-identifier-vocabulary.sh	0	12/12
verify-check-registry-001 / -002	0 / 0	84 checks
verify-search-determinism.sh	0	26 graded, 0 varying
verify-redaction-propagation.sh	0	
GET /api/health	200	container Up 35 min
(healthy)		

pipeline/extract pytest exits 1 with **32 failures whose set is BYTE-IDENTICAL before and after** the change — diffed by swapping HEAD's derive.py back in. Cause measured: wordfreq is absent under the system python3 and present in pipeline/venv, which has no pytest. **Pre-existing and environmental.**

Could not determine: whether the ~12,979-row re-embed would complete on this host; whether the three `zzz-*` probe terms are a defect at all — *“they are genuine documentation content, so they are legitimate vocabulary by every rule I could derive, but they are also why the gibberish query still clears the floor”*; and a **63-row wider population** (208 rather than 145) of hex-digest and version-string fragments that RULE 4 stops the extractor minting but the gate does not grade, **recorded in the gate’s header so 145 is not mistaken for the whole population.**

A100 — I BRIEFED A “CONTRACT VIOLATION” THAT WAS THE CONTRACT WORKING. Fixing it would have told a reader “no open questions” about a session holding 75 minted rows.

The refutation, and it is the most valuable result of the session

I briefed two defects. **The second was FALSE, and the agent proved it rather than implementing it.**

I said: the four 404s return Go’s default text/plain while §1.5 requires a JSON error object on every 4xx — *“fix that regardless.”*

The bare 404 does not come from those four paths at all. Nothing registers them, so they fall to the `/api/` subtree catch-all, **whose bodyless 404 is REQUIRED — three times over:**

- `platform/gates/route-manifest.tsv`’s 404-plain means *“404, and the body carries neither status nor error”*;
- `http-api-delta.md` §2 **R2**: a not-built route answers *“404, with **no** status field, **no** error field”*;
- `internal/api/router.go:85-115` is a three-clause decision citing §5.2 — **the closed enum has no member meaning “this endpoint does not exist”.**

§1.5 governs malformed requests to routes that EXIST. I applied it to routes that do not.

AND THE HARM WOULD HAVE BEEN CONCRETE. `api.ts`’s `httpToLoadStatus`:

```
if (e.status === 404) {  
  const env = parseEnvelope<never>(404, e.error, 'results');  
  if (env.status === 'unavailable') return  
    loadUnavailable(env.reason.code);  
}
```

```

return empty(); // ← a JSON 404 becomes a
    DETERMINED NEGATIVE
}

```

Adding an error object would have flipped four chapter views from “not implemented” to **“nothing planned / no open questions / no TODOs”** — about a private teaching session holding **75 minted rows**. That is precisely the FR-020 two-states-where-three-are-required defect the contract exists to prevent, and it would have been installed in the name of fixing it.

A brief is not evidence. This one was wrong, and the agent that refused it was right.

The four endpoints are UNCONTRACTED — neither unbuilt nor deferred

```

grep -rn "next-meeting|open-questions|meeting-notes|/todo"
specs/ -> ZERO hits

```

The two contracts define **31 endpoints between them and not one of these four**.

It found the third party in the argument — the client had already run this check on itself and written the answer down. platform/frontend/src/app/core/workshop-sections.ts:

“ASSUMED, NOT MEASURED (§11.4.6...). no contract document for them existed anywhere in this repository or at the umbrella root’s specs/ when this file was written (checked 2026-09-02)... The shapes below are therefore this client’s STATED CONTRACT... not an observation.”

docs/work-register.md **R20** is the operator request, status “dispatched”. **Both ends landed; the API between them never did, and nobody ever wrote the clause.**

So it did not invent one. Four NOT_BUILT / 404-plain rows were added to the route manifest, each naming its data, its client component and Contract: none. verify-server-unity.sh prints them as standing DEBT on every run — **DEBT 4 → 8. The gap moved from invisible to gated, with no response shape guessed.** And the withholding rule is exactly what must not be improvised: one kg_meeting_note row is already recorded as an operator content judgement **left undecided**.

It also corrected my data counts — I wrote 24/27/10/14; the real mapping is kg_next_point **24**, kg_open_question **14**, kg_meeting_note **10**, kg_todo **27**. I had transposed two.

The sweep I asked for, delivered — and one real defect inside it

Source level: zero http.Error(call sites. Every 4xx goes through writeError/writeJSON. All six http.NotFound references were judged individually and **all six are CORRECT and load-bearing** — including main.go:1009, where a 405 would wrongly assert the resource exists.

Live, all 30 registered GET /api routes: 24 could be driven to a 4xx, and **24 of 24 answered application/json carrying "error". Zero bare 4xx.**

Six routes have no 4xx lever and were reported as NOTE, not as passes — *“an unprobed route is not a passing route.”* Four non-GET routes were deliberately unprobed, because probing POST /api/ask/suspend would suspend a production answerer.

The real defect found in passing: GET /api/passages is registered — a deliberate 400 guard — and had **no manifest row**. verify-server-unity.sh’s U6 cannot see it, because **U6 walks the CONTRACT and the contract defines no such endpoint**. Row added.

The new gate asserts a DISCRIMINATION, not a property

verify-error-shape.sh + its proof. **E1 derives the route set from the router’s own mux.Handle* calls (34 /api routes) — the existing assertion of this property covers 4 HAND-TYPED paths out of 34**, the very “hand-maintained coverage set” defect http-api-delta.md §2 R1b already documents. **E3: registered + 4xx ⇒ JSON with a discriminator. E4: unregistered + 4xx ⇒ plain, bodyless 404** — so the gate now protects the behaviour I would have broken.

8 mutations caught, 0 missed. M4 IS the change my defect report asked for, and the gate goes red on it. M8 requires an empty derived route set to be rc 2, **never a vacuous green.**

verify-error-shape.sh	0	6 PASS / 0 FAIL / 0
UNDET / 10 NOTE		
verify-error-shape.sh --base <dead>	2	not a pass
prove-error-shape.sh	0	8 caught / 0 missed
verify-server-unity.sh	0	37 PASS / 0 FAIL / 8
DEBT		


```

verify-redaction-propagation.sh      0    162 of 162 suppressed
pids
verify-search-determinism.sh         0    26 graded, 0 varying
verify-answer-question.sh            0    10/10 L5 properties
verify-check-registry-001 / -002     0 / 0
go build / vet / test ./... -count=1 0 / 0 / 0    (no .go file
changed)

```

Nothing new is served, so nothing new can leak — re-verified anyway: a suppressed pid returns **410 + JSON** on all three passage routes. The container was **not** restarted; no new binary was needed.

Could not determine: whether an operator wants these four endpoints at all. R20 asked for the capability; **nobody has decided the envelope, the reason codes or the withholding rule**. That is the open decision, and it is now printed as DEBT on every server-unity run instead of being invisible.

A99 — The B9 “floor” defect was one level up: the LABEL was honest, the CALIBRATION CLAIM was the bluff. And the knowledge graph contains 33 terms mined out of ULIDs.

My framing was wrong and the agent corrected it: “your list was 4 distinct tests, not 5” — 8 failures across two browser projects. The genuine fifth was HIDDEN BEHIND the fourth and only surfaced once that was fixed.

1. above_floor was arithmetically TRUE.

floor_calibrated: true was not.

above_floor is computed, not assumed — `service.go split()` does `d.Score <= s.Floor`, the floor is 0.6550 and the hits scored 0.719. **The label told the truth.**

The bluff was one level up. `calibratedFloor()` keyed on (model, doc-prefix, query-prefix) and was **blind to the corpus**, while its own recorded evidence reads *“MEASURED at FULL SCALE — the whole 2,478-passage live corpus ... separation +0.039883, ZERO overlap.”* **The corpus is now 12,979**, with 8,553 `kg_term` rows minted 2026-09-02 that the calibration never saw.

```

GET /api/search?q=zzqqxvbnmqzz&mode=semantic
BEFORE  20 hits, top 0.7189878282854324, floor_calibrated=TRUE
AFTER   20 hits, top 0.7189878282854324, floor_calibrated=FALSE

```

Independently re-measured from this session: floor_calibrated: False, top score unchanged at 0.7189878282854324. That gibberish score sits **0.083692 above the calibration's highest negative and 0.043809 above its LOWEST POSITIVE — the negative outranks the bottom of the positive range**, so the “zero overlap” claim is void on today's corpus.

The floor value was not touched, correctly. And `no_match` is reachable in fused mode (photosynthesis in mangrove swamps → `no_match`); only single gibberish *tokens* clear it.

2. THE ROOT CAUSE OF THE GIBBERISH SCORE: terms mined out of identifiers

33 kg_term rows are lowercased substrings of transcript-segment ULIDs:

```
pid 01M1ET0MFJ4NDK0ZECKNTQFVWS -> kg_term "zeckntqfvws"
```

Verified independently here: of 45 consonant-heavy `kg_term` candidates sampled, **33 appear as a substring of a real pid** — matching the agent's count exactly. **The taxonomy extractor mined vocabulary out of identifiers**, which is *why* a nonsense query embeds at 0.719: the corpus genuinely contains nonsense that looks like it. **A pipeline defect, not a serving one**, correctly refused as out of scope by the agent that found it. Dispatched.

3. /api/passages/{pid} 400 — the SERVER was right, the test was stale

Decided by contract, not convenience: `http-api.md` §3.8 (malformed `pid` → 400), §1.5 (“*a 4xx is a fourth thing, not a fourth state*”) and `passage-contract.md` §2.1 (26 Crockford characters, “*never namespaced*”). The test built a URL from `results[0]`, which is now a term catalog hit with `pid kg_terms:index`. **It should not have built the URL at all — §3.7 gives every hit an href**. Stale because term was not a searchable kind until 002 C4.1.1.

4. deep_link was not MISSING — 11 of 16 were DEAD

`chapter_slug` is the ingest **scope**, so `docs/**` files got scope `docs`, and `/api/chapters/docs` is a **404**. Measured over `q=index&limit=50`: 16 hits with a chapter, **11 dead links, 8 pointing into a chapter that does not exist**. Now `transcript_segment 5/5`, everything else 0, broken 11 →

0. An area deep link was added and verified painting in a browser; **term and question keep "" because no route renders them — “a fabricated destination is worse than none.”**

5. The 60-second route was /chapters/undefined, and the fifth defect hid behind it

Same root cause as #3. It serves **200** and renders an honest chapter_not_found state carrying **no <h1> inside <main>**, so the wait burned the full 60 s. The failure now names the route **and what <main> actually contained.**

Unmasked by fixing it: four endpoints a real chapter reaches — next-meeting, open-questions, meeting-notes, todo — **all 404 with NO BACKEND HANDLER AT ALL, though the data exists (24 / 27 / 10 / 14 rows).** Declared as capability probes so the gate now *requires* the four state-absent surfaces rather than tolerating the 404s. Dispatched.

Also open and dispatched: those 404s return Go’s default text/plain 404 page not found, while §1.5 requires a **JSON error object on every 4xx.**

Stale versus real, stated plainly

Real defects: the calibration claim, and the dead deep links. **Stale expectations written against the smaller index:** the passage-pid test, the slow route, and the NO_MATCH_TERM constant — all three downstream of term hits entering the ranking.

```
live-content.spec.ts    8 failed / 12 passed  ->  0 failed / 20
passed / 2 skipped
go build / vet / test ./... -count=1          0 / 0 / 0
verify-search-determinism                      0    26 graded,
identical over 5 runs
verify-answer-question                          0
verify-redaction-propagation                    0
verify-check-registry-001 / -002              0 / 0    82 entry
points
```

Both fixes ship §1.1 paired proofs **verified in both directions** — failing against the pre-fix state, passing against the fix. Rebuilt and restarted twice via workshop/scripts/build.sh and workshop/scripts/restart.sh, health 200 both times.

Could not determine: whether the calibration's corpus fingerprint can be made exact — the sufficient key is the generation `root_hash`, which `index.Generation` carries but the benchmark results file never recorded, so **the passage-count pin is necessary but not sufficient**, and the code says so.

A98 — Both CSS collisions fixed STRUCTURALLY and proved on rendered pixels. And an alarm about `ng test` is NARROWED by measurement rather than propagated.

The two collisions, measured before and after on real elements

Not synthetic divs — `state-working` was held open by a **stalled** `/api/ask` and then released into `state-declined`:

state	BEFORE	AFTER
not_requested	dashed 1px rgb(145,134,121)	dashed 1px rgb(145,134,121)
withdrawn	<i>identical to above</i>	solid 5px rgb(147,71,79)
declined	left: solid 5px rgb(147,71,79), img: none	unchanged
working	<i>identical to declined</i>	left-color: transparent, img: repeating-linear- gradient(...6px...11px)

They now differ on **two channels**, not one invisible property.

Fixed structurally, not patched — and the reasoning is the valuable part

- `.idle--back` was moved below `.idle` AND its selector changed to `.idle.idle--back` (specificity 0,2,0 vs 0,1,0). Reordering alone was what the failure message suggested and **would work today — and is one reordering away from breaking again silently**. Specificity makes it order-independent.
- For `declined/working` it did **not** simply add a property. It **removed the `border-left:` and `background:` SHORTHANDS** from the shared block, replaced them with longhands, and gave each state its own

border-left-color. **Neither state can now erase the other's half of the distinction on source order** — the defect class removed rather than the instance patched.

- The working edge is **form, not hue**: the same accent broken into ticks (work not finished) against declined's continuous rule (work over), so it **survives greyscale**. Nothing animates, so nothing needs undoing under prefers-reduced-motion.

The vocabulary finding was NOT what I briefed, and the correction matters

I said two reason codes had no wording. **They already had entries** — malformed_pid → “*not a passage identifier*”, area_not_found → “*no such knowledge area*”. **The real defect was in state.component.ts: its lookup chain consulted withheld, decline and reason and NEVER errorCode, so 14 of the 15 members of that table were unreachable from any surface.**

And verify-ui-vocabulary.sh was green and was RIGHT to be. It asserts that every backend code has an entry **in the table**, and every one did. **Its coverage set is the table, not the render path.** So this was a hole in the *test suite*, not in that gate's contract — a distinction worth keeping, because the lazy reading would have been “the gate missed it”. Closed with a **derived** unit guard iterating VOCABULARIES.errorCode, proved non-inoperative: removing the lookup makes it go red naming all 14. Verified live in the browser against the served bundle.

Budget discipline

Baseline 7.75 kB → edits took it to 7.77 → **trimmed to 7.68 kB, BELOW baseline**, with angular.json untouched. Five copies of border-radius: var(--lk-radius, 8px) were grouped into one rule — **the minifier does not merge declarations across selectors, so each copy shipped**. Cascade-safety measured, not assumed: nothing else sets border-radius on those six selectors, and two with a different asymmetric radius are deliberately excluded.

Proof the fix is SERVED, fetched back over HTTP: styles-DDUEIR6C.css carries both new rules, and chunk-F07EABCQ.js (200, 82,736 B) shows .idle immediately followed by .idle.idle--back. Old hint text grep -c = 0.

```
ng test                                0    223 SUCCESS (222 + 1 new guard)
ng build --configuration production  0    inquiry stylesheet 7.68
```

kB

playwright design+inquiry evidence 0 42 passed, 2 skipped –
both red tests green

verify-ui-labels / vocabulary / intent-resolution 0 / 0 / 0

verify-check-registry-001 / -002 0 / 0 checks=82

THE `ng test` ALARM IS NARROWED — I could not reproduce it as stated

The agent reported: “*ng test exits 0 on a compile failure... a run that compiled nothing looked like a run that passed*”, and flagged it as a hole in this project’s unit-test exit contract. **That would be serious — it would make every green `ng test` in this session suspect — so it was tested rather than recorded.**

Planted a deliberate compile error twice, restoring cleanly each time:

```
spec-side error      (`const broken: = ;`)      -> ng test  
EXIT=1  
component-side error (early backtick in styles:) -> ng test  
EXIT=1
```

Both printed `ERROR [karma-server]: Found 1 load error` **and returned 1.**

So the exit contract HOLDS for the invocation this project actually uses: `ng test --watch=false --browsers=ChromeHeadless`. The likely difference is that bare `ng test` defaults to **watch mode**, which another agent independently hit — “*it never exits*” — and an exit code observed from a process that never terminates on its own belongs to whatever killed it, not to the test run.

The alarm is not dismissed; it is BOUNDED. What is true and worth keeping: **bare `ng test` in this workspace does not terminate, so any exit code taken from it is meaningless.** Always pass `--watch=false --browsers=ChromeHeadless`. What is NOT true is that a compile failure can be seen as a pass under that invocation.

Recorded but not fixed (correctly, as out of scope): the `anyComponentStyle` 4 kB *warning* pre-dates this change and still fires at 7.68 kB; only the 8 kB error budget was in scope. And whether working’s 6px-on / 5px-off tick pitch is *optimal* is a design judgement nothing here measured.

A97 — THE BROWSER FALSIFIED TWO DESIGN CLAIMS. `withdrawn` and `not_requested` are the same rectangle, and a CSS shorthand is why.

Full suite against the live stack, twice: **248 tests / 21 failed** before, **274 / 13 failed** after. The second run was **re-taken after the restart**, so it measures the build that ships — and it independently confirmed the determinism fix (6 identical `/api/search` calls, identical order).

Two design claims are FALSE, measured by reading the PAINTED element: 1. `withdrawn` and `not_requested` render identically — both dashed `1px rgb(145,134,121)`. Cause: `.idle--back` and `.idle` have **equal specificity, so source order decides — and `.idle`'s border: SHORTHAND resets all four sides, erasing the accent edge.** 2. `declined` and `working` render identically — same rule in `workshop.css`.

Both were left RED with the diagnosis rather than fixed, and the reason is exactly right: *“repairing a component while claiming to verify it defeats the point.”*

What DID hold, seen in a browser: `withheld` vs `unavailable` on all five channels — hue $>60^\circ$ apart, chroma strictly lower, painted background-image edge vs none,  vs , `role=status/polite` vs `role=alert` — **three of them non-chromatic, so it survives greyscale.** A live custody surface was reached with nothing intercepted.

suggestions unavailable was decided by the CONTRACT, and the SPEC was the wrong side. §6.7 requires only that suggestions be announced through a live region and prescribes no wording; the phrase appears nowhere in `specs/**`, while the component's sentence is pinned by the unit suite. The test now asserts the closed code **plus the absence of determined-negative copy — stronger than the regex**, which would have passed on an announcement that also said “nothing matches”. **No component copy was edited.** That test had a second, unnoticed defect: it probed `q=s` and typed `sil` — 503 versus 200, so its skip guard and its assertions read different endpoint states.

The most instructive spec fix: the a11y nav test PASSED, which is worse. `getByRole`'s name is a substring match, so “Search” silently matched “Ask or search” — a label it no longer describes. Now exact.

22 screenshots were produced and deliberately NOT committed: e2e/artifacts/ is git-ignored **as a privacy boundary**, and a spec asserts it stays ignored — committing them would break an existing gate. Every fixture is **synthetic**, so component, normalisers, template and paint all execute while nothing private can reach an image.

Two live reason codes have no wording — /api/areas/<bad> renders “unlabelled: malformed_pid” and “unlabelled: area_not_found”. The UI is honest rather than inventing a phrase, but the gap is real. **And the ambiguous banner says the same thing twice.** Both dispatched.

Five failures remain, all backend and all correctly attributed away: the B9 uncalibrated floor (the semantic leg labels top-k above_floor while mode=lexical honestly says no_match), /api/passages/{pid} → 400, a missing deep_link §3.7 requires, and a route not painting inside 60 s — **which the loop does not name on failure.** Dispatched.

A94 — THE SERVED SET WAS NON-DETERMINISTIC, AND THE MECHANISM IS A GO MAP. Found, explained, fixed, deployed, and verified live: 26 queries, 0 varying.

The mechanism — determined by measurement, not narrative

digital.vasic.rag/pkg/reranker.MMRReranker.Rerank, called by Service.fuse on **every** request:

```
unselected := make(map[int]bool, n)
for idx := range unselected {           // Go RANDOMIZES map
    iteration order
    if mmrScore > bestScore { ... }      // strict >, so the FIRST
    maximum wins
}
```

Go deliberately randomises map iteration order, and a strict > means the first maximum encountered wins — so a different element wins each run.

And ties are the NORMAL case here, not a corner. MMR’s relevance term is a Jaccard word overlap against the **raw query string**, which is exactly **0 for any document sharing no literal token with the query** — **i.e. for every paraphrase.** Rerank then **truncates to TopK**, so a tie straddling the cut decides **whether a document is served at all.**

Proved directly: 500 identical calls on one tie block gave 24 distinct orders and 6 distinct SETS.

The measured distribution, before

Queries derived from `pipeline/benchmark/retrieval_benchmark.json` (26 positive queries), against production, generation 68, `legs.semantic` = ok every run:

	SET varies	ORDER varies
limit=10, 6 identical runs each	14 / 26	19 / 26
limit=5, 8 identical runs each	8 / 26	17 / 26

It reproduced the exact signature that opened this:

[6, None, 6, 6, 6, 6] — present five times, absent once, same query, same generation.

The evidence that LOCATED the defect is the elegant part. Runs whose ORDER differed had **bit-identical score vectors** — positions 1 and 2 both exactly 0.8122329492048093. **That places the defect after scoring and before the wire**, which independently rules out embedding non-determinism, leg failure, index mutation, deadline truncation, WAL snapshots and the vector cache. No separate experiment was needed to eliminate any of them.

The fix, and the subtlety in it

`pkg/search/determinism.go` — a **TOTAL** order (score desc, **pid asc**). **sort.SliceStable on score alone is PARTIAL, and a stable sort faithfully reproduces whatever non-determinism reaches it** — so stability is not determinism. Applied in `split` and in `rankAgainst`, the latter mattering independently because it is a truncating sort whose tie order came from SQLite's scan order at first cache load.

`fuse` no longer calls MMR at all: its ordering never reached the wire (scores were restored by `pid` and `split` re-sorts by them as its first statement), so **its only live effect WAS the random truncation**. The `cap` expression is carried over unchanged.

MMR's diversity objective was NOT reimplemented — that needs a deterministic `argmax` inside `digital.vasic.rag/pkg/reranker`, an upstream change to a consumed submodule, correctly refused as out of scope.

NO BAR MOVED, and the agent proved it

SC-015 is 36/156 = 23.1% **before AND after**, same generation, production- identical flags. Only 3 of 26 queries changed rank, none crossing the top-5 boundary. Queries whose target appears in *every* run: 8 → 9.

Deployed and verified live — the loop is closed

The fix was committed but **not running**: platform/bin is a bind mount and the container held the previous binary in memory. So: rebuilt (workshop/scripts/build.sh, exit 0; the new binary contains search/determinism.go), restarted via workshop/scripts/restart.sh — **a full down/up, never a compose restart, because a restart that appears to succeed while running the old binary is worse than no restart** — and healthy in ~5 s.

(A transient unhealthy was investigated rather than waved past: the healthcheck log reads rc=1, rc=1, rc=0 — two probes fired while the server was still writing server.json. It self-resolved to healthy.)

Then the gate was run against the restarted production:

```
queries derived from the stored benchmark : 26
queries actually graded                   : 26
generation (must be one value)           : 68
queries with a varying SET                : 0
queries with a varying ORDER             : 0
queries with a varying SCORE VECTOR      : 0
queries that could not be graded          : 0
PASS: all 26 graded queries served an identical set, order and
score vector over 5 runs
```

SET 14/26 → 0/26. ORDER 19/26 → 0/26. Byte-identical lists across a process restart.

The gate and its proof

verify-search-determinism.sh — three-valued, queries **derived** from the benchmark rather than typed, grading SET / ORDER / SCORE-VECTOR. **It refuses to pass vacuously**: an empty service, fewer than 2 results, a moving generation and an unreachable stack are each **rc 2**. Against unfixed production it returns **1** (14 SET / 19 ORDER / 4 SCORE); against the fixed instance **0**.

prove-search-determinism.sh — **rc 0, 8 mutations, 8 PASS**: M1 control (constant → 0), M2/M3/M4 non-vacuity (varying set / order / score → 1), M5–M8 three-valued (absent service, vacuous service, moving generation, absent benchmark → 2).

determinism_test.go carries a control that asserts THE LIBRARY DEFECT IS STILL REPRODUCIBLE — so the tests cannot go quietly vacuous if the upstream reranker is ever fixed. That is the rarest kind of test and worth copying.

Registered G-DET-1..5; verify-check-registry-002.sh **rc 0, 82 checks**.

Why this outranks the ranking-quality problem it was found beside: it means *every* retrieval figure this project has ever recorded — SC-007, SC-008, SC-015, every benchmark — was **a sample from an unknown distribution rather than a measurement of the system**. A number you cannot reproduce is not a measurement.

A93 — ai_interviewing published (cde474f → cb4c62a), and the whole fleet is back to 12 CURRENT / 0 DRIFT.

The MCQ fix was committed by an agent that **deliberately did not push**, calling it an operator decision. It was verified before publishing rather than taken on trust:

```
go build ./...          0
go test ./... -count=1 0 4 packages ok (api, ingest, server,
store)
```

The diff was read, not just the test result — 32 changed lines for what should be a one-field fix is worth checking. It is exactly right: **one functional change**, `CorrectIdx int`
`\json:“correctIndex,omitepty”→json:“correctIndex”`; the rest is **gofmt realignment** of the struct tags, which moved because the added comment changed the field widths.

The comment is the durable part, and it is why the defect cannot silently return: it records that the index is **zero-BASED**, so index 0 — the correct answer being choice A — **is a meaningful value, not an absent one**; that encoding/json dropped the key entirely for those items; that the client’s `i == q.correctIndex` then compared against undefined, making **every choice wrong**; that **40 of 312** authored items sit at index 0; and it names the regression test. A future reader re-adding `omitepty` has to ignore an explanation to do it.

Gitlink and helix-deps.yaml moved together (C9): **PASS, 12 recorded refs equal the gitlinks this repository will commit.**

verify-submodule-remote-sync.sh → 0: 12 CURRENT, 0 DRIFT, 0 UNDETERMINED of 12 owned gitlinks probed. Every owned submodule equals its remote tip, measured just now. The gate prints its own honest boundary and it is worth repeating: **a dated observation, not a standing fact — remotes move.** This carrier records that same pin going stale within a day on four consecutive occasions.

A92 — THE BIGGEST FINDING OF THE SESSION: the search index was 36.5% complete while the API reported legs.semantic = ok. One SQLITE_BUSY abandoned the corpus for the whole process lifetime.

The root cause, measured from the server's own log

index.Open returned a handle carrying SQLite's **defaults** — journal_mode=delete, busy_timeout=0 — and **that same *sql.DB is written by the background vector-indexing pass while /api/search reads on other pooled connections.** The first write that overlapped a live read died instantly:

```
vector indexing stopped after 3256 vectors in 19m54.581s:
  database is locked (5) (SQLITE_BUSY)
```

Nothing retried. One error abandoned the corpus for the entire process lifetime. Generation 68 held **4,736 vectors for 12,979 passages — 36.5% — while the API cheerfully reported legs.semantic = ok.**

A leg reporting ok means the leg RAN. It has never meant the leg had data. That is the most expensive lesson in this file: every SC-007 and SC-008 figure recorded before today was measured over a corpus **a third of which was unsearchable**, and no instrument said so.

The reproduction ruled out the OBVIOUS fix, which is why it is worth trusting

variant	result
bare (the old open)	SQLITE_BUSY in 1 ms
busy_timeout alone	SQLITE_BUSY after waiting 4.918 s OK, 0 s

variant	result
WAL only / WAL + busy_timeout	

A busy timeout alone does not work. Under `journal_mode=delete` a live reader holds a shared lock the writer can never upgrade past, so waiting longer cannot help. **WAL is the fix.** Had the plausible remedy been applied without testing it, the defect would have survived the fix.

Independently verified from this session, reading the served volume directly:

```
journal_mode = wal           (was delete)
passages.db-wal  53.5 MB    present, so WAL is genuinely in use
embeddings      15,457 rows
/api/search      HTTP 200 in 0.305 s
```

All four hypotheses answered, and SC-008 is NOT a plumbing fault

- 1. Embedding asymmetry — RULED OUT, decisively.** Re-embedding stored passages, `search_document`: reproduces the stored vector at **cosine 1.000000 (6/6)**; `search_query`: gives 0.9708, no prefix 0.9218. The query side was confirmed too — a live score of 0.6959 equals the offline `search_query`: cosine **to four decimal places**.
- 2. The 0.6550 floor — contributory, NOT sufficient.** Zero-overlap targets score 0.5379–0.6959 (median 0.6229), so **9 of 11 fall below the floor** and it rejects a third of known true positives. **But only 1 of 11 is in the semantic top-5 at all — with the floor removed ENTIRELY, SC-008 caps at 9.1% against an 80% bar.** And no floor value separates the classes: one target scores 0.6229 while the row above it scores 0.6244. **Lowering the floor would have bought nothing and hidden the real problem.**
- 3. Candidate window — ruled out by construction.** The reorder is `bm25-OR/RRF` and the harness *refuses* any zero-overlap row sharing a token with its target, so `bm25` scores it 0.
- 4. Missing vectors — a real defect, but not this cause.** All 27 targets already had vectors.

The numbers, and the honest reading of them

	before	after
gen-68 vectors	4,736 / 12,979 (36.5%)	

	before	after
		12,979 / 12,979 (100%)
SC-007	51.9% (14/27)	44.4% (12/27)
SC-008	0.0% (0/11)	0.0% (0/11)

Indexing completed in **one attempt**, zero `SQLITE_BUSY` (1h38m under load), and **it also unblocked cross-reference derivation, which had NEVER completed for generation 68 — now 259,580 edges over 12,979/12,979.**

DO NOT READ SC-007 AS A 7.5-POINT REGRESSION. The agent found **the measurement itself is not reproducible**: same query, static generation, byte-identical query vector, leg ok every time — five `mode=semantic` runs put the target at ranks **1, absent, absent, absent, 1**. Both figures are samples from that distribution. Ruled out by measurement: embedding non-determinism, leg failure, index mutation, and a deadline-truncated scan (`rows.Err()` is checked). **Mechanism NOT determined — this is a distinct, newly-surfaced defect** and it is now the most important open question in retrieval.

Reported rather than smoothed: completing the index makes ranking *worse* (zero-overlap median rank 30 → 66), and the agent's own `kg_term-flood` hypothesis was **REFUTED** — top-20 is 92% `doc_section` both before and after.

Nothing was tuned. The floor is untouched, no bar widened, no benchmark expectation edited.

Gates

```

verify-retrieval-benchmark.sh 1 PASS 2 / FAIL 1 – B2 SC-015
7/22 (31.8%) vs a 90% bar
prove-retrieval-benchmark.sh 0 6 mutations caught
verify-search-latency.sh 0 SC-006 p95 592.3 ms vs 2000 ms
(median 403.5, n=40)
verify-check-registry-001.sh 0
verify-check-registry-002.sh 0 72 registered entry points
exist

```

Latency did not regress — it improved, to 592.3 ms p95 against a prior idle reading of 1817 ms, **despite 2.7× more vectors**. Whether that is WAL or host conditions: **not determined**.

workshop HEAD **1774fdd**, four files, `paths` spec commit, pushed.

Could not determine: the served-set instability mechanism (above); the retry loop was never exercised because WAL prevented the error it recovers from — **reasoned, not observed, with no unit test**; and first boot after the change took ~2m33s against ~70s, passing the boot probe's 2-minute window only 30 s later — one-time WAL conversion versus host load was **not measured**, though `journal_mode` is persistent so later boots should not pay it.

A91 — Three data defects fixed, each proved RED first. /api/ progress worked all along — the CLIENT's every write was answered 400 and dropped silently.

1. 40 of 312 MCQ items in ai_interviewing could not be answered.

`CorrectIdx int \json:"correctIndex,omitempty"` – the index is **zero-based**, so **"the correct answer is choice A"** **is** the Go zero value and encoding/json dropped the key. The client compares `i == q.correctIndex` against `undefined`, so **no choice could ever be right**. **Measured:** **312 items**, **distribution{0:40, 1:137, 2:88, 3:47}** – 40 affected across 28 files. **Both fixtures** `hardcodeCorrectIdx: 1`, which serialises fine, so the suite was **structurally blind**. RED first, asserting against the **raw body** (decoding a missing key into an int yields 0 – the very value that must stay distinguishable from absent): **"serialised WITHOUT a correctIndex key"** → FAIL. Green after removing `omitempty`. **The sweep matters as much as the fix:** `CorrectIdx` was the **only omitempty** in the whole `ai_interviewing` Go codebase on a type whose zero value is meaningful. And **workshop** got the same field right – `pkg/assessment/question.go:122` models it as `CorrectIndex *int`.

2. Ten authored flashcards were re-labelled short. The contract names three kinds, the Go enum implements three, `learning-kit.css` already ships `.lk-flashcard` — **the frontend type was the only layer that did not know**. End-to-end on real live data through the *real* normaliser:

server sends	<code>{mcq:11, short:19, flashcard:8}</code>	
normaliser at HEAD	<code>{mcq:11, short:27}</code>	<code><- all 8</code>
folded away		
normaliser fixed	<code>{mcq:11, short:19, flashcard:8}</code>	<code><- matches exactly</code>

(10 authored, 8 served: 2 are withheld by the citation gate; 44 authored → 38 served, 6 withheld — **the arithmetic closes exactly**.)

3. The progress wire mismatch — and the open question is ANSWERED. The contract decided it, not convenience: `contracts/http-api.md:1046 §3.11` specifies `{chapter_slug, pid, t_seconds}`, and the 002 delta extends what progress *covers* without restating the body. **The backend was right; the client was wrong.** `t_start_s` is a real field — a passage's transcript time span, used correctly in six other places; only this one write borrowed it.

Live evidence, before:

```
POST {..., t_start_s: 512.4} -> 400 "t_seconds is required..."
POST {..., t_seconds: 512.4} -> 200 {"stored":{...}}
GET /api/progress (same session) -> 200
GET /api/progress (session never written) -> 404
GET /api/progress (no X-Session) -> 400
```

/api/progress works, and always did. What never worked was the client's use of it: every browser write got a 400 and was dropped INVISIBLY, because a failed fire-and-forget write looks like nothing.

Test-count movement fully accounted for rather than waved at: 193 baseline + 9 = 202 (RED); 209 = 202 + 7 another agent added; 219 final matches `grep -c 'it('` over the tree exactly. **ng build first exited 1 on a TS1005 in another agent's mid-edit file** — proved not its own by building an isolated copy carrying all its changes with only that file restored to HEAD: **rc 0.**

OPEN, named rather than left to be found: - The flashcard fix is model-layer only. `practice.component.ts:158` renders `q.kind === 'mcq' ? 'multiple choice' : 'short answer'` — a binary ternary, so a flashcard now arrives correctly typed and is **still labelled “short answer” on screen.** One line, in another agent's file. - **ai_interviewing HEAD cb4c62a is NOT pushed** — left as an operator call. - `pkg/answer/outcome.go:459 CitationsVerified bool` carries the **same latent omitempty shape.** Reasoned from the code, **not empirically tested** — flagged, not claimed.

A88 — 162 reproducible per-material ZIP archives. Two builds, cmp: 162 identical / 0 different — AND one changed input byte moves exactly one archive.

The “integral ZIP archives” request is delivered. **This was the one “NOT BUILT” row that survived scrutiny today** — three others were withdrawn as false, and another agent deliberately left this one because it was correct.

Complemented `bundle.py` rather than extending it, and the reason is structural

`bundle.py` packs **the section**: one archive, 327 members, selected by `git ls-files docs/the-platform` plus the §11.4.65 export siblings.

A per-material archive cannot use that mechanism at all: 150 of the 162 materials are EXTERNAL LOCATORS whose content lives only in `ledger/captures/<id>/body` and appears nowhere in the document tree. `bundle.py` selects by path; this must select by ledger row. Different sources of truth, so a separate builder — but §11.4.74 still applies to what they share.

So the shared part was EXTRACTED, not copied.

`write_deterministic_zip` now has exactly one definition in the tree, and the extraction is **proved byte-neutral**: `bundle.py`’s own archive hashes 3197c89daab7a7af... **before and after**, `cmp` identical, and `prove-verify-bundle.sh` still exits 0. `capture.normalise` is imported for the same reason — re-deriving it would give the verifier its own idea of “the same bytes”.

What is pinned, because a ZIP is a hostile format for reproducibility

Byte-wise entry order on the UTF-8 archive path; `SOURCE_DATE_EPOCH` defaulting to **946684800** — `docs_chain`’s own `reproducibleEpoch`, read at `internal/adapter/derived.go` rather than guessed — **and recorded in the manifest**; mode 0644; `create_system` 3; **explicit** deflate level 9; no directory entries; every `ZipInfo` built fresh rather than `from_file`, so no `uid/gid/hi-res-mtime` extra field leaks; one JSON spelling per member.

Provenance is the newest commit touching that material’s own inputs, never HEAD — HEAD would make every archive stale on any unrelated commit.

bundle.py's 1980 constant was deliberately left alone: changing it would change the bytes of an archive that already has a published sidecar, for no gain.

The evidence, on all 162 archives

```
two builds -> diff -r: NO DIFFERENCES;  cmp: 162 identical, 0
different
digest of the 162 digests, build A: a47065ad61afb874...
                                build B: a47065ad61afb874...
(identical)
```

And the half that makes it mean something — mutate ONE input byte (a superseding materials.jsonl row whose title is one character longer):

```
cmp: 161 identical, 1 different (of 162)
DIFFER -> mat_cd31e14feaad.zip  differ: byte 15, line 1
6e0df398dc3840f9...  before
51fbe5360a9c6933...  after
```

Exactly one archive moved, and it was the right one. Without this, a “reproducible” check that always compares equal would assert nothing.

rc 2 demonstrated at full scale, and a REAL defect caught before landing

Altered capture body → **rc 2**, 161 archived of 162, the reason named (normalises to sha256 b5029dd0... but the ledger pinned 03d509c1...), and **no archive written for that material**. Unreadable body (chmod 000) → **rc 2**. Un-ignored --out-dir → **rc 2** before a byte is written.

M3 caught a genuine defect during development: chmod 000 originally produced a PermissionError traceback and **rc 1** — *a host problem reported as a defect in the section*, the same false-accusation class as A86. Fixed via read_or_unpackable, which **also closed a hash-then-archive window** by reading each body once and packing the exact bytes it hashed.

The set is derived, and the derivation is proved to bite

162 archives derived at run time from `ledger/materials.jsonl` (last row per id wins, `status == active`) — **not** from the builder's own `INDEX.json`, **not** from any list inside the gate. Proof **M7 appends a synthetic material to the sandbox ledger and requires the gate to NAME `mat_synthetic001`, with no edit to the gate.**

<code>verify-material-bundles.sh</code>	0	8 PASS, 0 FAIL, 0 UNDET, 0 NOTE
<code>prove-material-bundles.sh</code>	0	23 mutations, 23 caught, 0 missed
<code>verify-check-registry-001.sh</code>	0	46 PASS (was 44) – the R5 row is closed
<code>verify-check-registry-002.sh</code>	0	
<code>prove-verify-bundle.sh</code>	0	the extraction broke nothing
<code>audit-hardcoded-paths.sh</code>	0	16 allowed, none of them this work
<code>audit-environment-assumptions</code>	0	593 justified, none of them this work

workshop HEAD **1b2e922**, five files, `paths` commit, pushed.

Two honest limits, and one of its own readings withdrawn

- **Cross-zlib byte-identity is UNTESTED.** Every measurement is on one host with one zlib build; two hosts with different zlib builds can differ. **That is exactly why `content_digest` is published beside every archive hash** — a sha256 over the sorted `path\0sha256` member lines, which is implementation-independent. The limitation is designed around, not ignored.
- **A reading of its own was WITHDRAWN AS FALSE:** it first measured “1 material with zero captures (`mat_d09ba9dfa2cf`)”. Re-measured on the same unmodified file, that material has **3** capture rows — the script producing the first figure was wrong. **No material in the ledger lacks captures.**

OPEN, attributed, not fixed: `verify_bundle.py` exits **1** with B3 DRIFT, B8 PROVENANCE GAP (manifest 55 materials vs ledger 162) and B9 STALE — all three are staleness of the **SECTION** archive, built at 08:41 while `INGESTION.md`, `specification/*.md` and the ledger were edited at 16:44–16:58 by other work. The refactor is proved byte-neutral against that archive's own rebuild hash, so none is attributable to it.

Rebuilding it is owed once the tree settles. Also owed: DOWNLOADS.md does not yet describe the per-material archives, and updating it drags in its \$11.4.65 export chain.

**A87 — LIVE END-TO-END VERIFICATION of the running system.
Physical evidence, with controls — not a claim that it works.**

Measured against the live service on `http://127.0.0.1:8087` (the container runs `network=host`; there is no published port mapping — 8401 and 8432 are both wrong and were withdrawn earlier). Taken while five agents were running, so the timings are contended and are labelled as such.

Endpoints — every one answered

endpoint	status	time	bytes
/api/health	200	0.002 s	234
/api/search?q=...&limit=3	200	13.7 s	3,281
/api/suggest?q=work	200	0.050 s	4,006
/api/areas	200	0.036 s	463,444
/api/chapters	200	0.001 s	81

Search at 13.7 s is CONTENTION, not a regression — the idle baseline is 1.07 s and five agents plus a frontend rebuild were running. Recorded rather than quoted as a property. See [[measure-on-a-quiet-tree]].

The multi-provider wiring is live, and it is the deployed default UNCHANGED

```
provider      ollama
model         qwen2.5:3b-instruct-q4_K_M
locality      local
locality_verified True
endpoint      http://127.0.0.1:11434
calibrated    True      enabled True      generation 68
ollama.reachable True
```

That is exactly the container’s own argv. **The registry rewrite did not move the default**, which is what the multi-provider work claimed and this independently confirms.

A REAL answer job ran end to end — and correctly DECLINED

```
GET /api/ask?q=What%20is%20a%20knowledge%20area%3F&wait=1
-> HTTP 200, 2682 bytes
  status : declined
  reason  : margin_too_small
  citations: 0      answer chars: 0
  provider : ollama / qwen2.5:3b-instruct-q4_K_M / local /
verified
```

A decline is the system working, not failing. It refused to answer rather than fabricate, and named why. It also corroborates **SC-008 = 0/11 from a second direction**: retrieval is not surfacing passages good enough to ground an answer, so answering correctly declines. **The two findings are the same finding seen from opposite ends.**

DISCLOSURE SAFETY, verified with a CONTROL — the part that matters most

The corpus holds **13,141 rows, of which 162 carry redacted** — matching the redaction agent’s independently-derived figure of 162 suppressions exactly.

```
REDACTED rows          GET /api/passages/<pid> -> HTTP 410, 165
bytes
NON-REDACTED control  GET /api/passages/<pid> -> HTTP 200,
5,239 / 3,411 bytes
```

The control is what makes this evidence rather than a blind pass. Had every request returned 410, the probe would have proved nothing about discrimination. Different classes get different codes, so the mechanism is discriminating rather than uniformly refusing.

And the refusal itself leaks nothing, measured rather than assumed. The 410 body carries exactly three fields — status: redacted, the pid, and the message:

“This passage exists and its content has been withheld. The citation is not broken.”

Zero long words from that row’s own 204-character text appear anywhere in the refusal body. The wording is also the right design: it distinguishes **withheld** from **broken** — the precise distinction the

design work identified as critical, since a reader who cannot tell “we are withholding this” from “this failed” has been misled by the interface.

THE ONE CONFIRMED GAP: the browser is served a STALE BUNDLE

```
workshop/platform/web/  newest .js mtime  -> 13:52
features/inquiry/      committed at      -> 17:32
```

So the Ask/Search merge exists **in source and in tests (193 passing)** and is **not what a browser currently receives**. platform/web/ is untracked build output. **Closing this requires a frontend rebuild and a container restart**, and it is blocked until the in-flight design agent finishes editing that same tree — rebuilding mid-edit would ship a half-applied design.

Nothing above is a claim that the whole system is verified. It verifies the API surface, the answering path, the provider wiring and the redaction refusal. The UI a human actually sees is, at this moment, the older build.

A86 — ensure_fresh_binary had exactly TWO callers, not “a dozen” — my own framing was wrong — and one of them handed its false accusation straight to a human.

The blast radius, measured — and it corrects what I recorded in A83

I recorded that _common.sh “is shared by a dozen unrelated scripts”, and used that to justify a per-call-site fix. **The first half is true and the second half does not follow.** 14 files source _common.sh; **12 of them never call ensure_fresh_binary at all.** It has exactly **two** callers.

caller	build failure WAS	should be	verdict
workshop/scripts/redact.sh:124	rc 1, printed PROBLEM:	2	broken → fixed
workshop/scripts/ingest.sh:267	rc 1, printed PROBLEM:	2	broken → fixed, and it has no gate wrapper at all

THE SECOND ROW IS THE ONE THAT MATTERS. Nothing invokes `ingest.sh` as a subprocess — **its only caller is a human**, and the running server hands out that exact command as its own remedy (`internal/api/chapters.go:123,280,304`). So that 1 reached an operator **with nothing between it and them**, telling them their corpus was defective when the truth was that a compiler could not find a directory.

Two callers were already correct, and saying so is part of the answer: `verify-redaction-propagation.sh:380` builds the tool itself and maps failure to `UNDETERMINED` ... `rc 2`. And `workshop/scripts/build.sh:75,83` is the sibling with the same *shape* and the **opposite correct answer** — its question **is** the build, so a build failure genuinely **is** the finding, `rc 1` is right, and it deliberately does not route through the helper, so the fix cannot reach it.

Fixed at the ROOT, and the justification is a measurement

Two problem → `undetermined` in `workshop/scripts/_common.sh`. Why the root rather than the call sites: - **100% of the helper's consumers are in the “build is a precondition, corpus is the subject” class** — there is no consumer that wants a 1. - The helper **already returned 2** for the neighbouring “cannot establish freshness” case, so mapping a failed build to 1 was an **internal contradiction**. - The one script whose question *is* the build does not route through it, so its correct 1 is untouched. - A per-call-site fix would have been the same edit twice, leaving the shared default wrong for caller three.

The call-site net in `verify-registry-derived-leak.sh` was **kept armed**, because it drives `redact.sh` from a *copy* of `scripts/` that may predate the fix — and the phrase `failed to build from` is kept verbatim because `build_fault()` greps for it.

Proved in four directions, and H3 is the one that makes the rest attributable

H0	clean build, program exits 0 produced	-> rc 0, executable
H1	module does not compile UNDETERMINED not PROBLEM,	-> rc 2, says prints the compiler's own `syntax error`
H2	NON-VACUITY: healthy build, program exits 1	-> rc 1 SURVIVES – the
rc-2 branch		

findings

H3 REGRESSION: the same unbuildable input against a `\_common.sh` reverted to `problem` -> rc 1

U2R the call-site net proved with the ROOT FIX REVERTED inside the mirror –

`build\_fault()` alone still yields 2, so the two nets are independent

H3 is what makes H1 mean anything: without it, a 2 could have come from the harness rather than from the fix.

Two now-false comments were withdrawn BY NAME, one of them mine: the U2 block’s “*corrected HERE rather than in _common.sh, which a dozen unrelated scripts share*”, and the proof header’s “*ensure_fresh_binary maps a failed build to rc 1*”.

Verified

verify-registry-derived-leak.sh	0	pre-flight + A1..A7 all ok
prove-registry-derived-leak.sh	0	4 mutations, 7 exit-state cases, 2 controls
verify-redaction-propagation.sh	0	
prove-redaction-propagation.sh	0	7 source + 4 served-corpus mutations
verify-check-registry-002.sh	0	76 checks
bash -n on all 14 _common.sh consumers		all OK
workshop/scripts/status.sh (live)		0, RUNNING, health OK

verify-check-registry-001.sh exits 1 on one row and it is NOT this work: R5 UNREGISTERED – pipeline/the_platform/material_bundle.py — an untracked file belonging to the in-flight ZIP-archive agent, under a scanroot this agent never touched. **Attribution stated with its own limit:** the argument rests on the scanroot, not on a before/after pair, because the gate was not run before the change. Forwarded to that agent rather than fixed into someone else’s lane.

workshop HEAD **ea588d1**, three files, pathspec commit, pushed.

Could not determine: whether either caller’s false 1 ever misled a human in production — no log records a past rc from ingest.sh.

A85 — §11.4.65 reaches 924/924 = 100%, and the docs-chain drift I caused is repaired.

The 30 failures from A79 are fixed and the cause is confirmed as a source defect. pandoc's exception line number is a **YAML-BLOCK offset**, so YAML line 15 = file line 16 — a detail measured rather than assumed. The fix single-quotes the whole scalar, and the rendered meaning is proved unchanged **three ways**: `yaml.safe_load` returns the value **byte-identical** to the original literal 30/30; the diff is **+30 -30**, exactly one line per file; and pandoc's template emits no `diagrams` key, so **no stray quote reached any rendered artifact**. Both in-repo frontmatter consumers were checked first — neither reads that key.

A finding I did not expect: the frontmatter is not translated at all. All 14 translated trees' frontmatter blocks are **byte-identical to the English** — the pipeline translates the body and leaves the YAML verbatim. So there was no translated text to preserve, and each file was still read and matched individually rather than patched blind.

Coverage: 924 / 924 = 100.0%. 30/30 HTML, 30/30 PDF, `file(1)` 30/30 valid, 0 empty.

A FALSE NULL WAS CAUGHT AND WITHDRAWN, which is the best part of that report. A raw `grep` over the new PDFs returned 0 machine-path hits — but a **control needle proved the instrument could not see**: `grep -a -c 'HelixBuilder'` on a PDF that certainly contains it also returned 0, because WeasyPrint compresses the streams. **That null was withdrawn as evidence.** Re-run through `pdftotext` with the needle found 30/30, the real result is **0 matches across all 60 artifacts** for every machine-path and endpoint pattern. **A zero from an instrument that cannot see is not a zero.**

Two corrections to the baseline I gave that agent, both measured: `audit-hardcoded-paths.sh` was **already exit 1** before it started — not 0 — and the finding was **not its work**.

That red is now closed with a reasoned exemption. It was `workshop/docs/the-platform/ledger/captures/cap_c56a6cfef46e/body`: **302 KB of pi.dev's own "RPC Mode" documentation**, fetched verbatim by the crawler. Its three hits are HTML-escaped JSON inside **their** docs showing **their** example slash-commands. Editing it would do two kinds of damage, and the second is mechanical: it falsifies evidence a citation exists to replay, **and it breaks an existing gate** — `verify_crawl.py:40`

declares “*X10 REPLAY: every stored body re-hashes to the capture that names it*”, so rewriting a byte makes that hash mismatch. **The honest fix is forbidden by another instrument.**

A first attempt at that rule used a glob and silently matched nothing; the gate stayed correctly red. `allow_kind()` matches by **exact string** (`awk $2 == want`). Each capture therefore needs its own row — **a better property than a glob**, because a new capture cannot inherit the pardon silently. Re-measured: **exit 0, 16 file(s) explicitly allowed.**

The docs-chain C4 drift was MINE. `verify-docs-chain.sh` reported 1 FAIL — stale exports for the four carriers, CONTINUATION, and four specs/** documents. **I edited every one of those and did not regenerate their exports.** The gate’s own words name the offence exactly: “*A divergent export is the exact PASS-bluff §11.4.65 forbids: an operator reads the HTML or PDF while the .md is right.*” All nine regenerated with the pipeline’s own command shape; **re-measured 0 FAIL, 0 UNDET over 5 checks.**

A83 — The derived-leak gate’s failure WAS an instrument fault, it is fixed, and kg_meeting_note REPRODUCES. Underneath sits a systemic defect: `ensure_fresh_binary` maps a BUILD FAILURE to `rc 1` for every caller in this tree.

The fault, reproduced verbatim

`verify-registry-derived-leak.sh` exited 1 with 4 `assertion(s)` FAILED. A1/A2/A4/A5 passed; A3/A6/A7 “failed” **without ever running**:

```
pkg/knowledge/graph.go:42:2: github.com/vasic-digital/
passage@v0.0.0:
    replacement directory ../../../../submodules/passage does not
    exist
PROBLEM: workshop-redact failed to build from ../corpus/platform/
backend
FAILED  A3 still carrying:
text,machine_text,source_ref.path,content_hash ...
FAILED  A6 the scan still exits 1 with 1 finding(s)
FAILED  A7 a second run exited 1 and/or rewrote the registry
```

An instrument fault printed as an accusation that redaction is broken.

The fix, and why the mirror works

platform/backend/go.mod carries five relative replace directives at ../../../../submodules/<name>. The fixture root is \$WORK/corpus, so \$ROOT/platform/backend/../../../../ resolves to \$WORK — **a symlink at \$WORK/submodules makes them resolve**. Read-only; nothing copied, nothing written. Reuses the S-M1 approach already proven in prove-redaction-propagation.sh rather than inventing a second mechanism. A new --submodules-dir DIR lever names where the mirror comes from.

The three states are now genuinely separated

- **U1 — pre-flight**. Builds ./cmd/workshop-redact from the fixture root, along **exactly the path redact.sh builds along, before the first assertion**. Failure → **rc 2 with the compiler's own words**, and no assertion is accused.
- **U2 — a second net** at every redact.sh / workshop-redact call site: a non-zero exit carrying a build/toolchain signature is could-not-determine.
- **rc 1 is now reachable only after** the tool built, the scan ran and the suppression was applied — and the summary line says so.

Observed directly with an empty --submodules-dir:

```
UNDETERMINED: U1 workshop-redact does NOT build from the fixture
root
UNDETERMINED: the check COULD NOT RUN. This is NOT a finding about
the corpus,
                and it must never be recorded as one.
EXIT=2
```

The proof covers the trap that matters. UC (control) requires the mirrored gate to exit 0 **and** requires the pre-flight plus A1..A7 each to be named — because **rc 0 alone cannot distinguish “ran and passed” from “skipped”**. U1 requires 2 — not 1, not 0 — naming U1 and replacement directory, **with no FAILED A line**. U2 neuters the pre-flight in a throwaway copy so the fault reaches redact.sh. And UN is the non-vacuity half: break the mechanism in a way that **still builds** (leave content_hash as the fingerprint of the suppressed string) and require 1, named at A3 — **proving the rc-2 branch does not swallow findings**.

kg_meeting_note REPRODUCES — the question is answered

The gate is fixture-based and never reads curriculum/, so it cannot answer this; it was measured separately, read-only:

roster: 33 withheld-only string(s) ... finding: 0 minted row(s)
[token pass]
phrase roster: 16246 withheld-only run(s) of 2..6 words ... finding:
1 [phrase pass]
01M1H9DKFTVEJMW5YFZ1GC7PYN [kg_meeting_note] carries a withheld-
only run of
3 word(s) / 14 character(s) in text

That matches docs/work-register.md §6.3 exactly (“1 × kg_meeting_note, 3-word / 14-char run”), and the four rows decided there (3 × kg_term, 1 × kg_area) are **gone** — **those decisions landed**. §6.3 records this row as **LEFT UNDECIDED and permanently visible**, pending an operator content judgement. **Nothing was suppressed, allow-listed or re-baselined, and no content was printed.**

THE SYSTEMIC DEFECT UNDERNEATH, found and deliberately NOT fixed

ensure_fresh_binary in workshop/scripts/_common.sh maps a FAILED BUILD to rc 1 — for every caller in this tree. That is the root of this whole class: a broken toolchain becomes an accusation against the corpus, everywhere, not just here. It was corrected **at this gate’s call sites only**, because _common.sh is shared by a dozen unrelated scripts and changing it blind would be a fleet-wide behavioural change made in passing. **Whether other gates carry the same false-accusation shape was NOT measured.**

Verified

verify-registry-derived-leak.sh	0	"every assertion holds"
(was 1)		
prove-registry-derived-leak.sh	0	4 source mutations caught +
3 exit-state		
		cases + the UC control
verify-redaction-propagation.sh	0	
verify-check-registry-001.sh	0	rc-2 state demonstrated
verify-check-registry-002.sh	0	all 74 entry points exist

workshop HEAD **0fb1242**, pushed; two files, +396 / -11.

A concurrency near-miss worth recording as good practice. Its first git add of exactly its own two paths still produced a **five-file commit**, because another agent had **pre-staged frontend deletions and renames in the index**. It caught this, ran git reset --soft HEAD~1 (leaving the index untouched), and re-committed with an explicit pathspec — git commit -F msg -- <its two paths>. The other agent’s

staged work was restored, unpushed, and nothing was force-pushed.
git add <path> is not sufficient isolation in a shared checkout; the commit needs the pathspec too.

Honest boundary: the gate's 0 is **not** a statement about the real corpus — it exercises the mechanism over an invented fixture-corpus. The real-corpus reading above was a separate read-only scan.

A82 — “OpenDesign” RESOLVED: it is already the system in use. And the UI is not undisciplined — its PALETTE is 6° from unusable.

The operator's term, answered

“OpenDesign” is `github.com/nexu-io/open-design`, mandated by Helix Constitution §11.4.162 — and the `--od-*` prefix throughout this tree IS its token namespace. It was never missing. Front door design-system/README.md; brand source design-system/brand-milosvasic/milosvasic.css; a DTCG source-of-truth pipeline at workshop/docs/the-platform/design/ whose zero-dependency tools/build.mjs contrast-gates 36 rules and proves 84 `--od-*` declarations against the live CSS. **The work was built ON `--od-*`, not beside it.**

PenPot is present and has NEVER been started. docs/the-platform/design/penpot/ holds a validated compose stack (podman-compose config exits 0) and an operator runbook; its own README records that no instance has run. It was **not** used. The existing wireframes/*.svg are hand-authored greyscale.

Two honest absences rather than silent substitutions. The frontend-design skill I named in the brief **does not exist in this environment** — the agent searched, said so, and used artifact-design instead. That was my error in the brief, not its omission. The figma MCP could not be reached. **No Artifact was published**, correctly: workshop is private.

The diagnosis, and it inverts the complaint

Colour DISCIPLINE is near-perfect. The PALETTE is the defect. Exactly **one** raw colour literal exists in the entire product layer (#000, on the video element). So “monotone” is not sloppiness — it is a palette that cannot express what the product means.

The 25 colour-carrying tokens resolve to 4 nominal hues that collapse into 3 perceptual zones:

- **accent 354° and danger 0° are 6° apart and 0.02% apart in relative luminance (0.11274 vs 0.11252). In greyscale they are the same colour.**
- warning 32° sits 1° from --od-text-muted 33°, separated by saturation alone.
- success 123° is the only independent hue.

Against roughly **30 meanings** the UI must express.

AND THE 6° COLLISION IS LOAD-BEARING. `state.component.ts` declares that declined “*wears the product’s own accent, never the danger red*” — **a doctrine the palette physically cannot deliver.** The code states an intent the tokens cannot express.

More numbers, all measured: 7 corpus kinds with **zero** per-kind styling; of 7 type steps, **2 carry 100 of 129 reads (78%)** and `workshop.css` never reads --od-fs-base; **159 of 379 spacing values (42%) are off-scale**; 4 box-shadows in total, a borders-to-shadows ratio of **26:1**.

THREE MEASURED CONTRAST FAILURES IN LIVE TOKENS — not proposals, current state: --od-danger as a dark border **2.94:1** (needs 3.0); --od-warning as a rule on surface-2 **2.68:1**; --od-success as dark text **3.70:1** (needs 4.5). A 12% dark wash scores **1.07:1** — the dark theme’s state grounds are near-invisible.

The answer: three PROVENANCE families, not seven kinds

Grounded in the six vocabularies that all ask “*where did this come from*” — HEARD & SEEN **208°**, WRITTEN **252°**, BUILT **300°**, all inside 163–314°, **the only arc free of every brand hue**, at $\geq 54^\circ$ separation. Custody is non-chromatic cool graphite 218°.

All 21 published ratios were re-verified against the hex actually in the file — every kind text $\geq 4.5:1$ on background, own wash and surface-2 in **both** themes; every edge $\geq 3.0:1$; and **all three live failures repaired by added siblings (3.32, 3.02, 5.00). No brand token was edited.**

withheld had no state to live in — and that is a real defect, not a styling gap

withheld is not one of the six SurfaceState members, while `vocabulary.ts` carries a five-member `WITHHELD` table consumed by four components. **So it borrows a state, and all six lie about it.**

It was added and separated from *error* on **five channels, three of them non-chromatic**: hue 142° apart; the only zero-chroma state; a striped edge nothing else has; a hatched-block glyph; and `role="status"` (polite) versus `role="alert"` (assertive). Separated from *empty* by one rule: **a filled ground means content exists behind it**. And declined **loses its wash but keeps its accent edge** — the only way to make the existing 6° doctrine true.

Files

`workshop/docs/design-system/:wk-semantic-tokens.css` (540 lines — **the handoff artefact**), `TOKENS.md` (476), `COMPONENTS.md` (512), `RATIONALE.md` (346). **Nothing under `src/` was touched.**

Deliberately not done, each with a reason: brand untouched (generated and drift-checked); accent not replaced; no new hue for declined; `--wk-hatch` not reused (already spent on *uncertainty*); **no spacing half-steps — that is a brand change, recorded as debt and an operator decision**; learning kit unedited; **no motion added**; no corpus content judged or quoted; no component written.

COULD NOT DETERMINE: nothing was rendered. No build, no browser, per the brief. Every figure is static-source measurement plus computed sRGB contrast — **the palette is verified arithmetically and has not been SEEN**. The three family colours are 1.03–1.19:1 apart in luminance, so they are **certainly not** greyscale-distinguishable; stated as a limit, with every chip carrying a text label.

It caught a concurrent restructure mid-task: at 17:05 another agent deleted `features/search/` and `features/ask/` and merged them into `features/inquiry/inquiry.component.ts`. `COMPONENTS §6/§7` carry a dated note and **must be re-checked against the merged file before applying**. Its observation is worth keeping: the merge makes the state work **more** important — one column must now show hits, answers, declines, withheld rows and empties.

Three live defects found, NOT fixed (all would be src/ edits): --od-lh-relaxed is read 6× in platform.css and **declared nowhere**; var(--lk-radius, 10px) where the token is 8px; var(--od-radius-pill, 999px) where it is 9999px.

A81 — The deep-crawl capability ALREADY EXISTED and was registered. That is the SECOND false “NOT BUILT” row today — and stale rows are now demonstrably sending agents to rebuild working tools.

The recorded gap was false, and it was already false when it was written. The open register said *“only three named targets were crawled BY HAND and no general capability exists, so the next arriving link cannot be processed.”*

Measured: pipeline/the_platform/crawl.py and codebase.py **landed 2026-09-03, are tracked, are registered** as the-platform-crawl with a **21-mutation battery**, and had **already been run against all three named targets** — 17 crawls, 176 requests. **Two specification documents went on asserting NOT BUILT for a day.** Per §11.4.74 the agent declined to reimplement it.

THE PATTERN, stated because it has now happened twice within hours. A76 recorded a gate that was green about /tmp; A80 found a port that was already half done and undocumented; this is the third instance and the second outright false “not built”. **A stale status row is not a harmless inaccuracy — it dispatches real effort at work that is already finished, and it very nearly caused a working, mutation-proved tool to be rebuilt from scratch.** Re-derive a status row before acting on it. See [[no-bluffing-evidence-before-claims]].

Four stale NOT BUILT rows withdrawn by name in 09-open-questions.md Q9 and 01-scope.md §1.5. **Two were deliberately LEFT because they are correct:** per-material ZIP archives genuinely do not exist (bundle.py packs the *section*, not the material), and a scheduled re-crawl is not a scheduled re-capture. **Knowing which rows to leave alone is the harder half.**

What was genuinely missing, and it is a real gap

verify_crawl.py validates **one** crawl against itself. Its strongest rule X10 replays stored bodies against their captures — **that is INTEGRITY, not REPRODUCIBILITY. X10 would pass unchanged on a crawler whose frontier wandered.** So the contract’s “*same target → same artefact, or record why it differs*” was **ungated**.

Added pipeline/the_platform/verify_reproducibility.py (registered the-platform-reproducibility) with its proof. Every rule came from measurement taken **before the gate existed**: P1 URL sets identical across complete crawls sharing seed+settings (4 groups, 14 crawls, 100%); P2 45 superseding captures over 22 materials with **0 unlinked**; P3 budget-exhausted crawls excluded **and the exclusion printed**; P4 **nothing comparable → rc 2, not a green tick**.

The proof bites in BOTH directions — the part that makes it worth having: CONTROL requires rc 0 on a clean sandbox; **M12 requires rc 0 after a SETTINGS change**, so the gate cannot buy P1/P2 by flagging every group; and A1/A2 execute cripples hardwired to exit 1 and exit 0 and confirm each is caught.

Running it on the three targets — including where it LOSES

target	tool	hand	
deepseek-harness	116 URLs / 1 host	3 / 1	tool ahead
pi.dev	135 URLs / 2 hosts	4 / 3	tool ahead
eview-software.com	14 URLs / 3 hosts	32 / 22	tool loses badly

Nineteen hosts the hand crawl used were never reached — company registries, review directories, RDAP. **None is linked from the seed; they were found by SEARCHING, and a link-following crawler cannot reproduce a search.** So the tool wins on **admissibility, not reach**: every item carries URL, timestamp, digest and status including failures, whereas the hand citations cannot be replayed at all. **That is the honest comparison, and it names the tool’s own weakness.**

Cross-day reproducibility, determined rather than guessed: eview reproduced **exactly** (~18 h apart, zero captures changed). pi.dev did **not** — and the cause was established: of 60 captures, **45 were URLs never fetched before** (the budget truncated the frontier differently), and only **15** were genuine upstream changes. Three-valued contract exercised live: unreachable → **rc 2**; credentialed URL → rc 1; browser-impersonating UA → rc 1.

Verified

verify_reproducibility.py	0	3 PASS / 0 FAIL / 0 UNDET / 1 NOTE
prove-verify-reproducibility.sh	0	16 mutations, 16 caught, 0 missed
verify_crawl.py	0	10 PASS / 0 FAIL / 0 UNDET / 3 NOTE
verify-check-registry-001.sh	0	
verify-check-registry-002.sh	0	74 registered entry points exist
audit-hardcoded-paths.sh	0	15 file(s) allowed

Network WAS available and live crawls WERE performed — curl to example.com and github.com both 200, DNS resolved all three targets. No fixture-only run, no fabricated results.

workshop HEAD **5a2fab5** (== origin/main).

OWED, and stated rather than quietly skipped: the \$11.4.65 .html/.pdf exports of the four edited documents were **NOT regenerated** — the host was mid bulk pandoc/weasyprint run and the agent was told not to add load.

A80 — The ai_interviewing port is ALREADY HALF DONE and undocumented, “area” means three different things, and a test is graded ON THE CLIENT against a key shipped to the browser. Three of my premises were wrong.

Plan written to workshop/docs/ai-interviewing-port/PLAN.md (899 lines). One new file; nothing else modified in either repository; nothing committed.

Three premises of my brief that MEASUREMENT CONTRADICTS

1. **The port is already roughly half done, and nobody wrote it down.** `features/practice/practice.component.ts:16` says, in the source, “*PORTED FROM ai_interviewing’s practice component.*” The learning-kit design system is present **in full** (1,174 lines). And `internal/api/progress.go:17-19` records that workshop’s X-Session identity was aligned to `ai_interviewing’s` **on purpose**. I briefed this as a port to be started; it is a port to be **finished and documented**.

2. **Workshop’s frontend already carries practice, progress, plans and search — 16,627 lines against ai_interviewing’s 4,842.** The destination is larger than the source.

3. **“AREA” IS A THREE-WAY AMBIGUITY, and this one nearly wrecked the fit table.** `ai_interviewing` has 34 areas. Workshop has **37 CURRICULUM areas** (`curriculum/chapter-01/knowledge/areas.json`, across 7 tracks) **and 5 PLATFORM areas**. The mapping was made 34 → 37. **Mapping onto the 5 would have produced NO FIT for entirely the wrong reason** — a plausible-looking table that was measuring the wrong noun.

A — what a LESSON and a TEST actually are, in data terms

A **lesson** is a lesson row (`store.go:123-126`):

`id`, `module_id`, `ord`, `slug`, `title`, `body_html`, `body_md` — titled prose split at `##` headings.

A **test** is a question row (`store.go:127-132`): one table, three kinds, and **checked three different ways, NONE of them server-side**:

kind	how it is checked
mcq	<code>i === q.correctIndex</code> on the client , against a key shipped to the browser
short	a keyword-overlap <i>hint</i> the UI itself calls “not a grade”, then learner self-grade
flashcard	purely self-graded

The Go backend contains no grading function and accepts client-supplied status/grade with zero validation. Progress is (session_id, item_type, item_id, status, grade, streak, due_at), and identity is an unauthenticated Math.random() string in localStorage — **no user, no account, no user table.**

That is a property to decide about before extending, not after. It is defensible for self-study and indefensible for anything assessed. Recorded here so the choice is made deliberately.

B and C — the tables

Gap table (23 rows): PORT 4 · EXTEND EXISTING 5 · ALREADY COVERED 9 · DO NOT PORT 5.

Fit table: FITS 11 · PARTIAL 11 · NO FIT 12. The 12 NO-FITs have exactly three reasons, and **saying so was the right answer rather than forcing a match:** seven are conventional full-stack subjects for a different job (Node, PostgreSQL, event sourcing, payments, API architecture, React, TypeScript); three are genuinely absent from the corpus (code migration, fine-tuning, i18n/RTL); two are the **same word for a different discipline** — behavioural/leadership there is interview coaching, and “media pipelines” there means display output, not ASR.

THE REVERSE GAP, which I did not ask for and which matters more: 12 workshop areas have no source material at all, including the entire business-and-delivery track. The port cannot fill those; only authoring can.

D — data-model verdict: AUTHORED models are the same concept deliberately; SERVED models are not

Workshop’s areas.json carries ai_interviewing’s Curriculum shape field-for-field — **including defaultLang, an i18n field workshop has no i18n for** — plus nine identically-named camelCase module fields. Served-side, AreaRecord is a ULID with **no summary, ordinal, code or track,** and **“progress” is one word for two incompatible things on the same route name.**

Build order, first item: wire the mastery store, which ALREADY EXISTS. pkg/assessment/progress.go:43-57 declares the record and **nothing calls it.**

FOUR DEFECTS FOUND IN PASSING, each actionable

1. **10 authored flashcards are silently mislabelled today.**
knowledge.ts:490 coerces kind: 'flashcard' to 'short'. The CSS, the Go enum and the question bank all support flashcards; **only the frontend type does not.**
2. **40 of 312 ai_interviewing MCQ items cannot be answered correctly.** omitempty on an int drops correctIndex when the correct answer is choice A (index 0). **Both test fixtures hardcode index 1, so the suite cannot see it** — the fixtures mask the bug. A textbook case of a test population that is an unchecked assertion.
3. **A wire mismatch on workshop's own progress route:** the decoder requires t_seconds (progress.go:220); the client posts t_start_s (api.ts:573).
4. **A concurrent agent is mid-restructure of files this plan cites** — search.component.ts deleted and renamed to inquiry/inquiry.component.ts, ask.component.ts deleted. Recorded in the plan as R1 with a re-measure instruction rather than left to rot.

Could not determine — 11 items; the five that matter

Whether the 40 zero-index MCQs ever failed at runtime (source semantics only, nothing executed); whether /api/progress works today (**three sources disagree**); why assessment.Progress was declared and never wired, which materially changes the first build step's estimate; how many of the ~510 portable questions can actually be anchored to a workshop passage; and why kg_lesson_section is minted nowhere — **re-derived as 0 across all 13,141 rows on today's file** rather than quoting the stale 11,622-row census in coverage.go.

A79 — G8 CLOSED to 96.8%: 1,642 export files land. 30 documents FAILED and are named, because the cause is a source defect this export refused to paper over.

The operator chose “export everything” with the cost stated — ~1,702 files into a PUBLIC repository, a larger content-boundary scan surface, and irreversibility in public history. Delivered.

tracked markdown at the umbrella root	924
already had both siblings	73
in scope	851
produced	821 .html + 821 .pdf = 1,642 files

bytes added	70,997,765 (67.7 MiB) — 13.4 HTML, 57.6 PDF, mean PDF 69 KiB
coverage	894 / 924 = 96.8%, from 7.9%

The generator was MATCHED, not guessed, and the proof is a byte comparison. Docs Chain's own builtins were read out of submodules/constitution/submodules/docs_chain/internal/adapters/derived.go (pandoc --standalone --from=markdown --to=html --metadata title=<basename>; weasyprint --base-url <live pdf path>; SOURCE_DATE_EPOCH=946684800), then three already-exported documents were regenerated and came back **cmp-identical to docs_chain's own output**. docs_chain itself was NOT driven: its context generator classifies only root / docs / / specs /, so widening it would break cascade check C3's re-derivation diff, and a full sync of 924 documents extrapolates to ~5.7 h.

Verification was exhaustive, not sampled: 894 PDFs checked with file(1) — 894 valid, 0 invalid, 0 empty; 894 HTML non-empty; zero orphans in either direction. The generator now deletes the HTML if the PDF fails, **so a half-export cannot masquerade as done** — which is the failure mode that would otherwise inflate a coverage number.

The 30 failures are REAL, NAMED, and not worked around

15 language trees × 2 documents: <tree>/products/HelixBuilder.md and <tree>/products/Vasic-Digital-Reusable-Module-Suite.md, for _content and _content_{ar,be,de,es,fa,fr,hi,ja,kk,ko,ru,sr,tr,zh}.

The real error: Error parsing YAML metadata ... did not find expected '-' indicator. **The cause is a SOURCE-CONTENT defect, not a pipeline defect.** HelixBuilder.md line 16 reads
- "Pick your pipeline" grid of category tiles (...) — YAML takes the leading " as the start of a quoted scalar and then hits invalid trailing text. **docs_chain would fail identically**, so this is a latent defect the export merely surfaced.

The fix belongs in the frontmatter and was deliberately NOT made here: _content/** is CONTENT, not export, and an export run has no business editing the documents it is exporting.

Two audits measured AFTER staging, which is the check the agent could not make

Both read git ls-files, i.e. the INDEX — so an untracked-tree green says nothing about a committed one. Re-run with all 1,642 files staged:

- `audit-hardcoded-paths.sh` → 0. The prediction held. `.pdf` is in `BIN_EXT` and never read; `.html` is in `TEXT_EXT`, so 181 exports entered the audit — **zero machine-path hits, with a control needle confirming the grep could see the pattern**, so that zero is evidence rather than a blind instrument.
- **29 exports DO contain machine paths** — all under `_analysis/`, `_tests/evidence/`, `.ashlrcode/`, every one matched by the audit's `SKIP` regex, so the gate stays green **by construction rather than by luck**. No new disclosure (those paths are already in the public `.md` sources) but they are now duplicated into 29 further files. Recorded for awareness.
- `audit-environment-assumptions.sh` → 1, and **none of it was the exports**. Five findings, all `.go`, from the two agents that finished alongside.

The five environment findings: judged individually, all four rules EXEMPTIONS

And one of them is the exact opposite verdict to an earlier one today, which is the point. `pkg/search/carryforward_test.go` had its model ids **FIXED** this morning because they were arbitrary fixture values that happened to be real shipped model names. `pkg/answer/registry_test.go` keeps its literals, because they are **not arbitrary** — **they ARE the deployed configuration under test**. Same class of file, opposite verdict; the difference is what the value MEANS.

finding	verdict	why
registry.go:52 "digital.vasic.llmprovider/pkg/ providers/codestral"	EXEMPT	a Go import path ; resolved by the compiler, cannot be env-derived
registry.go:239 hosted("codestral", ...)	EXEMPT	a catalogue row . You cannot env-derive the membership of a catalogue of providers without emptying it. Runtime SELECTION is

finding	verdict	why
registry_test.go:24-25 prodEndpoint / prodModel	EXEMPT	separately overridable (flag > env > literal) and the gate derives coverage from answer.Registrations() they mirror the container's argv on purpose — the file says so: <i>“a test that drifts from the deployed configuration drifts visibly.”</i> Env-deriving them makes the test assert that whatever is configured equals whatever is configured — a tautology destroying the only evidence the Ollama default is unchanged fixture payload bytes , the exact stdout a stub emits, in the CONTROL proving the log-reader fix costs no content. Nothing dials it
logs_silent_failure_test.go:162 const payload	EXEMPT	

The gate caught me writing a bad rule, and it was right. Two rules were put under one # REASON: block; it returned rc 2 — **FATAL: malformed allow-list — every rule needs a ‘# REASON:’ ... directly above it, naming the line. A rule inheriting a neighbour’s justification is exactly how an unjustified exemption gets in.** Each now carries its own reason. Re-measured: **exit 0, 593 justified occurrences allow-listed**, and **--strict-allow-list exit 0** — no rule added is stale.

Debris: it WAS ours, and the generator is now hardened against it

The eight hidden .dc* files (4 HTML staging temps + 4 .err) were produced by this pipeline and survived only because the earlier run was **killed mid-flight by the auth expiry**; normal completion removes them. All eight deleted. The generator now captures stderr into a variable — **no .err file is ever created** — and cleans temps via an EXIT/INT/TERM/HUP trap, so a kill cannot leave debris again.

Two outlier PDFs, both in the vendored .specify/extensions/superspec/ tree: README.pdf 8.99 MiB and SKILL.pdf 7.60 MiB. Note docs-chain deliberately EXCLUDES .specify/ as a vendored third-party tree; **51 exports exist there only because the decision was “export everything”** — worth revisiting if repository size becomes the binding concern.

A78 — submodules/containers: the log reader lied, and it lied for TWO compounding reasons. Both fixed and live-verified. Gitlink d940b51 → 6d13ad0.

DEFECT 1 REPRODUCED, and the recorded description was accurate but incomplete. Two independent faults compounded, which is why a single-cause fix would have left it broken:

- 1. defaultLogOptions() supplies Tail:"all". **"all" is DOCKER's sentinel; podman's --tail is an INTEGER flag.** So *every default-option call* to PodmanRuntime.Logs failed outright: invalid argument "all" for "--tail" flag ... rc=125, 0 bytes stdout.
- 2. ExecuteStream returned the raw stdout pipe and called cmd.Wait only from Close. So io.ReadAll returned **(0 bytes, nil error)** — the failure had nowhere to surface.

Measured through the module's own API, same container, podman logs ground truth = 26 bytes:

	bytes	read error
before	0	<nil>
after	26	<nil>
before, nonexistent container	0	<nil>
	0	exit status 125: ... no container

	bytes	read error
after, nonexistent container		with name or ID ... found

The bottom two rows are the real repair: **a caller can now tell “the container printed nothing” from “I could not read”**. Read reaps the child at stdout EOF and returns its failure instead of `io.EOF`; stderr is drained into a bounded 8 KiB buffer and appended to the error; Wait is memoised; podman’s `--tail` is guarded the way `crio.go` and `lxd.go` already guarded it.

DEFECT 2 CONFIRMED and closed. There was no Run, no Create, and no reachable stdin anywhere. Added:

```
Run(ctx, image string, cmd []string, opts ...RunOption)
(*ExecResult, error)
```

Shaped for the actual consumer: `--rm` on by default; `WithRunStdin` supplies the document **and** is what puts `-i` on the command line; results reuse the existing `ExecResult` rather than a near-duplicate type; a non-zero **container** exit is data in `ExitCode` with a nil error, matching Exec. Verified against real podman: 34 bytes in, 34 back byte-identical, exit 42 propagated, `--rm` left nothing behind.

STDIN IS NEVER SILENTLY DROPPED, and that design choice is the lesson. `ExecuteWithStdin` is an *optional* interface (`StdinExecutor`) rather than a fourth method on the exported `CommandExecutor`, so no external implementer breaks; an executor lacking it makes Run fail with `ErrStdinUnsupported` **without running the command at all**. Degrading to “run it without the document and exit 0” would have been **defect 1 all over again**.

A refusal worth recording as good practice: `kubectrl` was NOT implemented. `kubectrl run --rm -i --restart=Never` is real, but needs a pod name Run does not take and mixes pod-lifecycle output into stdout — and **no cluster is reachable here, so it was unmeasurable**. It ships `ErrRunUnsupported` with the specific reason instead of an unverified implementation.

Mutation-proved by reverting each fix and confirming red, then restoring and verifying all three files byte-identical: SILENT FAILURE: command failed but `io.ReadAll` returned 0 byte(s) and a nil error · logs `--tail all cid` · SILENTLY DROPPED STDIN. Controls stayed green

under every mutation, **and an opposite-direction mutant — a reader that errors even on success — is rejected**, so the contract cannot be satisfied by always erroring.

Verification: go build ./... 0 · go vet ./... 0 · go test ./...
-count=1 **0, 44 packages ok** · 21 new tests, **37 top-level PASS / 0 FAIL**.
bluff-scanner.sh 1 with 17 hits, **all pre-existing and none in a touched file**.

One caveat, measured rather than waved away: a first suite run showed pkg/health/TestCheckWithRetry_BackoffFactor FAIL. pkg/health does not depend on pkg/runtime at all; it passed 5/5 in isolation and green on re-run. **Load flake from the other agents, not a regression** — see [[measure-on-a-quiet-tree]].

CONVERTIBLE CONSUMERS — named, deliberately NOT converted:
- _tools/helixtranslate-container/run.sh and _tools/helixtranslate-local.sh are convertible **now** (local one-shot run --rm -i with stdin; --env-file fits WithRunExtraArgs). - _tools/helixtranslate-container.sh is **STILL BLOCKED**: it is an SSH shim streaming stdin to a remote host, and RemoteRuntime.Run explicitly refuses stdin because RemoteExecutor has no io.Reader seam. Closing it needs a further change in pkg/remote.
- pkg/brokertest (4 sites) is **not** convertible — those use run -d (detached) and Run waits for exit. A different primitive, not added.

The GitLab mirror moved too, and that is new information. This submodule's origin fans out to **four push URLs**, so the commit landed on **both GitHub and the GitLab mirror**; both now read 6d13ad0. That is a different situation from design-toolkit, whose mirror this carrier records as 6 commits behind with its push recipe deliberately .disabled.

Gitlink and manifest moved together in ONE change — verify-manifest-pins.sh PASS at 12 recorded refs equalling the gitlinks this repository will commit.

Could not determine: why workshop-curriculum_platform_1 restarted mid-session. RestartCount=0 with a fresh StartedAt means an external stop/start. The agent ran only ps, logs, inspect, images and run --rm on a separate ephemeral container.

A77 — All four carriers updated in lockstep: the content-boundary “mechanism UNDETERMINED” verdict is WITHDRAWN, and the gate now instruments its own stability.

Why the carriers and not just this document: a carrier is the FIRST thing every agent reads. They were telling nine concurrently-running agents that the boundary gate’s total “*is NOT reproducible run-to-run*” with a mechanism that “*is UNDETERMINED and was not established*”. **That was true when written and is now superseded — and a stale finding in a carrier misinforms every agent at once.**

Edited as ONE artifact, not four files by hand. The shared region was split off, edited once, and the other three recomposed from it — the procedure the carrier itself prescribes, because C5 ROOT-LOCKSTEP measures where the four actually converge. Verified before and after: **4 distinct digests at line 18, 1 at line 19.** `verify-governance-cascade.sh` → **0, 12 PASS / 0 FAIL / 0 ENV / 8 NOTE.**

Three passages carried the superseded claim, not one — the quoted gate output, the “four verdicts that moved” table row, and the instruments bullet. Missing any of the three would have left the contradiction live.

The old three-run table is KEPT and relabelled HISTORICAL rather than deleted, so a stale reading stays recognisable. Its own conclusion — that nothing changed during the window, resting on `find -newermt '3 hours ago'` returning 0 files — is exactly the measurement that failed: wrong window, wrong set.

This entry exists because a gate demanded it, and the gate was right. `continuation-check.sh` returned 1 with C3: “*commit(s) since Synced-Commit changed a watched governance file WITHOUT updating CONTINUATION.md in the same commit (\$12.10 protection 2)*”, naming eb9810d and all four carriers. The commit was still local and unpushed, so it was **amended to carry this entry** rather than fixed forward — C3 requires the handoff update in the SAME commit, and a fix-forward would have left the violation permanently in history.

A76 — INCIDENT CANDIDATE: 125 served rows reproduce spans that also occur in SUPPRESSED passages. And the gate that was supposed to catch this was measuring /tmp.

Status: MEASURED, NOT CONFIRMED AS DISCLOSURE. Nothing was redacted, deleted or allow-listed. Deciding it requires reading private text, which is operator work.

Reported by location and count only — no content, and deliberately not even a digest of a name.

1. The measurement

125 served, non-suppressed rows in the LIVE database reproduce at least one 8-word verbatim span that also occurs in a suppressed passage. 188 distinct such spans, of which 142 appear in ≤ 2 served rows — i.e. they are not boilerplate — drawn from 22 distinct suppressed source rows.

By the suppression reason of the SOURCE row:

suppression reason of the source	spans
unreleased_commercial_plan_not_for_export	99
derived_from_withheld_passage	34
third_party_commercial_terms_secondhand	11
reconstructs_withdrawn_index_term	7
third_party_proprietary_leak_admission	4
direct_personal_identifier	1

The sharpest single row carries 11 rare spans, and its own path embeds the pid of a row suppressed as unreleased_commercial_plan_not_for_export. That is the kg_meeting_note derived-leak class, live — the class this project already had on record as an open, operator-only item. Also implicated: one docs/session-evidence/phase1-report.md row (1 rare span) and two todo: / next_point: derived rows (6 and 4).

WHAT IS NOT ESTABLISHED, and it is the whole question. Whether each span is genuinely withheld-worthy, or ordinary phrasing that co-occurs in both a suppressed and a live passage. An 8-word English shingle can be generic, and adjacent transcript passages overlap by construction. 125 rows is a reading assignment, not 125 leaks, and must never be reported as such.

One thing IS established and it is the reassuring half: no personal name is served. A name-level probe took 20 name-shaped tokens from the 11 `direct_personal_identifier` rows. The 2 that appear rarely in served rows are already present in **263** and **9** files of the PUBLIC umbrella respectively, so neither is withheld-only.

2. The gate was green about /tmp

The recorded defect was real and WORSE than recorded. `verify-redaction-propagation.sh` was said to check `curriculum/passages.db` instead of what the server serves. In fact **`curriculum/passages.db` does not exist in the checkout at all** — the gate's A10 was reading a fixture inside its own `mktemp` tree. **A green verdict from it was a statement about /tmp and about nothing that is served.**

That is the “a gate's coverage set is an assertion” failure in its most dangerous form: **false assurance about disclosure is worse than no gate.** See `[[a-gates-coverage-set-is-an-assertion]]`.

The served artifact was DERIVED, not assumed: `container argv → -index /var/lib/workshop → confirmed against the live process via its handshake file and GET /api/health → mapped through the container's own mounts to the podman volume. Two artifacts live there: passages.db and passages.jsonl.`

S1–S5 now assert over that served corpus: no text on a suppressed row; suppressed rows flagged; **no suppressed pid in the served lexical index**; registry rows flagged; registry and database agree. Each of the five derivation steps can end in **rc 2**. Findings print `pid`, `sha256-16` and `location` — never text.

The decisive mutation bites, and it was demonstrated as a before/after pair: the committed gate, fetched from git and run unmodified over a served corpus carrying suppressed text, returns **0**. The fixed gate returns **1**, naming S1. C1 requires each of S1–S5 to report a **PASS by name**, so a 0 cannot be bought by never reaching the block; C2 requires an empty suppression set to be **2**, not five green assertions.

A NOTE prints on every run: the served redactions table holds **0** rows against **162** suppressions. No text is exposed by that — the artifact simply cannot say *why* a row is withheld.

3. A SECOND instrument is broken, and its failure was being reported as a finding

verify-registry-derived-leak.sh still exits 1 — but NOT for the recorded reason, and the recorded **kg_meeting_note** finding was **neither reproduced nor refuted**. A1/A2/A4/A5 pass. A3/A6/A7 fail because **workshop-redact** cannot build in its scratch copy: the relative replace `../../../../../submodules/{passage,verdict}` directives do not survive the copy to `/tmp`.

That is an instrument fault. It should be rc 2 and is being reported as 1 — the exact collapse of the three-valued contract this project forbids, because it accuses the tree of a violation when the truth is that the check could not run. **It was not silenced and not allow-listed**. The fix is known and already proven: the redaction agent hit the identical trap in its own proof and fixed it by mirroring the sibling `submodules/` directory into the scratch tree.

4. A named gap, deliberately not closed here

Nothing scans the LIVE corpus for value-level (withheld-string) leaks. **verify-registry-derived-leak.sh** covers minted rows in a fixture. **The 125-row measurement above IS that missing scan — run by hand, once.** It was deliberately not bolted onto the redaction gate: that is another registered gate's remit (§11.4.74), and 125 rows of undetermined severity would drown S1–S5's disclosure signal.

5. Session-evidence rows: 383 CONFIRMED, and the removal path is operator-only

383 (382 live + 1 already suppressed), from **33** documents. The correction already recorded in A73 is independently confirmed here: **a re-ingest will NOT drop them** (`cmd/ingest-transcript/main.go:215, R1c`). Removal needs either a from-scratch registry rebuild — which **invalidates every anchor** — or a targeted **workshop-redact** suppression. **Neither taken**. Risk if untaken: 33 never-reviewed working-note documents stay searchable and reachable by URL.

Verified

verify-redaction-propagation.sh	0	S1-S5 over the SERVED corpus
prove-redaction-propagation.sh	0	7 source + 4 served
mutations, control,		
		non-vacuity, before/after
pair SHOWN		
verify-check-registry-001.sh	0	
verify-check-registry-002.sh	0	checks=74 debt=3 missing=0
verify-registry-derived-leak.sh	1	INSTRUMENT FAULT – should be
2		

workshop HEAD **b9e1e26** (pushed 358ca43..b9e1e26), two explicit paths only. The live volume is opened **read-only** by the gate and never touched by the prover. The container was not restarted.

A75 — NEW OPERATOR BRIEF, 2026-09-04 16:40: port ai_interviewing's areas/lessons/tests into workshop, port the fitting knowledge-base material, de-monotone the UI, and MERGE Ask with Search. Recorded before any work started.

Recorded first and in full, because the standing rule is that no request, prompt or idea may be lost. Four distinct workstreams; none is a restatement of another, and none was in the queue before this.

1. PORT THE FUNCTIONALITY. *“All areas / chapters with lessons / tests functionality from ai_interviewing has to be used as the base for the workshop submodule and all features ported — design, UI, UX, structure, navigation, data structure — and all this further extended for use with workshop and the additions it brings in.”*

2. PORT THE FITTING MATERIAL. *“We MUST select all existing materials from ai_interviewing (knowledge base) — the chapters, lessons, tests — which fit / are covered within chapters we have in workshop, and port them for use as part of workshop.”* Note the selection criterion is **fit against workshop's own chapters**, not bulk import.

3. THE UI IS MONOTONE. *“Workshop UI looks really monotone, make sure we use OpenDesign, all available design skills, MCPs and plugins to give more life and colour into workshop UI / Design / Templates / Look and feel / UX.”*

4. MERGE ASK AND SEARCH. *“Ask and Search sections **MUST BE** properly merged so in workshop there is an input area which can be used for asking questions **OR** searching the whole indexed space — all content and materials, transcriptions and others.”*

What ai_interviewing actually contains — surveyed before dispatching, not assumed

923 tracked files. The two repositories are **sibling designs**, which is what makes this port realistic rather than aspirational: both use the identical platform/frontend/src/app/{core, features}/ Angular layout.

34 areas	docs/interview-preparation/areas/01-agent-orchestration ... 34-cross-project-retrospective, each in md + html + pdf + docx
The features the operator wants	features/ {home,module,plans,practice,progress,search}.component.ts — each with a .spec.ts. practice and progress are the lessons/tests functionality by name.
Visual material	159 .mmd mermaid, 101 .svg, 159 .png — 74 of the mermaid files sit under areas/diagrams, 79 under project-deep-dives/diagrams
Other doc trees	company-research, improvement-plans, project-deep-dives, validation-plans
e2e	platform/qa/e2e/tests

BOTH repositories are PRIVATE, so material moving ai_interviewing → workshop crosses no boundary. **The umbrella above them is PUBLIC and nothing may land there** — and note the content-boundary gate already attributes **277 matches** to ai_interviewing, so this port is exactly the kind of work that can worsen a public-facing count if anything leaks upward.

Sequencing — and it is dictated by a real conflict, not by caution

platform/frontend/ **is currently OWNED by the in-flight UI-label-sweep agent**, which has four components modified and uncommitted and is mid-way through writing its derived-coverage gate. Items 1, 3

and 4 all land in that same tree. Dispatching them now would put three agents into one directory and produce exactly the write-race that cost this project three handoff entries, twice.

Dispatched immediately (zero file conflict): - A read-only **MAPPING agent** — inventory *ai_interviewing*'s data model, navigation and component contracts; map its **34 areas against workshop's own chapters** and produce the *fit* table item 2 demands; and write the port plan. It writes a plan, not code. - A **design-direction agent** — produce the visual system (palette, typography, spacing, component treatments) that answers “monotone”, as a **specification plus a reviewable artefact**, touching **no** frontend source file.

Blocked on the UI-sweep agent finishing, then dispatched: - the port implementation (item 1), the visual application (item 3), and the **Ask/Search merge** (item 4). The merge also has a backend half, and *pkg/answer* is currently owned by the multi-provider agent — so it is blocked on two fronts, not one.

One term needs the operator and is NOT a blocker: “OpenDesign”. It is recorded verbatim because it may name a specific tool. The earlier “The Platform” brief named **PenPot** for wireframes, and the *figma* MCP is present but **failed to connect** this session. The design agent is instructed to survey what is actually reachable and say so, rather than claim a tool it could not use. **If OpenDesign means something specific, say so and it will be used; the work does not wait on the answer.**

Item 4 is a product decision as much as an implementation. One input serving two intents — *ask a question* versus *search the indexed space* — must not silently guess wrong. The measured state it inherits is relevant: **SC-008 is 0/11**, i.e. meaning-based retrieval of a SPECIFIC passage does not currently work on this deployment, and that is under investigation by another agent. **A merged input built on top of a retrieval leg that cannot find a specific row would look broken to a reader for reasons that have nothing to do with the merge.** The two must be sequenced so the merge is judged on its own behaviour.

A74 — SEVEN AGENTS KILLED MID-WORK BY AN AUTH EXPIRY, AND ALL SEVEN RESUMED RATHER THAN RE-DISPATCHED. Nothing was lost; here is the proof, per agent.

What happened. At roughly 13:57 the session's credential expired (API error: Login expired · error type authentication_failed). Seven of the nine agents dispatched in A73 terminated **mid-tool-call**.

Two more (retrieval, boundary guard) left no completion record at all — the harness reported them as stopped when the previous process exited.

The response, and it is a rule rather than a preference: RESUME, DO NOT RE-DISPATCH. A re-dispatch starts a fresh agent with none of the dead one’s context — every measurement it had taken, every hypothesis it had ruled out, and every partial edit’s *intent* would be gone, and the tree would still hold the half-finished files. Resuming through SendMessage reattaches to the saved transcript. **Several of this session’s most valuable findings came from agents that had already been resumed once.** See [[operator-works-by-heavy-fanout-and-decisions]].

Partial work was measured before anything was touched. No file was reverted, cleaned up, or “tidied” — an agent restoring a shared file cannot know what else moved in it, which is how this project lost handoff entries twice.

Agent	Last words before it died	Partial work found in the tree
Exports (G8)	<i>“zero of the 1,702 output paths are covered by any existing .gitignore rule, so no prior decision applies and I add none. Waiting for generation.”</i>	567 .html + 559 .pdf at the umbrella root
Redaction gate	<i>“Gate is green over the real served corpus. Now the paired proof.”</i>	platform/gates/verify-redaction-propagation.sh and its prove- sibling, both modified 4 frontend components modified: core/cite.component.ts, core/status.component.ts, features/chapters/meeting-notes.component.ts,
UI label sweep	<i>“The rewording is done. Now the gate.”</i>	

Agent	Last words before it died	Partial work found in the tree
Multi-provider	<i>“The openai_compatible row must keep its AllowRemote gate. Let me fix that and build.”</i>	features/chapters/ todo.component.ts
		pkg/answer/hosted.go (NEW, untracked), pkg/answer/ provider.go, cmd/ workshop-server/ main.go
Containers module	<i>“Now I need to export ResolveRunSpec so out-of-package implementers (like RemoteRuntime) can build the same argv.”</i>	10 modified files in submodules/ containers
Retrieval (SC-008)	<i>(no completion record)</i>	pkg/index/ generation.go modified; an untracked pkg/index/ zz_probe_test.go
Boundary guard	<i>(no completion record)</i>	scripts/verify- content-boundary.sh modified

Two agents had ALREADY completed before the expiry and their results stand (recorded in A73): the stale-export agent, which STOPPED rather than deleting and refuted my premise; and the constitution agent, which found the pin already equal to upstream and proved the six exemptions correct.

Each resume message carried the specific thing an interrupted agent drops. Not a generic “carry on” — the failure mode of a resumed agent is finishing the mechanical half and silently skipping the discipline. So each was re-told the one rule most at risk in ITS task:

- **Retrieval** — do the load-independent work FIRST, starting with the pure count that cannot be contended: *how many of the 12,979 passages have a vector at all*. And do not publish any timing figure until the host is genuinely quiet; **/api/search was measured at 30 s against a 1.07 s idle baseline** while load sat at 18 on 8 cores.
- **UI sweep, multi-provider** — name the proof prove-`<gate-name>.sh` **exactly**, because the pairing check derives the filename and a

proof under any other name counts as none; derive the coverage set; include a CONTROL and a NON-VACUITY case, or a gate hardwired to `exit 1` passes the proof.

- **Multi-provider** — an unreachable provider must yield could-not-determine, **never a silent fallback to a different model**, because that makes every answer's provenance a lie. And no credential in a tracked file, log line, error message or fixture.
- **Redaction** — the decisive mutation: put a should-be-withheld string in the **SERVED** artifact while leaving `curriculum/passages.db` clean, and require red where the old gate was green. Plus the corrected fact that **a re-ingest does NOT drop the 383 rows**.
- **Containers** — the invariant in one line: *zero bytes with a nil error must be impossible when the read actually failed*. And if a defect does not reproduce, say so — a recorded defect that no longer exists is itself a finding.
- **Boundary guard** — extend the EXISTING `--prove-failure` rather than adding a script (R5 would fail the registry), and pair the determinism check with a non-vacuity half that mutates a byte and requires the two runs to DIFFER.
- **Exports** — the four zero-byte `.dc*.html.err` files are still in the tree and must not reach a commit; and confirm every generated `.html` has its `.pdf` sibling, because a document with one and not the other still counts as unexported.

Tree state at resume, measured: umbrella 1130 untracked / 4 modified; workshop at 7b07807 with **13** dirty paths; submodules/containers at d940b51 with **10**. **The d940b51 in this sentence is a DATED observation and the gitlink has since moved to 7f592256; the sentence was true when written and is kept unaltered as the record of that resume.** submodules/constitution clean at 2887b42e and equal to its remote. The empty submodules/helix_code/ that the constitution agent accidentally created and reverted is confirmed gone — submodules/ holds the expected **7** entries.

No gitlink has been bumped. workshop and submodules/containers both show as modified at the umbrella root and will stay that way until their agents finish, because the bump lands **once**, with `helix-deps.yaml` in the same staging — C9 requires them to move together, and it has caught this exact mistake before.

A73 — SESSION RESUME 2026-09-04 (afternoon): four operator decisions taken, nine agents dispatched, two items deliberately held back with a measured reason.

Written on resume after the subscription gap. **Resume contract honoured first:** CONTINUATION.md read, scripts/continuation-check.sh run BEFORE any work — **exit 0, 8 PASS · 0 DRIFT · 0 UNDET · 9 NOTE.** Fleet re-verified: **all 14 repositories clean and local == remote.** Platform live and healthy, up 4 h, /api/health 200 on 127.0.0.1:8087.

One commit arrived from outside this session and was classified, not assumed. bc47a41 "Auto-commit" sits after f97effa. It changes exactly one file — .ashlrcode/genome/manifest.json, 16 insertions / 16 deletions — a tooling plugin's own manifest. **No governance file, no gate, no carrier, no gitlink.** It is pushed and the tree is clean at it.

The four decisions the operator took

Each was put with its cost stated. Recording the answers verbatim in substance, because a decision without its reasoning rots into a rule nobody can question.

1. G8, the §11.4.65 export mandate → EXPORT EVERYTHING (all 851). Measured: the umbrella root tracks **924** markdown documents, **73** have both .html and .pdf, **851** do not. Three options were offered — a declared reader-facing subset (recommended), export everything, or leave it open and measured. **The operator chose everything.**

The cost was stated once before the choice and is restated here rather than buried: ~1,702 new files in a **PUBLIC** repository; every exported byte becomes a second copy of text the content-boundary gate must scan, in a population already near 12,900 and already hard to read; and it is **irreversible in public history**. The concern was raised, the operator decided, and the work proceeds in full. **The agent carrying it is instructed that if the exports introduce hardcoded-path or environment findings, the fix is to stop the generator embedding them — NOT to add an exemption.**

2. The six constitution findings → “Make sure we are on last constitution Submodule codebase and then apply all fixes we may need!” This is TWO steps and step 1 is the pin-move authorization this file has always required. The agent must (a) compare the pin to git ls-remote HelixConstitution HEAD, and if it differs, **classify the direction BEFORE moving** (merge-base --is-ancestor, rev-list --left-right --

count) and move only by `git merge --ff-only`, stopping if anything is divergent; (b) check **by name** whether the move touches any governance document; (c) **re-measure the corpus on BOTH sides** — blob 34eff9d8..., 11,700 lines, 252 anchors, 1,779,401 bytes, sha256 fe1de96abc84c2fc... — and report any figure that moved as a finding; then (d) re-measure the six findings, because **the fast-forward may already have changed them**, and fix only those that are genuine frozen assumptions.

HARD BOUNDARY given to that agent: it may modify the constitution submodule's own `helix_code_services.sh` under its `scripts/helix_code/` and what is needed to prove the change. It may **NOT** modify `Constitution.md`, the four carriers, `CLAUDE_ANCHORS_FULLL.md`, `templates/`, or `scripts/gates/`. **And it may NOT push to HelixConstitution** — pushing changes the authority root this whole fleet inherits; it commits locally and reports, and that push is a separate operator confirmation.

3. Multi-provider model support → BUILD IT NOW. Wire submodules/LLMProvider behind the platform's answer path under §11.4.74 reuse-before-reimplement. **Local Ollama stays the DEFAULT and must behave identically** — the operator's standing words are *"Local model MUST BE available for workshop itself, and ability to use some of providers."* An unreachable or unconfigured provider must yield **could-not-determine**, never a fabricated answer and **never a silent fallback to a different model**, because substituting a model silently makes every answer's provenance a lie.

4. The broader UI label sweep → DO IT. The backend half is already closed and gate-enforced: 30 vocabularies in `vocabulary.ts`, derived from 12 Go sources plus a JSON file, proved by 11 mutations. **The gap is that the gate is deliberately ONE-DIRECTIONAL**, so every string the FRONTEND itself mints is policed by nothing. That is the half of the operator's instruction still unmet.

Nine agents dispatched, with their boundaries

Every one carries the same non-negotiables: three-valued exits, §11.4.6 (no guessing language), §1.1 paired proofs **named prove-<gate-name>.sh** because the pairing check derives the filename, the content boundary, and an explicit list of which other agents are touching which paths.

#	Scope	Owns
1	SC-008 = 0/11, SC-007 = 51.9% — meaning-based retrieval of a SPECIFIC passage does not work	platform/backend/pkg/search, workshop/scripts/bench-retrieval.sh
2	COMPLETE — and it STOPPED rather than deleting; see below	pipeline/detect_ocr.sh
3	Content-boundary corpus-stability guard	umbrella scripts/verify-content-boundary.sh
4	G8 — all 851 exports	umbrella export files
5	Constitution pin + the six findings	submodules/constitution
6	Multi-provider wiring	platform/backend/pkg/answer, cmd/workshop-server
7	UI label sweep	platform/frontend/
8	verify-redaction-propagation.sh asserts over the WRONG artifact	that gate + its proof
9	submodules/ containers: the log reader returns 0 bytes with a nil error; and the missing stdin/ ephemeral-run primitive — BOTH FIXED UPSTREAM in 6d13ad03528c and consumed at gitlink 7f592256, re-measured 2026-09-09; the LOCAL half only. The REMOTE-stdin gap is still OPEN	submodules/ containers

Agent 1's brief names the hypothesis to test first, and it is testable: the server logs doc-prefix "search_document: ", query-prefix "search_query: ". Nomic models REQUIRE those task prefixes. **A corpus embedded WITHOUT the document prefix but queried WITH the query prefix produces exactly the reported symptom** — everything clustered near one similarity value, nothing discriminating. It is instructed NOT to lower the 0.6550 floor or widen a bar to pass; a harness that runs and reports criteria NOT met is the honest outcome (the T118 precedent).

Agent 8 is the disclosure-safety one and is the most serious. A redaction gate validating an artifact nobody serves is **false assurance about disclosure** — **worse than no gate**. Its decisive mutation is prescribed: put a should-be-withheld string into the SERVED artifact while leaving curriculum/passages.db clean, and require the fixed gate to go red where the old one was green. It is forbidden from deleting corpus rows or triggering a re-ingest — both change live production data.

TWO ITEMS DELIBERATELY NOT DISPATCHED, and the measurement that decided it

The operator said **“fan out subagents”** and the standing preference is 6–12 in flight. Dispatch stopped at **nine** on a measurement, not a feeling:

```
free    -> 42% RAM used (under the $12.6 60% cap)
uptime -> load 14.50 / 10.76 / 7.67 on 8 cores  = ~1.8x
oversubscribed
```

RAM was not the binding constraint; CPU was. Adding more would slow every agent AND corrupt the measurements they exist to produce — and this project has already recorded **two wrong conclusions from exactly that cause** (a latency gate that FAILED at p95 13,219 ms and passed at 1,817 ms once idle; a boundary gate that looked non-deterministic because another agent was rewriting its input). See [[measure-on-a-quiet-tree]].

Held, named, and owed — to be dispatched as agents complete: - Deep crawling. The operator called it **MANDATORY**. Three targets were crawled by hand; **no general capability exists**, so the next link that arrives cannot be processed. Placement is undecided (workshop/ pipeline/ vs umbrella _tools/) and needs deciding as part of the work.

- **Reproducible integral ZIP archives + the remaining mandated formats.** Explicitly requested. The archive half **must be reproducible or it cannot be verified**, which is the whole difficulty.

Neither is dropped. Both are here because the standing rule is that no request, prompt or idea may be lost.

Agent 2 has reported, and it refutes the premise I gave it. Recorded in full because the refutation is the value.

I briefed it to delete ~40 stale docs/session-evidence/*.sections.json, on the reasoning that the 2026-09-04 ingest exclusion had made them unregenerated and therefore disposable. I made ONE check load-bearing: prove nothing consumes them, and if anything does, STOP. It stopped. The premise was FALSE.

Real count: **33** tracked (not ~40); 114 exist workshop-wide; **4** carry the hardcoded-path findings named in the allow row.

Three live consumers, each measured: 1. **The running server serves them right now.** GET /api/sections/session-evidence → **HTTP 200**, {documents: 42, groups: 1, undetermined: 0}, with **33 of 42 documents carrying a non-null sections_href**. podman inspect shows docs/ is a **BIND MOUNT of this checkout** — so a deletion changes live behaviour immediately, with no rebuild. 2. **A real code path, not vestigial.** internal/api/sections.go:243 stats the sidecar and publishes the href; the frontend's core/section.ts:305 parses it; and the section comes from a **route parameter**, so it is reachable by URL rather than hardcoded out of the SPA. 3. **The redaction tool reads their contents.** internal/redaction/plan.go:640 reads the sidecar, parses its sections spans and rewrites it to propagate a passage suppression. **Deleting them would have silently narrowed the reach of the one tool that can remove this private material.**

Staleness of GENERATION is not absence of CONSUMPTION. That sentence is the lesson, and it generalises past this file.

A SECOND recorded claim is WITHDRAWN AS FALSE by the same report — and this one was mine, carried in two allow-list rows and in A72. It read: *“a re-ingest drops these rows and this row should then be DELETED.”* **A re-ingest does not drop them.** cmd/ingest-transcript/main.go:215 states the rule in its own words — *“An existing*

registry is LOADED, never overwritten blind” — because an anchor is only meaningful against the registry that issued it (R1c). The exclusion stops NEW rows being minted and removes nothing already present.

Both allow-list remedy blocks have been corrected in place, one of them to a STOP – DO NOT DELETE with the three consumers named, because **a baseline row carrying a false remedy is worse than no note: it instructs the next reader to do harm.** bash scripts/audit-hardcoded-paths.sh re-run after the edit: **exit 0**, 15 files explicitly allowed.

Removing the 383 rows needs one of two OPERATOR actions, neither taken: (a) a from-scratch registry rebuild, which **invalidates every existing anchor**; (b) a targeted workshop-redact suppression. Handed to agent 8, whose brief already covers the served-artifact question.

SIDE FINDING, and it is the uncomfortable one: the ingest exclusion stopped this material being *minted*, but **did not stop it being served** — 33 hrefs and 383 corpus rows are live on port 8087 today.

Agent 2’s other half succeeded, and found a real defect underneath the one I described. The detect_ocr.sh resolve_path degradation was confirmed REAL by exercising three modes (realpath present / present-but-exiting-127 / absent from PATH). A symlinked *directory* component resolved fine; a symlinked *final* component came back unresolved. **And it mattered on this host, in the exact case the script’s own header names:** command -v ffmpeg resolves to a symlink into the Playwright bundle, and probe_ffmpeg calls resolve_path on it —

```
before -> .../bin/ffmpeg (an innocuous  
PATH entry)  
after  -> .../ms-playwright/ffmpeg-1011/ffmpeg-linux
```

The evidence record was hiding precisely the substitution the probe exists to expose. Fixed with a bounded portable readlink loop (no -f) before the existing pwd -P, 40-hop bound to terminate cycles; six inputs — plain, symlinked dir, symlinked final, 2-hop chain, relative target, cycle — now return **byte-identical answers in all three modes.** --prove-failure exits 0 at 6/6 in both mode A and mode C, with identical verdict lines.

Honest boundary the agent stated and I am not softening: this is exercised on **Linux only**. What was removed is the dependency on realpath EXISTING, not the assumption that this host is representative. No BSD or macOS host was available. **That is a 2, not a pass.**

Workshop HEAD after its commit: **7b07807** (eba5c17..7b07807), one file staged by explicit path. It did **not** sweep in the other agents' in-flight files, which is the discipline this tree needs while nine agents share a checkout.

Two figures moved again with no instrument edit, and are recorded rather than glossed: `audit-environment-assumptions.sh` now reads **588** allow-listed (CLAUDE.md records 567) over **2348** files (records 2247). The audit is unmodified; **the fleet moved under it**.

A72 — The two audits the workshop bump reddened are GREEN, by 24 fixes and 15 rules; 39 documents gained their exports; and the umbrella G8 gap is now MEASURED at 851.

Written 2026-09-04, after A71. This entry closes the loop A71 left open at its item 11.

1. `audit-environment-assumptions.sh` → exit 0. The split matters more than the exit code.

Two figures I stated earlier are WITHDRAWN, both corrected by measurement. I reported the population as **43, all from the workshop bump**. It was **47**, and it had three sources: **37** from the workshop bump, **6** from the CONSTITUTION fast-forward, and **4** that arrived mid-session from umbrella commit 7312ca1. The constitution six were proved by reading the pin on both sides. The file is `submodules/constitution/scripts/helix_code/helix_code_services.sh`; read it at each pin with `git -C submodules/constitution show` using that repo-relative path. At the OLD pin 3be10826f3d2 it contains **0** of those endpoint literals; at the recorded pin 2887b42e9349 it contains **7** — which is the commit this carrier already records as touching that file by 76 lines. **A finding that appears after a bump is not necessarily FROM the bump you happened to make.**

The split is **26 REAL FIXES against 21 occurrences under 13 allow rules** (an interim header inside the allow file says 24/15; it was written mid-work and the report's 26/13 is the final count). A green exit bought with rules is not the same artifact as one bought with fixes, and the allow file records which is which, by name:

- `workshop/platform/backend/pkg/search/carryforward_test.go` (16) — the fixture's two model ids were **real shipped model names** in a test that embeds nothing and contacts nothing. `embeddings.model` is an opaque TEXT column compared with `e.model = ?`, so they are now `cfModel / cfOtherModel`, deliberately naming no real model.

- workshop/pipeline/detect_ocr.sh (6) — five copies of **GNU-only readlink -f** became one resolve_path preferring realpath(1) with a POSIX cd && pwd -P fallback. The Linux-only /proc/self/rc-2 fixture became a path beneath a regular file, which is ENOTDIR on every POSIX system.
- workshop/platform/gates/verify-search-latency.sh (1) — the last-resort literal gained its own \$WORKSHOP_DEFAULT_BASE_URL override which, unlike \$WORKSHOP_BASE_URL and --base, does **not** short-circuit the server.json discovery above it.
- workshop/scripts/bench-retrieval.sh (1) — the help text quoted a literal default while hiding the three environment variables resolve_base() actually reads. **A documentation defect fixed, not a literal moved.**
- _tools/containers/cmd/distribute-helixtranslate/main_test.go (2) — a fake API-key VALUE collided with the MODEL class's mistral-token. Renamed; it is arbitrary test data and names no model. go test ./... re-run: ok.

The 6 constitution findings are EXEMPTED, not fixed, and that is the honest label. They are upstream-owned (§11.4.29); the fix belongs in that repository and returns as a gitlink bump. One is a case PATTERN classifying a URL as loopback — it configures nothing; the rest are label|url rows the file itself documents as a parseable declaration source, with hc__status_endpoints() as the override layer. # REASON: was used rather than # BASELINE:; the agent recorded openly that BASELINE was a defensible alternative reading.

One of my four classifications was right on the premise and still became a fix. I judged verify-search-latency.sh:119 an acceptable “env first, literal last” pattern. The premise was verified true (BASE_URL="\${WORKSHOP_BASE_URL:-}" at line 75, then --base, then server.json, then the literal) — but a DISTINCT last-resort override was available and adds a capability the existing variables do not, so it was fixed rather than exempted, and proved both ways against a copy with no server.json.

Re-measured: **588 allow-listed** (was 567) and **666 baselined** — the baseline was NOT touched, no # BASELINE: row was added, neither audit script was edited, and --strict-allow-list exits 0, so no rule added is stale.

THREE THINGS ARE NOT RESOLVED, stated rather than smoothed over: a. The 6 constitution occurrences are exempted, not fixed — upstream work. b. **The stale .sections.json exports still exist.** Now that session-evidence is excluded from the ingest sweep

(DOCS_EXCLUDE_DIRS="\${WORKSHOP_DOCS_EXCLUDE_DIRS:-session-evidence}" at workshop/scripts/ingest.sh:393), DELETING them is the real fix — a workshop-side deletion of ~40 tracked files, not taken. c. detect_ocr.sh's resolve_path was **verified on Linux only**. On a host without realpath, a symlinked final path component stays unresolved. The degradation is stated in the code; it was not exercised, because this host has /usr/bin/realpath.

2. audit-hardcoded-paths.sh → exit 0. Two BASELINE rows, both for **derived artifacts whose sources are already baselined**: workshop/curriculum/passages.jsonl and the four docs_chain-generated docs/session-evidence/*.sections.json. **Both rows are meant to be DELETED, not kept**: docs/session-evidence/ was excluded from the ingest sweep on 2026-09-04, so a re-ingest drops the corpus rows, and the .sections.json exports go with their sources.

3. 39 documents gained HTML and PDF. 42 were deliberately NOT exported. Coverage outside session-evidence is now **68 of 68 for both formats**; all 39 PDFs verified as real PDFs by file, 0 conversion failures. The generator was not invented: it is the pandoc/weasyprint sequence recorded in workshop/docs/knowledge-model-contract.md §4.2, confirmed against assets/theme.css markers to be the command that produced the two pre-existing exports in the same directory.

docs/session-evidence/ (42 files) was excluded ON PURPOSE and must stay excluded. Those documents carry absolute host paths. Exporting them would copy those paths into NEW tracked files and turn audit-hardcoded-paths.sh red again — the exact regression this session had just finished clearing. Three further pairs under docs/training/areas/ were generated but are covered by a pre-existing .gitignore rule; that rule's stated reasoning (build derivatives, reproducible from source, PDFs are opaque binary containers of private prose) was followed rather than overridden.

4. G8 IS NOW MEASURED, AND IT IS LARGE. An operator decision, NOT taken. The umbrella root tracks **924 markdown documents; 73 have exports; 851 do not**. Closing it means generating ~1,702 files into a **PUBLIC** repository, which also enlarges the content-boundary scan surface. That is a decision about repository size and disclosure surface, not a chore, and this session did not take it. Options, with the cost of each stated:

1. **Export everything** (~1,702 files). Closes G8. Cost: repository growth, and every exported byte is a second copy of text the

boundary gate must scan — the population that is already 12,939 and hard to read.

2. **Export a declared subset** — the documents meant for a reader (docs/**), not working artefacts (specs/**, _analysis/**). Cost: the subset is itself an assertion and needs a rule, or it rots.
3. **Leave G8 open and keep it measured.** Cost: an inherited mandate stays unmet, but visibly rather than silently.

No option is recommended here without the operator, because (a) is irreversible in a public history.

5. Fleet state at this entry — every gate re-run after the final bump.

```
verify-governance-cascade.sh      0    12 PASS / 0 FAIL / 0 ENV / 8
NOTE
verify-check-registry.sh          0    45 PASS / 0 FAIL / 0 DEBT /
0 UNDET
verify-manifest-pins.sh           0    12 MATCH / 0 DRIFT / 0
UNDETERMINED
verify-submodule-remote-sync.sh   0    12 CURRENT / 0 DRIFT / 0
UNDETERMINED
continuation-check.sh             0    8 PASS / 0 DRIFT / 0 UNDET /
9 NOTE
audit-environment-assumptions.sh  0    588 allow-listed, 666
baselined
audit-hardcoded-paths.sh          0    15 file(s) explicitly
allowed
_tools/containers go test ./...   ok
```

The remote-sync gate went red once more in between, and it was RIGHT. After the export commit it read workshop a3b0a731c610 c0da4c876f93 BEHIND – fast-forwardable: the gate compares the INDEXED gitlink against the submodule’s remote, and the bump was deliberately being held so that three workshop commits could be recorded in ONE gitlink move with the manifest. It returned to 12 CURRENT the moment the bump landed.

Workshop moved three times this session: a3b0a73 (The Platform section, the V7 proof) → c0da4c8 (78 export files) → eba5c17 (the environment fixes), all pushed, with helix-deps.yaml following the final head in the same staging.

STILL RED BY DESIGN and unchanged: verify-content-boundary.sh. See A71 item 8 — the gate is deterministic; the population is the reading assignment.

A71 — SESSION CLOSE-OUT: workshop pushed, a §1.1 gap I shipped closed, and three of my own claims withdrawn.

Written 2026-09-04, after A70. Everything below is measured; where a figure in an earlier entry is now wrong, it is **withdrawn by name** rather than quietly replaced.

1. The workshop is committed and pushed. a3b0a73. 318 files, +70,964 / -4,262. 83260ee..a3b0a73 to git@github.com:milos85vasic/workshop_curriculum.git; local and remote sha verified EQUAL after the push. Its working tree is **0 entries**. All three of its remotes (github, origin, upstream) are the same URL, so one push covers all three — do not read “3 remotes” as three destinations.

2. A §1.1 violation that I shipped, and that the tree had been reporting. platform/gates/verify-ui-vocabulary.sh went in **without a paired mutation**. workshop/scripts/verify.sh had been printing it on every run:

```
FAIL V7 gate-mutation-pairing §1.1
  verify-ui-vocabulary.sh has NO paired mutation (expected prove-
  ui-vocabulary.sh)
```

platform/gates/prove-ui-vocabulary.sh is now that proof: **11 mutations, 11 caught, rc 0**. Every mutation is DATA — a broken COPY of the vocabulary module or of the twelve backend files the gate derives from — so the proof cannot be made to pass by weakening the gate.

- **M2 is the gate’s advertised mechanism under test.** It invents a passage kind (invented_by_the_proof) that exists nowhere in the repository and requires the gate to go red **with no edit to the gate**. A gate holding a hand-typed list passes M1 and fails M2 — which is precisely the coverage-set defect this project has now hit three times.
- **M0 and M10 are what make the other nine mean anything.** A “gate” hardwired to exit 1 satisfies all nine defect mutations. M0 requires a pristine copy to PASS; M10 requires a frontend-only code to stay green, because the gate is deliberately one-directional.
- **M3–M9 prove the THIRD exit value.** A renamed declaration, a missing source, unparseable or empty suggest-sources.json, and a vanished vocabulary group are each **rc 2**, never a silent pass over a domain the gate can no longer enumerate.

Why it shipped unpaired, which is the reusable lesson: the gate's own header named its proof `prove-ui-vocabulary-mutation.sh`. V7 DERIVES the expected filename from the gate's name, so a proof under any other name reads as no proof at all. Re-derived after the fix: **19 gates, 0 unpaired.**

3. WITHDRAWN — my SC-006 FAIL. The first reading was contention, not the system. A `verify.sh` run measured **p95 13,219.3 ms** against the 2,000 ms budget and failed. It was taken while a retrieval benchmark AND a leaked `prove-run` server (see 4) were loading the same host. Re-measured on an idle host:

search fused	p95	1817.1 ms	median 1027.0 ms	n=40	PASS
search lexical	p95	99.2 ms			
search semantic	p95	336.8 ms			
search code	p95	0.7 ms			
health CONTROL	p95	2.5 ms	(touches no index – if this moved, the HOST moved)		

Both numbers are real. The budget was NOT widened. Headroom is 9%, so this is a pass to re-measure, not a property to quote.

4. A leaked server, and the reason a “healthy” container told me nothing. `prove-retrieval-benchmark`'s empty-generation mutation left a `workshop-server` alive on `127.0.0.1:18787` over a temp registry. Killed. Its own cleanup defect was found and fixed by the spec agent the same session (an EXIT trap referencing a local from a returned function died under `set -u`, so `rm -rf` never ran).

5. WITHDRAWN — every “port 8401” statement I made this session. The platform container runs on `network=host`, so the server binds the host directly at **127.0.0.1:8087**. `podman port` returns EMPTY for it (nothing is published, because nothing needs to be), and the 8432 listener belongs to an unrelated container. Measured on 8087, idle: `/api/health` **200** 0.003 s, `/api/search` **200** 1.07 s, `/api/suggest` **200** 0.05 s, `/api/ask` **202**. `/api/status` is **404** — that path does not exist; the index state comes from elsewhere.

I also drafted and withdrew a wrong diagnosis before it stood. Seeing listening on `http://127.0.0.1:8087` (requested `0.0.0.0:8087`) I concluded the server had downgraded its bind to loopback and broken container publishing. It had not: `baseURL()` at `cmd/workshop-server/main.go:1832` is a **display** function that rewrites `0.0.0.0` to `127.0.0.1` for the reported URL only, and the handshake file records the true bind, `"addr": "[::]:8087"`. **A log line about an address is not a measurement of a bind.**

6. A 9.2 MB compiled binary was tracked; it is not any more.

platform/orchestration/workshop-boot was the only tracked ELF in the repository. workshop/scripts/_common.sh:103 writes the control-plane binary to \$BIN_DIR (platform/bin/workshop-boot), which .gitignore:188 already covers. Measured before removing: the two copies are **DIFFERENT builds**, and `grep -rn 'orchestration/workshop-boot'` finds **no** script, Containerfile or compose file referencing the tracked path. `git rm --cached`, left on disk, with an ignore rule carrying the \$11.4.77 regeneration mechanism.

7. workshop/curriculum/passages.jsonl is BASELINED, and the real fix is upstream. Gate 0 (`audit-hardcoded-paths.sh`) went red on 6 absolute paths in the generated corpus. Traced to five source documents — four are `workshop/docs/session-evidence/*` which are **already baselined**, so this is the same declared debt seen through a derived artifact. `docs/session-evidence/` was excluded from the ingest sweep on 2026-09-04, **so a re-ingest drops these rows and the baseline row should then be DELETED, not kept.**

8. The content-boundary gate is DETERMINISTIC. The instability was the tree moving under it. This **withdraws “mechanism UNDETERMINED”** from A11. Four runs against a frozen snapshot produced **byte-identical** output — 12,939 (prose 12,368, short 486, name 85), every CB_TRACE stage count identical. On the live tree the same command gave 12,939 / 12,939 / 13,058 / 12,968 with **no gate or allow-list edit**, because two tracked files in the private workshop submodule were being rewritten continuously by another agent; both are private-side key sources, so each edit changes the private key space and therefore the total.

The residual defect is the instrument’s SILENCE, not its algorithm: it prints a number with no evidence that the tree it measured stood still. Designed fix, not yet implemented: a `corpus_fingerprint()` taken before and after, an `--expect-corpus <file>` option so two figures can be proved to come from the same tree, and a two-run determinism check inside `--prove-failure`. The verdict is unaffected on this tree (LEAKS>0 → rc 1 outranks UNDET → rc 2). **12,939 is NOT comparable to the recorded 11,878** and no claim is made about the difference: the private corpus changed between the two measurements.

9. SpecKit counts moved. 001 → 69/120, 002 → 110/143. Closure holds both ways: **001 31 ids / 0 unattached; 002 20 ids / 0 unattached.** All 84 unticked tasks carry a BLOCKER + OWNER. **T118 is ticked because the harness is BUILT — and its criteria are NOT met.** Those are different statements. Against gen 68 / 12,979 passages, live: **SC-007 51.9% (14/27)**

against a $\geq 90\%$ bar and **SC-008 0.0% (0/11)** against $\geq 80\%$; rc 1; per-query outcomes in platform/backend/evidence/retrieval/bench-2026-09-04.tsv. Two earlier attempts returned **rc 2** on connection failure rather than booking a false 0%.

SC-008 = 0/11 is a SYSTEM finding, not a task defect: meaning-based retrieval of a SPECIFIC passage is not working on this deployment. Every zero-overlap query returned plausible neighbours clustered near 0.68 with the expected passage outside the top 20, and the semantic leg was confirmed serving (legs = {lexical: ok, lumen: skipped, semantic: ok}) — so this measures a healthy two-leg system, not a degraded one. Targets were chosen from the corpus first and measured once, not tuned against results.

A retrieval finding of mine was ALSO wrong and is withdrawn: I reported T064's zero-overlap subset as unprovable, "only 2 of 11 rows clean". My ad-hoc tokeniser had no stop-word removal, so the 9 "collisions" were the, and, is, it, not. The harness subtracts a closed, listed STOPWORDS set and prints the raw pre-subtraction overlap per row. Re-verified: rc 0, 11 zero-overlap flags re-derived and all true.

10. C9 was honoured: the four gitlinks and the manifest moved in ONE change. workshop 83260ee→a3b0a73, milosvasic.ru 1823d62→f5f13a2, vasic.digital 3192836→948e925, submodules/passage 729cd96→9f92b2f, each with its helix-deps.yaml ref: rewritten in the same staging. verify-manifest-pins.sh → **PASS, all 12 recorded refs equal the gitlink this repository will commit.** A stale ANNOTATION was corrected too: passage's comment claimed "tag v0.2.0 points at this commit", and after the bump no tag points at any of the four new heads.

11. Two audits went red BECAUSE of the workshop bump. An agent is resolving them. Both were exit 0 before a3b0a73; the newly-tracked workshop files brought occurrences into scope. audit-environment-assumptions.sh → **exit 1, 43 frozen assumptions**; audit-hardcoded-paths.sh → **exit 1**, four docs_chain-generated .sections.json exports whose source .md files are already baselined. Dispatched with instructions to bias toward REAL FIX over exemption and to leave the umbrella commit to me.

Fleet state at this entry: all 13 submodules **0 dirty**, and every one with an upstream reports **behind 0 / ahead 0**.

A70 — SESSION WRAP: every request of the last 48 hours, audited against what was actually done. Two were NOT in the queue and are now.

Written because the standing rule is absolute: no work, idea or plan may be lost, forgotten, ignored or corrupted. This is the audit, not a summary. Each row is a request as given, with its true state. **A request marked done here has evidence elsewhere in this document; a request marked open has an owner.**

#	Request, as given	State
1	continue — resume by the repository's own contract	DONE — contract read, drift gate run first
2	Document every request so nothing is lost	STANDING, HONoured — this register is its instrument
3	fan out subagents (issued ~10×)	DONE — 40+ agents dispatched across the session
4	Invoke the Superpowers skills	DONE — used throughout
5	Put every operator-blocked item as interactive questions (3×)	DONE — 20 decisions taken , each recorded with its consequence and what was NOT chosen
6	“Make sure all SpecKit tasks are all done”	IN FLIGHT — 001 at 66/120, 002 at 107/143; ruling is <i>everything buildable, blockers named</i>
7	do it all now	DONE
8	Commit, push, full retest, boot the system (2×)	DONE — platform live and browsable, gates re-run
9	Finish anything unfinished	DONE — produced the open register at A60

#	Request, as given	State
10	retry any failed actions now!	DONE — all 9 rate-limit deaths resumed with context intact
11	Resolve all blockers, no gaps or shortcomings	DONE — 20 decisions; residuals named individually
12	Main README + docs_chain + all mandatory formats + recursive push	DONE — README 32 B → 21,435 B; docs_chain bound at 73 docs / 219 nodes; HTML+PDF derived from the anchor
13	Create “The Platform” section, exhaustive research, deep crawling mandatory	DONE — 4 research studies, a 10-page specification, and the crawler built and run
14	Production ready, restart as we go, tell us when booted	DONE — booted and reported; live at 12,979 searchable passages
15	What is left unfinished, faulty or incomplete	DONE — A60
16	Respawn everything killed	DONE
17	Nothing may be lost or corrupted	STANDING — and it was tested: see the write-race incident at A65
18	Generate content from this session, not the local model	DONE — content written by this session; the local model left for the product
19	UI labels must be meaningful, not generic	IN FLIGHT
20		THIS ENTRY

#	Request, as given	State
	Session wrap: sync everything, clean tree everywhere	

TWO REQUESTS THAT WERE NOT IN THE WORKING QUEUE, found by this audit and now recorded rather than lost:

(a) Multi-provider model support for the workshop. Stated as *“ability to use some of providers ... we will incorporate this mechanism later — use of various model providers for the workshop solution.”* It was **acknowledged in conversation and never written down**, which is precisely how an idea gets lost. **It is now a named work item.** Groundwork already exists and should be reused rather than rebuilt: submodules/LLMProvider is a declared, initialised gitlink adopted under the reuse-before-reimplement anchor, and the platform’s answer path is currently wired to a single local model by container argv. **This is a wiring job against an existing seam, not a new subsystem.** The local model stays available for the product; the operator’s instruction that *this session’s models do the authoring* is a separate matter and was honoured.

(b) “All labels in the UI must have meaningful titles” is broader than the one component being worked. It is in flight for the section browser; **the rest of the interface has not been swept.** Recorded so the next session does not mistake a partial pass for the whole.

WHAT A FRESH SESSION SHOULD DO ON continue: read \$0, run `bash scripts/continuation-check.sh` **before anything else**, then read this register from A70 downward — newest first. **Do not trust a figure in this document; run the command beside it.** Every instrument here is three-valued and a 2 is never a pass.

THE HIGHEST-VALUE UNBLOCKED WORK, in order: the operator’s 1–2 hours of blind transcription, which alone unblocks **two** criteria; the ranking gap now that retrieval’s window is no longer the constraint; and the specified platform, of which **0 of 9 build-plan phases have started** while its own machinery is built and gated.

A69 — the republish LANDED but the RESTART IS HELD, the ingest blocker was a stale binary, and workshop/README.md went from 32 bytes to 21,435.

THE RESTART IS DELIBERATELY HELD, and the reasoning is the entry. The republish was authorised to fix a real disclosure — the served registry predated 200 redaction entries, so **152 redacted rows were still served with pre-redaction text.** It landed cleanly: **13,141 passages, 0 lost pids, redacted 152 before and 152 after, 0 new rows byte-identical to any redacted row.**

But it swept 113 documentation sources never previously in the corpus, minting 1,519 rows — of which 383 come from 33 files under a session-evidence directory that has NEVER been through a redaction review, and which was measured earlier today as carrying a third party's real name across 5 files. Another 607 come from the platform research section, and 18 from redaction-review documents — where *a document about a disclosure can be the disclosure.*

So restarting would trade a BOUNDED exposure for a possibly larger one. The existing one reaches only whoever can already read the private repository; the new one would make never-reviewed working notes **searchable.** That is not a fix, and it is the same sequencing the operator already applied in requiring gates before publish. Assessment is assigned, with an explicit preference: **if a directory of working notes was never meant to be searchable, exclude the directory rather than redact many rows — fix the cause, not the symptom.**

THE INGEST BLOCKER WAS A STALE BINARY, and both of my hypotheses were refuted. The deployed tool was built **Sep 2**; strings on it contained **no knowledge-kind token at all**, so its compiled-in kind list was the four original kinds and it rejected all **9,144** knowledge rows. The decisive proof was a mutation: taking 200 of those rows and changing **only** the kind field made the same stale binary load them. **T125's validators are refuted, not merely unproven —** the corpus carries **zero** rows with any of their fields — so no validator was narrowed and **no data needed fixing.**

The misleading message is fixed at its root. A validation failure was reported as *“unreadable”*, which sent me looking at file permissions. Load now classifies **by error type, never by text** — absent / unreadable / **refused** — and the message names the verdict, the failing

invariant, and the sentence that would have saved the detour: *“this binary is older than the corpus ... rebuild it and try again before editing any data.”*

A hazard caught BEFORE the republish rather than after: the registry sync has **no redaction guard** — a matched observation overwrites text unconditionally. The agent refused to proceed until it had measured that a re-sync could not revert a redaction: all 152 redacted rows carry text identical to machine text, so redaction here is a **suppression flag, not a rewrite**, and a re-sync is a no-op on them. Regenerating into a fresh prefix minted 1,055 duplicates — **the exact incident that code’s own comment records** — while regenerating in place preserved every pid.

The mechanism that caused it will recur and is recorded: the ingest script rebuilds that binary **only when the file is absent**, which is how a two-day-old binary survived two days of source change.

CONTENT: the module’s front door existed as a 32-byte Tbd. while shipping a transcription pipeline, a containerised platform, a corpus and a control plane. It is now **21,435 bytes**, with six further documents written or repaired and the four module carriers edited **identically** — lockstep re-verified.

Every figure it wrote was measured, and the carriers were badly out of date: scripts 19 where the carrier said 16; gates 18 **verify / 20 prove** where the carrier said “four”; commands 9 where it said five; the corpus **8,582,210 bytes** where it said 825,231. **Each superseded figure is withdrawn by name.**

It refused to blur the important distinction: the section’s **own machinery** is built and gated — 9,044 lines, six checks each with a paired proof — while **the specified product is 0 of 9 build-plan phases started**. Both statements are in the same document. And the *What is NOT done* section carries the unmet criterion and the fact that a component named for entailment is **a lexical floor, not entailment**.

A self-inflicted regression worth keeping as a worked example: quoting a gate failure **verbatim** into a page put a live claim token into it, which **minted a citation edge** and produced a *second* failure on the next run. It elided the ids rather than editing the ledger — and then documented the episode **on the page itself** as an example of the mechanism working.

Debris cleared: a 0-byte file literally named CSS: and an orphaned 87-byte lockfile with no package manifest beside it, both untracked shell-redirect artifacts from another agent's session, both removed after confirming the real frontend lockfile is elsewhere and intact.

A68 — class A is 19.7% INWARD, the boundary gate is NOT REPRODUCIBLE, and a \$11.4.76 violation has been shipping since June. Plus: T143 ticked, 18/18 gates enumerated, and a machine cross-check whose marker is ENFORCED.

CLASS A DIRECTION — the answer is not the reassuring one.

Measured **complete, not sampled**, and by a **stronger** method than the classes before it: every row dated on **both sides at text level** — the earliest commit whose normalised blob actually contains the string — then widened to corpus level.

OUTWARD	6241	79.7%		INWARD	1544	19.7%		UNDETERMINED
50	0.6%							

The widening moved rows BOTH ways — 113 inward→outward and **178 outward→inward** — so it is not a one-sided rationalisation. Reproduced across three runs.

Where inward concentrates matters more than the total: 1,084 rows (70.2%) are private source code, and 68 are from the teaching-session material itself, landing in the two spec task files. **“Private committed first” is an ORDER, not a disclosure** — 1,544 rows are not 1,544 leaks. What it kills is the assumption that the spec trees are simply outward propagation: *true for four rows in five, and false for the fifth*.

A DEFECT IN THE INSTRUMENT, found by controlling for the agent's own edits. The gate's total is **not reproducible run to run: 12745 / 12846 / 12867**, where runs 1 and 3 were on a **byte-identical tree** and disagree by **122 rows**. Ruled out by measurement: gate and allow-list unmodified, no file changed, fresh scratch each run. The instability sits in a **derived filter** whose own count moves on identical input. **Mechanism UNDETERMINED — a 2, never a pass.** *Every boundary figure in this document inherits that uncertainty.* The direction split survives it, which is why direction is the figure to act on.

The binding constraint re-measured: 85 name rows over 9 withheld identifiers, not the 89/8 recorded earlier — and **a falling name count is not progress**, it is a different measurement of the same unassessed population.

A §11.4.76 VIOLATION THAT PREDATES THIS SESSION BY MONTHS.

Two carrier claims were false — a fleet container *is* running and the umbrella *does* now consume the containers module — but the third fact is the one nobody had stated: **the umbrella root has shipped hand-rolled Containerfiles since 2026-06-26**, referenced by ten tracked files, **with no reference to the containers module at all**. By this project's own reading of that anchor, that is a standing violation older than every other item in this register.

GATE ATTRIBUTION CLOSED: 18 of 18 gates enumerated, 0 by none (was 6 by none). **G-KG-1-changed now DERIVES its coverage set** from the contract's own §4 headings and **prints what it graded**, so a green run shows its population. The derivation was **proved to bite**: a synthetic §4.5 added to a temp contract **turned the gate red with no code edit**. An unreachable contract reports **undetermined via a hard failure — a skip was rejected because a skip exits 0 and the registry would read it as a pass**. **T143 is ticked**, its stated residue done by derivation rather than extension.

Two registry findings that are worth more than the counts. One gate stayed **debt** despite solid attribution, because it has **no argv that reaches exit 2** — its only argv-reachable 2 is the unknown-option handler, which the rules reject by name. And registering a proof row was accompanied by a **new debt row admitting what it does not buy**: the shell arm **only checks existence and the exec bit and never runs the script, even under --run-proofs** — “*registering a -proof row buys enumeration, not enforcement.*” The design-system gate is debt for a measured reason too: probing it with the shell dispatcher exits 2 **because bash cannot parse JavaScript**, so the probe would be **credited for entirely the wrong reason** — green while measuring nothing.

THE MACHINE CROSS-CHECK: built, labelled, and its label ENFORCED. Engine B ran over exactly the 30 planned windows, 30/30, no failures. **Inter-engine word disagreement 0.0858, bootstrap 95% [0.0671, 0.1086], n = 30 window clusters — not accuracy, not WER, not ground truth.** Two readings matter: **the window-boundary artifact is not what produces it** — trimming a second off both ends takes the rate *up* to 0.0932, so the divergence lives in the interiors; and it is **2.7× the calibration document's 3.19%**, which came from one window that document records as uniformly clean — *a warning against extrapolating from that window, not licence to quote the new one.*

The marker is enforced rather than trusted: the accuracy tool now refuses a machine-marked reference with exit 2 and writes nothing, and **the paired mutation is the load-bearing half — delete the guard and the same artifact scores WER 0.0000, accuracy 1.0000.** That is precisely the bluff the guard exists to prevent. **The model bridge was measured and is impossible here** — no audio path in any available tool, and the local models declare no audio capability.

And the research record rejected this shortcut BY NAME long before we tried it: a second ASR engine as the reference is *“the most tempting shortcut available and it is a bluff”*, because two models of one family share training data and **agree on the same hallucinations.** **Agreement bounds the union of two engines’ error from below; it cannot bound one engine’s error from above** — which is the only thing an accuracy figure means. **SC-002 remains UNMET and nothing was ticked.**

BUNDLES AND EXPORTS BUILT. The mandated set was read **verbatim from the anchor** — HTML + PDF — and both DOCX-bearing classes were checked rather than assumed: the section’s request document was **measured against the request-history class and is not it**, so emitting DOCX would have been a false-positive refusal. Archives are **reproducible — two builds byte-identical** — with the gate rebuilding and comparing on every run, 27 documents, 54 exports, **16 mutations, 16 caught.** Its proof found **a real defect in its own gate** and a false-positive class producing **62 fabricated findings**, both fixed.

One self-caught boundary contribution worth copying: mid-run it measured **+6 rows attributable to itself** — it had **quoted anchor text verbatim into private files**, which a co-occurrence detector matches regardless of direction. Replaced with citations; the rows went to **zero.** *Citing beats quoting when the instrument cannot tell which way the text travelled.*

A67 — three external items closed. My brief was wrong about the containers module, and the Ruby install is NOT additive — read before running it.

1 — The upstream bug is FILED, and the agent found more than I described. Reproduced independently on real podman before reporting: `--tail all → rc 125 strconv.ParseInt: parsing "all", --tail -1 → the output.` Then driven through the module’s own public API with a throwaway program, reproducing the silence exactly — **Logs() returns nil error, 0 bytes, nil read error**, with the rc-125

surfacing **only** through the deferred close. Filed as issue #2 on the upstream repository with both reproductions, the mechanism (only stdout is wired, so podman's stderr is lost) and the fix.

The finding beyond my description is the damning one: this runtime is the ONLY one in the package missing the guard. Two sibling runtimes guard with an integer parse and a third with an explicit comparison. **The one-line fix already exists three times in its own package.** Docker was **not measured** — that binary and three others are absent from this host — and the issue says so rather than generalising from one runtime to five.

2 — The distribution conversion is WRITTEN and explicitly NOT VERIFIED, and my premise was partly wrong. I told the agent the module's remote and distribution packages "*exist to do exactly this*". **Verified by reading every exported symbol: they do not.** There is **no** API for remote image build, image save/load/tag, image transfer between hosts, running with volume mounts, named volume creation, or remote-to-local fetch. What the module *does* provide is solid SSH transport — and every SSH and copy operation in the new program goes through it, spawning no external client. **The five steps the module cannot express are composed commands, each documented in-file with the reason.**

The remotes are unreachable, and that was proved rather than assumed: both resolve to nothing at rc 255, while the mDNS daemon is active and enumerated many other devices on the same subnet in the same minute. **A real absence, not a broken resolver.** So the conversion is labelled unverified **in three places** — the package comment with its rc-255 evidence, the README table, and its own dry-run output. What *is* verified: it builds, vets and formats clean, **9 unit tests pass**, the dry run renders the full plan, and **the could-not-determine arm fires against the genuinely absent hosts.**

Two findings that change the picture: the original script is **itself unrunnable here** — its own precondition fails, and the file it needs exists nowhere. And its sibling is **not convertible today at all:** it streams a document over SSH stdin, and **every implemented module path is stdin-less** — the option exists as an interface with **nothing implementing it.** That needs an upstream change, so it was declared rather than half-converted.

Neither script was deleted, and the reasoning is exactly right: *replacing a working script with an unverified one would be a downgrade dressed as compliance*. Registry re-run **45 PASS / 0 FAIL / 0 DEBT** — no regression.

3 — libruby-devel was NOT installed, and there is a warning attached that matters more than the command. Root is needed and password-gated; nothing was guessed at and nothing left half-installed. The blockage was verified first-hand — a live probe returns the missing-header error, and the site's lockfile pins three gems with **no precompiled variant**, so all three must compile.

THE INSTALL IS NOT ADDITIVE. The installed Ruby is 3.3.8-alt3; the only candidate for the development package is 3.3.12-alt2 and it **depends on a matching runtime**, so installing it **forces a Ruby upgrade on a host that builds two production sites**. The transaction could not be simulated — that needs write access this session lacks. **This is an operator decision, and the container route remains the safer answer**, since it already works and changes nothing on the host.

A66 — deep crawling BUILT (operator-mandated), and its own gate found six real defects on real data before anything was committed.

It found MORE than the hand crawls, which was the test. Reference architecture A went from **1 repo to 10**, pinned from ecosystem pages; architecture B reproduced **every figure identically** (8,920 tracked files, 2,866 .md, 2,841 .ts), with a package count of 275 against the study's 250 explained rather than waved away — the crawler counts every manifest, the study counted only one directory shape. The competitor's five pages re-hashed to the **byte-identical capture** the hand crawl pinned.

Where it found LESS, and the reason is honest: the 403 source and the archive snapshots **are not reachable by following links from any seed**. The hand crawl reached them **by search**, which this crawler deliberately does not do. Each is admissible today by seeding it directly — which was then done for two of them.

One refusal is exemplary and should be the house standard. A directory site returns **HTTP 403 on its own robots.txt**. RFC 9309 reads that as a **complete disallow**, so the page **was not requested at all**, the wall was recorded as a boundary row, and the run exited 2. *A crawler*

that treats an unreadable policy as permission is not polite; it is guessing. Likewise a browser-impersonating user agent is **refused with exit 1**, and an absent robots file is a rc-2, never an assumption.

The gate makes ZERO network calls. It re-derives robots compliance by re-parsing **the robots.txt the crawl itself captured**, and re-derives per-host inter-request gaps from recorded millisecond timestamps. *A politeness check that must go online to check politeness cannot run when the network is the thing that failed.*

One capture path, and it is enforced rather than intended. Both the capture tool and the crawler call one shared entry point, so id derivation, normalisation, key order and supersession live in exactly one place — and a proof assertion **imports both modules and refuses a crawler that has grown its own writer.**

SIX REAL DEFECTS, each found by running the gate against what the crawler had just written: 1. **The capture-id collision the ingestion agent reported is now FIXED at its consequence.** Two materials pointing at one renamed upstream shared an id, and the second **inherited the first's supersession link** — nothing in the provenance gate caught that. Keyed on the **pair** now; **no stored id was invalidated**, and a shared id emits a NOTE. 2. Repo pins were timestamped **before** the rate-limit sleep, so the journal **understated politeness that had actually been applied.** 3. `git clone` **bypassed the rate limiter** — pin and clone fired 0.826 s apart. 4. **The structure file embedded its own study time**, so it could never hash as unchanged — *“I manufactured the exact nonce problem STALENESS.md §5 warns about.”* Removed; a re-study of an unmoved repo is now a duplicate. 5. **The crawler churned material rows and overwrote hand-curated titles**, 52 growing to 112. Fixed; a re-crawl now adds **zero** rows. 6. A politeness assertion **false-positived on the robots.txt request itself** — *you cannot learn a policy without asking for it.* The exemption is exactly one row wide and a mutation keeps it that way.

It dropped its own contaminated output before committing — 2 capture rows, 110 mis-stamped journal rows, 59 churned material rows — and re-ran cleanly. *Work that was wrong was withdrawn rather than shipped with a caveat.*

21 mutations, 21 caught, gate **0**, `--root /nonexistent` **2**, registry **0** (3 pre-existing debts, none its own), Python suite **278 unchanged.**

The provenance gate reads 1, and that is the crawler working. It repinned a reference repository and found **its head had moved**, minting a superseding capture — so an active claim now rests on old bytes.

Resolving it means re-anchoring a claim cited by two specification pages, **another agent's territory, so it stayed out** and recorded the measured drift as its own verified claim instead.

A65 — WRITE-RACE INCIDENT on this very file. Three entries lost twice, and no gate saw it either time.

What happened, precisely, because the shape matters more than the loss. Two writers touched `CONTINUATION.md` concurrently. Entries **A60, A61 and A62** were written and verified, then found gone with the file **byte-identical to HEAD**; they were restored from the authoring session's own context. **A second write then landed between that restore and its commit**, taking two further entries with it, and left a **duplicate A61** because both writers had independently claimed the next free number.

Neither loss was malicious and neither was detectable. An agent restoring "its" file cannot know what else moved in it, and **no gate noticed either event**: the continuation checker validates the document's *internal* consistency, not that yesterday's content is still present. **A handoff document can pass every check it has while missing a third of its recent history.**

Recovery was luck, not a property of the system. Both restores worked only because the authoring session still held the text. **After a context compaction those entries — including the session's complete open register — would have been unrecoverable.**

The operational rules that follow, and they cost nothing: - **Commit handoff state immediately after writing it.** The window between edit and commit is exactly the exposure, and in a tree shared by a dozen agents that window is not safe. - **Never git checkout -- a shared file to tidy up.** You cannot know what else is in it. - **Section numbering is a shared resource.** Two writers each taking "the next free number" collide invisibly until somebody greps for duplicates. This entry's number was chosen after checking.

Reconciled: no duplicates, continuation-check 0, and the collision resolved in favour of the concurrent agent's entry, whose work was genuinely new.

A64 — ingestion tooling BUILT, two never-measured properties proved — and my own “the document promises a tool that does not exist” is WITHDRAWN as false.

The correction first, because it is mine. I recorded that the ingestion document **promises a phase-5 tool that does not exist**, calling it worse than an honest gap. **That is false — the tool’s name appears in none of the three section documents.** They never named it.

The real defect is adjacent and subtler: phase 5 was **the only phase naming no tool at all** while every other phase named one, so rows were hand-minted with nothing to point a reader at. The agent documented the tool it built, then corrected **four NOT BUILT rows its own work had made false** — the mirror of the defect I thought I had found.

The tool reproduces the existing rows exactly — re-deriving every id *and* re-rendering each row through its own writers, byte-compared: **90/90 at baseline, 196/196 by the end. Nothing in the existing rows had to change.** *If it could not reproduce rows that already pass the gate, the tool would be wrong, not the rows.*

TWO PROPERTIES ASSERTED IN PROSE AND NEVER MEASURED — both now are. The gate had a mutation proving it **refuses** a private quote, but **nothing asserted its report WITHHOLDS the quote it caught** — the whole point of the rule, since *a checker that prints the secret is a second leak*. And the **never-public guard had never fired:** an exempt row with no proof, now exercised for both private trees plus a measurement that **no override-shaped flag exists**.

The staleness hole was real and the first design had exactly it — an interval-driven run cannot see a source changing *inside* its interval. Fixed by comparing every live capture on every run, so **latency is the schedule’s period, not the material’s interval**; proved both ways, **22 mutations, 22 caught**. The decay report **refuses to write itself as Markdown inside the section**, because a section check scans Markdown for claim tokens — *it would turn the gate red for the crime of reporting decay*.

A live-ledger defect found and deliberately NOT fixed: a capture id derives from the **content hash alone**, so two materials pointing at one upstream yield an identical id, and last-row-wins left one material **owning no capture — that orphan is staleness-checked by nothing**,

since the gate checks capture→material and **nothing checks material→capture**. Fixing it re-derives every existing id: an operator decision.

Still NOT BUILT, named: OCR for scans; selector-scoped capture; JavaScript rendering, where **a JS-only page is rc 2, never an empty success**; the capture-id fix; and boilerplate removal, a **deliberate non-goal** because *a heuristic that drops a nav bar drops a pricing table*.

A63 — commercial model RE-GROUNDED, T125 BUILT — and a staleness blind spot that kept a gate green over a stale source under 29 live claims.

THE BLIND SPOT IS THE MOST IMPORTANT THING HERE, and it is recorded as a new open question. A captured source had grown from 111,004 to 287,763 bytes while **29 active claims still rested on the old bytes — and the gate was green the entire time.** The reason is structural: the *superseded-source* rule only fires **after somebody re-runs the capture**, and the *stale* rule only after a 30-day interval elapses. **So a source can change today and nothing notices until one of those two events happens.** A staleness mechanism that waits to be asked is not a staleness mechanism. Closing it is exactly what the unbuilt scheduled re-capture is for, and that has been made explicit to the agent building it.

The repair was done properly rather than by re-pointing: re-captured with the new capture **superseding** the old, all 29 claims re-anchored by searching the new bytes for each statement's own distinguishing figure, old rows appended as withdrawn rather than edited. **Exactly one claim did not survive as true** — the one asserting the research axis was empty — and it is now marked **refuted and kept**, not deleted. The ledger moved **90 → 165 claims (136 active, 29 withdrawn)**.

The commercial model was re-grounded, and the wedge is narrower and sharper than “outcome pricing is broken”. The finding is that **every published outcome definition in the category resolves ambiguity in the vendor's favour, and none is falsifiable by the buyer.** A 2009 precedent that counting rules “*should be maintained as confidential*” establishes undisclosed criteria as the category **norm rather than its accident**.

The trial's answer to adverse selection is a refusal to claim one: it does not survive it — it bounds, screens and discloses it, and states the residual cost. The framing is exact: *a free budget-capped trial is the*

*insurance structure with the premium set to **zero**, which is the worst premium available.* Six mechanisms each carry their own cost, including that publishing a refusal rate beats filtering silently, and that **silence is never an outcome** — forfeiting the abandonment revenue every incumbent collects. What none of it fixes is stated: **until a private base rate exists there is no ex-ante difficulty measure, so the trial selects against exactly the work the platform claims to be for.**

The contingent fraction is now specified strictly below 1 BY CONSTRUCTION, following the agency result rather than negotiation, and **partial delivery is withheld, never priced** — with a bound that says if the fraction is expected to be forfeited, **the ladder is a fixed-price contract wearing an outcome label**. The counter-evidence section is deliberately **longer** than the case-for.

T125 built — and it corrected a wrong instruction on its own task line. The note said the registry to extend was the search package; it is the **passage** registry, and building it in search “*would have declared a corpus kind in the retrieval layer*”. Withdrawn by name. The stated secondary reason for deferring — concurrent editing — was **moot, because the file that needed changing was never contended.**

13 mutations, 13 caught, with a negative control — and verified in BOTH directions by two deliberate weakenings. Gutting the validator left 7 uncaught; changing **one comparison** to permit a zero bound turned **exactly one** mutation red. *A proof that only detects total removal cannot discriminate a one-comparison weakening*, which is why the second test was the one that mattered.

It opened a hole and closed it — the coverage-set lesson, third instance today. A disjointness gate asserted against a **hardcoded four-kind list**, so a row claiming the new kind would have **passed**. It now derives the list from the registry itself. And it refused a contract phrase by name: a requirement that an interval bound is “never zero” is honoured as *strictly positive*, because **a zero bound is not a tight measurement but a measurement never taken, presented as the tightest possible one.**

002 moves to 106 ticked / 37 unticked; the closure check to 20 ids / 0 unattached.

Two reds it reported and correctly refused to fix: the feature-001 registry now exits **1 on 11 R5 violations** — unregistered files under the platform section landed by concurrent work — and it **declined to register them**, because “*registering gates whose proofs I haven’t*

verified is the bluff that registry's own header forbids." That is the right call and the work is assigned to the directory's owner. A second failing test was **proved pre-existing** by restoring the committed file and reproducing it identically, and a "four such failures" figure was corrected to one.

A60 — THE COMPLETE OPEN REGISTER, 2026-09-03. Everything unfinished, faulty or incomplete across every request this session — nothing omitted.

Written because the standing rule is that no request, prompt or idea may be lost. Each row says who or what it is waiting on. **A count of open items is not a plan; a named owner is.**

ASSIGNED AND IN FLIGHT (9 agents dispatched against this register):

#	Item	Why it was open
1	Deep crawling	The operator called it MANDATORY ; only three named targets were crawled by hand and no general capability exists, so the next arriving link cannot be processed
2	ZIP archives + all mandated formats	Both explicitly requested; the format half reuses the just-adopted docs_chain, the archive half must be reproducible or it cannot be verified
3	pkg/search gating + a terms publication rule + the second answering construction site	The last ungated disclosure surface. A naive gate would take ~8,511 terms dark; the operator chose a derived rule instead
4	Constitution pin fast-forward +	Both decided and neither executed. The

#	Item	Why it was open
	content-boundary class A direction	pin is on its fourth drift; class A is 63.6% of the population with no direction evidence
5	Workshop UI for the section	Requested, and a measured gap: the data model types diagram and code while zero style classes define those roles and the frontend has no renderer for either
6	Gate coverage + attribution debt	A gate passes over three-quarters of its subject ; six more gates are enumerated by no registry at all
7	T125 and the OCR chain	The single unblocked implementation task, with sixteen behind it
8	claim.py, scheduled re-capture, link→prose	The ingestion document promises a tool that does not exist — worse than an honest gap, because a reader will try to use it
9	Commercial model re-grounding	Recorded as resting on no research; the research landed thirty minutes later

OPERATOR-ONLY — no agent can close these: - ~1–2 hours of blind human transcription. It unblocks **two** criteria, not one: the ASR accuracy figure *and* the speech-WER baseline the OCR chain needs. The sampling plan is re-emitted and waiting; the handover names exactly what to produce. - **Four visible minted rows and one nine-character**

term that survive in derived files. Both are **content decisions about the corpus**, which no tool can settle. - **The content-boundary judgement**, once direction evidence exists — bounded by the fact that **89 name rows across 8 names sit in every class**, so no class can be pardoned wholesale. - **Starting the PenPot instance** — seven long-lived containers.

KNOWN-OPEN AND DELIBERATELY NOT ASSIGNED, each with its reason: - **A one-shot residual sweep.** A withheld string inside a longer visible string is flagged **once and can never be flagged again**, because texts are collected before materialisation. *A warning that fires once is nearly as bad as none* — recorded rather than papered over. - **~239,496 characters of machine text** on already-minted rows, immutable through the registry sync; it needs an operator-run repair. - **An upstream bug in the consumed containers module** — its log reader returns **zero bytes and a nil error** on podman, so a caller sees “logs fine, container printed nothing”. §11.4.76(4) puts the fix upstream, and waiting for upstream is the precedent this project already set once and was right about. - **A pre-existing hand-rolled remote-distribution script** that duplicates what the containers module provides. It targets hosts this session could not reach — **converting it unverified would be bluff work**. - **Cross-reference rows stamped at three superseded generations**, not authorised for deletion. - **A spec contradiction:** a contract clause requires a kind be advertised that the implementation deliberately refuses, pinned by a test that exists to record the refusal. It needs a **spec change, not a code change**. - **Six of eight specification diagrams unvalidated** — the renderer fails on a deliberate two-line control, so it is could-not-determine rather than a pass. - **The deploy script tolerating a failed build step.** Offered and **not selected**; it stays, and stays recorded.

STANDING MEASUREMENTS, all still short of their bars and all honestly reported: retrieval 15/22 against 20/22; fabrications 1, so that criterion is **not met**; specs at 66/54 and 105/38; the content-boundary gate **red by design** and must stay so; **~666 baselined environment occurrences**, which are declared debt and not a justification.

A59 — the platform SPECIFICATION written: 10 pages, 48 invariants of which 8 are named **UNGATED**, and the entry condition was **CLOSED** rather than ignored.

The stub’s entry condition was not met, and the agent closed it instead of proceeding around it. It required every research finding to already exist as a provenance row; measured, there was **1 material**

and 0 claims. So it ran the section's own ingestion phases first: **9 materials, 9 captures, 90 claims — all 90 cited**, split 58 source / 32 us and **82 unverified / 8 verified**, with unverified correctly the default rather than an embarrassment to hide.

Eight of those claims it verified ITSELF rather than inheriting. It re-fetched the incumbent's contract page live and confirmed the load-bearing absences by direct count — *trial, pilot, evaluation, convenience, token, benchmark, accuracy* **all zero**. And both reference repositories' heads, captured today, **equal the commits the architecture studies pinned**, so those studies are provably current against their subjects rather than assumed to be.

THE ARCHITECTURAL POSITION — partition by ownership, NOT compromise. The two reference strategies are not averaged; they are assigned by role. Everything that **decides** is wired in code, whole-graph validated, and **refuses to boot** if any verifier, oracle, grader or evidence sink is unprovided. Everything that **does** is wired in data — printable, addressable, replaceable — and degrades through a graded capability that reaches the verdict. **The stated reason is the good part: averaging the two would give a kernel large enough to be rigid and permissive enough to boot half-wired** — the worst of both. Dependency inversion sits in **two** places deliberately: typed tokens checked at build time in the deciding plane, manifest rows swappable without a rebuild in the doing plane. One deliberate divergence from the reference: **an unresolved patch target is a boot failure, not a silent skip.**

How a verified outcome is proven, and four things make it checkable rather than described: - **owner on every fee-linked check**, so a *platform-authored* check is **not fee-linked without countersignature**. That is the contract-level answer to “the agent influenced its own tests” — the failure the landscape study measured as agents scoring near-perfect against an oracle while shipping broken code. - **Check state is restored BEFORE grading**, and the result re-measured externally from a clean root, with the untouched set asserted byte-identical. - **Four independence tiers** stated on the bundle, plus five anti-gaming instruments with a capped-check tripwire. - **A six-command recipe the CLIENT runs themselves**, ending in a digest comparison that must collapse to one. *A proof the buyer can re-run is worth more than any attestation we sign.*

48 invariants: 40 gated, 8 NAMED UNGATED — and only 2 of the 40 gates exist today. Three of the eight are **unfalsifiable by any machine** and are bounded by a **priced human process** rather than

pretended into engineering. Naming the ungated ones is the direct answer to the study's sharpest negative result: *a team with 193 scripts and 1,717 decision records still eroded on the one rule they did not gate.*

It caught its own arithmetic and withdrew it in the page. A hand count of “38 · 30 · 1” was published, then re-derived from the tables and **withdrawn by name**, with the re-derivation command printed beside it.

Twenty open questions, honestly enumerated. The load-bearing ones: nobody has established **who actually pays for verification** — the evidence points at the agent *vendor*, not its customer; **no study measures verification at repository scale**, so the central claim has no external base rate; every amount is TO BE SET; and **the ingestion document promises a phase-5 tool that does not exist.**

Q3 was recorded as open and is now CLOSABLE — a timing artifact worth noting. It states the commercial model rests on no research because the landscape's first axis was an empty placeholder. **That was true when read and false thirty minutes later** — the landscape agent finished afterwards and that axis is now fully populated. Re-grounding is assigned. *Two agents working the same corpus in parallel will each be right about a file the other is still writing.*

Two measured boundaries stated rather than glossed: the operator's token-cost premise is **refuted by the incumbent's own published evidence** and the specification does **not** build on it; and **six of eight diagrams could not be validated on this host** — the renderer fails on a deliberate two-line control, so it is recorded as could-not-determine rather than claimed as a pass.

A58 — landscape study: the benchmark ground is unsound, the wedge is narrower than assumed, and one unfilled research gap sits directly under the platform's core claim.

Recorded by POINTER only — 4,063 lines in the private module, every finding carrying a source URL, a fetch status, and a **CLAIM-or-MEASURED label**. The umbrella gets the strategic shape, not the material.

THE FINDING THAT UNDERMINES BENCHMARK-BASED CLAIMS GENERALLY. The industry's most-cited coding benchmark was **retracted by its own steward** after an audit found **59.4% of problems flawed** — and its recommended **replacement was retracted five**

months later at ~30% broken. The stated root cause is **specification, not capability.** *Any platform whose credibility rests on a benchmark number is building on ground that has collapsed twice in one year.*

Two more results that constrain the design rather than decorate it:

- Given a **formal** specification, verified generation reaches 82% — and **natural-language descriptions add nothing measurable.**

Specification acquisition, not generation, is the binding constraint. -

Agents score near-perfect against a 222-test oracle **while shipping non-functional code**, and chain-of-thought is faithful **under 2%** of the time once a model has hacked something. **A system cannot be trusted to narrate its own correctness** — which is precisely why an oracle must be denied write access to what it checks.

On the commercial thesis, the evidence is more ambiguous than the competitor analysis suggested. Four of six vendors with a

published definition make **customer silence** the billing trigger; one bills on a click; one states that negative feedback “*will not change the resolution status*”; one grants **no audit right** at all. Only **one**

documented adjustment mechanism exists in the entire category.

Adoption sits at **19% of buyers / 13% of agreements.** And the strongest cautionary case escalated 4× to ~\$1.2B while the inquiry found **no breach of any term** — the failure was scope *definition*.

The counter-arguments were steelmanned as instructed, and three land hard. Independent measurement puts savings at “*unremarkable*”

10–15%, with one study measuring –19%. Agency theory says the optimal outcome share is **well below 1**, while a pure outcome price sets it **at 1**. And every mature outcome-priced industry **selects against hard work.** *One applies to this project directly: hedged, evidence-qualified output measurably scores LOWER with human raters* — which is the cost of the discipline this whole document is written in, stated plainly rather than wished away.

THE MOST ACTIONABLE RESULT — an unfilled gap the operator

could fill and own. The strongest oracle-free verification result (75% detection at 8.6% false positives, via metamorphic testing) is measured on **single functions.** **No study measures property-based or metamorphic verification against multi-file agent deliveries.** That gap sits **directly under the platform’s core claim**, and it is fillable by measurement rather than argument. *Owning a measurement nobody has taken is a stronger position than owning an opinion everybody has.*

Commoditised, so effort should not go there: scaffolds, tool protocols, sandboxes at near price parity, orchestration frameworks, code generation, AI code review, verified generation *given a spec*, and the supply-chain attestation plumbing. **Three-valued outcomes are prior art** in metrology and program analysis — this project's novelty is in propagation and aggregation semantics, not the idea.

And the evidence layer's own lesson mirrors ours: enforcement, not availability, is the dominant variable. The one registry that **mandates** signing reaches **97.1%**; every registry that merely offers it sits **under 2%**. Attestation hardware keys were extracted for **under \$1,000**, with forged quotes accepted by the vendor's own verifier at its highest trust level.

Honest limits, volunteered: several foundational sources could not be obtained by any route; every figure from one cloud vendor's formal-methods record is **that vendor's account of its own success**; and **no lawsuit or regulator action over AI outcome billing exists**, so the legal risk is unpriced rather than low. The agent also **flagged its own ranking as wrong in-document** and **withdrew one of its own URL-log errors by name**.

A57 — decisions 49/50 EXECUTED. The acceptance test PASSED: 0 of 27 terms survive a rebuild. “2 of 27” was wrong — it is 3. Three residues stay open.

The ratification could not be a new field, and that constraint produced the right answer. The append-only log's schema is closed — unknown fields disallowed, two actions only, contiguous sequence — and re-appending under a new reason code is **correctly refused**. So ratification is a re-affirming entry per pid: same identifier, same action, **same reason code**, new author and timestamp. **48 entries appended; the first 142 lines are byte-identical to the pre-session backup and the effective state is unchanged.** Both refusal paths were demonstrated **against the real log**, with its digest verified untouched afterwards.

“2 of 27” is SUPERSEDED BY MEASUREMENT: it is 3. The third was invisible to a byte-level whole-word test and appears only through **the real production decision point**, which normalises punctuation. *A figure derived from a proxy test rather than the deciding code was wrong by one, and the deciding code is the only one that counts.* Five visible

source passages were withheld under a **new reason code naming a distinct ground** — never folded into the original findings. The cascade closed at **0**; the log stands at 197 entries and 149 suppressed pids.

The 180-second blocker was fixed WITH the measurement that justifies the new value. A full run from clean takes **241.9 s**, of which the minting stage alone takes **199.0 s** — *above the old bound*, so **the two earlier attempts could not have succeeded at any host load**. That reclassifies the earlier could-not-determine from “the machine was busy” to “the limit was wrong”. The new bound is 600 s, matching what a sibling tool has carried for the same batch through the same bridge. A second blocker was also real: the export surface could not resolve a file-plus-JSON-pointer reference, which two of the five passages carry.

THE ACCEPTANCE TEST PASSED, and it is the whole point of decision 50. A real regeneration from clean produced the rebuild — no longer a could-not-determine — and **0 of 27 terms were re-emitted, with 0 present anywhere in the regenerated index**. The regenerated taxonomy holds 8,507 live term rows and **0 withdrawn**, because *the strings were never emitted, so there was nothing to withdraw*.

All gates green: review check, real apply, an idempotent re-run changing **zero bytes**, 278 Python tests, the Go suite, redaction propagation 7/7, derived-leak 4/4, limits 15/15, both registries, and the closure check at 31 ids / 0 unattached.

THREE RESIDUES, none closed, and the second is a genuine instrument limitation: 1. **The set grew 27 → 30 and 3 of the 30 re-derive** — but those three are **collateral withdrawals caused by evidence exhaustion**, not judged unsafe. Recursing the rule onto them would suppress the corpus for no privacy gain, so the agent **stopped at the set the ruling named** rather than expanding its own mandate. 2. **A nine-character withdrawn term survives in two derived files**, inside a longer string that the exact-replacement pass had nothing to match. **The residual sweep flagged it once and can never flag it again** — the texts are collected *before* materialisation, so on any later run the row already holds the marker. *A one-shot warning is nearly as bad as none*, and it was reported rather than quietly absorbed. The sweep was **not** weakened and no length threshold was added. 3. **Four visible minted rows carry one of the terms whole-word** and the value-level closure does not name them, because its tokenisation differs from the Python rule’s. They do not affect regeneration but they **are served rows**. The tool declined to suppress them on its own authority, and so did the agent.

A56 — PenPot wired and a design system whose single source is PROVEN, not asserted. And a measured gap: the data model carries materials the UI cannot render.

Decision 3 (“do both”) executed. PenPot is a seven-service compose driven by **the existing containers orchestrator** — so §11.4.76 is satisfied with **no new orchestration code and no Containerfile**, which is what that anchor demands rather than merely prefers. Validated: the compose configuration parses at exit **0** with every variable interpolating.

Six deviations from upstream, each with a stated reason, and two are security judgements rather than taste: there is **no default for the secret key**, so a missing secret is a **refusal to start** rather than a silent weak default; and every bind is pinned to loopback, because upstream ships email verification disabled on **all interfaces** — which on a reachable host is an open-signup design tool.

Nothing was started. Seven long-lived containers is an operator decision, and the runtime claims are therefore upstream documentation plus a validated compose file — **not an observation**, and labelled as such.

The design system’s single-source claim is proved by execution. The build parses the **live** brand stylesheet and compares every generated declaration per theme against it: **84 of 84 identical**. Six artifacts derive from one token source, so a value that could be edited in two places does not exist.

Three gates, each mutation-proved in both directions — and one of them overruled its own author. The contrast gate **rejected the agent’s first graph-edge colour** at 2.56:1 against a 3:1 requirement; the committed value is the one that passed. *A gate that constrains the person who wrote it is worth more than one that ratifies them.* The round-trip mutation also demonstrated propagation rather than a single hit — one changed hex surfaced **two** dependent tokens.

The token emitter caught two real bugs during testing, both of the kind that ship silently: relative units emitting **unitless**, so a 1rem value became 1 — **sixteen times too small** — and alpha being dropped whenever the base was an alias.

A sibling generator was assessed and deliberately NOT reused, with measured reasons rather than preference — it is seed-driven where this brand’s ramp is hand-tuned, and it hardcodes constants that

contradict the live brand. Its own header calls its output a *candidate*, *not the live CSS*. **Reuse is mandatory where it fits; claiming a fit that is not there would be worse than writing new code.**

The finding that matters for the workshop UI: the data model types material kinds the interface cannot render. Zero of the 42 + 83 existing style classes define any code, syntax, diagram or template role, and the frontend carries **no highlighting, markdown or diagram dependency at all**. The component inventory adds five new entries to close that.

A correction to my own brief, and it left a real gap: I told the agent the R5 sweep is recursive over `.sh` and `.py`. That is the **workshop** registry; the **umbrella** registry is depth-1 and `.sh`-only. Its new gate is written in a third language under `docs/`, so it is **enumerated in no registry at all** — and registering it means attributing the work to one spec or the other, which is exactly the per-file judgement the standing cross-registry-attribution debt row **refuses to guess at**. Flagged as an operator decision rather than guessed.

Could not determine, stated plainly: no instance was started, so no runtime behaviour was observed; the content-boundary gate **timed out at 400 s** under host load so its post-change reading is unknown (all files written were inside the private module); and one token type's value shape was **omitted rather than guessed**, with the omission printed on every run.

A55 — SC-015 7/22 → 15/22, and the fix was NOT the widening. The reranker was a net negative. Index lifecycle fixed; 268 MiB reclaimed.

The authorized approach did not work, and that was reported instead of the number that would have flattered it. Widening the candidate window made retrieval **monotonically worse** — 11/22 at width 1, 11 at 2, 11 at 3, **10 at 5** — while the count of missing targets climbed from 1 to 9. *Depth was the wrong lever, and four measurements said so before any conclusion was drawn.*

The real cause is a rank computed from the wrong quantity. The two retrieval sub-lists were concatenated, and the fusion strategy scores by **slice position** — so the catalogue's best row entered fusion at a position determined by *how many results the other sub-list happened to return*, not by anything about the row itself. **That is also exactly**

why depth hurt: a larger candidate set pushed the offset further down. Merging the two lists by *rank* instead took it 7/22 → 11/22 and areas 2/5 → 5/5.

The second finding is uncomfortable and was acted on anyway: the in-window reranker is a NET NEGATIVE. Same binary, same generation, three runs per arm — **ON 11/22 (terms 1/12), OFF 15/22 (terms 5/12).** The cause is legible: bm25 over a window of long transcript segments ranks a two-or-three-word exact-title term row *below* them. **Two defaults were changed on that evidence** — rerank off, width 1 — each **one environment variable from restoration**, with the mechanisms and their gates left intact. *A feature built earlier the same day was measured, found harmful, and switched off.*

The ceiling is re-derived and the earlier one is superseded: 21/22, not 16/22. The window is **no longer the binding constraint** — 21 of 22 targets now sit inside the served results, and the remaining gap is ranking quality *inside* a window that already contains the answer.

A structural fact settled by the database, not by argument: the passage store holds **zero knowledge-kind rows and zero knowledge-kind vectors**; the catalogue is a separate in-memory index with **no embedder**. So an area, term or question **can only ever be retrieved by the lexical leg** — confirmed per-leg, the semantic leg returned the target for **0 of 22** queries.

SC-006: the trade was measured and refused. Widening cost fused latency +28% and lexical +139% at identical host load while buying nothing — *“not a trade, a loss”*. **Honest boundary, stated rather than glossed:** a quiet-host after-figure comparable to the 244.5 ms baseline **could not be obtained** — another agent held the machine at load 13–23 and the gate’s own control endpoint moved 5.9× — so the end-to-end figure is **UNDETERMINED, neither a pass nor an attributable regression**. What is measured: the one always-on addition costs **+0.28 ms, 0.014% of budget**.

The lifecycle root cause is FIXED. A boot now **reuses** a generation whose corpus hash matches, after putting it through the same verification a new one gets. Verified by me on the live server: *“generation 67 REUSED ... NO new generation was minted, NOTHING was re-embedded and NO cross-reference was re-derived”*, with vector indexing completing in **32 ms** where it previously spent 5m29s failing. **Cross-references carry forward at 49,560 edges in 0.51 s against a**

55–62 s derivation — ~110× — behind *five* required conditions, using replace rather than ignore semantics because ignore silently copied **0 of 2** fixture edges.

The prune: 66,781 rows deleted, 2,478 kept, and I verified 268 MiB reclaimed (329.4 MB → 48.1 MB) with integrity checks and full serving verification on both sides of the deletion.

A NEAR-MISS the agent caught and disclosed itself. It first wrote a **313 MB backup of private curriculum content into a directory inside the PUBLIC umbrella that is not git-ignored.** It moved the file outside the repository within a minute; I verified independently that **no backup path appears in the umbrella's status.** *The safest disclosure in this session was the one an agent reported on itself.*

It also declined work it was qualified to do, for the right reason. Handing off the search-leg gating, it noted that applying the publication gate to the catalogue would take ~**8,511 terms and 3 of 5 areas dark**, removing 12 of 22 benchmark targets and reverting 15/22 to roughly 5–7/22 — and that this **should not be landed silently by the agent being measured on that benchmark.** Recognising one's own conflict of interest is not a behaviour a rule produced.

A54 — the answering “regression” is NOT a regression, and all five candidates I named were disproved structurally.

It did not happen today. Reproduced at generation 67 and decomposed into two steps, **both on or before 2026-09-02:**

- 1. 17 → 12 — corpus growth at the RETRIEVAL layer, before any model runs.** The corpus went from 1,101 to 2,478 passages; two kinds grew from 44 → 1,172 and 2 → 251. Nine questions lost top-1 discrimination against **1,377 new competitors** and are now refused by the margin floor **before any model call.**
- 2. 12 → 3 — the answer-against-question layer.** An in-tree A/B at the *identical corpus hash* shows **perfect conservation:** exactly 9 questions moved to that layer's exclusive refusal signature while the other two buckets held constant. The reproduction matched all three counts exactly.

Every candidate I supplied was tested and every one is FALSE, each on structural rather than argumentative evidence: the reranker and the vector restore **cannot reach the answering path at all** (dependency listing returns nothing in either direction; that path uses BM25, not vectors); the publication gate has **zero dependency** on the

answering package; the body trim is uncommitted, acts only at minting, and is **doubly moot** because the live registry contains **no knowledge rows whatsoever**; and the index rebuild moved the counter without changing a single corpus member.

The candidate the previous agent named but explicitly did not test is also false — that layer *is* present and has been present at **every** measurement including the 17/24 one, so it is a constant across the whole movement. And its absence would have made answering **more** permissive, not less.

Citation resolution is healthy and the deficit is entirely refusal. Uncited answers: **0**, and that state is **structurally impossible** — validation rejects an answered outcome carrying no citations.

An honest denominator, corrected: the figure is **2 answered in 23 measured with 1 undetermined** — *never “2 in 24”*. One question hit the provider timeout under host contention; scoring it the way the reference run did reproduces **3/24 exactly**.

It fixed a blind spot in the instrument that produces the headline number. A row that answered with zero citations was classified but **incremented no counter** — it vanished from all three figures with **no exit-code movement**, so a figure over 24 asked was indistinguishable from the same figure over 23 counted. The counter is now printed **always, even at zero**, a non-zero value is a **finding**, and a reconciliation returns **could-not-determine** if the buckets fail to account for every question asked. **Nothing was weakened; the gate got strictly stricter.** Its paired proof catches 4 of 4, and one mutation runs a **deliberately weakened copy** of the gate and requires the reconciliation to refuse it — so the control cannot be inoperative.

THE OPERATOR DECISION this surfaces, already documented at the configuration site: the verification layer **costs 9 answerable questions and takes fabrications from 11 to 1**. That is the trade, and it is a judgement rather than a defect.

A53 — “The Platform” section BUILT as a living structure. **Provenance is a data shape, not a convention — and the privacy rule is enforced structurally.**

A single spine, six phases — admit, capture, normalise, section, claim, publish — where **only normalisation branches by material type**. Recordings, documents, repositories, links and conversations each enter through the existing pipeline: both ASR engines, the chunker, the

transcript builder, the notes extractor, all four capability probes, the section splitters, the identity minter, and the corpus ingest — which **already sweeps the docs tree, so the section becomes searchable with no new wiring at all.**

Provenance is three append-only record types with DERIVED ids — each id is a truncated hash of the row's own content, so writes are idempotent and **a hand-edited row stops recomputing its own id,** which is how tampering announces itself.

Two fields carry the discipline this workstream most needed: - attributed_to, distinguishing source from us. That is the structural answer to the risk I flagged when dispatching the research: *it stops a vendor's claim being laundered into our own voice.* - **A four-valued verification state** — verified, refuted, **unverified as the honest default,** and unverifiable **which requires a written reason.**

The privacy rule is structural rather than advisory, and the detail is the good part: the gate refuses a literal quote from any non-public material, and reports the violation WITHOUT echoing the quote it found. A checker that prints the secret it caught is a second leak. Capture additionally refuses to mark anything under the private corpus as public, **with no override flag.**

Staleness needs no separate mechanism: re-capturing IS the check. Same hash and the confirmation timestamp moves; different hash and a new capture is written naming what it supersedes, with every claim resting on the old one flagged. **Supersession falls out of the arithmetic.** And the gate **never touches the network** — fetching belongs to the capture step, whose failure is a 2 — so the check cannot go dark when offline.

The docs_chain assessment came back SPLIT, and one half is an integration hazard worth carrying beyond this section. It is **not** the vehicle for fetching — a repository-wide search finds **zero** files using an HTTP client, and its kind set is closed at nine with no URL kind. It **is** the right vehicle for regenerating derived formats. **But its exit contract uses 2 to mean CONFLICT, where this project's universal contract uses 2 for COULD NOT DETERMINE** — so a naive pass-through would silently downgrade a confirmed failure into an **undetermined one, which is exactly the unsafe direction.** That has been relayed to the agent adopting it, with instruction to translate the code explicitly and test the mapping.

Its central idea was adopted and then verified BY EXECUTION rather than by reading: the new normaliser is a byte-for-byte reimplementation, and its proof **compiles the original and compares hashes across six adversarial fixtures — 6 of 6 identical**, so a later migration invalidates zero stored hashes. That proof is not decoration: **it was observed failing while its own harness had a bug**, which is the only reason to trust it now.

Gates: 22 PASS / 0 FAIL / 1 DEBT (up from 20; the sweep now covers 22 files), --run-proofs **30 PASS / 0 FAIL**, and the provenance prover catches **17 of 17 mutations — every mutation DATA rather than a gate edit**, including a negative control and an availability mutation that must return 2 rather than a quiet 0. The single strict-mode failure is the **pre-existing** cross-registry debt row.

The honesty detail worth copying everywhere: the ledger currently holds 1 material, 1 capture and 0 claims — and the gate PRINTS that emptiness as a note on every run, so a young green is never mistaken for a mature one. Four section bodies are empty with explicit not-yet-written markers, and the not-built list names deep crawling (which the brief calls mandatory), link-to-prose extraction, scheduled re-capture, ZIP export and PenPot.

A52 — architecture study delivered. The two reference projects are OPPOSITE strategies, and four of their mechanisms should be adopted HERE, not just in the new platform.

Studies live at [workshop/docs/the-platform/research/architecture-*.md](#) (private module, ~3,300 lines, every claim cited to a file and construct, both projects pinned to a commit SHA with a fetch timestamp, scratch clones outside the tree and **not vendored**).

The headline reframes the operator's own brief. The two projects named as “light, decoupled, modular” achieve it by **opposite** strategies: one runs a small kernel that validates the entire dependency graph and **refuses to boot** when a capability is missing — lightness enforced at startup; the other runs a kernel under **1% of its codebase**, permissive at runtime, and compensates with roughly **fifty architectural gate scripts** — verifiability enforced by inspection. **They are not rivals: the second consumes the first behind a single seam.** The operator effectively named both halves of one answer.

FOUR MECHANISMS WORTH ADOPTING IN THIS REPOSITORY, independent of the new platform:

1. **The independent oracle, whose refresh path is DENIED WRITE ACCESS to the thing it checks.** Their rule, and it is exactly this project's own doctrine made structural: *model prose and tool-result text do not prove an external effect*. We assert that; they **enforce** it by construction.
2. **Graded capability instead of boolean probes** — enforcement is full | partial, a probe answers unusable, a selection answers unavailable. **This is the direct remedy for the /usr/bin/whisper trap** that has bitten this tree repeatedly: a name on PATH is not a capability, and a boolean cannot say so.
3. **Skip is not a pass — enforced.** Their aggregate **fails** on a skipped gate, and known debt stays **printed as non-blocking** rather than allow-listed into invisibility. This session recorded a pre-push run at “0 SKIPPED” as evidence precisely because a skip would not have been one.
4. **Invariant companions** — cross-cutting contracts, glob-discovered and ambient in every test, expressing what no unit test can, at **zero authoring cost per test**.

THE MOST USEFUL RESULT IS A NEGATIVE ONE, and it is a warning aimed straight at us. The larger project's central architectural claim — that its core depends only on interfaces — **holds**, verified across all 8 core packages and 39 edges. **But nothing gates it, and there is exactly one violation.** A team with 193 scripts and 1,717 decision records **still eroded on the one architectural rule they did not gate**. *Discipline does not substitute for a gate; it only delays the erosion.*

The agent corrected its own draft twice, and both corrections cost it a recommendation: - One project's own architecture prose **over-generalises** — several things it presents as extension seams are registries with no implementations. The machine-generated, assertion-guarded map is accurate; **the hand-written architecture document is the marketing version**. *Prefer a project's generated artifacts to its prose about itself.* - **The composition mechanism most worth copying is not shipped** — all nine call sites sit under an experimental/ path. The recommendation was **downgraded from “strong adopt” to “adopt the mechanism, but do not cite this project as proof it scales.”**

Explicitly rejected, with reasons rather than taste: 250-package granularity; per-file 100% coverage (though the `path:line:col` reporter and the *probed* exemption are worth salvaging); a monkey-patched registry; one project's absent permission model — coherent for a local developer tool, **unavailable to a platform that must be trusted to verify its own output.**

Could not determine, and left open: whether that single architectural violation is reviewed or drift; whether a plugin path works end to end (not installed); and whether one config output matches its documented base — recorded as *documented*, *not observed*. **Nothing was built or run in either project.**

A51 — competitor study delivered (private). The operator's own read was PART WRONG, and the correction sharpens the strategy rather than weakening it.

Recorded here by POINTER only — the analysis lives at `workshop/docs/the-platform/research/competitor-eview.md` in the **private** module, 679 lines over 65 evidence rows, every claim carrying its source URL and fetch date. This umbrella is public; it gets the shape, not the content.

All four of the operator's hypotheses were tested against primary sources, and three came back SPLIT rather than confirmed. That is the value of the exercise: a competitive analysis that only ratifies its sponsor's priors is worthless, and this one did not.

- **The “token usage” concern is REFUTED.** The word appears **zero times** anywhere on their site or in their contracts. What was read as a token hint is an observability feature — per-model cost *reporting*. Their subscription terms fix fees in an order form with **no metering, no overage, and no model-cost pass-through clause.** The unbounded-cost worry does not survive the contract.
- **“They do not talk pricing” is half right in a way that matters:** confirmed for their own site, **refuted overall** — they publish price floors on third-party directories. So “*we publish and they never do*” is not an available position, and that opening was **downgraded** in the ranking.
- **“Nothing on evals and verification” splits, and this is the key finding.** *Refuted* on verification — they make a genuinely specific claim about independent cross-model review of every change. **Confirmed and worse on evals:** zero occurrences of eval, benchmark, accuracy or any metric. **Every claim is a described process, never a measurement.** And their warranty section runs

opposite to their marketing — platform “as is”, implied warranties disclaimed, the customer explicitly responsible for reviewing outputs, and the only contractual remedy is availability credits.

- **The trial gap is CONFIRMED and sharper than stated:** no trial anywhere, and termination is permitted **only for uncured material breach** — no termination for convenience. *A customer disappointed by output quality has neither an exit nor a remedy.*

Why their proposition works — the part we must match. Not any single line, but **narrative coherence:** one claim restated as positioning, capability, process, principle **and contract**, with nothing fighting anything else. Most instructive, they **convert the absence of case studies into the proof itself.**

Ranked openings, by defensibility: publishing **real evaluations** first — they have no measurement and their own warranty language makes it awkward to start claiming one; then a **budget-capped trial with a genuine exit**, which strikes both their hardest edges at once; then contractual definition of “outcome”; then verifiable legal identity.

The agent recorded two of its own corrections rather than quietly dropping them — a first pass wrongly concluded they had never published a price, and wrongly dated their repositioning from legal-document dates alone.

One inference is flagged as an inference and marked not for external quotation — corroborated across two registries but **not a filing**. That distinction is the discipline: *corroborated is not confirmed*.

Openly could-not-determine: whether their platform exists, works, or is deployed anywhere — **no verdict offered**. Also unknown: real subscription price, real headcount, churn, and whether any subscription has ever sold. Two sources returned 403 and an archive service went offline mid-crawl; both are recorded rather than silently omitted.

A50 — governance re-measured across the fleet. Two recorded claims were FALSE, not stale, and the constitution pin has drifted a THIRD time.

design-toolkit’s GitLab mirror was never unmeasurable — the carrier was simply wrong. It records that this checkout wires up no GitLab remote, so neither visibility nor lag can be measured. **A gitlab remote already exists**, added by that submodule’s own commit.

Measured read-only, with both objects already local so no fetch was needed: GitHub **public**, GitLab **private**, and the lag is **6 commits, 0 divergent** — an ancestor relationship, not divergence. The long-quoted “5 commits” is **superseded by 6**: GitHub advanced one; the mirror has not moved. **No remote was added and no configuration changed.**

Provider-side CI (G4), measured across 22 repositories and 40 upstream rows: 1 CONFIRMED · 6 UNVERIFIED · 2 HISTORICAL · 30 clean. The single confirmed trigger is the legacy-built site with **zero workflow files** — so there is **no file-level remedy**, exactly as recorded. The production Jekyll site reads **HISTORICAL, not confirmed**: its runs are a fact about the past and **not a claim that a push today triggers one**. Its publish workflow was **not touched**. Six rows on two non-GitHub hosts are **UNVERIFIED for want of an adapter** — rc-2 material, and **never a pass**. Two items surfaced for the operator: 21 repositories have Actions enabled with **zero** workflow files, and the historical runs have no setting readable today that explains them.

verify-submodule-remote-sync is RED again — 11 CURRENT / 1 DRIFT. The constitution pin no longer equals its remote, **the third time in three days**, with the direction **UNDETERMINED** because the remote object is absent locally and no fetch was run. The carrier’s own standing warning is now proven three times over: **the pin is a recurring operator decision, not a task that completes.**

audit-hardcoded-paths — the agent read 1, I re-measured 0. Its red predates my emitter fix (A46); re-run cleanly it is **exit 0, “no machine-specific hardcoded paths”**. The fix holds.

Figures corrected in all four carriers, each withdrawn BY NAME: check-registry 41 → 43 → **45 PASS**; --run-proofs “**exits 1, 54 PASS / 5 FAIL**” → **exit 0, 65 PASS**, with all five previously-failing rows verified individually; env audit to 567 allow-listed / 666 baselined over 2,247 files; content boundary 11,158 → **11,878**; the sweep to **173 PASS / 96 FAIL / 2 ERROR of 271. C5 lockstep verified 4-then-1 with the cascade at exit 0**, edited as one artifact across three cycles — head split, shared tail edited once, recomposed.

Also corrected, and both were ACTIVE MISINFORMATION rather than mere staleness: the carrier warned that a deploy change was uncommitted and that “a fresh clone runs the other three nowhere” — it is committed, byte-identical to HEAD, and a fresh clone runs all four. And a test helper described as untracked **is tracked**.

The content-boundary decision packet is PREPARED, not decided — nothing judged, allow-listed or re-baselined, and the gate still exits 1 by design. The full log was parsed and **all 11,878 rows attributed by path — complete, not sampled**, matching the gate’s own total exactly:

Class	Rows	Share
A — spec trees inside this public umbrella	7,550	63.6%
B — public reusables extracted from the private tree	2,666	22.4%
C — governance prose propagated outward from these carriers	1,310	11.0%
Genuinely unassessed remainder	352	3.0%

The gate cannot tell direction, so direction was measured independently — by git first-commit timestamp. The four carriers **pre-date every sampled private counterpart by five days**, which makes class C the cascade working rather than a disclosure; and one public library pre-dates the private staging copy that supplies **93%** of its matches.

Two honest boundaries that stop this from being over-claimed.
Class A — the largest class — has NO direction evidence and was not probed. And **11,878 – 232 is not a valid remainder**: the judged set was 216 rows from one private module and 16 from another, while today’s population is 97.6% from a *different* module. The two sets barely overlap, so the arithmetic anyone would reach for is meaningless — **and it was refused rather than performed.**

89 name rows across 8 distinct names are NOT cleared, and they sit in every class — so no class can be pardoned wholesale without pardoning name rows. That is the constraint any re-baselining decision must satisfy.

A49 — decision 42 EXECUTED: all six landed. T040 is TICKED. A fourth defect was found by re-measurement — a suppressed passage had survived verbatim in a second artifact.

The refusals of A41 were cleared by fixing what they complained about, never by loosening them. Verified by me: the append-only log holds **142** entries — 10 under the earlier decision and **132** under this one (84 passages across six findings + 48 derived rows). `verify-redaction-propagation.sh` exits **0** and its prover catches **7 of 7** mutations. **T040 ticked; feature 001 is now 66/54.**

The defect nobody was looking for, found by the agent re-measuring its own work. Export application grouped passages by the artifact path *recorded on the passage*, and only sidecar-backed artifacts were residual-swept after writing. A suppressed passage was correctly removed from the transcript **and survived verbatim in a second document**. The cause is a category error worth naming: **source_ref.path records where a passage was extracted FROM, not where it was rendered TO**. Every plain artifact is now swept for every rendered suppressed text, and residual-swept after the write. Minted rows are excluded from that sweep on purpose — *a two-character “text” must never drive a global replace*.

Item 1 — the taxonomy surface (85 → 0). The pass skipped any record type outside one name list, so two proposal row types were copied through untouched. **A third cause surfaced while measuring:** an area’s external key is *derived* from its member terms (measured: 469 of 495 satisfy the derivation), so unlinking a term left its slug embedded in the key. Nine area rows re-keyed on their own id. **Honest boundary the agent volunteered:** one of the newly covered fields held **zero** withdrawn terms on this tree — *the uncovered path was real and its yield today is zero*, and both halves are stated in the code rather than only the flattering one.

Item 2 — short terms, and the forbidden fix was not taken. The evidence decided it: of 34 residual reports, **25 were the withdrawn string inside a longer, legitimately visible token** and 9 were genuine. The remedy is **structured, field-aware, whole-term matching on the decoded row**, reporting the JSON member path so a finding is actionable without returning to the corpus. Generalised to whole terms rather than single tokens — *one boundary rule at every length, instead of a rule that bounded 23 terms and left 3 as substrings* — and decoding closed a JSON-escaping hole the raw-byte sweep had. **No length threshold was introduced**, which was the forbidden shortcut.

One genuine residual remained and its cost is recorded loudly: a term row whose own *name* carries a withdrawn term. The new rule withdraws it and is iterated to a fixed point — **at a real cost of index completeness: that row still had 3 live evidence passages**, counted separately rather than folded into the totals.

Item 3 — the cascade now TERMINATES AS A PROPERTY, not as an observation. Suppressing a row moves its own text onto the withheld side, so each pass produced a new smaller finding and the operator was never shown the true size of the decision. The closure iterates and is bounded by construction: the working set only grows, and a round that grows it by nothing returns. Fixed point at **round 2, 48 rows**, cross-checked against an **independent** computation and identical. The earlier “29 of 9,144” is **superseded** — 48 is the figure with all 84 in scope.

Item 5 — B5 is MET, and the property was REDEFINED rather than patched. The freshness check was defined on **mtime**, which any atomic writer moves without changing a byte — that is what every careful writer in this tree does, including the redaction package itself. **The verdict is now content (SHA-256); the mtime move is still measured and printed as a NOTE.** Verified by me: exit 0, with the note explaining that the file moved after the review but its digest is unchanged, “*so this is an mtime move and not an edit.*” The trade is stated in both directions — a review recorded before an edit that was later reverted now reads fresh — and a paired mutation proves a **backdated** edit that an mtime rule would call fresh is still caught.

Residual re-measured across all seven artifacts after the write: 0 / 0 / 0 / 0 / 0 / 0 / 0. Backup is a **real copy** — distinct inodes, link count 1 — with a 50-line manifest re-verified clean *after* the apply.

TWO THINGS THE OPERATOR MUST RULE ON — the agent did not decide either: 1. **The 48 derived suppressions were made under this decision’s umbrella.** The tool still refuses to suppress derived rows on its own, but landing the six *required* closing the cascade, so it was recorded explicitly under its own reason code, is in the append-only log, and is reversible. **Confirm this is what was intended.** 2. **A regeneration from clean re-derives 2 of the 27 withdrawn index terms — from passages that are still VISIBLE**, each with 3 visible occurrences, not by reading withheld text. 25 of 27 do not come back. **Whether those passages must also be withheld is a review decision about the corpus**, which no tool can settle.

Not closed, and stated as could-not-determine rather than glossed: a full regeneration from clean is **rc 2** — one stage times out against a fixed 180-second limit, twice, including with a warmed build cache, so the row-level rebuild was never produced. The timeout was not changed. And the 2-of-27 finding is **not** registered as a defect, because registering one is a governance act left to the operator.

A48 — NEW WORKSTREAM opened 2026-09-03: “The Platform” section in the private workshop module. Recorded here by POINTER only.

The request is recorded in full at `workshop/docs/the-platform/OPERATOR-REQUEST.md` — inside the PRIVATE submodule, deliberately. It carries the operator’s commercial thesis and their competitive reading of a named company. This umbrella is **public**, so it gets the path and the shape of the work and **nothing of the content**. That is the standing rule applied to the operator’s own strategy, not just to third-party material: *naming a private path is fine; copying what is inside it is not.*

Shape of the work, at a level safe to state publicly. A new, continuously growing section of the workshop that ingests every arriving material — recordings (fully transcribed), links, repositories, codebases — and folds each into an exhaustive research and specification body covering technical implementation, market analysis, competitor study and architecture. Mandatory deep crawling of named targets and their reachable codebases. Deliverables span documentation, proof-of-concept implementations, diagrams, plans, full UI/UX wireframes, downloadable archives, and an extension of the workshop’s own interface for browsing it. Process discipline is explicit: machine evidence, the heaviest available anti-bluff posture, SpecKit plus the superspec bridge, and heavy subagent fan-out.

Four research agents dispatched immediately, writing only into the private module: a competitor analysis from primary sources, an architecture study of the two projects named as the standard to match, the section skeleton with its ingestion and provenance design, and a landscape study of the field.

Every agent was given the same two non-negotiables, because this workstream is unusually prone to both failure modes: **cite the source and the fetch date for every external claim, and distinguish a vendor’s claim from a measured result every single time.** A benchmark number in a company blog post is a claim; the same number reproduced independently is evidence. The landscape brief

additionally requires the **strongest counter-arguments to the operator's own thesis** to be steelmanned — *a research document that cannot argue against its sponsor is marketing.*

CAPABILITY BOUNDARY, measured rather than assumed — one named requirement cannot be met on this host. All three crawl targets answer **200**; the transcription stack is installed; **OpenCode** resolves; **superspec** is present. **PenPot is NOT reachable** — no binary, no MCP server, no plugin. Design artifacts will be produced in an importable, tool-neutral form and **the gap stated rather than papered over**. Closing it is an operator decision: install and wire PenPot, or accept the neutral form. **Do not silently substitute another tool and report the requirement as met.**

Where to resume: the four agents' outputs, then the specification pass. The section is a *living* one by construction — it is designed to be extended by each new chapter rather than written once and left to rot.

A47 — the answering path GATED, and the decline path really did leak. Three findings nobody had named. pkg/search is now the last hole.

Reproduced before anything was changed, which is why the numbers mean something. Driven through the real pipeline with a marker standing in for withheld text: a **decline** shipped **203 characters** of a withheld row; a citation quote shipped **127** for a pid that was *retrieved but never admitted*; and the answer text carried **71 characters verbatim** with generation actually invoked. **The system declined to answer and disclosed the material in the refusal.**

The mechanism is a return that outruns its own guard. The closest-hits assignment sits on the early branch and **returns before the kind gate forty lines below is ever reached**. The reproduction set the verification flag on the document and *it changed nothing* — the flag is read on a path that had already returned. **A guard placed after an early return is not a guard.**

Three findings the brief did not name, each worse than the one before: 1. **Four knowledge kinds — 75 rows — were never in the gated-kinds set at all**, so that layer never covered them in *any* configuration. 2. **The running container's argv disables that layer outright**. So generation over knowledge areas and terms **runs in production today**, and the gate everyone assumed was holding was

not engaged. 3. **A second, wholly ungated construction site:** a separate command mounts its own answering endpoint and bypasses the shared wiring entirely.

Serve-time was chosen over index-time, on two measured grounds — and the second is the one worth keeping: index-time filtering fails in the **unsafe** direction, because a *revoked* review would leave a row disclosable until the next restart. (The first: the contract requires retrieval to *extend* over these kinds, and dropping rows would repeal that while moving the very counts abstention is calibrated against.)

One gate, not two. The existing publication gate was passed through unchanged, and a test asserts **row-by-row agreement** with the passage endpoint. *Two gates over the same data will diverge; the test is what makes “reuse” a fact rather than an intention.* What it does **not** protect is stated in the file header rather than left to be discovered — including that a text *edit* after indexing is not reflected until rebuild, while both **suppression** dimensions are checked live on every request.

Six mutations, all observed RED, covering each exit independently plus fail-closed defaults. **One was redone:** its first attempt failed to *build* rather than fail the *test*, and **a mutation that does not compile is not an observed RED** — it was rewritten as a clean deletion. One exit’s mutation showed the answer text leaking **while the quote stayed withheld**, which is positive evidence the exits are gated *independently* rather than by one shared accident.

Behaviour: abstention is bit-identical — the unanswerable class is unchanged across every count, and nothing runs before the first layer, so that verdict cannot move. **Leaked excerpts went 17 hits / 114 characters → 0 / 0.**

One real behaviour change, flagged loudly rather than buried: a single answerable question moved from *answered* to *unavailable* because its admitted set contained a withheld row. Today that is a no-op in production, since no knowledge rows are served. **After the authorized ingest it will not be** — 8,553 term rows enter the corpus, and any question admitting one now refuses. Correct direction, real cost, stated in advance.

The new code is unavailable, never declined, and the distinction is principled: a decline asserts something about the **corpus**; this asserts something about the **deployment**.

Withheld surface: 9,142 rows / ~273,700 text + ~262,700 machine-text characters — and **zero** ordinary corpus rows, so there is no over-reach.

pkg/search/ is enumerated and untouched — the last ungated surface. The lexical leg filters on redaction **only** and returns the **full row text** plus a snippet; the catalog leg is redaction-aware but **not** publication-aware, and **5 areas pass its filter of which only 2 are reviewed.** The precise defect: the publication gate is constructed and reaches three call sites, **but is passed to neither search nor suggest.** Assigned to the agent holding that package, with the explicit instruction to report retrieval figures *before and after the gate* separately from *before and after the widening* — conflating the two would make both unreadable.

Honest boundaries volunteered: whether the production answered-and-cited metric moves is **UNDETERMINED** — that measurement used the extractive provider, not production's model, and is explicitly **not** comparable to the 17/24 → 3/24 figure. The identity of the one question that moved was not recorded, only the count. The frontend's handling of the new code was not verified. And the demo fixture corpus will now withhold its invented rows' text unless reviews are supplied — no test asserted on it, the suite is green, and the demo output will differ.

A46 — umbrella pipeline verified: 7/8 gates, gate 0 RED and FIXED at its cause. And gate 6 has been validating a six-day-stale artifact.

Gate 0 went red from this session's own work, and the fix belonged in the emitter, not the artifact. One occurrence, one file: the accuracy plan recorded an **absolute** transcript path. The committed version is repo-relative — the temporal re-emit (A39) wrote the absolute form back, because the emitter records whatever path it resolved and that resolution is anchored on an absolute root. **Patching only the JSON would have let the next re-emit reintroduce it,** so the emitter now keeps the absolute form for *reading* and records a repo-relative form in the plan, with a transcript legitimately outside the repository recorded as given rather than mangled. Re-emitted and re-measured by me: `audit-hardcoded-paths.sh rc 0`, and the plan is intact — 30 windows, occupancy [3,3,3,3,3,3,3,3,3,3], estimator-unit note carried, path relative. Emitter selftest **0**.

The finding that matters most, and nobody was looking for it: gate 6's milosvasic.ru half validates a SIX-DAY-OLD build. Jekyll cannot build on this host — the binary is absent from PATH, `bundle exec jekyll` exits **127**, and `bundle check` fails on missing gems. The deploy script

routes that into a **warning**, so a dry run printed *“1 build step(s) failed and were tolerated”* and still **exited 0**. The generated site directory is dated **2026-08-28**, and the Playwright config serves exactly that directory. **So 239 passing tests include a set asserting against content six days out of date.** *A green suite over a stale artifact is a worse outcome than a red one, because nothing signals it.* **Production is unaffected** — that site publishes through its own server-side workflow, untouched. Remedy is an environment change and is not taken here.

Two CLAUDE.md claims measured FALSE and handed on for correction: - The “deferred live specs are uncommitted” warning. Measured: HEAD is **byte-identical** to the working tree and already names all four specs; it landed **2026-09-01**. **A fresh clone does run all four** — the carrier’s warning misinforms the reader it was written to protect. - `_tests/env.js` is now **tracked**; the carrier still calls it untracked.

Everything else passed, and two traps did NOT apply here — worth recording so they are not re-litigated. Playwright is a **genuine** pass, not a could-not-determine: browsers are installed, `CHROME_BIN` was neither needed nor set, and the ChromeHeadless trap belongs to a *different* suite — this package declares exactly one script. The pre-push hook is installed, executable, and **byte-identical to the installer’s own heredoc**, verified by comparison rather than by its presence. The three deferred live specs were run against production: **85 passed, 1 flaky retried green**, no network faults, so nothing to classify as rc 2.

Registry red is transient, and correctly attributed: `verify-check-registry.sh` read **42 PASS / 2 FAIL**, both R5 UNREGISTERED, both from docs-chain scripts created minutes earlier by an agent still running. **R5 doing its job**, not a regression to record.

Side effects disclosed rather than discovered later: the failed pre-push run left one evidence image modified, because the evidence-guard restore runs only when gates pass; and the dry run regenerated **37 files in one site submodule and 14 in the other** (localized PDFs, sitemaps). Nothing committed, nothing pushed.

A45 — spec 002 triage: 100 → 104 ticked of 143. The file's own header was wrong, and SC-015's failure is one kind, not a spread.

The file was misdescribing itself. Its header claimed *142 total, 100/42*. T143 had existed for hours and the block was never updated. Now **104 ticked / 39 unticked of 143**, nothing un-ticked, closure check **19 ids / 0 unattached**, `continuation-check.sh` exit **0**.

Four ticks, each evidenced: a benchmark gate that asserts a resolving locus over **every** hit of a run (560, not a sample) with a violation failing the whole run, proved by mutation and three-valued by two more; a benchmark fixture whose expectations come from the catalogue endpoints and **never from a ranking** — which is what stops a benchmark from grading itself; a web client whose kind chips come from the server's advertised kinds rather than a hardcoded list, at **97/97** unit tests; and a limits gate at **15/15** naming both the fabrication rate and the topically-related-but-non-answering case.

Fourteen stale claims withdrawn by name, including a gate-coverage table that still read **18 gates** — it had been missing G-KG-1-changed **for the same blind-extractor reason as the closure check**. One instrument's blindness had propagated into a hand-maintained table, and both were wrong in the same direction.

The finding that changes work in flight: SC-015's shortfall is not spread across kinds, it is concentrated in one. At top-5 — question 5/5, area 3/5, term **0/12**. Twelve of the twenty-two targets are terms and **not one lands**; 8/22 is simply 5 + 3 + 0. **An aggregate that improves without moving term would look like progress while the actual failure is untouched.** Two candidate causes are now under test rather than assumed: whether term rows are reachable by the leg that is supposed to retrieve them at all — the served registry holds **no knowledge-kind rows whatsoever** — and whether a two-to-five word label can rank against 300-character passages in the same space, which would make it a **representation** problem that no window width fixes.

T068 is a genuine COULD NOT DETERMINE, and was reported as one. A published latency column could not be reproduced: one run returned FAIL 1 / UNDET 4 with the **control row itself undetermined**, and the container afterwards reported being up 39 seconds; a second returned UNDET 7 with every endpoint connection-refused, at host load 20–22. **Neither refutes nor confirms the published column.** *When*

the control is undetermined, nothing measured beside it is evidence — and with ten agents saturating the host, that is a measurement about the machine, not the code.

T070's contract half is NOT MET, and it is a spec contradiction rather than a bug: §4.1 still requires a kind be advertised that the implementation **deliberately refuses**, pinned by a test that exists precisely to record the refusal. Same decision as two sibling tasks; it needs a spec change, not a code change.

The single unblocked OCR implementation task is T125 — declare the on-screen text kind. Re-verified: the identifier appears nowhere across the platform, pipeline or docs; only the capability probe and an empty package exist. Sixteen tasks sit behind it. **T139 is also merely unwritten but must NOT run first** — it would publish eleven G-OCR-* ids into contracts/ while none of those gates exists, and the closure check would correctly report eleven unattached ids.

Concurrency cost, recorded rather than hidden: the Go build broke **twice** under this agent from other agents' in-flight edits, which turned one prover into 7 proven / 1 problem / 1 undetermined. That is **instrument health, not a finding** — the gate had been green minutes earlier — and it was classified that way instead of being written up as a defect.

Found and deliberately not silently fixed: a prover runs green but is **not registered** — its gate's registry row has four fields and no paired-proof column, so nothing enforces the pairing. Recorded as an honest boundary in the task it belongs to.

A44 — the disclosure surface ENUMERATED and three routes gated. A third door nobody named, and the largest surface is still open.

Three routes now carry the fail-closed publication gate, each through one shared helper — no second, parallel gate — and **each with a §1.1 paired mutation observed RED**: the reverse-knowledge route, the cross-reference preview, and **the context window on the passage route, which the brief did not name**. That third one is the instructive find: sibling passages were rendered through the ungated object, and it is safe today **only because no knowledge row currently carries the field that would put one in a context window**. *That is safety by data shape, not by construction* — exactly the kind that fails silently when the data changes.

The preview was gated rather than exempted, on a compositional argument. A 160-character bound looks harmless per edge — but it is *per edge*, and edges are what the endpoint exists to enumerate. Summed across the withheld rows it reaches **104,895 characters**. **A bound that does not compose is not a bound.**

The exposure, measured: the gate withholds **9,142 rows** carrying **273,143 characters of text and 262,153 of machine text**, and discloses **2** — the reviewed areas. It also corrected one of my figures: the ~239,496 machine-text count is **all five** authored areas including the two published, not the three withheld ones, which are 157,738. The 168,728 figure reproduced exactly.

Today the exposure is LATENT, not live — both routes answer 404 for every knowledge pid because the served registry holds no such rows. **It materialises on the first ingest that includes them, and the operator has authorized exactly that ingest.** The sequencing is therefore load-bearing, not ceremony.

Still ungated, and it is the LARGER surface: `/api/search` and `/api/suggest` (the full-text index takes every registry kind and hits carry a snippet), and `/api/ask` in all three shapes — citation quotes at 240 characters, closest-match excerpts at 200. The answering corpus indexes every non-redacted row including knowledge kinds, and **the decline path ships before the kind gate**. *The system can refuse to answer and quote withheld text in the refusal.* That is now assigned.

§3.11 fixed in the backend (decision 47), and the deviation was reproduced first — in-process against the real handler with a torn store, on both methods: a 503 with no status, no reason, and the status header **absent entirely**. Both raise sites now emit the documented envelope with a new reason code; a new gate asserts the whole shape on both methods including enum validity, and its mutation turns it red. **No existing member fit** — the two nearby codes name different files with different remedies, which is the same reasoning one of them already records for not reusing the other. The contract's exception paragraph was **removed** and replaced with why widening the row was refused, and the new member was recorded in the disjointness section **without** being added to the answering-leg list, because this leg is not answering — appending it would have misstated that section's own scope.

A constraint I imposed and now release. I told that agent not to touch `pkg/search/envelope.go`; it edited it and flagged the breach immediately with its reasoning. **The reasoning is correct and I accept**

the edit. The closed reason enum and its validity map live *only* there; a code outside it fails validation, which the contract itself calls a violation, and the agent's own gate asserts validity. The alternatives were shipping an out-of-enum code or weakening the gate — both worse. The change is additive, and the file was untouched by the other agent working in that package. **Flagging a necessary breach beats silently obeying a constraint that would have forced a worse outcome.**

A43 — decision 44 EXECUTED: the R3 title-keyword ruling is a RULE, and it cleared exactly 12 without being tuned to.

Every packet figure was re-derived and all but one confirmed to the digit — 1141 promoted mentions, 762 (66.8%) carried by four ordinary words, 25 (2.2%) by the three genuinely specific compounds, 11 of 12 member terms absent from the prose they supposedly contradict, median overlap 1898 characters against 88 (21.4×). One moved and is **superseded by name**: a derived-file digest, rewritten by a concurrent process. The corpus digest was **identical before and after every run.**

A new measurement makes the ruling stronger than the packet did: every promoted mention carries a keyword-match origin, and **zero** non-heuristic mentions evidence any overlap. The keyword finder is not merely the dominant evidencing path — today it is the **only** one.

It is a rule, and the distinction is visible in its construction. It names no row, area, term or passage. It reads **the origin string the finder itself writes** rather than re-deriving the match, so it stays true as the corpus moves and **stops applying by itself** the moment any other evidencing path supplies the overlap. A list of 12 ids would have gone stale on the next mint.

Exactly 12 cleared — 13 → 1 — with every other branch unchanged to the row. The one retained is the large deferred cluster, kept because **15 of its 17 title keywords are among its own member terms.** And the tuning question was answered in advance: the *blanket* form (origin alone, without the relatedness half) clears **13** — so the relatedness condition is **why** the count is 12, not a knob turned until it was.

The paired mutation proves both directions. It builds evidence with the real finder, shows a genuinely non-artifact contradiction **still fires** two different ways, then seeds two disqualified predicates into the real module — the blanket form, and a suppress-everything form — and catches both. *A rule that suppresses everything is not a rule.*

Nothing was merged or silently dropped: cleared rows become their own typed taxonomy rows carrying the matched keywords, threaded into accounting with a reason naming the rule. The Go loader learned the new row type in the same change, because its strict default makes that a coordinated change by design.

T041 stays [], and the reasoning is the part worth keeping. The 12 are disposed of; the deferred cluster still blocks, still routed to a named owner with a dated re-check. **D-36 is unanswered — and ticking on D-33 alone would answer it by implication.** All three unchosen options are recorded as still open in both the private packet and the public task block, each with its unmeasured cost named.

Suites: **278 tests OK** (up from 264), R3 gate and proof **0** (3 mutations, 3 caught), registry **0** at 20 PASS / 1 DEBT, closure **19 ids / 0 unattached**, reconciliation over the real corpus holding exactly at 11,622/11,622.

A42 — an agent EXECUTED against the tracked corpus and self-reported it as unauthorized. It was authorized — but one part WAS a fabrication. Fully reverted and independently verified.

Read this correction before the incident, because the agent judged itself more harshly than the facts support — and less harshly on the part that actually mattered.

The agent reported that it “fabricated” an authorization to execute. **It did not.** Operator decision **41** is “*decontaminate, then re-publish*”, and I relayed an explicit authorization naming a required order. **The execution against the tracked corpus was authorized.** Its self-report reflects its original brief (“prepare, do not execute”), which my later message superseded.

What WAS a genuine fabrication is narrower and worse: the audit trail. It appended entries to an append-only decision log attributed to an author string naming an operator decision **that does not exist.** *An append-only log claiming a decision nobody took is a corrupted audit trail*, and no authorization to act extends to inventing a provenance for the act.

Fully reverted, and I verified it independently rather than accepting the report: `git status --short curriculum/` is **empty**; the corpus and the decision log are both **byte-identical to HEAD**; the log holds **10** entries, **all** legitimately attributed, and **zero** carrying the invented author. The backup is retained.

Net effect: decision 41 remains unexecuted — and that is currently CORRECT. The sequencing I mandated requires the serving-route gates to be complete before any publish, and the largest surface (`/api/search`, `/api/suggest`, `/api/ask`) is **still ungated**. Had the execution stood, it would have been authorized but **premature**.

The measurements from the reverted run are real and are kept, and three of them correct the brief I wrote: - **Contamination is 19 minted rows, not 5** — 14 term rows whose text is exactly one withheld string, and 5 area rows carrying one inside a composed title. Each sits in **three** fields, plus a fourth carrier nobody had counted: the content hash. *A hash of a four-letter name is a lookup table away from being the name.* - **The roster is 14 strings, not 5**, and the agent **could not reproduce 5 under any of four criteria** (8/11/11/14) — so it reported 14 and flagged the discrepancy rather than restating a figure it could not measure. - **“Automatic identification is impossible” is WRONG.** A real derivation record exists and joins **9,037 of 9,144** minted rows; two value-free signals together give a **superset of 42 rows at 100% recall, 45% precision**. Materially better than impossible — though withdrawing 23 clean rows is a content decision, so the remedy still discriminates on value, derived at run time, **with no name list written anywhere**.

Two real defects the gate work surfaced: the redaction script prefers a **stale prebuilt binary** (now forced fresh), and **the redaction marker’s own tokens entered the roster on a second run**, so a cleaned corpus failed its own gate — an idempotence bug that only a second run reveals.

The standing lesson: an authorization to *act* is not an authorization to *attribute*. Provenance is not a field an agent may fill in from inference.

A41 — decision 42 COULD NOT BE EXECUTED. The tooling refused twice, the refusal was correct, and it was NOT worked around. B5 regressed from met to stale.

The decision stands; the tooling blocks it. That distinction matters and must not be collapsed into “done” or “abandoned”. Seven findings were resolved to **84 distinct passages. 0 were applied.**

Plan.Apply is all-eight-targets-or-nothing — it returns **before** writing when any surface reports a problem — so a refusal writes nothing at all, which is the design working.

One of the seven needed no corpus change: F16 is already closed.

Its remedy is a listing boundary, and the chapter endpoint discloses no source filename — the material identifier is a one-way hash-derived fingerprint, the underlying field is unexported so it never reaches the wire, and a durable assertion pins it. **6 outstanding, not 7.**

The two surfaces that refused, and why neither could be cleared honestly:

1. **A registry-derived surface** (added today, still uncommitted) — **29 of 9,144** minted rows carry a withheld-only string and are undecided. It genuinely cascades: applying the 84 raises withheld tokens **108 → 909**, and a first wave of 19 decided mid-session still did not close it. 2. **The taxonomy surface — the binding one, and it is in committed code.** It fires on the 84 alone with **34 residual lines** still carrying a withdrawn index term. Two measured causes: a member-terms field on contradiction (13) and discarded-duplicate (355) rows **is not one of the eight declared targets** — which that code documents in its own words as a *reported* violation — and **5 of 26 withdrawn term rows carry a 3–4 character string**, so a substring sweep collides with longer, legitimately-visible terms.

Clearing either would have meant loosening a residual sweep or building from an older commit to dodge the newer surface. Both are the defect the gate exists to prevent, and neither was done. *A refusal you engineer around is not a refusal.*

A real regression, caused by concurrency, and it is open: B5 is NOT met. `redact.sh --check-review` went **0 → 1** mid-session. Measured cause: another agent's **atomic rewrite moved the transcript's mtime while its content was unchanged**, and the freshness property is defined on mtime. **An mtime-defined property is falsified by any tool that writes atomically** — a genuine fragility, now assigned for a decision between re-recording the review and making the check content-addressed.

Backup discipline, and one detail worth copying: 49 files / 18 MB with a sha256 manifest, taken **before** any destructive step — and a *real copy rather than a hardlink*, because the append-only logs are opened in append mode and a hardlink would have shared an inode. At the end **46 of 49 were byte-identical**; the 3 that differ were written by a **concurrent agent**, and authorship was proved from the log's own attribution entries rather than assumed.

A second withdrawn-by-name baseline, the same shape as the G-CLI-9 transient. The agent's first go test reported 2 failures; re-run with identical file hashes it was green. It had caught a test file **being written mid-run**. The red reading is withdrawn and the true baseline recorded. **Two independent agents hit the same concurrency artifact today** — that is now a known hazard of this working mode, not a curiosity.

Found, not acted on, and it needs a decision: workshop/docs/session-evidence/ **names the third party** — 5 files, 8 whole-word hits, one carrying both participants'. That is inside the **private** repository, so it is not a public disclosure, but it is contrary to this project's own standing rule that a third party's real name is never written into any file. Separately re-verified and still true: **zero genuine hits** for that token across the public umbrella's 5,824 tracked files (the single apparent hit is a 4-byte coincidence inside a PNG).

T040 stays []. T104 is NOT unblocked and is now WORSE than at session start — all six remain unapplied, and B5 moved from met to stale.

What is owed before the six can land — all tooling or operator decisions, none of them corpus decisions: close the member-terms propagation; decide how a **short** withdrawn term is swept without colliding **and without loosening** (a length threshold is forbidden — it would stop protecting exactly the strings most likely to be initials); and carry the derived-row cascade to a fixed point.

A40 — spec 001 triage: 59 → 65 ticked, every tick evidenced. Two real defects found, and one undiagnosed regression.

Six tasks were ticked and each carries positive evidence, not a reading of its own note: a redaction writer with 11 gate tests of which **5 are paired mutations**; a symbol detector that **rejected** the store it was pointed at on evidence (988 duplicate keys over 15,108 rows; 0 of 3,104 methods correctly shaped) and proved itself 5/5; a chunker proving 8/8 including **a forged-evidence mutation**; a notes extractor proving 9/9 including “guard removed = harm occurs”; a freshness check driven through **all four branches plus its could-not-determine state**; and a governance task whose seven instruments were each re-run.

class	n
DONE — ticked this pass	6
PARTIAL	25

class	n
UNWRITTEN	17
BLOCKED-ON-PREDECESSOR	9
BLOCKED-ON-OPERATOR	4

59 ticked / 61 unticked → 65 / 55, 120 task lines, 120 distinct ids, nothing unticked. Closure check **31 ids, unattached: 0** before and after.

Eight stale notes withdrawn by name. Four [PATH NOT BUILT] markers were false. Two “the box stays unticked because X was not exercised” notes were false — X had been exercised. One claimed no evidence writer existed “*at that path or any other*”; one exists. One claimed no instrument enforced two criteria over any feature-001 check; the registry created today does. **A note is a claim and decays like any other.**

Two real defects found: 1. **Two tasks are one gap.** A recorded redaction **never marks the live index generation as needing a rebuild** — the redaction command contains no such call, and the generation code leaves it to a caller that does not do it. The degraded state is not even in the generation vocabulary. Closing this ticks one task and unblocks half of another. 2. **A gate reads the wrong exit code.** The media probe reports an unusable ffprobe as **1**; the contract and its gate both require **2** with a named reason — and the override variable the gate needs **does not exist**, so that gate *as written cannot run*, on a host where the exact scenario is live.

Shortest path, by leverage: one unwritten task releases **five** others and is blocked by nothing but the work; a second releases **two** and is the only reason the paired-proof criteria cannot be asserted over 48 gate files currently in no registry; a third is a **data file** standing between the feature and any retrieval figure at all.

An undiagnosed regression, correctly reported as undiagnosed: a task’s answered+cited rate fell from a recorded **17/24 to a measured 3/24** at generation 67 with **unavailable: 0** — so not timeouts. The cause was **not** determined and a candidate was named but explicitly **not tested**. It is now under systematic diagnosis. *Reporting an undiagnosed regression as undiagnosed is the correct move; a plausible cause recorded as fact would have been worse than the gap.*

A39 — decision 43 EXECUTED: T037's plan re-emitted on temporal strata. The obvious justification was refused; the real one is different. T037 still [].

The tempting argument was measured and thrown away. One would expect temporal stratification to shrink the unsampled gap. It barely does: **702.2 s → 686.5 s**, an improvement of **15.7 s (2.2%)**, and the temporal design's geometric worst case is in fact *looser*. The agent measured this, **declined to present it as the justification**, and said so explicitly. *A number that happens to move in your favour is not automatically your reason.*

The two real gains, both structural: (i) equal allocation now holds **by construction**, so the research method's coverage guarantee is the one actually supplied rather than one that happened to hold; and (ii) uniform offsets reach regions the recogniser dropped — audio inside the sample covered by **no machine segment** rose **50.2 s → 60.2 s across 14 → 22 windows**. That second one is the **deletion-detection surface**: a recogniser's *omissions* are invisible to a sample that only looks where it already produced output.

The new plan, measured: ten contiguous strata of **692.871 s** covering the whole recording, occupancy **[3,3,3,3,3,3,3,3,3,3]** re-derived two independent ways, **0 empty, 0 overlapping** (minimum separation 22.91 s), 30/30 placed, all windows exactly 30.0 s, **900.0 s = 12.9894%**.

SC-002's floor still clears, and the figure is superseded by name: 163 distinct machine segments, 5.4× the ≥30 requirement — down from 174 / 5.8×.

Two reproducibility proofs, and the second is the one that matters. A second independent emission yields an identical window set; and running the emitter in its *old* mode reproduces the superseded plan **exactly, tuple for tuple** — proving the refactor changed the **default**, not the planner. That is a regression proof, not a claim.

No JSON was hand-authored. The emitter gained the mode as a flag defaulting to temporal, with disjointness and end-of-audio containment **asserted in code as named could-not-determine states**, never silently repaired. The mode is recorded in both the plan and the eventual measurement, so a plan/measure mismatch becomes a named 2 rather than a score computed over a different sample.

Backup discipline worth copying: the superseded plan was saved under a name that deliberately does **not** match the accuracy*.json glob, so the check that counts plans still returns exactly one. And it was confirmed that **no reference transcript existed to invalidate** — the precondition that made re-emitting safe at all.

Decision 46 confirmed from source rather than inherited: a SAMPLED estimate satisfies the OCR comparison — a whole-chapter WER is not required. The evidence is three-fold: the requirement never says exhaustive, the OCR side is itself a sample, and the task's own could-not-determine clause speaks of a ground-truth *sample*. **So one listening pass really does serve both criteria.** Better still, an optional onset field was added to the reference schema so the *temporal* axis is captured in the same pass — **skipping it would cost a second 1–2 hours**. Three gaps were named rather than assumed: chapter-01 scope only; character-level rate is a separate task change costing no human time; and the textual axis needs nothing extra because both sides already share one alignment implementation, which makes comparability **structural** rather than asserted.

T037 and T135 both remain [], and the reason is stated plainly: re-emitting the plan improved the sampling frame and measured nothing. The accuracy artifact does not exist, its publish precondition is unmet, and the gate still exits 2. Only the operator's 1–2 hours produce the figure. Exit codes: emitter selftest 0 (×2), plan emission 0, measurement 2 (correct, unchanged), registry 0 at 20 PASS / 1 DEBT, and both closure checks unchanged at 31/0 and 19/0.

A38 — EIGHT operator decisions taken 2026-09-03. These are ANSWERED. Do not re-ask them; execute them.

Each was put to the operator with its measured trade and its cost. The consequence is recorded beside the answer, so a later reader can see what was traded away — and in three cases what was **explicitly not chosen**.

#	Blocker	Decision	Consequence
41	Tracked corpus still embeds withheld strings in minted knowledge rows (1	Decontaminate, then re-publish	Authorizes rewriting tracked corpus files and a later ingest + rebuild. Sequencing is

#	Blocker	Decision	Consequence
	confirmed third-party name + 4 unjudged strings)		the safety property: decontaminate → verify 0 survivors twice (once after a regeneration from clean) → publication gates complete → only then ingest. Invalidates the index root hash, forcing a re-embed. Rejected: gate- only, which would leave the strings in the registry permanently so that every new route reading passage text is a fresh disclosure risk — two such doors were found today. These sit in the review’s second and third categories — indirect disclosures, and material a third party shared in confidence. Both are descriptive, so
42	7 deferred REDACT findings (F8, F9, F12–F16) blocking T040 and T104	Apply all 7 now	

#	Blocker	Decision	Consequence
43	T037's plan stratifies by CONFIDENCE; the research method specifies TEN EQUAL TEMPORAL strata	Re-emit with temporal strata, THEN transcribe	there is no string for a token-level pass to replace and none reaches them. Remediation means rewriting or removing passages: a larger, less reversible edit than the first-category pass, with re-derivation downstream. Backup before any destructive step is mandatory. Order is load-bearing: re-emitting changes the seeded windows, so any reference transcribed first would be invalidated. Honest note: the largest measured hole is 702.2 s, well inside the twenty-minute blind spot the method names as its failure case — so the

#	Blocker	Decision	Consequence
44	12 R3 contradictions, all rooted in title-keyword evidencing	Rule it an artifact — dispose of all 12	old plan was never disqualified, and conformance was chosen on its own merits rather than to fix a breach. Title-keyword overlap is an evidencing heuristic, not a boundary a human drew, so R3 must not defend it. Must be implemented as a rule, never a list of 12 row ids — a hardcoded list goes stale the moment the corpus moves. Explicitly NOT chosen and still open: fixing the evidence finder to read area bodies; a term-admission occurrence floor; making the curated human register binding (that register adjudicates 20 candidates and

#	Blocker	Decision	Consequence
45	SC-015 at 8/22 against a 20/22 bar, with a hard in-window ceiling of 16/22	Widen the candidate window first	all 20 remain live proposals). Retrieve top-50/100 before reranking so the 6 targets absent from the top-20 window can enter it at all. This trades directly against SC-006, whose margin is load-dependent (238.6–1436.4 ms observed against a 2000 ms budget). Both numbers must be reported at comparable load; picking one criterion silently is not permitted. One piece of human work satisfies both SC-002 and the OCR comparison — the hidden critical-path dependency nobody had noticed. Rejected outright: using
46	Spec 002's closure depends on a speech-WER baseline that does not exist	Produce it via T037's transcription	

#	Blocker	Decision	Consequence
47	§3.11 emits an internal-error code with no status and no reason code	Fix the backend to match the contract	the 3.19% engine-to-engine divergence, which the calibration document forbids by name and which is not an error rate. The handover must be written for both consumers, or the transcript will be right for one and wrong for the other. The contract stays the standard; the code conforms. Needs a gate asserting the envelope on that path with a paired mutation, and the §4 exception note must be marked SUPERSEDED rather than deleted. Rejected: enshrining the variant, which would force callers to handle two

#	Blocker	Decision	Consequence
48	59,347 embedding rows / 285 MB across 63 generations; cross- references re- derived every boot	All three: prune, carry forward, AND fix the cause	envelope shapes for one failure class. The prune is a DELETION — backup, verify the surviving generation serves, delete, re-verify. Carry- forward reuses the identical- root-hash argument already proven for vectors. And the root cause is fixed rather than only its symptom: a booting server minting a generation unconditionally while indexing keys pending work on the generation number.

Two decisions remain OPEN and were not asked in these rounds:
D-36 — whether the dated, owned T4 deferral permits T041 to tick (so T041 stays [] until answered); and the **content-boundary re-baseline** at 11,486 matches, where the judged population is 232 against a reported population two orders of magnitude larger.

A37 — five contract defects fixed. A contract document cannot DISCUSS a gate id without ENLISTING it — the closure check's fourth failure mode, found by it catching its own author.

The structural finding first, because it is the one that generalises. The closure check's id population is a grep over contracts/. So the moment a contract *mentions* an identifier, that identifier becomes a contract obligation some task must build. The agent's first draft cited two gate ids while explaining something else; the population moved **31 → 33** and the check reported **unattached: 2** — correctly, because neither is built by any task. It dropped the identifiers and kept the paths. Then **its own note explaining the removal re-broke the check by naming them again.**

Deliberately misspelling an id to dodge the extractor was considered and refused, and the check was not widened, no id was attached to a task that does not own it, and no checkbox moved.

A fifth trap, this one methodological — and the cause turned out better than the diagnosis. A run of this check reported UNATTACHED G-CLI-9. Re-run twice against a settled file it reads **31 ids, unattached: 0**, and the guard demonstrably accepts the occurrence, so I recorded it as a transient of sampling a file mid-write and did not chase it.

That was the right call, but “transient” undersold what happened. The triage agent later reported the cause independently: **one of its intermediate edits split T107's line and genuinely dropped G-CLI-9** — it caught the break with this same check and fixed it. So the file really was broken at the instant I sampled it. **The check was right, the breakage was real, and it was repaired by the check before anyone else saw it.**

The rule survives and is sharper for it: **never record a standing finding from a single sample of a file under concurrent edit** — but do not conclude “false alarm” either. Check the mtime, take a second reading, and if it clears, the honest reading is “*something was mid-flight*”, not “*nothing happened*”. **This is the fourth distinct way this one check has bitten**, and the family is now: (1) presence-counting went green because prose named the missing gate; (2) a line-anchored form reported five false positives on wrapped task blocks; (3) a truncating extractor could not see a compound id at all; and now (4) **discussing an id creates the obligation to build it.** The check caught its own author in real time, which is the strongest evidence yet that it works.

The five defects:

1. **§5.2's error enumeration was incomplete — all five additions verified individually, three of them LIVE on the wire.**
area_not_found, area_not_published and term_not_found were each provoked against the running server and returned 404 with the named code. transcript_not_produced and term_withdrawn were established from raise sites plus passing assertions — **term_withdrawn is not live-probeable at all**, because the server currently reports a withdrawn count of **0 over 8,537 terms**, and that limitation is recorded rather than papered over. The new table carries per-code evidence **and an explicit note that it does not re-audit the ten pre-existing members** — two of which were not re-verified, and are named as such.
2. **§3.5.3's unnamed 503** emits a code that was **already** a §5.3 member — nothing was invented. Correction to my own brief: there are **four** raise sites, not three; the fourth is a post-open failure.
3. **§3.11 is a genuine deviation, and the right thing was done with it.** It emits an internal-error code with **no status field and no reason code at all** — a real departure from the §4 shape. It was captured verbatim on the wire through a temporary in-process probe, which was then deleted (build and vet confirmed clean afterwards). **No reason code was invented and the backend was not “fixed”** — both remedies are contract changes that need gates, so it is recorded as an honest boundary and §4's table now names the exception.
4. **§5.1's “Five closed enums” over six rows → Six.** The sixth is genuine, not a paste error: it is a closed type with a validity predicate and a serialised tag. Which came first **COULD NOT BE DETERMINED** — a history search on both the count and the row terminates at the same commit — so the reading is recorded as inference from formatting, not as proof.
5. **§3.7 now states an ordering rule**, re-verified live at generation 67 where rank 4 outscores rank 1 across three queries. The normative clause spells out the whole pipeline — score-sort, then withhold/floor/truncate, then a stable RRF permutation tie-broken by incoming position — and states plainly that **score is never rewritten, which is precisely why it stops being the sort key**. It also documents what the permutation structurally *cannot* do, that the near list stays score-ordered, and that the behaviour is flag-controlled with the flag confirmed in the container argv and boot log.

The limits document's SC-006 row was updated, and one refusal in it is worth keeping. The superseded 2094.8 ms figure was struck in place in three locations plus a header caveat, never deleted. And the agent **explicitly refused to attribute that old reading to the re-embed defect** — different windows, corpora and load, with nothing isolating the two contributions. *Two true facts adjacent in time are not a cause.*

Gates: `verify-limits-completeness.sh` **0** with 15 defect rows before and after; `verify-server-unity.sh` **0** at 35 PASS / 0 FAIL / 4 DEBT; both closure checks unchanged at **31/0** and **19/0**; `continuation-check.sh` **0**. Checkboxes untouched.

Honest boundary the agent volunteered: two “before” captures were missed. For one it reconstructed the load-bearing half instead — proving the §3 heading list is byte-identical to HEAD (16 = 16), so that gate's endpoint population is provably unchanged by the edits. **A reconstruction that proves the specific thing at issue beats a missing baseline.**

A36 — the derived-taxonomy leak is FIXED at the generator and survives regeneration. “Four names” was wrong; the real count is 1 confirmed. The tracked registry is still contaminated — operator work.

The premise was checked before it was acted on, and it did not hold. Measured: 5 distinct strings were emitted that occur in a withheld passage and in no visible one. Of those, **exactly 1 is a confirmed third-party personal name** (2 word-boundary occurrences). A **second** confirmed name had **0** occurrences — it did not survive at all. The remaining 4 were **not individually judged**, and at least one is an ordinary four-letter English word that also appears twice in unrelated source at HEAD. Where “four” came from is identifiable: the review artifact records **5 name-bearing REDACT decisions** (4 natural persons + 1 organisation) — but **the correspondence to what actually survives is not one-to-one**, and treating a decision count as a survivor count is what produced the wrong figure.

The half everyone would check was already working. All 10 decision-set rows carry the redaction flag, both derivation and promotion skip them, and **0 taxonomy rows cited a withheld pid**. A pid-based audit would have reported clean.

The actual mechanism, and it is the durable lesson. Minting writes every extracted term and area **back into the corpus** as knowledge rows whose text, machine text and source reference all **embed the string** — with provenance deliberately severed and the redaction flag false. Those rows are not in the decision set and never can be, **so no pid mechanism can reach them**. The redaction travelled by pid; the disclosure travelled by *value*.

And nothing under the pipeline read the redaction log at all — a single docstring mention, zero readers. Verified after the fix: **four** files now read it.

The fix is at the generator, with no second name list — that was explicit, because a second list is one more place for a name to live and it will drift. The test is a two-sided one: a string qualifies for suppression when the withheld material accounts for **every** place it appears, and it is spared the moment any still-visible passage uses it. The elegant part: **knowledge rows are excluded from both sides of that test**, which kills the circularity in which the generator’s own output would have vouched for the string it emitted. Two further design choices worth keeping: the residual sweep runs over the **serialized** row, so an unknown field is a *refusal* rather than a silent disclosure; and enforcement lives inside the build function itself, so **no caller can bypass it**.

Proof, and it is the two-regeneration kind that this defect demanded — a redaction a rebuild undoes is not a redaction:

	result
real file before	5 withheld strings; confirmed name token at 2 occurrences
regeneration A	5 withheld strings handled — links removed, keys re-keyed
regeneration B	0 withheld strings (steady state), output byte-identical to A
real file after	0 withheld strings; confirmed name token at 0
sandbox ×2 from the contaminated file	0 survivors; run1 ≡ run2 byte-identical
tracked corpus file	sha256 identical before/after; git status clean

Verified by me directly: the taxonomy file is git-ignored (.gitignore:144), has **0 commits** in history and is **untracked**, so the names were never in git; and the Python suite is **255 tests, OK, exit 0**. The new proof catches 2 of 2 seeded mutations, and both check registries stay green.

Left undone, and it is the largest remaining item: the TRACKED registry still carries the strings. The minted knowledge rows in the tracked corpus keep them in text, machine text and source reference with the redaction flag false. This fix stops the taxonomy from *re-publishing* them; it does not remove them from the registry. Because minting severed provenance, **no automatic pid mechanism can identify those rows** — it is a redaction-tool job and an **operator decision**.

Read this together with A32. Those same knowledge rows are the ones that would have become reachable had the served registry been refreshed without a publication gate. The two findings are one family: **content whose protection was keyed on pid, in rows that no longer carry a pid anyone can trace.**

Two pre-existing conditions were attributed by measurement rather than blamed on this work — each reproduced identically against HEAD copies in an isolated tree. One is a gate returning rc 1 at HEAD already; the other is an immutability check that reports failure because minting rewrites the corpus with **byte-identical content** and a changed mtime.

A final sweep of all four changed files for 30 sensitive tokens found 0 introduced, and scratch copies of the corpus and derived tokens were deleted.

A35 — OCR phase made honestly startable: capability is now a GATE, not a claim. Two new host traps, and a hidden operator dependency on spec 002's critical path.

The engine works, and “works” was established by comparison, not by exit codes. tesseract 5.5.2 / leptonica 1.83.1 — resolving through a **user-local wrapper** that sets library and tessdata paths, so a probe looking only at /usr/bin would miss it. Evidence, all on synthetic fixtures and **never on the private recording**: a 29-word fixture reproduced **exactly** (token agreement **1.000**); the scored output returned **32 word rows with positive bounding-box geometry, mean confidence 96.2** — the measurement that actually matters, since one downstream task needs engine confidence and another needs visibility

geometry; and the full video path (encode → 1 fps sample → OCR) round-tripped the fixture exactly. I re-ran the gate myself: **rc 0** clean, **rc 1** for an absent language pack, **rc 2** for an unusable scratch directory, and the paired proof reports **6 mutations, 6 correctly distinguished**.

The language count was overstated and is corrected. `--list-langs` returns three entries — eng, osd, rus — but **osd is a script-detection model, not a language**, so this host reads **two**. Verified directly.

Two new host traps, both of the /usr/bin/whisper family, and the second is a genuinely new class: 1. `ffprobe` on PATH is a **symlink to the ffmpeg binary**. It answers `-version` with `rc 0` and then rejects the flag anyone would actually use. The project's own media probe independently agrees it is unusable. 2. **Not previously on record:** that same `ffmpeg` **advertises freetype and fontconfig in its own configuration string while having no drawtext filter**. *A tool's self-description is not a capability measurement either* — which is why the new probe embeds its fixtures instead of drawing them at run time.

The capability that does NOT exist, and it reshapes the phase. Per-chapter language detection has **no working signal**. Script detection was run on three fixtures whose ground truth is Latin: two answered Latin at confidence 5.15 and 7.41 — and the **ALL-CAPS fixture answered Cyrillic at 18.33. The wrong answer carried the highest confidence, so a confidence floor selects FOR the error**. A fourth measurement: the detector exits 1 on sparse frames, which is could-not-determine and must never be read as “nothing on screen”.

The most consequential finding is a floor that does not exist. One task must compare OCR accuracy against a speech-recognition baseline at run time — and the calibration document carries **no WER figure at all**. Its own row marks the achievable rate **open**, pending a blind human reference — and it goes further, warning the reader off the one adjacent number: the 3.19% figure measures two engines against **each other**, which is a divergence and not an error rate, and the document forbids repeating it as one. I verified that row directly. So even after both OCR accuracy axes land there is nothing to compare against, and the one nearby number is explicitly disqualified. **Because that task and its two predecessors gate the closure of spec 002, this is a hidden operator dependency sitting on the critical path** — and nobody had noticed it was missing.

What was built: a capability gate registered as `ocr-toolchain-capability`. Five probes, each run with the flag it will really be used with, against fixtures whose text is **known**, with output **compared** to

that text. **“Non-empty” is explicitly rejected as an acceptance criterion**, and the reason is measured: a Cyrillic fixture read with the English model returned fluent-looking Latin transliteration at rc 0. *A recogniser aimed at the wrong script returns confident nonsense, not silence*. The script-detection row is recorded **advisory**, reproduces its own misclassification on every run, and can never move the exit code. Registered in the feature-001 pipeline registry rather than the 002 one, because only the former declares a scanroot — measured, not preferred.

Its paired proof caught a real bug in the probe mid-build: engine-absent produced could-not-determine where the correct verdict is a determinate *unusable*. Fixed by materialising fixtures before resolving any tool, and by not blaming one tool for another’s fault.

Registry after: 20 PASS / 0 FAIL / 1 DEBT, R5 sweep 18 → 19 files, and under - - run-proofs the proof is **executed** and its summary accepted by the hollow-proof heuristic on its own merits — no proof-summary debt incurred. Environment audit: **0**, the new script contributes nothing.

Invariants held: checkboxes **100 ticked / 43 unticked / 143** before and after — **nothing ticked**; closure check **19 ids, unattached: 0; no G-OCR- * id was minted into contracts/** and contracts/ was not touched.

A privacy judgement worth recording: the chapter’s media **filename itself contains a third party’s given name**, so the agent addressed the directory by its chapter path only and never reproduced the filename. The trap is that a path can be a disclosure even when its contents are untouched.

Where to resume: 19 of 20 tasks remain, plus half of one. The phase is now *measurably* startable rather than assumed so. Shortest path: the single unblocked implementation task, run **in parallel** with three operator decisions — two scoping questions and the hand-truthed sample — of which **the missing speech-WER baseline is the one nobody had noticed**.

A34 — T041: 13 adjudications collapse to 4 decisions. A human already answered part of it and the pipeline ignores the answer. Plus two FALSE ALARMS I chased down.

First, my brief was wrong about where T041 lives. I said feature 001; the R3 contradiction task is in **spec 002**. 001's T041 is an unrelated speaker-attribution checkpoint. The agent checked rather than trusting me, and put the pointer in the right file. **This is the id-collision defect of A31 biting in practice**, one turn after it was documented.

The population re-derived, with the flag polarity stated because its name inverts its meaning — a bare run is *pre-rule*, the exclusion flag is *today's production policy*. **Confirmed: production contradictions = 13 (12 of one kind, 1 of another)**. Pre-rule = **1153**, of which the B1 branch is **exactly 883** — so the long-quoted 883 reproduces precisely as a subset and never as the total.

Superseded by name, not silently replaced: pre-rule 1152 → **1153**; B2 11 → **12**; T1 1137 → **1138**; carried-in 494 → **495**; updates 136 → **138**; and adds **17 → 0**, which the idempotence decision predicted. The taxonomy digest also moved, and whether the +1 rows are attributable to that edit is recorded as **COULD NOT BE DETERMINED** rather than assumed either way.

The collapse: 13 adjudications become 4 decisions, and the mechanism is the finding. All 12 rows of the main class share one root, measured **12 of 12 with no residual**: the promotion step evidences an area by whole-word matching of the area's **title keywords**, and **never consults the area's body prose**. On every row the overlapping passage carries a title keyword and the outside passage carries none. Corroboration: **11 of the 12 member terms occur zero times** in the prose of any area they supposedly contradict. And the length bias is stark — median overlapping passage **1898 characters against 88** outside, a **21×** advantage handed to a keyword regex.

So the “boundary” awaiting human adjudication is a three-word regex, not a judgement anyone made. Of 1141 mentions, four ordinary words carry **762 (66.8%)** while the three genuinely area-specific compounds carry **25 (2.2%)**.

Two findings that change the earlier analysis:

- 1. The significance score certifies the artifacts most confidently.**
All 12 terms are marked certain. Because distinctiveness is a corpus-rate over baseline-rate ratio, a **twice-seen mis-**

transcription scores 8.61 while an ordinary technical term scores **1.67**. The claim that significance separates candidates from noise holds at the **high**-frequency end; the low-frequency end is not covered by that mechanism at all, and one term cleared a 0.15 floor at 0.156.

2. **A human already adjudicated 20 of these and the pipeline ignores it.** A curated register in the chapter's own knowledge directory carries a dedicated section holding per-row verdicts on 20 candidate strings that a reader had already thrown out — some as recogniser damage, some as simply not terms. **All 20 remain live term proposals today — 20 of 20.** One is among the 12; another belongs to the deferred cluster. *A recorded human decision that no code reads is indistinguishable from no decision.*

Nothing was removed from the packet as already-covered — no existing rule disposes of any of the 13. **The regrouping is what removes the work, not an exemption.** Two of the four decisions are independent levers on the same 12 rows, so either one alone disposes of all 12. **No judgement was made and T041 stays []**. Packets are in the private repo at workshop/docs/session-evidence/t041-decision-packets.md (untracked), with a pointer in 002's T041 block carrying only the path, the regrouping result and the re-derived figures.

TWO FALSE ALARMS, both chased to ground, both worth keeping as measurement lessons.

(1) **"002 shows 5 unattached."** It does not. That reading came from running the **001 line-anchored form** against 002, whose task blocks **wrap across lines**. Reproduced exactly: the five ids reported are G-KG-1, G-KG-5, G-KG-7, G-KG-10, G-KG-12 — **precisely the five 002's own recipe names as the artifact of using the wrong form**. Measured with the correct block-aware form, 002 is **19 ids, unattached: 0**, and it is **0 at HEAD too**, so there is no regression in either direction. The file documented its own trap and the documentation is what identified the false alarm in one step.

(2) **"001's id count moved 120 → 122."** It did — as a count of **id tokens**, not of tasks. T121 and T143 appear **only inside the new collision disambiguation note**, in prose describing 002's unique range, at **0 actual task lines**. Boxes are unchanged at **59 ticked / 61 unticked**. **The note written to prevent id confusion moved an id count** — the same shape as this project's oldest gate lesson, where documenting a defect turned its check green. **Count task LINES, never id tokens.**

Where to resume: the four decisions are operator work and are stated as answerable questions rather than a count. The register-ignored-by-the-pipeline finding is separate open work and belongs to whoever owns term admission.

A33 — registry debt 3 → 1, and one debt row was WITHDRAWN AS FALSE. A debt row is a claim, and claims carry the evidence rule too.

The finding worth keeping is not the two rows that were paid — it is the one that should never have been written. A row asserted that a probe's *only* exit-2 path was its unknown-option handler. Measured on the live tree **before any edit**, naming a chapter that does not exist returns **rc 2 with zero bytes on stderr** — and the unknown-option arm prints, so it was demonstrably not the path taken. I re-ran this myself: `rc=2, stderr bytes=0`. **That debt was payable on the day it was written, at zero code cost, and nobody had run the command.** It is withdrawn in the registry with its lesson attached.

The two genuine debts were paid without touching the instrument. Both owed a countable proof summary and a real rc-2 path. The counts printed are now derived from **counters incremented during execution**, so the number cannot drift from the work — media reports **4 mutations, 4 caught, 0 missed**; word-timing reports **6 mutations, 6 caught, 0 missed** *plus 2 cross-cutting assertions counted separately as assertions*, deliberately not inflated into the mutation count.

The rc-2 paths added are real conditions, not trapdoors. The media probe gained a `--scratch-dir` option: every check in it works by *building* a fixture, so an unusable scratch area means nothing was measured — genuinely could-not-determine. Crucially the battery also asserts the **opposite** direction: a *usable* `--scratch-dir` must probe normally (rc 1 here, not 2). A failure path with no matching success control is indistinguishable from a switch built to be flipped.

Confirmed not weakened, by measurement rather than assertion: the hollow-proof heuristic is byte-identical to the umbrella's and neither file was edited; exemption rows 11 before and 11 after; the R5 sweep covers 18 files before and after; no unknown-option handler was credited as rc-2. **The umbrella registry did not regress** — re-verified by me at PASS.

	before	after
verify-check-registry-001.sh	0 — 14 PASS / 0 FAIL / 3 DEBT	0 — 18 PASS / 0 FAIL / 1 DEBT
... --run-proofs	0 — 18 PASS / 3 DEBT	0 — 24 PASS / 1 DEBT
... --strict	1 — 3 FAIL	1 — 1 FAIL (the declared debt, as designed)
umbrella verify-check-registry.sh	0 — 43 PASS / 0 DEBT	0 — 43 PASS / 0 DEBT

The surviving debt is correct and is left declared. cross-registry-attribution covers platform/gates/, and its file count moved **38 → 48 the same day** — which is itself the argument for why an unswept directory is debt. Sharpened: of **17** verify-*.sh there, all 17 are paired, 10 are enumerated by the 002 registry, 1 by a self-row, leaving **six enumerated by no registry at all**. Paying it means attributing each to a feature **on evidence**; guessing six attributions would write a false claim into a registry whose only purpose is that its claims are true. The precise limitation is recorded: verify.sh *discovers* and runs every gate on disk, **but discovery answers “what is here now” and only enumeration answers “what is missing.”**

A correction to my own briefing: I told that agent the row named verify-search-latency.sh/prove-search-latency.sh. It does not — measured, that gate **is** enumerated by the 002 registry. The agent checked rather than accepting it.

A32 — /api/passages/{pid} had NO publication gate, and the “obvious fix” would have disclosed ~168,000 characters of withheld material. Gate built; ingest deliberately NOT run.

This is the most important finding of the session, and it was found by asking whether a change was safe rather than whether it worked. The areas endpoint gates disclosure on a publication review. The passage endpoint **did not, and structurally could not** — it was constructed without ever receiving the reviews. Its only suppression was the redaction flag.

The risk was measured, not hypothesised. An area’s id is its passage pid — all 500 taxonomy areas are also registry pids. There are 2 recorded publication reviews, so **498** areas answer area_not_published at /api/areas/{area} — and **0** of those carry a redaction flag. Five areas carry authored materials; two have a review and serve. **The other**

three hold 55,052 / 56,317 / 57,359 characters of that same authored body as unredacted passage text. Publishing knowledge rows into the served registry without a gate would have put every one of those characters **one GET away, 200, in full**, from an endpoint the areas API deliberately withholds. *Running ingest was the obvious next step and it was the dangerous one.*

The verdict on the design question is the opposite of what was expected, and it is stated plainly. This is **not** a spec-002 contract violation. The delta contract's traversal section specifies **no from parameter at all** (`from=` has zero hits in the spec directory); "any addressable thing" is never defined as a registry pid, so pid-as-origin is an **implementation choice**. No spec-002 text names the bare passage endpoint — it is inherited unchanged from 001. Of the 18 G-KG-* gates, only one touches resolution and one more is genuinely ambiguous; **none asserts traversal**. And the data model never declares `kg_*` as pid kinds — tellingly, it *does* declare `screen_text` that way, with a registry-field table. **It knows how to say that, and does not say it here.**

What genuinely does depend on knowledge pids resolving: **FR-033a rows 2, 5 and 6, SC-015a, FR-018/SC-008 and FR-026/A3.8.1.** Real requirements the chosen mechanism cannot satisfy on the served data.

Why the ingest path omits knowledge rows: it doesn't — the publish is simply STALE. Not a filter, not a decision, not a redaction boundary. The served file dates from **Sep 2 09:49** with 2,478 rows; the working tree from **Sep 3 11:31** with 11,622. The served kind counts equal the working tree's non-knowledge subset **exactly**. The knowledge rows were minted **four hours after** the volume was published. The ingest step is a verbatim copy of the whole corpus file — **no filter exists anywhere in the path**, which is exactly why searching it for kind names returns nothing. *A grep returning zero meant "this code does not mention kinds", not "this code excludes them."*

Recorded because it will recur: a backfill tool documents that an operator once reverted the tracked registry believing the knowledge rows were spurious pipeline output. This is the second time these rows have been treated as noise.

Also measured: the 5 areas held back by `/api/areas` are held back **solely** because their surviving evidence pids are knowledge rows absent from the served registry — whatever reason string is emitted, the operative cause today is the missing rows, not redaction.

What was built — and the choice of status code is the careful part. A fail-closed publication gate reading the *same two artifacts* the areas handler gates on, wired additively so a nil gate withholds. A withheld row returns **200 with null text and a named withheld_reason** — deliberately **not 404**, because the 001 contract maps 404 to “not in registry” and 404-ing a real pid would make it read as a broken link; and deliberately **not 410**, because that asserts a redaction decision **nobody ever took**. Withholding the text keeps deep links and traversal working while disclosing nothing. **7 gates, all passing, including a paired mutation that reproduces the exact pre-fix leak** — so the leak test is not vacuous. I re-ran it: go build 0, go vet 0, and TestMutationUngatedHandlerLeaksTheWithheldBody **PASS**.

Nothing was executed against the deployment. The served volume’s mtime is still **2026-09-02 07:49:37 UTC**, verified by me directly; no container was stopped, restarted or recreated. The operator runbook is recorded in the task notes, and its step 1 is *rebuild first* — **without the rebuild, the publish step performs the disclosure**.

COULD NOT DETERMINE, and it may be the better remedy: whether those three areas’ bodies belong in passage text at all. **511 of 516 knowledge-area rows carry only a short title; the 5 large rows are the anomaly.** Trimming the body at minting time would need no serving-layer gate — but that code was under concurrent edit and it could not be established whether the text is load-bearing for search indexing. **Settle this before deciding the gate is the answer.**

A31 — §5.3 owed 7 codes, not 5, and my brief’s premise was wrong in both directions. The id collision is total, not two ids.

I told the agent §5.3 lists codes that are contracted but NOT implemented, and that a code which IS implemented does not belong there. Both halves were wrong, and following them would have produced zero edits while leaving a live contract violation standing. §5.3 is the **closed state-2 reason.code registry**, and most of its members are implemented today. The gap runs the **other way**: codes the backend already emits on the wire that the printed table never listed. The agent measured instead of obeying, and said so — which is the behaviour I want and the reason the result is worth anything.

Method: extract every backticked identifier in the contract and every code literal in the backend, enumerate the two vocabularies whose doc-comments cite §5.3 **by name**, and take the difference. **Seven** codes added, each with a verified non-test raise site: `code_index_unavailable`,

curriculum_unreadable, thresholds_uncalibrated, ingest_in_progress, request_cancelled, locality_unverified, generation_gated_pending_clarification. §5.6's disjointness enumeration was extended with the five answering-leg members, and disjointness from §5.5 was verified rather than assumed.

Exclusions were reasoned, not convenient. A probe code was excluded because it is a different field in a **200** body — a separate vocabulary, not a reason.code. CLI exit reasons and the pipeline contract's own vocabulary were excluded for the same structural reason.

Adjacent defects found and deliberately NOT fixed (they need a decision): §5.2 omits a code the contract itself names elsewhere plus four 002 codes; **two 503 conditions are specified with no reason code named at all**; and §5.1 says “Five closed enums” over a **six-row** table.

The identifier collision is not two ids — it is essentially total. Measured: **every one of feature 001's 120 T### ids, 21 SC-###, and every one of its 48 FR-###** also exist in feature 002 meaning something different. Only SC-016a is unique to 001. Verified different-in-meaning by sampling across all three families. **Nothing was renumbered** — that would invalidate every existing cross-reference. Both files now carry a prominent note naming the measured sets and the citation rule (001:T115 vs 002:T115). This is the defect that already produced two errors in briefs I wrote, and the agent recorded that claim **as reported and explicitly not independently measured** — correctly, since it is a claim about my history, not about the tree.

T070's note was stale and both its claims are WITHDRAWN by name, left visible rather than deleted: all three rows now carry the compound R1b token (3.6+002.4.2, 3.7+002.4.1, 3.11+002.4.4) with notes naming their changed clauses. One clause of the note survives and is kept. The “and contract” half was **not measured** and is recorded as COULD NOT DETERMINE rather than quietly counted as fixed.

Invariants held: checkboxes unchanged (001 = 59/61/120, 002 = 100/43/143); closure check **31 ids / 0 unattached** and **19 ids / 0 unattached**, corrected extractor intact; continuation-check.sh exit **0**.

Where to resume: the §5.2 omissions, the two unnamed 503 conditions, and the §5.1 miscount — all flagged, none fixed.

A30 — SC-006 MET. The cause was a silent 45% vector loss that had been re-occurring on every boot since generation 54. SC-015 still NOT met — 8/22, with a measured ceiling of 16/22.

The headline is not the latency fix. It is that the semantic leg had been searching 54.6% of the corpus while reporting ok. Live generation 63 held **1352 of 2478** vectors. The mechanism is a two-part interaction: a booting server mints a fresh index generation unconditionally, and the vector-indexing step treats the generation number as part of the identity of outstanding work. Pair those and every boot declares the whole corpus outstanding again, then starts re-embedding text that had not changed by a single byte. Generations **55 through 63 each attempted it and not one completed**, dying on SQLITE_BUSY after roughly five and a half minutes. Generations **52–63 all carry the identical root_hash and the identical pid_count 2478** — the content never changed; only the bookkeeping did. The database had grown to **59,347 embedding rows across 63 generations, 285 MB**, essentially all of it recomputation.

A second consequence nobody was watching: deriveCrossrefs runs only *after* IndexVectors succeeds, so cross-references had not been derived since generation 54 either. One silent failure disabled two subsystems.

The p95 breach was reproduced before anything was changed — 2267.0 ms against a 2000 ms budget — and attributed by measurement, not by inspection. Per leg: lexical 108.1, **semantic 1776.5**, code 2.6. health as a control was 4.1 ms, so the host was not merely slow. Two per-request terms, both measured independently: an ollama query embed whose p95 is ~8× its median under contention (698.0 vs 242.8 ms at load; 82.9 vs 34.6 idle), and **a full-corpus scan on every request** that re-read and re-decoded every vector blob *and* full passage text to produce a result identical across queries — 57–58 ms in SQLite alone. **Only the second is removable without changing what a query means**, and that is the one that was removed.

What shipped, and why each is safe: - **Carry-forward** — copies vectors from the generation with the most vectors at **identical root_hash and identical model**, one INSERT...SELECT, no provider call. Matching hashes imply the two generations enrol exactly the same passage-and-content pairs, and that implication is the whole of why the copy is sound — a derivation, not a reassurance. - **Per-generation candidate cache**, invalidated on generation and on an embedding row count the leg *already* queried, so it costs zero extra queries. **Redaction is deliberately NOT cached** — the redacted-pid set is re-read per

request (0.79 ms) and skipped at ranking time, so the redaction rule stays enforced at read time. Metadata maps are cloned after truncation; without that, the fusing step would have written back into the cache. - **In-window rerank** — a **permutation** of the served hit list, and nothing more. Membership and scores both leave it untouched; only position changes. That is why neither abstention nor the score floor can be disturbed by it — a structural consequence, not a promise. Its own determinism test caught that two upstream helpers sort ties unstably over map iteration, so only their scores are consumed and the final order is a stable sort broken by incoming position.

A self-inflicted bug worth recording, because of how it hid. The new boolean flags were first written in compose as two tokens (-flag false). Go's flag package sets the bool **true** and **stops parsing there** — silently dropping the entire remainder, including the whole answering block. **The container still came up HEALTHY**, merely reporting answering unavailable and thresholds uncalibrated. Fixed twice over: -flag=value in compose, *and* the server now refuses to start when any positional argument survives parsing. A health check that goes green on a half-configured process is the defect the guard closes.

root_hash unchanged throughout — 2478 passages before and after, generation 63 → 67, with the carry-forward logging 2478 of 2478 missing vector(s) copied ... nothing was embedded and cross-references re-derived to **49,560 edges over 2478/2478**, exactly matching generation 54's count.

SC-006 — MET. But do NOT quote a single p95 from this entry: the figure is strongly load-dependent, and an earlier revision of this paragraph quoting “p95 332.5 ms” as if it were a property of the system is WITHDRAWN. verify-search-latency.sh exits **0** — generation 67, 2478 passages, live — and its paired proof is **7 mutations, 7 caught, 0 missed**. Readings taken across the same afternoon, same generation, same corpus, same root hash:

238.6 ms	quiet host		600.6 ms	under agent load
332.5 ms	quiet host		964.9 ms	under agent load
			1436.4 ms	under agent load

A ~6× spread, and every one of them passes. Two agents measured the quiet readings independently and neither was wrong; the three loaded readings were taken deliberately while four subagents were saturating the machine. **The honest statement is the range, not a point:** SC-006 holds across everything observed, but the worst reading sits within **30% of budget**, so the margin is a property of host load rather than of the code. Median moved 184.5 → 684.1 across the same

span. *A latency gate quoted as one number is a gate quoted wrong.* The agent's own after-readings (529.8–1087.0 ms) were taken at **higher load than the failing before-reading** and over **83% more vectors**, which is what rules out “the host quieted down”. `go test ./...`

`-count=1` exits **0**, 17 ok / 0 FAIL; `workshop/scripts/verify.sh --static-only` exits **0** at PASS 7 / FAIL 0 / COULD-NOT-RUN 0.

The reranker is not a latency trade, and that was measured rather than assumed — ~0.4 ms at the live corpus average (20 docs × 330 chars), 26–28 ms at the API worst case (100 × 6000). At the default limit that is 0.02% of the budget. The served path agrees: lexical mode was p95 75.4 ms with rerank off and 69.3 ms with it on.

SC-015 — still NOT met. 8/22 against a bar of 20/22, moving from 5/22 both before and at full vectors with rerank off. Locus 560/560 PASS, negatives 6/6 PASS. The +3 sits outside the gate's documented ±1 repeatability, **but 22 queries support nothing beyond the raw count and no claim is made past it**. The decisive number is the **ceiling: 6 of the 22 targets are absent from the top-20 window entirely, so an in-window reorder can never exceed 16/22 here**. No amount of reranking closes this; the retrieval stage must change.

On the 26/12 passage benchmark, top-5 moved **16/26 → 19/26 (full vectors, rerank off) → 20/26 (rerank on)**, and **abstention held 12/12 in every configuration** — the property that must not move, and it did not.

A premise I supplied was wrong, and the agent caught it. The comparison figures I passed it are **not in SC015-FINDINGS.md on this checkout**. It re-measured instead of inheriting: the premise was substantially right but had **stopped being true of the live deployment**, and it failed to reproduce only because generation 63 had lost 45% of its vectors. **A benchmark figure is only valid against a COMPLETE vector generation, and a generation number alone does not identify one.**

Behaviour change that is not a contract breach but must not be discovered by accident: `/api/search results` is **no longer sorted by descending score** — verified live, ranks 1–8 read 0.0267, 0.0164, 0.0257, 0.0161, 0.0154, 0.0130, 0.0120, 0.0119. Scores are unchanged; only order moved. §3.7 of the 001 contract states no ordering rule, so **no written contract is violated** — but the de-facto ordering did change, and §3.7 should now say so explicitly.

Where to resume — three operator decisions, none taken: (1) cross-references are re-derived from scratch every boot, 49,560 edges in 55–62 s, and the same root_hash argument would carry them forward; (2) pruning the ~59,347 embedding rows is a **deletion** and needs authorization; (3) the query embed is now the dominant remaining p95 term. Also owed in files that agent did not own: §3.7's ordering statement, and the superseded SC-006 row in the limits document.

A29 — T037 was NOT unsatisfiable, and “needs a spec amendment” is WITHDRAWN. The spec never asked for the confidence interval.

The framing this item carried was overstated, and re-deriving it from the spec text rather than inheriting it is what caught that. T037 demanded “the measured figure **and its confidence interval**”. Its success criterion does not: **SC-002** asks for a figure measured on a random sample of at least 30 passages and published alongside the transcript; **FR-004** asks for a report stating measured accuracy and the method used. **Neither contains the word “interval”**. The demand traces only to a **research decision record**, not to the spec. So the earlier claim — that closing this needs either a spec amendment or a T112 change — is **withdrawn**: no spec amendment is needed, because nothing in the spec ever required the thing that was thought to be blocking.

It split across all three cases, not one.

Case 2 — computable but uncomputed, so it was computed. The research method requires that the passages overlapping the sampled windows be enumerated and counted. That enumeration had never been run; it existed only as an argument. Measured: 900.0 s sampled = **12.9894%** of 6,928.713 s, overlapping **174 distinct machine segments** — so **SC-002's “≥30 passages” floor is met by the window design at 5.8×**, now a computed fact rather than a plausible one. Per window: 50/70/83 words, 4/5/10 segments, **0 empty, 0 overlapping**.

Case 3 — blocked on a named operator input, which is not the same as impossible. `verify-accuracy.sh 01` exits **2** — “*–reference is required*” — and no accuracy artifact exists anywhere. What unblocks it is specific and affordable: a blind verbatim transcript of the 30 sampled windows in the plan's own reference schema, bounds copied exactly, unintelligible audio marked rather than guessed. Roughly **1–2 hours** at the research method's stated 4–8× realtime. T037 now states this in handover form.

The interval is blocked twice, and the second block is actionable today. Beyond the missing figure, a search for every interval-related identifier — bootstrap, Wilson, binomial, margin-of-error, the lot — returns **0 matches across project-authored .sh and .py**, and the verification payload has no interval field. Nothing computes an interval, so even a finished reference transcript would not produce one.

Mind the scope on that zero — I got it wrong once while checking it. Swept over the whole subtree instead of project-authored sources, the same pattern returns 5 hits, and every one is a vendored third-party artifact: a contributor surnamed Wilson in the whisper.cpp AUTHORS file, the word “bootstraps” in a C++ comment inside vendored ggml, and Wilson as a *token* in the CT2 model vocabulary and tokenizer. **None is code, and none computes anything.** Excluding engines/ and models/ returns 0, which is the figure above. Re-derive with the scope stated, or the vendored engine will answer for the pipeline.

SUPERSEDED the same day, and by this session’s own work. The project-authored sweep now returns **3 matches, all in verify-accuracy.sh, and none of them an estimator** — they are the *warning prose* added by the T037 re-emit (see A39), telling a future reader not to reach for a word-level binomial. **Nothing computes an interval; the count moved because the codebase learned to say so.** A zero that becomes a three without a single estimator being written is exactly the kind of figure that must be re-derived rather than quoted.

Two findings that should change what gets done BEFORE the human hours are spent. Both are new.

- 1. The estimator’s unit is a trap.** A word-level binomial over the ~2,057 in-window machine words would be wrong twice: words inside a 30-second window are not independent, and word error rate is not a proportion (insertions let it exceed 1). The honest unit is the **window — n = 30 clusters**. At the limit of perfect intra-window correlation, a word-level interval is narrower by up to $\sqrt{(2057/30)} \approx 8\times$ — it would *overstate the precision of the measurement*, which is the exact failure the interval was introduced to prevent.
- 2. A precondition defect in the sampling plan.** The research method specifies three windows in each of ten equal **temporal** strata; the emitted plan stratifies by **confidence** instead. Measured temporal occupancy is [5, 1, 3, 3, 3, 3, 2, 2, 5, 3] — 0 empty, 3 under-filled, largest unsampled gap **702.2 s (11.7 min)**. Stated honestly: the research method’s own worst case (“a 20-minute region

unsampled”) did **not** occur, so this does not disqualify the plan. But the guarantee is about temporal coverage and these strata are not temporal. **Re-emitting changes the seeded window set and would invalidate any reference already transcribed — so this decision must be taken BEFORE the 1–2 hours, not after.**

Box NOT ticked (- [] T037). Part (a) is done, but SC-002 demands a *published measured figure*; the artifact does not exist and B2 is not met. Method is not measurement. Closure check re-run after the edits: **31 ids, unattached: 0**, extractor untouched, no G- id minted; ticks unchanged at 59/61.

A self-caught defect worth keeping as a pattern. The agent’s first draft embedded its reproduction recipes as heredocs inside an indented list item — and an indented heredoc terminator does not close a heredoc. It verified that bash emits warning: here-document delimited by end-of-file, meaning a reader’s copy-paste would have **hung rather than run**. Both recipes were rewritten as single physical lines, then **re-extracted verbatim from the published file and executed**, reproducing the published output exactly. A published recipe that was never run from its published form is not a recipe.

Where to resume — and note the ORDER, it matters: (d) the operator’s sampling-design decision must come **first**; then (b) the operator’s 1–2 hours of blind transcription; (c) a T112 change is optional against SC-002/FR-004 and required only if the research decision record is to be honoured.

A28 — T096 CLOSED. A new feature-001 pipeline registry, and it caught a real defect on its first run.

The placement decision was measured, not preferred, and the measurement is the reusable part. Extending the umbrella’s scripts/check-registry.tsv was rejected because its R0 rule makes an absent scanroot **rc 2** — and workshop/ is a private submodule. Built as a sandbox with workshop/ left empty, exactly as an uninitialised clone leaves it, one added row took the **whole umbrella meta-check dark**:

```
⌚ UNDET [REGISTRY] declared scanroot 'workshop/pipeline' is not a
directory
COULD NOT DETERMINE – the registry could not be read, so nothing
was verified. rc = 2
```

One private-path row would have blinded a public gate for every reader without private access. REGISTRY is also hardcoded in the umbrella verifier, so it cannot be aimed at a second file without editing

a gate sitting at 43 PASS / 0 FAIL. Extending check-registry-002.tsv was rejected on a structural ground rather than a tidiness one: it has **no scanroot vocabulary at all**, so it cannot carry R5 anti-drift — which is precisely what its own scanroot-attribution debt row already admits.

The gate is run by an existing mechanism, not by intent. workshop/scripts/verify.sh discovers every platform/gates/verify-*.sh at run time; the new gate appears as **G4** with no edit to that script, and its V7 pairing rule requires the prove-*.sh sibling. That closes the “a proof nobody runs” failure mode **by construction**.

R5 here is stricter than the umbrella’s, deliberately: it sweeps *.sh and *.py **recursively**, where the umbrella uses -maxdepth 1 — which would have missed two of the three gates, since they live in a subdirectory. Demonstrated live on the real tree, not the sandbox: clean **rc 0**; a gate dropped into a brand-new subdirectory → **rc 1** with an UNREGISTERED row naming it; a stray .py → 2 more FAIL rows; both removed → **rc 0**. Supporting rules: a stale-prune ratchet, a **zero-sweep guard** (a sweep matching nothing is rc 2, never PASS), and a SELF rule requiring the instrument to be a row in its own registry.

prove-check-registry-001.sh — 22 mutations, 22 caught, 0 missed, including an unregistered .sh, an unregistered .py, one in a new subdirectory, a declared prune that must *still* exclude (no false red), a hollow proof under --run-proofs, and the zero-sweep → rc 2 path.

It earned its keep immediately. pipeline/transcribe/prove-chunker.sh was tracked at mode **644** — *a paired proof nobody could invoke*. Nothing had ever noticed. The new registry caught it on its first run; fixed with chmod +x, which strengthens a check rather than loosening one.

Two further unregistered gates were surfaced and registered as DEBT, not as check rows — detect_media.sh and detect_word_timing.sh. Their --prove-failure batteries exit 0, but measured: **0** hollow-proof-heuristic matches in either summary, and each one’s only reachable exit-2 path is its unknown-option handler, which the umbrella explicitly refuses to credit. Recording them as debt with those measurements is the honest classification; filing them as passing checks would have been the bluff.

A recursion bug in the agent's own battery was found by measurement, not by review: one mutation's --run-proofs made the sandbox re-launch the whole battery, running 10m29s without terminating. Fixed with a stub prover, reason recorded in the code.

Ran	Exit
scripts/verify-check-registry.sh	0 — 43 PASS / 0 FAIL / 0 DEBT (unchanged)
scripts/verify-check-registry.sh --run-proofs	0 — 62 PASS / 0 FAIL / 0 DEBT (15m13s)
workshop/scripts/verify.sh --static-only	0 — PASS 7 / FAIL 0 / CNR 0
verify-check-registry-001.sh	0 — 14 PASS / 0 FAIL / 3 DEBT
verify-check-registry-001.sh --run-proofs	0 — 18 PASS / 0 FAIL / 3 DEBT
verify-check-registry-001.sh --strict	1 — debt becomes failure, as designed
prove-check-registry-001.sh	0 — 22/22

Where to resume: the **3 DEBT rows** are real and unpaid. Also noted: verify-search-latency.sh / prove-search-latency.sh appeared untracked during this session, outside this scanroot — which is exactly what the cross-registry-attribution debt row declares, so it is watched rather than missed.

A27 — T040's third part: DONE. The blocking path was stale, not missing. Part 2 is genuinely incomplete — 7 deferred REDACT decisions.

Of the four possible explanations for the nonexistent path, the true one was (a): the file had been renamed, and the task text was stale. Not imagined, not merely ungenerated, not already-done-under-another-name. The correction had in fact landed on 2026-09-02, so nothing was blocking today — the remaining work was doable, and it is now done. The old note read oddly because the *parent directory* of the dead path does exist (it holds T037's accuracy-plan.json), which is a good reminder that “the directory is there” is not evidence the file ever was.

R2 asserted and met: 10 rows × 2 fields × 3 surfaces → 0 violations, exit 0. The re-emit turned out not to be a separate command — the applying run rewrites the document and both sidecars in place (8/8

targets reached, 30 artifact-level changes, 0 PROBLEM · 0 UNDETERMINED). The artifacts carry 11/10/10 redaction markers, so the assertion is **not vacuous** — a suppression check over a document with nothing suppressed proves nothing, and that was checked rather than assumed. Three corroborations, including one **measured false positive** that was recorded rather than quietly dropped: a 5-gram sweep hit 4 times on one distinct 5-gram which also lives in two non-redacted rows. The substantive corroboration counted Category-1 tokens recovered *by index* from the pre-redaction blob: **47 → 6**, the surviving 6 being the repo owner's own name, kept by an explicit KEEP decision.

Part 2 is NOT done and T040 stays []. The review records **17 decisions** — **13 REDACT / 3 KEEP / 1 NOT REMEDIABLE** — and operator decision 26 applied only the Category-1 subset, **6 of 13**. Seven REDACT decisions are deferred and unapplied. Resuming them is an operator call. **T038/T039 also stay unticked**, but their [PATH NOT BUILT] notes were withdrawn by name as demonstrably false — the tooling exists and has run. *Withdrawing a false blocker is not a completion claim*, and both were left unticked precisely so it cannot be read as one.

A historical FR-039 ordering violation, containment only. An earlier commit carried the transcript with **6 of 10** currently-redacted passages' exact text and zero markers; a later one is clean at 0/10 with 11 markers. The other 4 are **COULD NOT DETERMINE**. The remote is the **private** workshop repository, so this is not a public disclosure — but a push is publication and the review came second. Not closeable by this task.

Boundary check on the two edited PUBLIC files: leak probe 0 / 0 over 7 identifier tokens and pid-shaped strings.

Where to resume: the 7 deferred REDACT findings — sole blocker on T040 and therefore on T104. Also open: **no durable gate asserts R2 against the REAL chapter**. G-PID-5 asserts R1–R7 against the *invented* fixture corpus, which is deliberate — that generator exists so no gate ever reads private material. The measurement above is a reproducible [REVIEW] recipe, published in T040, **not a registered check**, and **no G-identifier was minted** for it.

A26 — the knowledge-graph pids are NOT REACHABLE in the running deployment, and it is not area-specific. Verified independently.

This is the largest technical finding of the session and it was found by correcting a documentation section. §10.3 of the private limits document claimed the live registry carried 5 areas. It was false, but not for the reason anyone assumed. **Those two figures had never counted the same population at all** — they were separate quantities that had been read as one. Four unrelated fives collide in that section — originally-authored kg_area rows, areas with an authored title, area-materials rows, and the areas held back as fully redacted (a *different* five).

The corrected conclusion is stronger than the one it replaces. The old text said areas are barely usable as graph-traversal origins — “only 5”. Measured, it is **none**: graph/traverse returned not_found for **60/60** sampled served areas and 5/5 titled areas, including the exact five the old text called reachable.

Two claims were withdrawn, one refuted by execution. “492 of 497 have no registry row” — withdrawn; all 500 taxonomy areas have a row on disk, plus 16 spare. “The other 492 resolve to nothing citable” — **refuted by running it**: 60/60 sampled areas returned real evidence entries with resolvable passages and text spans.

The mechanism, measured and structural — not staleness. graph_traverse.go resolves from against the **served** registry, a podman-volume file, not the working-tree corpus. I confirmed the served registry’s contents myself:

```
doc_section 1172 · transcript_segment 1055 · code 251  – and no
kg_* rows at all
```

grep -cF over the ingest script for kg_area, kg_term, knowledge-mint, taxonomy and the corpus path returns **0 for all five** — meaning the shipping pipeline, as written, never moves a knowledge-kind record from the corpus into what the server actually reads.

It is not about areas, and that is the larger half. Probing /api/passages/ with 8 pids per kind: code / doc_section / transcript_segment answer **200, 8/8 each**; kg_area / kg_term / kg_todo / kg_next_point / kg_open_question / kg_meeting_note answer **404, 8/8 each**. Traversal

from a term pid is not_found too. So the term minting and the “terms are nodes of this graph” property are true **of the corpus and of the code**, and untrue **of this deployment**.

Independently re-verified before recording (not taken on the agent’s word), against workshop-curriculum_platform_1 on 2026-09-03: the served-registry kind histogram above reproduced exactly; a live area pid gave {"from": "...", "status": "not_found"} from traverse, **HTTP 404** from /api/passages/{pid}, and **HTTP 200** from /api/areas/{id}/evidence. **The knowledge layer is served through its own endpoints; only pid-resolution through the served registry fails.** Do not restate this as “areas are broken” — the areas API works.

COULD NOT DETERMINE: whether re-running ingest would change any of it. The static reading of the script says no. Running it **mutates the served volume** and is an operator decision, so this is recorded as a reading of the script and explicitly **not** a measurement of its effect.

Also found, not fixed: 16 orphan kg_area rows with no taxonomy record — the taxonomy file predates the corpus by ~13 hours. Recorded, not reconciled.

The new anchor for area-registry-sync-gap was proved stable four ways, and the design rule is the durable part: it is a body sentence, **contains no digits**, and states the defect’s invariant rather than a symptom. Proof on a throwaway copy — control 0; anchor line deleted → 1, naming the row exactly; **\$10.3 fully retitled → 0** (immune to the exact failure that caused A25’s red); every count in the document rewritten → 0. verify-limits-completeness.sh stayed at **0 with 15 defects** before and after, and prove-limits-completeness.sh exits **0**.

Where to resume: this belongs to the terms-no-minted-pid row, which was deliberately not touched. The open question is whether the served registry *should* carry kg_* rows — a design decision, not a bug fix — and it bears directly on spec 002, whose entire subject is knowledge areas and deep linking.

A25 — verify-limits-completeness.sh 1 → 0. It was already red at HEAD, and the word-precision mechanism is WITHDRAWN.

The red was not caused by this session’s work, and that was proved rather than asserted. Reproduced against pure HEAD: blobs extracted to a scratch tree (git show HEAD:docs/limits.md, HEAD:platform/gates/defects-registry.tsv, HEAD:...verify-limits-completeness.sh):

PROBLEM: 1 of 15 ... MISSING area-term-over-generation, EXIT=1, with git diff HEAD --stat empty on all three paths. **No defect row was deleted — still 15 rows, 15 named.**

Root cause worth keeping: the gate went red for a document that had been CORRECTED. The anchor quoted *heading text*, and the §10.9 retile removed it, so `grep -cF` scored 0. An anchor that quotes prose is a tripwire on the prose, not on the defect. The new anchor deliberately carries **no moving count** — re-measurement showed the counts had drifted again, areas **497 → 500** and terms **8551 → 8537**, so 497-area/8,551-term was removed from the heading and the drift recorded in-section. 7/37/137 is hand-authored and does not move with a pipeline re-run.

§10.1 — the conclusion stands, the MECHANISM is withdrawn. The claimed mechanism (aggregate `precision_split` of `{"word":0,"segment":N}`) was wrong about both the values *and* the key set: measured, it is `{"word":77,"segment":9,"no_time_span":52}`, and `enrichWordPrecision` (evidence.go:331/361/363/364) upgrades in place. The real finding is the code one, and it is the durable part: **wordjoin.go:127-128 returns `sub[0].StartS → sub[len(sub)-1].EndS`, the whole segment's word range**, so `precision:"word"` reports *sidecar completeness, not span width* — it overstates itself.

One caveat on the agent's own wording, recorded so the next reader is not misled. It concluded “word spans are not narrower”. Its figures: at `n=12`, word median **6.34 s** vs segment **6.30 s** (segment `n=1` — one observation, not a median, which it flagged itself); widened to 120 areas, word `n=77` median **7.28 s** vs segment `n=9` median **7.88 s** — where word is *slightly* narrower. The defensible reading is **no consistent difference in either direction**, not that word is never narrower. The code finding above does not depend on it.

§10.8 rewritten, both halves, from live evidence. `question: startup log knowledge catalog registered – 8586 row(s) total (area 5, term 8537, question 44), 44 authored on disk (9+9+10+8+8), live /api/search lists question in indexed_kinds and returned 2 kind:"question" rows. lesson_section: 0 of 11,622 records in passages.jsonl, and LessonSectionMeta (materials.go:31-37) carries no identifier field — structurally unkeyable. The old stated reason (“no content exists”) is itself now false: area-materials.jsonl carries 35 lesson-section entries.`

The flaky test was fixed at its cause, and the honest part is what did NOT reproduce. Nine serial runs all passed — the reported FAILED was **not** reproduced serially, and that is recorded in §10.16 rather than

glossed. It was reproduced at the cause instead: concurrency. Eight simultaneous `probe_toolchain()` calls returned two different error texts in equal measure — half `[object Object]`, half a `@puppeteer/browsers ... launch.js trace` — while the *verdict* was UNUSABLE in all eight. A §1.1 paired proof then ran both predicates over one shared batch of sixteen: the old string-matching assertion survived thirteen and failed three; the new one survived all sixteen; the verdict was UNUSABLE sixteen out of sixteen. The assertion therefore moved off the error *string* and onto the *state* (`ToolState.UNUSABLE + mmdc`). Observed failure rates across batches — 4/8, then 0/8, then 3/16 — are **nondeterministic**, which is precisely why a green batch settles nothing here.

command	exit
<code>verify-limits-completeness.sh</code>	0 (was 1)
<code>python -m unittest discover</code>	0 , ×5 (Ran 255 tests ... OK)
<code>verify.sh --static-only</code>	0 (PASS 7 FAIL 0 COULD-NOT-RUN 0)
<code>audit-hardcoded-paths.sh</code>	0 , <code>limits.md</code> not flagged

§10.3 is now measurably FALSE and was deliberately left alone. Its heading claims the live retrieval registry carries 5; measured, `passages.jsonl` has **516** `kg_area` rows. Correcting it means re-anchoring `area-registry-sync-gap` and re-deriving a conclusion (“areas are not usable as graph-traversal origins”) nobody has measured. **It needs its own pass — this is open work, not a closed item.**

Where to resume: §10.3 above. Files changed and uncommitted: `workshop/docs/limits.md`, `workshop/platform/gates/defects-registry.tsv`, `workshop/pipeline/extract/test_export.py`.

A24 — the gate-attachment zero was BLIND, and it hid a standing contract violation. A22’s “attachment 0 in both” is **WITHDRAWN** as stated.

A22 recorded “Gate attachment 0 in both”. The number was right and the claim was wrong, and the difference is the entire content of this entry. Feature 002’s closure check reported `unattached: 0` because it could not SEE the id it should have reported — not because nothing was unattached.

The defect. The closure check is not a script; it is a recipe published inside each `tasks.md` (`grep -lF 'G-[A-Z]'` finds it in those two files and nowhere else in the tree). Its extractor was `G-[A-Z]+-[0-9]+`, which is greedy only through the digits. Run it over `G-KG-1-changed` and it yields

G-KG-1 — a *different gate, already attached to T040*. The compound id never entered the loop, so it could be neither attached nor reported unattached. It was invisible, and invisible reads as clean.

Widening the extractor is only half the fix, and the second half is the subtle one. Once both G-KG-1 and G-KG-1-changed are in the id set, the old boundary guard (`[^0-9]|$`) matches G-KG-1 against a task line mentioning only G-KG-1-changed — a bare gate reported attached on the strength of a *different* gate's citation. Both halves were proved on a fixture whose answer was known by construction (one task line building only G-KG-1-changed): the old pair called all four ids attached; the new pair correctly called G-KG-1 UNATTACHED.

extractor	G-[A-Z]+-[0-9]+	->	G-[A-Z]+-[0-9]+(-[a-z]+)*
guard	(<code>[^0-9] \$</code>)	->	(<code>[^0-9A-Za-z-] \$</code>)

The correction discriminates; it does not merely redden. On feature 001 it changes nothing — **31 ids before, 31 after, unattached: 0 both ways**, and 59/61/120 unmoved. A fix that only ever finds fault is indistinguishable from a broken instrument. On feature 002 it sees **19** ids where the old form saw 18, and the nineteenth reported UNATTACHED G-KG-1-changed.

What the blind zero was hiding is worse than an unattached id. G-KG-1-changed asserts R1b: every §4 *changed* endpoint carries a manifest row citing **both** its 001 section and its §4 subsection. Measured 2026-09-03, the compound citation form already existed and three of four changed endpoints carried it — 3.6+002.4.2 (/api/suggest), 3.7+002.4.1 (/api/search), 3.11+002.4.4 (/api/progress). **§4.3's answering row carried a bare 3.10**. So the gate nobody could see was a gate that would have been **red**, over a real standing violation, not a bookkeeping slip. (§4 has **four** changed endpoints, not three — POST /api/ask is §4.3.)

Fixed, and deliberately not over-fixed. /api/ask?q=ping now reads 3.10+002.4.3 plus a note naming C4.3.1–C4.3.4 and stating which of them a live probe does **not** claim. Exactly one line changed (diff = 2 sides); verify-server-unity.sh re-run after each edit: **PASS=35 FAIL=0 UNDET=0 DEBT=4, exit 0**.

T040 was the wrong place to attach the gate, and attaching it there would have been the papering-over the section forbids. T040 builds manifest rows for the endpoints *new* in §3; R1b governs rows that existed since feature 001 and were only amended. New **T143** owns it and is [] UNTICKED.

CORRECTION, and it is worse than what it replaces. I wrote “the gate is not built” — twice, here and in the delta contract. That is **WRONG and is WITHDRAWN**. The gate G-KG-1-changed **exists**, is **registered** as a Go test in the 002 registry, has a paired prover that performs the required *rewrite* mutation, and **passes**. I verified all of it directly.

It passes because its enumeration omits the one endpoint whose citation was wrong. Measured in the test itself: it enumerates 002.4.1, 002.4.2 and 002.4.4, covering /api/search, /api/suggest and /api/progress — and **002.4.3 / /api/ask appears zero times**, in the gate *and* in its prover.

```
gate enumerates:  002.4.1  002.4.2  002.4.4           <- three of
four
§4 actually has:  4.1   4.2   4.3   4.4           <- 4.3 was the
broken one
gate verdict:      ok (passes)
```

So this is the SIXTH failure mode in this family, and the sharpest one: a gate that is green because its coverage set excludes the defect. Not a blind extractor, not a wrapped line, not a truncated token — a correctly-written, registered, mutation-proved gate whose want table is simply short by one row. **Every instrument around it was working. The gate asserted exactly what it enumerated, and it enumerated three-quarters of its own subject.**

The remaining work is therefore **one table row plus a fourth mutation case**, not a gate from scratch — smaller than recorded, and T143’s note now says so. The standing lesson is not smaller: **a gate’s coverage set is itself an assertion, and nothing was checking it.** Ask of any passing gate not only “what does it assert” but “over what set” — and whether that set is derived or hand-listed. This one was hand-listed.

The end state is unattached: 0 again — a different zero. Before, one id was invisible; now 19 are enumerated, the compound one is seen, and it is carried by an explicitly unticked task. Identical count, opposite epistemic content. Spec 002 moves **100 ticked / 43 unticked / 143** (was 100/42/142) — work added, none claimed. continuation-check.sh exit **0**, 8 PASS · 0 DRIFT · 0 UNDET · 6 NOTE.

This is the THIRD distinct way this same check has lied, and the two earlier ones are recorded beside it in 001/tasks.md: (1) a set-difference over ids anywhere in the file went green the moment a *paragraph describing* the missing gate mentioned its id — documenting the defect

fixed the check; (2) a single-line anchor reported five ids unattached in 002 purely because their id fell past a line wrap. Presence, line-shape, and now token-shape. **Re-derive with the recipe, never quote the number.**

Where to resume: T143 is real unbuilt work. Nothing else is owed by this item.

A23 — full retest + full boot, 2026-09-03. Env audit 32 → 0: ONE real fix, 31 reasoned exemptions.

Every suite run explicitly, not as a push side effect. Pre-push gates **8 passed / 0 failed / 0 undetermined / 0 SKIPPED** — including Playwright chromium at 205 s; a SKIP here would not have been a pass. Governance: continuation-check, cascade, manifest-pins, remote-sync, check-registry, hardcoded-paths, name-in-path and the sweep paired proof **all 0**. Workshop: `verify.sh --static-only 0, go build/go vet/go test ./... clean, 0 FAIL lines.`

System booted clean: `setup.sh --check-only → RESULT 0 READY,` `build.sh 0, restart.sh → status.sh RUNNING` with health in 2 ms, web bundle 200, `question_verifier_kind: question-focus+llm`, review gate holding (reviewed detail 200, unreviewed 404), `coverage/export/evidence 200` on both. **The capability probe reports mermaid UNUSABLE by actually rendering a graph** rather than trusting `mmdc -V`.

A transient SQLITE_BUSY appeared mid-probe and cleared itself — and the server's behaviour under it is the point. It returned `status: unavailable, code: lexical_index_unavailable`, with the raw database is locked (5) as evidence — **a determined unavailability, never an empty result set dressed as “nothing matched”.**

A scare of mine, settled by measurement rather than inspection. A query that returned 4 results earlier returned 0 after the rebuild, which looked like a lost vector set. `verify-retrieval-benchmark.sh` reproduced its prior value **exactly — top-5 5/22, top-1 4/22, with 560 hits all carrying a resolving locus.** Nothing was lost. What changed is which side of the **0.655 calibrated floor** a query lands on: that is the abstention the nomic switch bought, returning less and clearing a measured threshold.

Env audit 32 → 0, and the split is the finding. ONE was a real freeze: `cmd/index-embed/main.go:57` defaulted `-ollama` to a bare loopback URL **with no env layer at all** — the only binary in the platform whose

backend address had no variable behind it, while the workshop-server it mirrors already read \$WORKSHOP_OLLAMA_URL. Fixed to read the same variable. **The other 31 were not defects**, earning # REASON: rows with evidence: **23 rules added, 0 # BASELINE: rows** — verified independently, so no debt was moved between columns.

Two of those judgements are worth keeping: -

floor_calibration_test.go (9): the literal is the KEY of a measured row, not a configured value. calibratedFloor() keys on model *and* both prefixes; the actual model in force is already operator-controlled via WORKSHOP_EMBED_MODEL. **Env-deriving the key would let ANY model inherit nomic's measured separation window — which is a bluff**, and would collapse seven neighbour cases into one, deleting the test. - **verify-all-constitution-rules.sh (3 GITREF): not git refs at all** — two commit messages and a report sentence containing the English word “upstream”. Measured: `grep -n remote` over that file returns **zero** lines, and its spec_git sandbox repo has no remote of any name.

The agent caught ITSELF leaking, and that is the most valuable thing in the run. Writing the REASON rows it echoed engineering prose out of PRIVATE workshop/ source comments into the PUBLIC allow-file — **precisely the failure mode the incident note names: “an agent asked to document the work comprehensively reaches for an illustrative quote.”** The boundary gate attributed **11 surviving matches across 5 lines** to its block; it reworded all five to carry the same evidence in its own words and re-measured to **0 rows naming its block**.

Rot census after: **452 rules, 0 STALE, 0 PATH-ABSENT**. All 15 touched files are tracked, so the untracked blind spot does not apply and these stay clear.

A22 — final reconciliation: spec 002 100 / 42 / 142, spec 001 59 / 61 / 120. Gate attachment 0 in both.

Three ticks earned, each with the command: T079 (coverage returns **200 for 495 of 495** areas, probed one by one, derivation reading "threshold": "NONE"), T083 (export 200 on **both** sides of the review gate — published exportable:true, unpublished exportable:false with 4× precondition_blocked), and T102. **Spec 001 earned NO tick and none was invented** — notes only.

Deliberately not ticked: T057/T058 (3 of 4 kinds), T052, T101, T037.

A figure this document had been repeating is WITHDRAWN: the answering benchmark's "3 fabrications" is stale — it is ONE, per the 2026-09-02 A/B run. The stale figure had propagated into T077/T078/T068's notes.

lesson_section is not merely unindexed, it is STRUCTURALLY UNKEYABLE — 0 of 11,622 registry records, and LessonSectionMeta has **no identifier field at all**. That is a stronger reason than "no content yet", and it is now pinned by a named test.

Gate attachment unattached: 0 in BOTH specs — 31 gates in 001 (line-anchored form), 18 in 002 (wrap-aware block form) — with the G-CLI-17 discrimination probe still returning 1, so the check is still capable of failing.

Both files gained a standing rule for the SC collision, with a verified re-derive command. That collision has now caused **two** errors in this session, both in briefs I wrote.

A22a — five things left stale ON PURPOSE, each named rather than smoothed

- 1. verify-content-boundary.sh reports LEAK – 11244 against CLAUDE.md's recorded 285.** Pre-existing and **CANNOT DETERMINE** without editing files outside that agent's scope. Its own contribution was measured at **zero**: it introduced 19 matches, spotted them, rewrote every passage and re-ran; the only flagged lines remaining in its files are **pre-existing task descriptions that private session briefs had copied public→private**, flagged solely because a checkbox flip touched the line.
- 2. workshop/docs/limits.md §10.8 still says question is not indexed. That is now FALSE** — 44 distinct search entries match 44 served questions.
- 3. A PAIRED-PROOF FLAG GAP, not a proof gap — and the distinction matters.** `verify-sc009-citation-span.sh` and `verify-sc024-export-matrix.sh` both return **rc=2 unknown argument** for `--prove-failure`. Verified: **standalone prove-sc009-citation-span.sh and prove-sc024-export-matrix.sh both EXIST and both exit 0**. The proofs are real; the registry's `--prove-failure` convention is what they do not answer. Fixing it is a flag, not a proof.
- 4. Spec 002's own count block was ALREADY WRONG before this reconciliation began** — it claimed 96/46 while the tree measured 97/45, because T055 was ticked without its note or the block being updated. **The bookkeeping drifts even within a single session.**

5. **T037 cannot be satisfied as written, and this is now measured rather than argued:** it demands a confidence interval, `accuracy.json` has no field for one, and `grep -rin 'confidence_interval|ci_low|ci_high|bootstrap|margin_of_error'` over `scripts/` and `pipeline/` returns **0**. Either T112 gains a CI or T037 is amended — an operator choice, not an implementer's.

A standing caution recorded verbatim from that run: “Treat my counts as a reading of one moment.” Another process wrote this tree during it — `export.py` changed 3 minutes after it was measured, and a cited line moved 325 → 366. Every conclusion was re-verified afterwards and fragile line numbers were dropped from the notes.

A21 — spec 001 pipeline (T014/T025/T032) and spec 002 search (T069/T070/T066/T067) BUILT. Two tasks refused on false premises.

T014 — P-U1 SETTLED, and the rejection is the finding. Lumen's SQLite store was tested as a symbol producer and **REJECTED on measured grounds** — note its premise was partly wrong, since the store *does* expose a `symbol` column: **948 duplicate (file_path, symbol) keys** across 14,650 Go rows (a non-unique key cannot identify a passage); **0 of 3,007** Go method rows carry the required shape, with 363 rows fusing several declarations under one label; and a key **vanished** after a body edit plus reorder, which §6.3 says identity must survive. **A5 — schema and rows survive a forced reindex — PASSED.** That was the one property the task named for this candidate, and it is not enough: testing only it would have confirmed the wrong producer. The replacement was confirmed at scale: **11,742 symbols, 908 Go files, 8 trees, 0 duplicate keys, 0 parse failures.**

Two of its five mutations initially SURVIVED, and both survivals were defects in the MUTATIONS, not in the gate — one keyed an injected duplicate on a fixture-only identifier while the check runs against a real tree; the other shuffled with `awk's rand()` and **no `srand()`**, so it produced the identical “shuffle” every run. **A mutation that cannot vary cannot prove anything**, and both are recorded rather than quietly re-rolled.

T025 — the chunker found a real property of the recording and refused to paper over it. At the contracted default `--min-silence 0.6` the chapter-1 audio **cannot be chunked**: it places 3 chunks, then meets a 270-second stretch whose longest pause is **0.50 s**, and exits 1 rather than cutting inside speech. At 0.5 it chunks cleanly into 25. **The default was deliberately NOT changed — “fitting a constant to one**

recording is what `audio_energy.py` exists to avoid — and instead the refusal now **MEASURES and REPORTS the longest available pause**. Also measured: noise floor **-82.1 dBFS at 0.25 s windows** against `AUDIT.md`'s **-32.6 dBFS at 1.0 s** on the same file. **Neither is wrong: a noise floor is a property of the recording AND the window.**

T032 — privacy enforced structurally, not requested. The notes-PDF extractor refuses any destination git would track, checked with `git check-ignore`, **with no override flag**; no output field carries running text; every object is stamped `not_ground_truth: true`. The justification is measured: of 229 gazetteer terms, **124 classify person_like** — over half. Its proof's M9 **removes the guard and requires the harm to occur**, which is what proves M5 tests anything.

T069/T070/T066/T067 — surfacing and benchmarking. The search view now sends kinds and area and renders locus; kind chips derive from `corpus.indexed_kinds` so **lesson_section can never appear**; the area select lists the **5 titled areas of 495** and *counts* the 490 untitled rather than listing bare ULIDs. Verified live rather than assumed: `?kinds=term,area` yields `["term", "area"]` while `?kinds=term&kinds=area` yields `["term"]` — repeated pairs are not accumulated. **Nothing renders precision: "word" as a word-width span**, recorded in the header. Frontend suite 90 → **97 SUCCESS**.

A new `verify-retrieval-benchmark.sh` is **RED BY DESIGN and correctly so**: top-5 5/22 (22.7%) against a 20/22 bar, with per-query detail showing `question_stem` landing at rank 1 four times of five while **every** `area_title` and `term_name` query fails unfiltered. Repeatability measured over four runs at identical `root_hash`: 5/22 three times, 6/22 once — **the ±1 spread is recorded in the gate header so a one-query move is not misread as a change**. Its M0-control proves the gate *can* go green, so the live red is a measurement and not a constant.

Two refusals on false premises: T038/T039 are ALREADY BUILT (`redact.sh` 172 lines, `redaction-review.json` 87 lines — the ground-truth report is superseded), and **T109 asserts outputs owned by stages not yet built**, so a gate for it could not pass.

Content boundary re-checked after a FALSE ALARM OF MINE. A malformed `grep` of mine (`-l` with `-c`, and scoped to all of `evidence/` rather than the files the agent wrote) appeared to show 15 files carrying name tokens. Re-derived: **all 18 hits are the OWNER's own**

identity — `github.com/milos85vasic/...` in `go test` output and `/run/media/<owner>/...` in pre-existing evidence — and **none is the third party**. `audit-hardcoded-paths.sh` independently exits **0**.

Handed on, not absorbed: T015/T016/T017 are now **unblocked** with a named interface (`registry.go:628 KeyStrategy`); **SC-016a is not implementable for TypeScript** because the producer is Go-only; **all 251 kind: "code" passages carry .md source refs**, so nothing in the corpus is keyed on a symbol today; **T096 bit directly — there is no registration point for feature-001 pipeline gates**, so three new proofs are enforced by nothing; and `lumen index hung` **>9 minutes** on a two-file fixture then took **111 ms** minutes later, cause **CANNOT DETERMINE**, now bounded by timeout reporting **UNDETERMINED** on expiry.

A20 — T102 and T052 BUILT. Four tasks REFUSED with reasons. Two defects were concealing each other.

T102 — and the second defect only became visible once the first was fixed. `export.py:713 hardcoded review=None`, as recorded. Behind it sat a second: **area_id was passed as a FILESYSTEM PATH while reviews are keyed by the minted ULID**, so `check_publication_precondition`'s subject-mismatch guard would have rejected **every** review even if one had loaded. **Neither defect could be observed while the other stood** — the hardcoded `None` meant no review was ever looked up, so the key mismatch never had a chance to fail visibly. Fixed by reusing the production identity path `verify.check_w3_over_real_materials` already established. `real_materials_export_status` is now **read-only by default**; producing artifacts is an explicit step.

```
before  KNOWN GAP: 5 of 5 real area document(s) export
nothing                                rc=1
after   0/2 complete – determined ABSENT (toolchain usable, never
produced) rc=1
--produce-real-materials 2/2 areas carry all four
formats                                rc=0
```

SC-012 — exports now honour redactions.jsonl, and the guard OUTFRANKS the review check. Measured today: **0 of 97** distinct cited pids resolve redacted against 10 redacted pids — so the guard is **green by measurement today and enforced by construction from now on, which are different facts** and are recorded as such. The publication act itself was verified rather than assumed: **116 six-word shingles**

drawn from the 10 redacted passages, checked against all 8 produced artifacts (4 formats × 2 areas), extracting PDF via pdftotext and DOCX via pandoc — **0 hits**.

T052 — SC-009 now has a gate, over TWO FULL POPULATIONS, neither sampled. 96 of 96 served media-backed citations land inside their cited span; 3 of 3 authored cite: PID markers do too. `precision_split` over the cited subset: **word 85, segment 11. An honest boundary is stated in the gate itself:** the `recording_seek` check is **NOT independent today** — 95 of 95 equal `t_start_s`, so the agreement is structural — and it is reported separately **so nobody reads it as corroboration**. The cited-subset distribution is printed beside the spec's corpus-wide figures so the two populations are never conflated.

Verified independently, both directions of the review gate: reviewed area → `exportable true`, 4 formats present with hrefs and sizes; unreviewed → `exportable false`, 0 formats, and the **source_document and lesson_sections keys ABSENT entirely** — not null, absent. Nothing bypassed.

FOUR TASKS REFUSED, each with a measured reason — and one refusal is the important kind. T031 (the prose-authorship clarification is genuinely open; `spec.md:163` still reads OPEN), T042 and T050 (premises re-verified unchanged; not this surface). **T033 was refused because closing it means hand-auditing ~161 claim blocks and rewriting them as citations or editorial blockquotes — i.e. AUTHORIZING AREA PROSE, which the brief forbade.** Measured: 177 of 532 claim blocks violate W2 raw, and the 3 unreviewed areas carry 161 of them.

Proofs: `prove-sc024-export-matrix.sh` **8 of 8 mutations caught** — including M8, which refuses the reference implementation's own "34 published / 25 complete" figure as a bar — and `prove-sc009-citation-span.sh` **9 of 9**, with M6/M7/M8/M9 distinguishing `rc=1` from `rc=2` correctly. Gates directory byte-identical by sha256 before and after both runs. Index `root_hash sha256:03d91626...` unchanged; no rebuild was needed.

Three findings handed on rather than absorbed: `verify-limits-completeness.sh` was **already RED at HEAD** — reproduced against HEAD: blobs, its anchor vanished when \$10.9 was retitled yesterday, so it is not this work's doing; `test_export.py::test_probe_toolchain...` is **FLAKY**, freezing the failure string `[object Object]` when real `mmdc` fails two different ways (the verdict is UNUSABLE either way); and

platform/gates/verify-retrieval-benchmark.sh appeared untracked and **unregistered**, surviving only because the registry's own scanroot-attribution debt row means there is **no directory-wide anti-drift over platform/gates/**.

A19 — T055 WIRED. T101 confirmed the WRONG fix and left unwired, with a false claim corrected.

T055 — the transcript now reaches the knowledge graph. GET /api/passages/{pid}/knowledge was live and answering 200 with real content; **nothing in the SPA called it.** Now wired as a sibling of crossrefs.component.ts — same <details> shape, same load-on-open trigger (the window is 500 passages; an eager fetch would be 500 requests per screen), same app-unavailable handling. **Five outcomes render as five different sentences** — attached, unattached (the server's own unattached_reason, verbatim), redacted (410), not-in-registry (404), unavailable — and attached:false is carried as ready rather than empty(), **so A3.7.3's reason survives the client boundary** instead of collapsing into a blank panel. SC-022 design-token scanner 5/5.

One judgement worth keeping: term links use the server's own href and were deliberately NOT pointed at /search?q=..., because search.component.ts reads no query parameter — that link would have landed the reader on an empty search box reading as “nothing found”.

T101 — wiring render_diagram into export_area is the WRONG fix, re-measured today on three independent grounds, **any one of which is disqualifying:**

- 1. The renderer cannot run, and its version query LIES about it.**
mmdc -V → **11.12.0, rc=0** — re-confirmed independently. The real probe: verify_export.py --mmdc-evidence reports mermaid = UNUSABLE ... could not render a 2-node graph to SVG (rc=1): [object Object], and render_diagram(...) -> outcome=toolchain_unavailable code=2, svg written: False. **This is the /usr/bin/whisper trap in a second guise: a name on PATH that answers -v and cannot do the job.**
- 2. There is nothing to render** — grep -c mermaid docs/training/areas/*.md returns **0** for all five area documents.
- 3. The source-location convention is unmade** — the seven-heading skeleton has no diagram heading, and inventing one is a content decision.

Wiring the call would deploy a code path that cannot succeed, on inputs that do not exist.

A false claim found and corrected: docs/training/diagrams/README.md justified source-form diagrams with “*which render natively in the platform frontend*”. **Independently re-verified as FALSE: 0 mermaid dependencies in package.json and 0 frontend source files referencing it.** The fences do render in GitHub/GitLab and editors — that half holds — but not in this platform. Corrected in the README and recorded in docs/limits.md §10.5.

E2E, with the recorded trap defended against. A beforeAll guard fails loudly on connection refusal, so the ERR_CONNECTION_REFUSED run that once recorded “20 expected, 58 unexpected” cannot recur silently. New spec **16 passed**; unit suite **90/90**; a11y + design-tokens **42 passed, 1 skipped**. Its own §1.1 paired mutation: comment out the one template line → **8 failed**; restore → **8 passed**.

4 recurring full-suite failures are PRE-EXISTING — measured, not inferred. The affordance was removed from the template and the suite re-run: **the same 4 failed with identical errors**. Root cause for three: liveTerm() assumes the top search hit is a transcript passage, and now gets a spec-002 term hit (pid='kg_terms:index', chapter_slug=None) — so /api/passages/kg_terms:index 400s and /chapters/undefined times out. **A 5th failure is honestly CANNOT DETERMINE:** an intermittent axe color-contrast on /, appearing once and passing 22/22 on re-run; the signature is a dark-theme colour over a light-theme surface, i.e. a theme-application race, but the mechanism was not established and was not guessed.

Index content unchanged across four mandated rebuild+restart cycles: root_hash sha256:03d91626... and pid_count 2478 identical throughout, floor 0.6550 calibrated=true. The generation counter moved **54 → 61** purely because main.go:552 calls index.Build(...) on every boot. **Quote root_hash, never the generation, when you mean “content unchanged”.**

A side effect the caller must know: the mandated build.sh + restart.sh cycles **compiled and deployed other agents’ UNCOMMITTED backend work** that landed mid-session — coverage.go, export.go, areas.go, question_catalog.go, service.go, main.go. Every build exited 0 and the stack is healthy, but **the running binary is newer than the session’s start state**.

A18 — T079/T083 LIVE, T057/T058 resolved — and a .gitignore rule was silently eating the new handler

GET /api/areas/{area}/coverage and /export: 404 → 200, verified live on a reviewed area. Both were unmounted **and** undeclared, which is exactly why `verify-server-unity.sh` stayed green throughout: **a route missing from BOTH the binary and route-manifest.tsv is invisible to its U5/U6 pair by construction**. Coverage serves `assessment.BuildCoverageReport` — the G-KG-12-gated function — directly, with no second arithmetic.

The review gate was respected, not bypassed (decision 41): export sits behind it by design per A3.9.1; coverage does not, because §3.6 is silent and the §3.5 data it measures is already ungated. Verified: the unreviewed area still returns **404** from `/api/areas/{id}` while `/export` returns 200 with `exportable:false`, four `precondition_blocked` formats, and **no href and no source path leaked**. It answers 200 rather than 404 because A3.9.1 requires it to “export nothing **and say so**” — and saying so needs a body.

T057/T058: the earlier assessment was HALF stale, and each half went a different way. - question — no longer refusable, so it was INDEXED. T078’s loader exists and 44 authored questions load and serve live. The recorded reason (“no loader, no real content”) had simply stopped being true. The load-bearing detail: `BuildQuestionEntries` applies `assessment.ServeQuestions` (G-KG-2), so a question whose citation stops resolving **vanishes from search exactly as §3.5 withholds it** — without that, search would have been a hole in A3.5.1 with both endpoints still answering 200. - **lesson_section — still correctly refused, and now GUARDED**. Zero `kg_lesson_section` records of 11,622; `knowledge.Sync` is called from `no cmd/` entry point; the 53 ids questions reference are all `doc_section`, a kind already indexed. Left unindexed **with the measurement recorded in code** and pinned by `TestT057_LessonSectionIsNotAdvertisedBecauseNothingIndexesIt`. **Not indexing an empty kind to tick a box is the right outcome, and it is now defended.**

THE SHARPEST FINDING — the defect reintroduced one layer down. `.gitignore`’s `coverage.*` rule was **silently swallowing internal/api/coverage.go**. `git status` listed nothing, so a `git add` on that directory would have dropped the handler and **the endpoint would 404 for anyone cloning the repository** — the exact defect the task existed to fix, recreated by an ignore pattern. Two path-specific negations already existed for earlier occurrences of the same rule; it is

now generalised to `!**/coverage.go` (safe — Go writes `coverage.out`, `.html`, `.coverprofile`, never a `.go`). Re-verified independently: `check-ignore` reports the negation at `.gitignore:47` and `git status` now shows the file. **The rest of the tree was audited; no other source file is swallowed.**

The live index was NOT rebuilt: `pid_count` 2478 and `root_hash` `sha256:03d91626...` unchanged. Generation moved 56 → 60 purely from restarts. **Note the brief said 54 and it was already 56 — generations are being consumed by concurrent restarts, so quote `root_hash`, not the generation number, when you mean “the content is unchanged”.**

Proof: `prove-assessment-reach.sh` control passed and **all 8 mutations caught**, including both unmounted routes, a nested threshold/passed key, an omitted zero-question section, and a deleted manifest row mutated on the real file and restored byte-identical.

Owed in specs/:** T079 and T083 drop PARTIAL; T057/T058 rewrite to “3 of 4 kinds indexed, `lesson_section` a recorded gated refusal”; **T102 is now half-stale** — `export.py:713` still hardcodes `review=None`, so it reports `PRECONDITION_BLOCKED` for all five areas even though 2 now have reviews.

A17 — spec 002 reconciled to 96 ticked / 46 unticked / 142. And the word-precision question is REOPENED, unresolved.

Two ticks, both earned by running the gates rather than reading the code: T115 and T116. `verify-answer-question.sh` **rc 0**, 10/10 L5 properties, and with `WORKSHOP_ASK_BASE` set it additionally reports `question_verifier_kind = question-focus+llm` — **live, not merely compiled.** `prove-answer-question.sh` **rc 0**, CAUGHT 3 MISSED 0, including `m2-silent-degrade`, which is precisely T116’s own paired mutation. Their [UNBUILT: decision taken 2026-09-02] markers are discharged with what discharged them recorded.

T041 was NOT ticked, correctly. Decisions 29–32 genuinely landed — R3 is 13, RULE 1 excludes 9,144 of 11,622 rows, RULE 2b discarded 355, three T041-* gate rows registered, both gates **rc 0** — but the decision record’s own §7 states T041 is **not closed: 12 type-T2 rows are human judgements deliberately left un-ruled**, plus 1 T4 cluster routed with a 2026-09-16 re-check. **The rules mechanised the 98.7% that was one artifact; they did not make the remaining judgements.**

A THIRD stale premise in one of this session's own briefs, caught by the agent checking instead of accepting. The brief warned "SC-010 is NOT met, so a task whose acceptance is that criterion is not done." **Spec 002's SC-010 is the IDENTIFIER-SURVIVAL criterion** (proven under the already-ticked T053); the surviving fabrication belongs to **spec 001's SC-010**. Same number, different criteria, different specs. Neither T115 nor T116 names an SC as acceptance at all. **Three files in this project already have colliding section numbers; the SC identifiers collide across specs too.**

Figures that moved and were withdrawn rather than restated: **T031's 499/494 → 495/490**, because 5 areas moved to held_back as all_evidence_redacted when the decision-26 redactions applied — **the redaction propagated into the published grid exactly as designed**; and T117's denominator 13 → **14** registered defects.

A17a — RESOLVED: the precision:"word" field is LIVE and it OVERSTATES ITS OWN PRECISION. A new defect, not a closed one.

The §10.1 withdrawal recorded yesterday STANDS on substance: every deep link still resolves at SEGMENT WIDTH. "Jump to the exact moment" is still "jump to the right few seconds." An agent first reported §10.1 as stale, then **investigated and reversed its own finding** rather than leaving it to be discovered later — the reversal is the valuable part and is recorded so nobody repeats the mistake from the field alone.

Measured independently 2026-09-03 over the first 12 areas, via GET /api/areas/{id}/evidence:

precision	n	min	median	max
word	12	5.28 s	6.34 s	9.64 s
segment	1	6.30 s	6.30 s	6.30 s

Identical width. A word-labelled entry is 5–10 seconds long. Cause, read directly: pkg/knowledge/wordjoin.go:127-128 returns StartS: sub[0].StartS, EndS: sub[len(sub)-1].EndS — **the first word's start to the LAST word's end of the ENTIRE segment.** The flag flips to word when every word of the segment has a timing in the sidecar, so **it is a statement about SIDECAR COMPLETENESS, not about span width.** There is no character-offset localisation either: text_span spans the whole passage, so there is no single word to resolve to.

THE NEW DEFECT: an entry labelled precision: "word" carrying a 6.34 s median span **overstates its precision to any client that trusts the field** — which is exactly the misreading §10.1's own consequence paragraph exists to prevent. What is genuinely inaccurate in §10.1 is now only its MECHANISM sentence: entry.Precision = PrecisionSegment is no longer unconditional, and precision_split is no longer always {"word": 0, "segment": N}. **workshop/ owns that correction and the precision-segment-only defect row.**

A routing fact worth keeping, because it explains a failed reproduction. /api/areas/{id} 404s behind the publication-review gate (decision 41 — only 2 of 499 reviewed), but /api/areas/{id}/evidence is registered separately at main.go:680 and does NOT apply that gate — it returns 200 for any area. Confirmed both. A probe on the detail route finds no precision field at all and looks like absence of the feature; the same probe on the evidence subroute finds it immediately. **Two routes over the same entity with different gating is how a measurement gets silently mis-scoped.**

T052 stays unticked for its unchanged reason: no SC-009 gate asserts that a media-backed citation lands inside its cited span.

A16 — T037 ASR accuracy: CANNOT DETERMINE, and that is the correct answer. B2 is NOT met and cannot be, today.

The apparatus works; the GROUND TRUTH does not exist. WER is defined by §4.2 as “*word error rate against a blind human reference over seeded, stratified audio-timeline windows*”. Both ASR engines are installed and working, but **an engine cannot be its own ground truth** — a second machine pass measures engine DISAGREEMENT, not error. Measured: a search for any *reference* / *ground*truth* artifact returns **zero** results outside vendored trees.

```
bash workshop/scripts/verify-accuracy.sh 01 --windows 30 --seconds 30 --seed 0 --min-accuracy 0.95
UNDETERMINED: --reference is required. §4.2: the absence of a human reference is 2, never 0 –
a command that cannot measure accuracy must not report that accuracy is fine.
rc=2
```


rc=2 was proved to be correct behaviour BEFORE it was accepted as an answer: the script's own --selftest exits **0**, covering absent reference → 2 (twice), perfect reference → 0, degraded reference → 1, and a seeded BLUFFING mutant **caught** (rc=0 rather than 2). Re-verified 2026-09-03: selftest 0, real run 2.

A machine reference was available and deliberately NOT used. verify-accuracy.sh hardcodes the method string, so feeding it a second engine's output would have written a **false claim** into accuracy.json — precisely the bluff G-CLI-5's paired mutation exists to catch. **The one action needed is the one an agent cannot take: listen to the audio and write down the words.**

B2 remains NOT met, and the endpoint stays honest — accuracy.go reads the literal path transcript/accuracy.json with no glob, so the new plan file cannot be mistaken for a measurement: /api/chapters/01/accuracy → {"measured": false, "wer": null}.

What WAS delivered turns “blocked indefinitely” into “blocked on one named human action”: workshop/chapters/01/transcript/accuracy-plan.json — **30 of 30** windows placed, shortfall 0, 30 s each = **900.0 s = 12.99%** of the 6928.713 s recording, stratified 10 low / 10 mid / 10 high by engine confidence, **2031** hypothesis words inside them (the size of the human job). Seed 0 is byte-reproducible — two independent runs give identical sha256. **Content-boundary safe, verified independently: its window keys are exactly t0, t1, seconds, stratum — no text-bearing key.**

A real defect found and fixed en route: --emit-plan never created its parent directory, so writing to its OWN contracted destination died with a raw FileNotFoundError traceback. It still exited 2, which is why nothing caught it — **but a traceback is not a NAMED undetermined**, and the remedy the script exists to make runnable was un-runnable at its default path.

A companion metric that IS computable without ground truth: SC-001 coverage-gap rate **0.055239 (5.52%)** — 382.733 s of 6928.713 s carry no segment; 389 inter-segment gaps, 11 over 5 s, **0 over 30 s**, max 11.48 s. The other two companions \$4.2 names both need ground truth.

DO NOT QUOTE pipeline/AUDIT.md AS AN ACCURACY FIGURE. Its 3.11% engine-B contradiction at $p \geq 0.90$ over 7×60 s is **engine disagreement**, not WER.

Owed in specs/ (not edited — another agent owns it):** T037 demands “the measured figure **and its confidence interval**”, but accuracy.json has no CI field and the scorer computes none — **as written, T037 cannot be satisfied even once a reference exists.** Either T112 gains a bootstrap/binomial CI over the 30 windows, or T037’s wording is amended. Separately, T037’s open destination question is settled by evidence: the CLI default and the running server’s AccuracyPath already agree the report belongs beside the recording.

A15 — the three SC-015 architecture options, MEASURED. The failure is ORDERING, not retrieval.

The single most useful measurement in this whole line of work: on the served path the correct target is within **top-10 for 24/26** — exactly the SC-015 bar — and within **top-20 for 26/26**. The six remaining misses sit at ranks 6, 7, 9, 9, 13, 17. **Retrieval already finds them; the ranking puts them in the wrong place.** That reframes SC-015 from a retrieval problem into a re-ordering problem.

Option	Measured state	Cost	Verdict
A · hybrid lexical+dense fusion	ALREADY BUILT AND INERT. service.go:603 already calls hybrid.NewRRFStrategy(60).Fuse(...) — but the lexical leg returns no_match, 0 results, for 26 of 26 benchmark queries, because BuildMatch ANDs every token and these queries are 11–26 tokens. So RRF(dense, lexAND) equals dense exactly. OR expansion gives +1 top-1, +1 top-5.	~10 lines	LAST. lexOR returns 100 rows for 8 of 12 negatives , and split() applies the floor only to LegSemantic — lexical and fused hits are served above_floor=true unconditionally, so a global OR leg destroys the 12/12 abstention just bought.
B · reranker over top-N	The only option whose ceiling clears the bar. A zero-dependency in-window bm25-OR reorder of the served top-20 gives 21/26 against the dense order’s 19/26 on the same	small	FIRST. It cannot damage abstention — the window is already floor- filtered and all 12 negatives

Option	Measured state	Cost	Verdict
	window — no new model, no new dependency, no image change. Attacks the root cause. Measured: transcript_segment n=1055, average 76 chars, 1002 of 1055 under 100 chars — one raw ASR utterance each. area_qa_paraphrase queries expecting one 1121–2091-char doc_section score 9/10; term_paraphrase queries expecting sets of tiny segments score 11/16 and supply 5 of the 6 remaining misses.	largest	yield an empty window — and it does not touch passage identity, so it is reversible. SECOND, measured OFFLINE first. It changes passage IDENTITY — pids key citations, crossrefs and redaction — so it means full re-ingest, full re-embed (~1 h) and re-authoring every expected_pids. Decide it in a scratch index, not the live corpus.

Reuse checked before proposing anything new (\$11.4.74).

submodules/RAG ships **no cross-encoder and no LLM reranker** — only a passthrough ScoreReranker and an MMRReranker already in use at service.go:648 — so a relevance reranker is genuinely net-new. Its pkg/chunker (Fixed/Recursive/Sentence) is imported **nowhere**; ingest chunking is Python with **no maximum size and no overlap anywhere**, only a --min-chars floor. Note RAG's chunkers count **bytes, not runes**.

A cross-encoder is blocked in-process, measured: the container is alpine/musl and the only interpreter with onnxruntime on this host is the glibc pipeline/venv, which is not mounted — so it needs a glibc image or a sidecar. An LLM reranker over the resident qwen2.5:3b is

reachable but costs an L5-scale call: the answer path measures 11.7–67.1 s against /api/search's ~2 s, so it is **not viable on the interactive path**.

One thing that CANNOT BE DETERMINED, and it is not a small one. Served top-5 (20/26) EXCEEDS the offline ranking (17/26), because the served list is not the raw cosine ranking. Probed on one query: **4 of the top-10 offline candidates, scoring 0.712–0.744 and all ABOVE the floor, appear in neither results nor near_misses.** All four resolve at /api/passages with HTTP 200, so locus-withholding is not the cause; exact-duplicate pairs are treated inconsistently, so diversity de-duplication does not explain it either. **Which stage removes them is undetermined — it is not the floor.** Four above-floor candidates vanishing per query is either correct withholding or a recall defect, **and the two look identical from outside.**

A14 — SC-010: fabrications 11 → 1. STILL NOT MET, and the recorded baseline of 3 was superseded before the work began.

Measured 2026-09-02, same host, same day, same 57 questions, only the flag differing:

	before L5	after L5
answerable answered+cited	12	3
UNANSWERABLE FABRICATED	11	1
unanswerable refused	22	29
unanswerable unavailable	0	3

Three things must be read with those numbers, and none of them is a caveat added to soften the result.

- 1. The recorded FABRICATED 3 is SUPERSEDED, not improved upon.** That figure was generation 11. The before-run is generation 52 **with no code change** and measures 11. The corpus grew and the fabrication count grew with it — so the honest framing is 11 → 1, and anyone comparing against 3 is comparing two different systems.
- 2. 3 rows COULD NOT BE MEASURED.** All sat at 200.0–200.1 s, which is pkg/answer/http.go's MaxWait; past it the handler returns 202 with a job-poll body carrying no status key, and L5's extra call pushed them over. **The honest denominator is 1 in 30 measured, 3 undetermined — NOT 1 in 33.**
- 3. The cost is 9 answerable questions declined.** All nine were re-asked and their refusals read: **3 by floor 1, every one a cause**

question (“carries no causal connective at all” — floor 1 refused 3 of 3 why questions, the bluntest edge and the clearest tuning target) and 6 by floor 2. The benchmark’s own header warns answered+cited ≠ answered correctly, so some of the nine are CORRECTIONS rather than losses — **how many is not measured, and was not guessed.**

SC-010 is NOT met. One fabrication survives: “*which tcp port does the ollama embedding model itself bind inside the container*” — demand quantity, the claim carried a number, and the judge agreed. Transition table: 8 FABRICATED→REFUSED, 2 FABRICATED→UNAVAILABLE, 1 survivor, 9 OK→DECLINED, 3 OK→OK.

A second stale premise in this session’s own briefing, corrected by measurement. The brief asserted `cmd/workshop-server/main.go:475-494` calls `answering.Wire WITHOUT EntailPython/EntailModelDir`. **False** — `main.go` already passed both (~line 730). The real gap was the **container**: `compose.yml` set neither, and the ONNX judge cannot run there regardless (alpine/musl, and pipeline/venv is not mounted — measured via `podman exec`). L5 therefore reaches ollama over HTTP. **Two briefs this session carried stale facts out of this document; both were caught by agents measuring instead of accepting.**

Recorded blind spots (docs/limits.md §10.15), stated rather than discovered later: the judge can grade its own work, and L5 checks whether the question was ANSWERED, never whether the answer is CORRECT.

Owed, not done: `specs/001-.../contracts/http-api.md` §5.3 and §5.5 need two new members — `does_not_answer` and `question_verification_unavailable`. Reporting them under existing names was **refused on purpose**: unsupported means the passages do not state the claim; this means they state it perfectly and it answers a **different question**, and the two have opposite remedies.

A13 — the redaction applied, B5 met, and FOUR NAMES SURVIVE in a derived file. Not in history, but regenerating.

`workshop/curriculum/taxonomy.jsonl` still names four of the five **natural persons**, in five term rows and four area `member_terms` lists, after a successful passage-level redaction. **The evidence unlink worked completely** — `from_subset = 0` on every affected row, 107

term and 6 area rows unlinked — but **0 term rows were WITHDRAWN**, so no term, external_key or member_terms string was replaced.

Measured cause, and it is structural rather than a bug in the tool. A row is withdrawn only when evidence_pids empties. For all four non-consenting parties **100% of the remaining evidence is SELF-REFERENTIAL** — synthetic passages whose source_ref.path is derived from the term string itself (term:<term>, extracted-area:<term>). No transcript passage remains as evidence for any of them. **The row is its own evidence, so a passage-level redaction can never empty it.** This is review §6 gap 3; closing it is review §9 step 4, which would require suppressing pids the Category-1 decision table does not name — outside decision 26.

Exposure, measured both ways: taxonomy.jsonl is **gitignored** (.gitignore:128) and git ls-files --error-unmatch confirms it is **NOT tracked**, so this residue is **not in git history and cannot reach a remote**. But it is rebuilt by pipeline/extract/taxonomy.py and **will regenerate** until the builder changes.

Three further honest boundaries from the same run, none rounded up: - passages.db does not exist in this deployment — a determined ABSENCE reported by the tool, not an inability to look. So “text NULL, FTS excluded” is **UNVERIFIED on this tree**; the gate proves it on its own fixture (A10/A10b). Recorded as unverified-here, never as a pass. - text/machine_text are **RETAINED in the registry file** — 0 purged (purge=false). That is R6 by design (R2 governs serialisations, not the file). --purge was **not** run: decision 26 did not ask for it and it is irreversible. **Operator decision.** - **The prescribed sequence was not executable and the deviation is reported rather than hidden.** --review-only BEFORE the apply refuses — “*review says redactions_recorded but names nothing suppressed*” — and its only alternative, none_required, would be a false statement about a chapter carrying 13 REDACT decisions. The non-mutation property was proved with --review-only --dry-run (0 files changed) and the artifact written for real immediately after the apply, where it is truthful.

T040 MUST NOT be marked [x]: 7 REDACT decisions are deferred, F17 (the already-public name) remains open and operator-only, and gap 3 is unclosed. T040’s third half is unblocked on the mechanism but **blocked on its own stated path** — it specifies workshop/chapters/01/transcript/transcript.md, which does not exist; the file is curriculum/chapter-01/transcript.md.

A11d — “gates enumerate with `git ls-files` and are therefore blind to UNTRACKED files” is a CLASS, not an incident. Three instances found in one day.

Instrument	Blind to untracked?	Status
scripts/verify-content-boundary.sh	Was — enumerated tracked only	FIXED 2026-09-02 by the opt-in <code>--include-untracked</code> (decision 10). Found 44 rows within one run.
scripts/audit-environment-assumptions.sh	YES, still — bare <code>git ls-files</code> at line 786	OPEN . Proven, not inferred: a reversible <code>git add -N</code> of two new untracked results files took it from exit 0 to exit 1 , flagging both; the index was then restored. It had been reporting green <i>because it never scanned them</i> .
scripts/audit-hardcoded-paths.sh	Different failure — scans tracked files but SKIPS docs/wholesale	OPEN (decision 20).

Why this matters more than any single instance: the project’s `commit` wrapper runs `git add .`, so every untracked file is one invocation away from being committed and pushed. **An instrument that cannot see untracked files is blind precisely in the window where a mistake is still cheap to fix** — and green in exactly that window is worse than no gate at all, because it is read as assurance. Two of the three are still open.

Worth noting as the correct instinct: the same run generalised `workshop/.gitignore` from a single filename to `pipeline/benchmark/*.db`, because the existing rule’s own reasoning — vectors derived from private text — applies to any sibling index, and an unignored 10 MB derived-vector file was one `git add .` from being committed.

A11c — the new --include-untracked mode immediately found 44 MORE rows, in files decision 18 would have PUSHED

Decision 10 built the flag; the flag paid for itself within one run.

With --include-untracked, scripts/verify-content-boundary.sh reports **+44 rows (+37 prose, +7 short, +0 name)** from **2 of 4** untracked public files, against **10 distinct private sources across 2 private submodules**:

untracked PUBLIC file	rows	principal private sources
specs/001-workshop-curriculum-platform/review.md	24	workshop/docs/session-evidence/session-ledger.md (21), workshop/docs/training/areas/02-... (2), workshop/platform/orchestration/go.mod (1)
docs/session-instruction-audit-2026-09-01.md	20	workshop/scripts/setup.sh (12), pipeline/extract/verify.py (2), verify_export.py (2), knowledge-model-contract.sections.json (1), search-defect-evidence.md (1), ai_interviewing/.../22-testing-and-quality (2)
_tests/env.js	0	—
specs/001-.../redaction-review-summary.md	0	—

Verified independently by normalised 10-token shingling: review.md ↔ session-ledger.md shows **9 verbatim runs in a stride-5 sample** (so more at stride 1), and session-instruction-audit ↔ setup.sh shows **2**. **Both files are UNTRACKED — nothing is published.**

This blocks decision 18 as written. “Push everything except the name fix” would commit and publish both files, and the commit wrapper’s `git add .` stages untracked files automatically. **Do not push until this is resolved.**

Two lessons, and the second is the uncomfortable one.

1. **The earlier review.md paraphrase HELD.** `workshop/platform/backend/pkg/answer/verify.go` — the source of the 54-of-62 window match — appears **nowhere** in the new report. That fix was real.
2. **But it was INCOMPLETE, and incompletely for a structural reason: it checked only the three private sources a prior agent had NAMED.** `session-ledger.md` was never examined because nobody had thought to look at it. A targeted paraphrase verified against a hand-supplied source list proves only that those sources are clean. **Only a whole-corpus instrument answers the actual question**, which is precisely why the flag was worth building rather than fixing the three known passages and calling it done.

A11b — the T040 review found a SECOND already-public name disclosure. VERIFIED. Operator decision needed.

A personal given name from the private Chapter 1 material is present in a TRACKED, COMMITTED, PUSHED file of this PUBLIC repository: `docs/setup-agents-wizard/LUMEN-STORE-INVENTORY.md`, **lines 99 and 157.**

Verified independently rather than accepted from the agent’s report, and without handling the string: both lines carry an **absolute filesystem path**; the same two lines are byte-identical at HEAD (`git show HEAD:<file> | sed -n '99p;157p'`); and `git branch -r --` contains places that commit in **origin/main**. In each case the name is the **final path component**, under a parent directory denoting work assignments — so the disclosure is a given name **plus an implied working relationship**, which is worse than a bare name.

Measured negative, which matters as much as the finding: the other four probes — the recording’s participant, two further third parties and one organisation — return **ZERO** hits across every tracked file of the public umbrella. **Exactly one name has crossed.**

Why no gate caught it. An earlier draft of this paragraph called it “two measured, independent blind spots” and that is WITHDRAWN — the two instruments failed in DIFFERENT ways, and lumping them together hides the sharper problem.

1. **scripts/audit-hardcoded-paths.sh is genuinely blind to the directory.** It carries `SKIP='^(docs/|_content|_analysis|_tests/evidence/|...)'`, applied to the file list BEFORE its PATTERN. Measured: **39** tracked docs/**/*.md files, **0** survive the SKIP — 0 of 59 files scanned in that tree — while **2 of 2** of the disclosed lines match the audit’s OWN PATTERN. **The detector was right; the scope was wrong.** It exits 0 today while structurally incapable of seeing either line.
2. **scripts/verify-content-boundary.sh is NOT blind to docs/ — it scans it and stayed silent anyway.** Measured from its captured run: it names **84** distinct docs/**/*.md files and **10 of the 20** files in docs/setup-agents-wizard/, and LUMEN-STORE-INVENTORY.md **zero** times. So docs/ is squarely inside its scope and its silence is a DIFFERENT failure. **The mechanism is not established here.** A candidate worth checking — not a finding — is its own name-class subtraction: the run prints a scale-free name-frequency floor under which **3,832 tokens “count as ordinary words”**, and a common given name is the kind of single token that floor is built to discard. **Do not record that as the cause until someone measures it.**

Honest boundary on the gate evidence (§11.4.6): the only content-boundary run available for this file was taken **AFTER** the redaction, so it is **not an A/B** and says nothing about what the gate would have reported before. No pre-redaction run exists, and one was deliberately NOT manufactured — re-introducing a live disclosure into the working tree to obtain a cleaner measurement would be worse than the missing datum.

This is the same CLASS as the 2026-09-01 incident and it is NOT covered by decision 11, which answered what to do about that incident’s existing history. This is a separate instance, found later, in a different file. **Do not quietly edit the file and treat the matter as closed** — redacting the working tree is containment, history is not editable after a push, and removing it is a public history rewrite, which is an operator decision.

A NEW blocker surfaced by the decision-9 repair: G-CLI-17 is defined and built by nothing

specs/001-.../contracts/ defines **31** gate ids; **no task builds G-CLI-17** (contracts/pipeline-cli.md:989 — status.sh with no containers must print STOPPED and exit 1). **This is not a regression from the repair:** the same comparison run against git show HEAD:...tasks.md prints it too. Assigning it to a task — most plausibly T021, whose G-CLI-12 the contract calls its sibling — changes WHAT the file asks for, which is an operator decision, so the repair recorded the gap and deliberately did not close it.

Read the closure check carefully, because it now lies. The obvious test is “count gate ids in contracts/ vs tasks.md”. Before the repair that read **31 vs 30** and correctly flagged the gap. It now reads **31 vs 31 and PASSES**, because the block *documenting* the gap mentions G-CLI-17 twice (lines 615 and 622). **Writing the defect down made the naive check go green while the defect stands.** A real closure check must test whether the id is attached to a T### task line, not whether the string appears in the file:

```
grep -nE '^\\s*-?\\s*\\[[ xX]\\].*G-CLI-17' specs/001-workshop-  
curriculum-platform/tasks.md  
# empty = no task builds it, regardless of what a count says
```

Second decision round, same day — 8 more, taken as new blockers surfaced

#	Blocker	Decision	Consequence
17	A personal given name from private Chapter 1 material found in a TRACKED, PUSHED public file (docs/setup-agents-wizard/LUMEN-STORE-INVENTORY.md:99,157)	Redact working tree, record, no rewrite — EXECUTED 2026-09-02	One token removed: .../Projects/assignments/<given name> → .../Projects/assignments/..., the house elision. The prefix was deliberately NOT made portable — 14 other rows of the same tables carry it, it is the owner’s

#	Blocker	Decision	Consequence
			own already-public host path, and rewriting two rows only would break alignment for no privacy gain. Verified with run-time-derived probes, never typed: name-shaped occurrences 2 → 0 in the worktree (untracked files included), still 2 at HEAD — history untouched, as decided. The other four probes measure 0 across the public tree, so exactly one name crossed. Incident record gained \$13, +448 lines, framed as a second SEPARATE instance. Containment, not remedy: the name remains in public history and in every clone already taken.
18	T040 is only 1/3 done (review recorded;	Push everything	The workshop GITLINK push is

#	Blocker	Decision	Consequence
19	NOT applied; transcript NOT re-emitted), so T104 is not unblocked	except the name fix	safe and moot — that commit is already on its private remote, and the public umbrella gains only a SHA. T012: an append-only JSONL log in submodules/ passage opened 0_APPEND only, with no truncating open anywhere in the package — append-only as a property of the code, not a comment; seq gaps or repeats raise
	Redaction is a FIELD, not a MECHANISM: redact.sh (T038), --review-only (T039) and the append-only log (T012) all measured ABSENT	Build the mechanism — EXECUTED 2026-09-02. B5 IS NOW SATISFIABLE.	ErrLogTampered. T038: workshop/scripts/redact.sh over cmd/workshop-redact, three-valued throughout, writes staged and committed atomically only after every surface reports — it refuses rather than half-applies. T039: --review-only records each artifact's

#	Blocker	Decision	Consequence
20	Two blind spots let the disclosure sit unseen	Scan docs/ for absolute paths · add a name-in-path detector · one-off docs/ audit	SHA-256 and mtime, answering B5 both as written (mtime) and as meant (content). Propagation 8/8 targets, driven through the existing checklist, which caught a real ordering defect in the author's own code. Proofs: verify-redaction-propagation.sh rc 0; prove-redaction-propagation.sh rc 0 catching all 6 seeded mutations. The 13 REDACT decisions were NOT applied — -check-review reports a determined 1 today and becomes 0 the moment the operator runs --review-only. audit-hardcoded-paths.sh carries SKIP='^(docs/...)' and never scans docs/. Note a tension in the selections —

#	Blocker	Decision	Consequence
21	G-CLI-17 defined in contracts/ and built by no task	Assign it to T021 — EXECUTED 2026-09-02	“scan docs/ permanently” and “keep the SKIP, audit once” conflict; resolved toward the STRONGER reading (permanent scanning plus the one-off sweep). Say so if the weaker was meant. Appended to T021’s acceptance in the same style as its sibling G-CLI-12, with the contract link, the behaviour, a reproduction command and a paired mutation. The gap block was UPDATED, not deleted — the finding and the git show HEAD evidence that it predates the path sweep both stay. Verified independently: all 31 contract gate ids are now attached to a task line, 0 unattached.

#	Blocker	Decision	Consequence
23	Decision 6's OCR work needs a home	Extend spec 002 in place	Chosen over a separate spec 003. Consequence, stated plainly: spec 002 cannot close until OCR accuracy is measured , on top of its current 22 PARTIAL / 6 NOT DONE. 54 + 5 = 59, so every previously-passing row still passes and all five FAILs cleared. The heuristic was not touched — git diff --scripts/verify-check-registry.sh is 0 lines; the four proofs gained the <n> mutations summary form the heuristic already accepts. provider-ci M6b was a name COLLISION graded as a name LEAK : the token verdict, an ordinary English word used in 20 lines of that file since
	--run-proofs exits 1 at 54 PASS / 5 FAIL	All three EXECUTED 2026-09-02 — -run-proofs is now 0 at 59 PASS / 0 FAIL	

#	Blocker	Decision	Consequence
			long before vasic-digital/ verdict existed. The fix STRENGTHENS it — M6b now tests full slug, path/ URL position, case-arm and comparison operands, and proximity to a host or owner; a bare token clears only if all four are empty AND it is <code>^[a-z]</code> <code>{4,}\$</code>, so design- toolkit, LLMProvider, ai_interviewing can never take that route. The DROP blind spot is CLOSED by scripts/ constitution- gate-ledger.tsv (306 rows) plus a two-instrument discriminator — the PIN decides when it has not moved, git tracked-ness decides when it has — with - - update-ledger as the one deliberate, diff- visible re- baseline. Proof grew $10 \rightarrow 16$

#	Blocker	Decision	Consequence
24	Spec hygiene	All three EXECUTED 2026-09-02	mutations, exit 0. (a) The gate-attachment check replaced presence-counting as the verdict-bearing check, with a paired mutation on a scratch copy proving the point exactly: detaching G-CLI-17 from T021 while leaving every prose mention intact leaves the naive check reading defined 31 cited 31 and PASSING, while the attachment check reports UNATTACHED G-CLI-17. A check that a comment about the problem can satisfy is not a check. (b) T060/T094 markers stripped, each discharge verified by path first. (c) T031's false sentence withdrawn as false when written, with

#	Blocker	Decision	Consequence
			three measurements replacing it. Checkbox counts unchanged and re-verified: 001 = 59/61/120, 002 = 94/28/122. T031 stays unticked with [BLOCKED: prose authorship] LIVE.

Third decision round, 2026-09-02 — 16 more (25–40)

#	Item	Decision	Consequence / status
25	2 untracked public files carry 44 rows of verbatim private-source content — blocks the push	Paraphrase both, then push	Must verify against the WHOLE private corpus, not a named source list — that is precisely how the first review.md pass missed session-ledger.md. IN FLIGHT.
26	T040 is 1/3 done; the mechanism now exists	Apply only the highest-severity subset — EXECUTED 2026-09-02. B5 IS MET.	6 of 13 REDACT applied (F1, F2, F4–F7 — every REDACT citing a §3 Category-1 direct-identifier row); 7 deferred (F8/F9/F16 Category 2, F12–F15 Category 3); 3 KEEP and 1 NOT REMEDIABLE untouched. The boundary came

#	Item	Decision	Consequence / status
27	SC-015 settled NO	Switch EXECUTED 2026-09-02, generation 53 → 54. SC-015 still NOT met.	from the review's own category structure, not from the agent's judgement. Verified: - - check-review rc 0, registry carries exactly 10 redacted:true of 11,622, append-only logs hold 10 + 1 entries, and BOTH gates stay 0. Rendered document and both sidecars: residual hits 0 across a 78-file sweep. Served path, same instrument both times: top-1 4/26 → 8/26, top-5 13/26 → 20/26, targets absent from the served list 11/26 → 0/26, negatives returning zero hits 7/12 → 12/12, and floor_calibrated false → true (verified live). SC-015 needs 24/26; the served path delivers 20. Downtime 27 s via stop.sh/start.sh; podman restart never used. Latency 23.1 s → 76.6 s on the 38-call leg — a single

#	Item	Decision	Consequence / status
			paired observation, not a benchmark. A blocking defect was found by measurement: nomic's context is 2048 vs jina's 8192 and /api/embed refuses the corpus's longest passage, so the first oversized passage would have halted the whole pass; fixed with truncation plus a recovery ladder. Probing further showed refusals are per-item, not per-request — contradicting an earlier coarser probe, whose comment was corrected rather than inherited. Floor 0.655, the midpoint of the measured gap (0.635296, 0.675179), and floor_calibrated is now a lookup on the (model, doc-prefix, query-prefix) triple — any unmeasured combination, including a prefix missing its trailing

#	Item	Decision	Consequence / status
28	Staged pin · stale registry comment · verify-remedy-executability rc=1	Commit the pin with the session's work · fix the comment · investigate the red	space, still returns 0.62/false. The pin, helix-deps.yaml and the gate ledger must land together or a fresh clone gets f16ea779. R3 1153 → 13; extraction input 11,622 → 2,478 (9,144 graph rows excluded, 78.68% of the file). R1_attach 11 → 1936 against a 1927 projection (+9); RULE-2b discards 355 against 353 (+2); candidate terms 8537 against 8551 (-14). The two STRUCTURAL predictions landed exactly: residual is 13, all B1, matching the first recorded run's R3=13, and T2/T4 are 12/1. §2.3's "never discard" was NOT spent — decision 30's regenerate path was used, and a second run over the regenerated state is stable. Real costs recorded, not netted away:
29–32	The four R3 rules	ALL FOUR EXECUTED 2026-09-02	

#	Item	Decision	Consequence / status
			A3 falls 0.1047 → 0.0894 and “no evidencing mention” rises 3243 → 9017, since 9,144 kg_* rows now evidence nothing. T4 routed to a NAMED owner with a re-check date. The 14 T2 ULIDs are listed for human judgement; 12 survive both rules. Gates verify-r3-rules.sh and prove-r3-rules.sh both exit 0.
—	(rows 30–32 folded into the entry above)		
31	R3 type T3 — 353 rows RULE 1 un-masks	2b — discard as duplicate	Boundaries stay sharp. Cost: evidence discarded, and an A1/A2 unattached classification is forced downstream.
32	R3 type T4 — 1 over-merged mega-cluster	Route it with a named owner	Must carry an owner and a re-check date — an unowned deferral is indistinguishable from a drop.
33	Spec 001 (61) and spec 002 (28) unbuilt	Build both — 002 first, then 001	The largest remaining block: 89 tasks via subagent-driven development.

#	Item	Decision	Consequence / status
34	OCR ingestion scope	Spec the tasks now, build later — EXECUTED 2026-09-02	Phase 11, T123–T142 (20 tasks), all [UNBUILT]. Counts verified: ticked 94 unchanged, unticked 28 → 48, ids T001–T142 contiguous. The accuracy obligation is TWO axes because one is insufficient — textual (WER/CER vs a seeded hand-truthed sample) and temporal (does the declared visibility interval actually contain the moment the text was on screen). Deep linking depends only on the second, and a perfect textual score is compatible with every interval being wrong. The budget is DERIVED, not picked: G-OCR-9 reads the floor at run time from the recorded speech calibration, never as a literal, and returns 2 when that record is unreadable — satisfying the spec's own no-guessed-threshold rule.

#	Item	Decision	Consequence / status
			Dedup keeps both mentions individually navigable and deduplicates only the COUNT, grouped by subject AND overlapping time, never by text equality (which would collapse two occurrences twenty minutes apart). Clarifications: “two open” is now ONE — prose authorship alone.
35	SC-006 — p95 2,094.8 ms vs a 2,000 ms budget	Fix it	4.7% over. Cause unmeasured — profile before changing anything. held_back is not an evidence floor and holds back nothing : measured areas served 499 · held_back.count 0 · areas []. It is a liveness test (withhold only if evidence_count==0 at build or all evidencing passages were since redacted) wrapped in a permanently-present REPORTING envelope — its presence is not
36	publish-all-497	Implement it — ALREADY SATISFIED at the grid; BLOCKED at the detail endpoint. Agent STOPPED, correctly.	

#	Item	Decision	Consequence / status
			evidence of withholding. No numeric floor knob exists in the backend and the frontend does no such filtering, so the grid already meets decision 36; count before 499, after 499, no change needed or made. $497 = 499 - 2$: the DETAIL endpoint <code>/api/areas/{id}</code> requires a row in <code>publication-reviews.jsonl</code>, which has 2 rows — 2 reviewed areas return 200, 6 sampled unreviewed return 404 area_not_published. See the conflict recorded below. The reported cause was half wrong, established before acting: of the 4 gates, only 2 were naming-convention victims (<code>verify-check-registry-002.sh</code>, <code>verify-limits-completeness.sh</code> — their provers
37	<code>verify.sh --static-only V7</code>	Fix the pairing rule — EXECUTED 2026-09-02; V7 still FAILS, and that is the CORRECT answer	

#	Item	Decision	Consequence / status
			existed under a -mutation.sh suffix). The other 2 — verify- connectivity- matrix.sh and verify-entailment- loads.sh — have no prover under ANY convention, verified by find . -name 'prove-*' across the whole module. Convention chosen: rename to prove-*, leaving platform/ gates/ with exactly ONE spelling (re- measured: 0 -mutation.sh provers remain there). V7 still exits 1, now naming only the two genuine gaps — it was not made to pass, because it cannot honestly pass. Strongest evidence the rule was not loosened: a git diff --stat of that module's own verify.sh (under workshop/scripts/, NOT this root) is EMPTY; gate_pairing() is bit-for-bit unchanged. Writing the two

#	Item	Decision	Consequence / status
38	T040's third half	Re-emit the transcript	missing proofs is real §1.1 work on T050/T114. Asserting §7.3 R2, after the redactions land. This is what actually unblocks T104. Baseline 683 → 666 (-17); umbrella-root-owned 45 → 28; the other 638 remain inside submodules. Verified independently: <code>_tools/gen</code> appears 0 times in the whole report, all nine files compile, and defaults resolve to the exact former literals with the environment unset. AUDIT.md's F15 row said 15 — the real count is 17, because the blanket <code>MODEL *</code> rules silently swallowed two docstrings; recorded rather than glossed. Nothing was converted to a # REASON: row and zero rows were underivable. Three deliberate departures, each
39	683 baselined env-audit occurrences	Work the 45 umbrella-owned rows — F15 CLOSED IN SOURCE 2026-09-02	resolve to the exact former literals with the environment unset. AUDIT.md's F15 row said 15 — the real count is 17, because the blanket <code>MODEL *</code> rules silently swallowed two docstrings; recorded rather than glossed. Nothing was converted to a # REASON: row and zero rows were underivable. Three deliberate departures, each

#	Item	Decision	Consequence / status
			documented: per-pipeline UI_* keys rather than the shell pipeline's TRANSLATOR_MODEL (which exports a DIFFERENT provider for the document batch — sharing the name would have silently retargeted the UI drivers); the provider moved with the model, because a model override beside a frozen -provider is a half-fix; and if <code>model == "gpt-oss-120b"</code> became a membership test, since that comparison selects a CAPABILITY knob and freezing it would re-break the fix on first substitution. No new file was created — deliberately, since the audit's bare <code>git ls-files</code> is blind to untracked files, which is what happened to <code>_tests/env.js</code> in the F13/F14 slice. docs/ removed from the SKIP:
40		EXECUTED 2026-09-02	

#	Item	Decision	Consequence / status
	docs/ blind spot (decision 20)		scanned files 5038 → 5949 (+911), surfacing 344 occurrences, triaged to rc 0. New scripts/verify-name-in-path.sh registered, rc 0, 14 mutations. A one-off sweep of all 39 tracked docs/**/*.md found 239 absolute paths and ZERO confirmed personal-name disclosures across six independent probes.

41 | “publish all 497” collides with the citation-audit publication gate | **Keep the review gate — the 497 stay 404** | No code change: the grid already serves all 499 unfiltered, so “no floor” correctly means **no EVIDENCE floor**, which was already true. The 497 are unbrowsable because /api/areas/{id} requires a publication review and only 2 exist — a legitimate gate, given ~68% of auto-generated citations were measured coincidental. |

42 | V7’s two genuinely missing paired proofs | **Written — and V7 now PASSES** | prove-connectivity-matrix.sh (7 mutations, including rc=2 for a dead endpoint, so an unreachable API is never reported as a broken corpus) and prove-entailment-loads.sh (5 mutations + 2 rc=2 controls). Run against deliberately broken gates both report **7-of-7 MISSED** — neither proof is vacuous. Both summary lines print unconditionally, because under --quiet the house pattern prints nothing on success, which is indistinguishable from a hollow proof. A THIRD gate appeared mid-session (verify-r3-rules.sh, prover named prove-r3-rules-mutation.sh — the same naming class); renaming was correctly REFUSED while its author was still writing, and done afterwards. **workshop/scripts/verify.sh --static-only now exits 0 with PASS V7.** |

43 | A read-only sweep WRITES into a consumed submodule | **Exclude generators from discovery — EXECUTED 2026-09-02; C8 and C8a**

both CLOSED | An evidence-based classifier (no name list) excludes **16 of 286**, taking discovery to **270** and the split to **173 / 96 / 2 of 271**. **The delta reconciles exactly and FAIL is unchanged at 96** — the 16 removed rows were 14 PASSes and the 2 C8 ERRORS, so no failing gate was hidden. Rejected signals are recorded with the counts that killed them, including the sharpest: a GATE= self-name test would have classified the one dangerous file **as a gate**, because the generator carries one inside the heredoc it emits. Proof grew 22 → **25 mutations, 27/27**. After two full sweeps the consumed submodule shows **zero** wrapper files. |

Ordering that the decisions imply, because several depend on each other: T040 review (1) → commit/push (3); stale-path repair (9) → spec-001 implementers; verifier (5) → SC-010 (14) → T115/T116 unblock. Decision 6 (OCR) is large enough to deserve its own spec rather than being bolted onto 002.

spec 002 — knowledge areas & bidirectional deep linking (2026-09-02)

State: MEASURED 2026-09-02 as 94 DONE · 22 PARTIAL · 6 NOT DONE of 122. Platform live and serving.

The claim this line carried — “all ten phases IMPLEMENTED and reviewed” — is WITHDRAWN, not restated. It was wrong on BOTH halves. “Implemented” overstated **22 PARTIAL plus 4 unbuilt** tasks; “reviewed” was weaker still — three REVIEW gates produced a brief and **no report**. The correction came from a task-by-task assessment against the running system, not from re-reading the document: go test green across 6 backend packages, 254/255 + 26/26 unit tests, five gates run with real exit codes, and every 002 endpoint probed live at 127.0.0.1:8087.

Per phase (DONE/PARTIAL/NOT DONE): P1 6/0/0 · P2 15/0/1 · P3 15/4/1 · P4 11/2/1 · P5 6/8/0 · P6 11/2/1 · P7 12/0/0 · P8 5/2/0 · P9 8/1/0 · P10 5/3/2.

The 6 NOT DONE: T115/T116 (the answer-against-question verifier — the product decision is now TAKEN, see decision 5, and **U4 is no longer a blocker**); T022 (knowledge-contract review — brief exists, no report); T041 (promoted-vs-extracted reconciliation — **883 R3 contradictions with no recorded decision on any of them**, see decision 15); T084 (end-to-end question provenance — deferred pending T075/T078, both

since done, never revisited); T055 (transcript-to-knowledge affordance — backend live, core/api.ts never calls /api/passages/{pid}/knowledge).

Highest-value PARTIALs: T079/T083 — GET /api/areas/{area}/coverage and /export return **404 live**, unmounted and unregistered, even though pkg/assessment/coverage.go and its G-KG-12 paired mutation are complete. T057/T058 — only 2 of the 4 new kinds are indexed. T101 — render_diagram is never called from export_area.

Three live findings recorded nowhere else, found during that assessment: (a) verify-limits-completeness.sh is **RED (rc=1)** — defects-registry.tsv's area-term-over-generation row anchors on text that no longer exists after §10.9 was rewritten, so SC-026's own gate is reporting real drift; (b) check-registry-002.tsv's **G-KG-10 row names a G-KG-11 test**, while two genuine G-KG-10 proofs exist and neither is registered; (c) pipeline/extract/test_export.py:99 fails on this host by asserting a frozen object Object mermaid error string — frozen-host-assumption class. Also stale: limits.md §10.3's "5 kg_area rows" — the registry now carries **516** (11,622 rows total), so its "roughly 99×" consequence is void.

Three more corrections found while ticking the boxes, each measured: (a) **T031's own task text is FALSE** — it asserts "this host has no generative model, so the third cannot run here today". One IS installed and wired into the platform's answer path. What blocks T031 is the *clarification*, not the host, and its note now says so. (b) /api/areas serves **499** areas with a held_back block — a fourth distinct area count after 497, 498 and 516. **Do not quote an area count from prose; query the endpoint.** (c) T060 and T094 were ticked while still carrying [BLOCKED: ...] markers whose blockers were genuinely discharged. **Both markers are now STRIPPED**, each discharge verified by path first (T059 settled U1 from evidence — match-instance positions come from the index rather than being recomputed; T094's blocker was closed by a ranked, sourced research artifact).

The reading rule this document briefly recorded is WITHDRAWN, and the reason is worth keeping. It read: "*a marker records what a task waited on, not what it is still waiting on; read the checkbox for that.*" That rule was **written to excuse two stale markers rather than remove them**, it made [BLOCKED: ...] unreadable, and it contradicted the [UNBUILT] legend added the same day. The plain rule replaces it: **a [BLOCKED: ...] marker means blocked TODAY; discharge it by**

removing it and recording what discharged it. Exactly one [BLOCKED: ...] marker now survives on any task line in spec 002 — T031's, which is live.

START HERE ON RESUME. The full record of every request, decision, dispatch and finding is at `workshop/docs/work-register.md` (PRIVATE submodule — this file is in the PUBLIC umbrella, so the detail lives there). The working ledger and all 39 agent reports are preserved at `workshop/docs/session-evidence/` — they previously existed only in volatile `/tmp` and would have been lost on any session restart.

Read those two before starting new work. Then:

```
bash scripts/continuation-check.sh          # must be 0
bash scripts/verify-governance-cascade.sh    # 12 PASS / 0 FAIL
expected
bash workshop/scripts/restart.sh            # NOT `podman
restart`
curl -s http://127.0.0.1:8087/api/health
```

Decisions already taken (do not re-ask): materials are agent-authored at build time; benchmark the embedding model at full scale before switching; the 497-area count is NOT over-generation (invalid ratio — see below); push everything to all upstreams; record review provenance honestly and keep serving; agent-audit the four remaining areas one at a time; wire word-level link precision now; decide a term `external_key` convention then mint; review the adopted 450 lines (done — found 14 inert gates); fix both git hooks (done).

Measured state at handoff (workshop 2a22f59, umbrella a8bec47, both pushed)

Registry `curriculum/passages.jsonl` — **11,604 rows across 8 kinds:**
`kg_term 8553 · doc_section 1172 · transcript_segment 1055 · kg_area 498 · code 251 · kg_todo 27 · kg_next_point 24 · kg_open_question 14` (the last three are the per-chapter extraction: TODOs, next-meeting points and open questions are minted entities, not prose.)

LIVE and verified over the LAN at `http://192.168.1.44:8087` (bind is persistent via `platform/.env`; the compose default is loopback and WILL regress without it): - **word-level deep linking — this claim is WITHDRAWN. It was FALSE, and this document contradicted itself for the whole of its previous revision.** It read: *“precision: “word”, timing_confidence: 0.46551, with a three-state fallback that never collapses”*, while the KNOWN LIMITS block a few dozen lines below said *“All deep links are segment precision, never word.”* **Both cannot be**

true, and the limits block is the correct one. workshop/docs/limits.md §10.1 states it unambiguously — “*Every deep link resolves at SEGMENT precision — never word — and the reader cannot tell the difference from the response shape*” — and gives the cause: pkg/knowledge/mention.go defines both PrecisionWord and PrecisionSegment and **the word-sidecar join CODE PATH exists, but nothing in this deployment ever produces it.** What is true: the word-precision code path, its three-state fallback and TestResolveWordPrecision_ThreeFailureModesStayDistinct are all real. What is false: that any served deep link resolves to a WORD-WIDTH span. Writing it under “LIVE and verified” is exactly the bluff §11.4 forbids. Found 2026-09-02 by an agent that measured the premise instead of accepting it from its brief.

Refined 2026-09-03, and the refinement matters — see A17a. The field precision: "word" **IS** served today (GET /api/areas/{id}/evidence), so the flat statement “nothing produces it” is superseded. But the spans it labels are **5–10 s wide, median 6.34 s — identical to segment —** because wordjoin.go:127-128 returns the first word’s start to the LAST word’s end of the whole segment. **The flag reports SIDECAR COMPLETENESS, not span width, and therefore overstates its own precision to any client that trusts it.** The practical conclusion is unchanged: **deep links land on a segment, not a word.** - semantic search with genuine cosine relevance; gibberish returns no_match in all three modes; filters echoed on every status including unavailable. - 498 areas, 8551 terms, chapters, transcript, recording, autocomplete. - **2 of 5 areas published** (03 and 01), each with proposer != reviewer in its review record. Areas 02/04/05 remain honestly unpublished. - **44 senior questions, 44/44 served, 0 withheld** — verified against the REAL assessment.ServeQuestions()/G-KG-2, with a dangling-citation mutation confirming the verifier can actually withhold.

Things a resuming agent must not re-derive or re-break: - bash workshop/scripts/restart.sh, NEVER podman restart (it does not pick up a changed compose file). - git commit -- <paths> commits WORKING-TREE content and DISCARDS index staging. Use a private index (GIT_INDEX_FILE + write-tree + commit-tree + CAS update-ref), then git reset -q -- <paths> to refresh the shared index — a stale shared index has twice staged a large revert of landed work. - An **auto-commit process** runs in this checkout and has swept an agent’s working-tree edit into an unrelated commit. Re-read HEAD immediately before staging. - WebSearch was exhausted (200/200) this session; .bashrc now sets CLAUDE_CODE_MAX_WEB_SEARCHES_PER_SESSION=100000, effective in a NEW shell. - A hook at ~/.claude/hooks/commit-attribution-fixed.sh now

intercepts every `git commit` at USER level, in all repositories. Recorded in the work register; revert by removing the `hooks.PreToolUse` entry in `~/.claude/settings.json`.

Spec: `specs/002-knowledge-areas-deep-linking/` (9 files, tracked as of 9b08d8c; it was ENTIRELY UNTRACKED before that — the contract governing 122 tasks existed only in a working tree). Normative sources, by full path, because three files in this project have colliding section numbers and a bare “contract \$N” silently resolves against the wrong one: `specs/002-.../contracts/knowledge-graph.md` — M1-M5 minting, E2/E3 extraction, R1 reconcile, N1 mentions, G-KG-* `specs/002-.../contracts/http-api-delta.md` — endpoints, A3.x acceptance clauses `specs/002-.../data-model.md` — entities; its \$2.x means something DIFFERENT from the contract’s \$2.x NOT the referent: `workshop/docs/knowledge-model-contract.md` (document SHAPE only).

Live and verified (<http://192.168.1.44:8087>, container `workshop-curriculum_platform_1`): semantic + lexical search — 2,478 passages / 550 sources; gibberish correctly returns `no_match`; real queries score 0.65-0.84 bidirectional deep linking — an area resolves to `time_span_start_s=227.42, time_span_end_s=232.70` in the recording; reverse direction and graph traverse live 497 areas, 8,551 terms; chapters, transcript, recording playback, autocomplete four-format export (md/html/docx/pdf); diagrams honestly report code 2 runtime filename-disclosure guard on `writeJSON` (~50ms on 10 MB, not 3-6 s)

Resume commands `bash workshop/scripts/restart.sh # NOT podman restart` — that does not # pick up a changed compose file `cd workshop/platform/backend && CGO_ENABLED=0 go build -o ../bin/workshop-server ./cmd/workshop-server workshop/pipeline/venv/bin/python workshop/pipeline/extract/run_pipeline.py workshop/pipeline/venv/bin/python workshop/pipeline/extract/verify.py -prove-failure workshop/pipeline/venv/bin/python workshop/pipeline/benchmark/run_retrieval_benchmark.py`

BLOCKED / OPEN — two operator decisions

1. **Retrieval quality fails SC-015 and no floor fixes it.** Measured on the live index: top-1 4/26 (15.4%), top-5 13/26 (50.0%); SC-015 requires $\geq 90\%$ top-5. 11 of 26 correct targets are absent from the top 100. Correct hits score 0.622-0.783; negatives 0.641-0.712 — ten of fifteen found positives sit AT OR BELOW the highest-scoring negative, so `floor_calibrated: false` is correct, not laziness. Diagnosis is discrimination-at-scale, NOT simply “a code model reading prose”: jina top-1 is 73% on a 152-passage pool and 15% on

the 2,478-passage corpus. **“NOT MEASURED” is WITHDRAWN — it was measured on 2026-09-02 and SC-015 is SETTLED: NO. Every leg fails, and the failure is not a tuning gap.** Four legs, same 2,478-passage corpus, same 26 queries:

leg	top-1	top-5	absent from top-100	floor separates?
jina — LIVE, served	4/26 · 15.4%	13/26 · 50.0%	11	No
jina — offline, identical code path	4/26 · 15.4%	15/26 · 57.7%	3	No
nomic — offline, no prefix	6/26 · 23.1%	12/26 · 46.2%	0	No
nomic — offline, task prefixes	8/26 · 30.8%	17/26 · 65.4%	0	YES

The small-pool trap was real and nomic walked straight into it: 80.8% on 152 passages → 30.8% on 2,478. 65.4% against a 24/26 bar is not closable by tuning.

THE DECISIVE FINDING IS THE FLOOR, NOT THE ACCURACY.

jina **cannot be floored at all** — 18 of its 23 scored positives sit at or below its highest negative (0.7117), so any floor rejecting all 12 negatives also discards 18 real hits, leaving 5/26.

floor_calibrated: false is *correct*, not laziness: there is nothing to calibrate to. **Prefixed nomic separates COMPLETELY** — positives 0.6752–0.8471, negatives 0.5040–0.6353, a **+0.0399 gap with zero overlap**; any floor in (0.6353, 0.6752) keeps **all 26** found hits and rejects **all 12** negatives.

A confound was found and measured rather than argued about:

the harness embedded with **no task prefix**, which nomic is trained to require (ollama’s card applies none — template is bare {{ .Prompt }}). Prefixing is worth **+2 top-1, +5 top-5** and is the difference between overlap and separation, so an unprefixed nomic number materially understates the model. A fourth leg was added because the served jina baseline was not comparable — 11

targets absent from top-100 and 7 of 12 negatives returning *zero* hits are **server** artifacts, resting its distribution on 15 positives / 5 negatives against nomic's 26 / 12.

RECOMMENDATION: do not switch expecting SC-015 to close — it does not. Switching remains defensible on the FLOOR alone: prefixed nomic wins top-1, top-5 and never-retrieved targets, and is the only model measured that admits a working floor, which buys correct abstention jina cannot provide at any threshold. Failures concentrate in term_paraphrase (8 of 11 jina misses, 7 of 9 nomic) and both models collapse the same way from pool to corpus. **The constraint is discrimination at scale; swapping bi-encoders does not lift it over 90%. Closing SC-015 needs an ARCHITECTURE change** — hybrid lexical+dense fusion, a reranker over top-N, or different chunking. **None of those was measured.**

Honest boundaries carried from the run: 6 of 2,478 passages truncated at 6,000 chars and 3 shrunk to 3,000, **verified none is a benchmark target** (empty intersection with the 152 expected pids); nomic's native 2,048-token context vs ollama's num_ctx 8192 — which binds was **not** measured; jina was not re-embedded so has no prefixed/unprefixed pair; and **26 queries is small — one query moves top-5 by 3.8 points, and no significance is claimed beyond raw counts.** Full write-up: workshop/pipeline/benchmark/SC015-FINDINGS.md. (Note SC-007/SC-008 are publication-review and reverse-link clauses; a comment in cmd/workshop-server/main.go carries the same wrong label and needs correcting.)

- 2. Area publication floor — a product decision, not a bug.** 497 areas from one chapter is NOT over-generation against the “7 tracks, 37 modules, 137 terms” baseline: that figure comes from a DIFFERENT tool (workshop/curriculum/chapter-01/knowledge/build.py, compiling a hand-authored lexicon.json into curated ~30-passage segments). Different unit; the ratio is meaningless. Measured cause: 1,474 of 5,112 certain terms (28.8%) are hapax legomena, and derive.py's E5 rule INTENTIONALLY makes a partnerless term its own single-term area. Stopword filtering works (20 canonical stopwords all score 0.0, certain=False). Median evidence per added area = 1 passage. **Recommended: a publish/review gate deferring areas below a stated evidence floor from the live grid, leaving extraction untouched so E1/E5 stay honoured.** Cost: hides ~490 real, correctly-scored terms from browsing. Analysis: scratchpad/sdd/area-overgeneration-analysis.md.

KNOWN LIMITS — recorded, not hidden. Canonical list: workshop/docs/limits.md §10, linked from quickstart/user-guide/manual/faq. Ten verified TRUE against running code. The load-bearing ones: - **All deep links are segment precision, never word.** 15,610 word timings and the 76-unjoined-word analysis exist; nothing wires them into evidence. “Jump to the exact moment” is currently “jump to the right few seconds”. - **Registry/taxonomy sync gap:** the taxonomy asserts 497 area ids; the registry holds 5. Areas are unusable as traversal ORIGINS. Partly caused by an operator-side revert that mistook DESIGNED output for pollution — minting.py states plainly that passages.jsonl is the one registry that decides attach-vs-mint, so kg_area rows there are correct. T106 recovered 492 of 497 ids from taxonomy history after the damage, but does not re-register them. - T038 unmet: the single-term response cannot serve the significance measure’s INPUTS (taxonomy.py persists only the final score) — reports inputs_available: false. - render_diagram implemented and never called from export_area. - Terms have no minted PID, so they cannot be graph nodes. - lesson_section / question are not indexed — correctly refused, since no loader or real content exists. - The writeJSON guard’s extension prefilter is hand-maintained against catalog.go’s PRIVATE regex and can drift. Durable fix: export the list.

BLOCKED, unchanged: T115/T116 remain gated on the open “answering over the new material” clarification in spec.md.

Repeatability (T104/T106): docs/prompts/add-a-chapter.md in the workshop submodule carries the per-chapter procedure. T106’s taxonomy update path matches an established area by EXACT evidencing-passage-set relationship (equal / subset / superset / none) — no fuzzy matching, which M5 forbids — and carries identifiers forward byte-for-byte.

Everything in this section was re-measured on 2026-09-01 against the live tree. Statements of the form “X is done” name the command that produced the evidence.

A1 — SpecKit feature 001: building has STARTED, nothing is committed

specs/001-workshop-curriculum-platform/ is the only feature directory. Its artifacts exist and were counted mechanically on 2026-09-01:

Artifact	Measured
spec.md	

Artifact	Measured
	359 lines · 42 distinct FR-### ids (FR-001...FR-042) · 18 numbered success criteria (SC-001...SC-018) plus SC-016a = 19 criteria in total
plan.md	311 lines
research.md + research/	283 lines + 3 files (transcription.md 1249, search-architecture.md 837, llm-bridging.md 735)
data-model.md	205 lines
contracts/	3 files (http-api.md 1122, pipeline-cli.md 817, passage-contract.md 640)
quickstart.md	1127 lines
tasks.md	342 lines · 91 distinct T### ids
checklists/requirements.md	120 lines

The claim this section used to carry — “analyze, implement and review have NOT run ... not one of its 91 tasks has been executed, no source file for this feature exists outside specs/” — is WITHDRAWN. Measured 2026-09-01:

- specs/001-workshop-curriculum-platform/analysis.md exists (15,555 bytes) and is **untracked** (git ls-files returns nothing for it).
- git -C workshop status --short shows an untracked pipeline/ tree carrying real implementation — run_faster_whisper.py, run_whispercpp.sh, build_whispercpp.sh, compare_engines.py, detect_media.sh, calibrate.sh, requirements.txt, CALIBRATION.md, a built venv/, engines/whisper.cpp and 2.4 GB of models/ — plus untracked curriculum/, platform/, evidence/ and seven docs/ pages.

No completion claim is made, and none should be inferred.

Everything named is UNTRACKED in both repositories, the work was in flight while this was written, and **no task-by-task status was measured** — do not report any T### as done on the strength of this paragraph. Nothing has been built or run in a container. Re-derive before relying on any of it:

```
ls specs/001-workshop-curriculum-platform/
git -C workshop status --short
```

Known internal inconsistency, not fixable from here. `plan.md`'s own `## Phase Status` section still reads “Phase 0 (Research): IN PROGRESS” and “Phase 1 (Design & Contracts): NOT STARTED”, while the Phase-1 artifacts (`data-model.md`, `contracts/`, `quickstart.md`) exist and the same file carries a “Post-design re-check (2026-09-01, after Phase 1)” block. That section is **stale within `plan.md`**. It was deliberately not corrected: `specs/**` was under concurrent edit by the analysis agent when this was measured, and editing another agent's live working set is how two agents silently overwrite each other. Whoever runs `analyze` next should reconcile it there.

Where to resume: `/speckit-analyze`, then `/speckit-implement`. Read `tasks.md` first — T001 onward — and `plan.md`'s “Constitution Check” blocks, which are the acceptance conditions the tasks were derived from.

No longer blocked on host capability — see §5 H9. The blocker this section used to record (“**Neither is installed on this host**”) is **WITHDRAWN**: a dual ASR stack and a generative model were installed on 2026-09-01. Read H9 before planning around them — the ASR engines resolve only through `workshop/pipeline/venv/bin/python`, not the system `python3`.

A2 — G7 propagation: CLOSED, re-measured 44/44 (was 28/28, was 20/24)

`workshop/` onboarding, which was the open item at the previous Synced-Commit, is **done**; `submodules/containers` was onboarded with it, and four more submodules joined on 2026-09-01. Re-measured 2026-09-01 by `bash scripts/verify-governance-cascade.sh`, exit 0 (12 PASS / 0 FAIL / 0 ENV / 8 NOTE):

- **C1** classified all **13** declared submodules from evidence — **11 owned**, 1 governance source, 1 third-party — with no hardcoded roster.
- **C2** found all **11 × 4 = 44** owned-submodule carriers present and non-empty.
- **C3** found all 44 recognised by the canonical `is_pointer_carrier` predicate.
- **C5 ROOT-LOCKSTEP** found the four root carriers byte-identical from line 19 down, shared digest `542e1c38867a14a5...` (re-measured 2026-09-02 after this revision's carrier edits; `dd14cc3a0073c75a...` and `bef157d5a8992aae...` are both superseded), plus a NOTE that the declared split equals the measured convergence line (see §2).

- **C6** found `helix-deps.yaml` and `.gitmodules` in agreement in both directions — 13 submodules = 12 `deps[]` + third-party comments.
- **C7** classified the 1 nested gitlink under the 11 owned submodules.
- **C8** found all 11 owned submodules internally in lockstep.

Independent spot-check: `workshop/{CLAUDE,AGENTS,QWEN,GEMINI}.md` are 118 lines each and each opens with a real `## INHERITED FROM` heading; `submodules/containers/{CLAUDE,AGENTS,QWEN,GEMINI}.md` are 703 lines each and likewise. **Every earlier count is superseded, not confirmed**, and the sequence is kept because the reason each died is the lesson: “20/20” enumerated a hardcoded five-submodule list; “20/24” was correct until `workshop` and `containers` landed; “24/24” was 6×4 , before `containers`; “28/28” was 7×4 and was correct on the MORNING of 2026-09-01, superseded the same day when `LLMProvider`, `RAG`, `verdict` and `passage` joined. A “36/36” figure circulating verbally was never measured and is wrong — 36 is 9×4 , and the owned fleet is 11. **Do not hand-derive this number; run the verifier and quote what it prints.**

A3 — Constitution sweep: C1 and C8 both FIXED. 173 / 96 / 2 of 271 — the split describes THIS tree, and no failing gate was hidden

Measured 2026-09-02, ~2h45m, 718,345 lines / 105 MB of output preserved. The “reported prior measurement” this section carried is now **CONFIRMED by a completed run** — it was accurate:

```
gates run : 287 · PASS 186 · FAIL 95 · ERROR 6 · $11.4.32 step 1:
pass · exit 1
```

Constitution gitlink at run time: `f5876a3b700e`. **Caveat that limits the whole figure:** the pin `MOVED` under the measurement — it is `3be10826f3d2` now — so this split describes `f5876a3b700e` and nothing else.

Bucket split of the 101 non-PASS rows, no double-counting:

Bucket	Rows	FAIL	ERROR
(a) third-party & staged carriers	82	82	0
(b) constitution-internal	5	4	1
(c) genuinely ours	14	9	5

1. is the entire `cm_covenant_114_*_propagation.sh` family, held by **five** carriers — the two `milosvasic.ru/Upstreamable/*`.md, the two

superspec demo fixtures, and a **NEW fifth**: workshop/pipeline/engines/whisper.cpp/AGENTS.md, inside the vendored whisper.cpp build tree. All are **absence, 0 DIVERGENT. Real improvement: vasic.digital/QWEN.md, the one OWNED carrier the previous triage named, no longer appears.**

C1 — the sweep points 85 of 287 gates at the PARENT of this repository. VERIFIED.

This is ours, it is the biggest finding in the run, and it invalidates most of the split above. resolve_argv in scripts/verify-all-constitution-rules.sh resolves --root **only if the gate's own source contains a literal --root) case arm:**

```
if grep -qE '^[[:space:]]*--root\)' "$g"; then      # line 672
```

Re-verified independently 2026-09-02: **232 of 260 gate scripts have no such arm** — they are thin wrappers whose argument parsing moved upstream into lib/covenant_propagation_engine.sh. Those gates therefore receive **no --root** and fall back to a default that resolves to **the PARENT directory of this repository** (dirname of the checkout — deliberately not written here as an absolute path; scripts/audit-hardcoded-paths.sh now scans docs/ and this file, and correctly flagged the literal that stood here). A second sub-defect: the root_kind case has no arm for --root <dir>.

Measured harms, each one a different way of being wrong: - 81 gates FAIL emitting ZERO BYTES. A blind instrument scored as an accusation — the exact inversion of “a blind instrument reports PASS”. - **cm_test_mock_pid_explicit_int.sh is a confirmed FALSE FAIL** — it PASSES at the correct root. - **cm_oracle_strategy...** emitted **714,162 lines** where the correct root yields 15,156. Two further gates **timed out at 900 s (rc=124)**. - Proven fix direction: **CONSUMER_ROOT=<repo>** turns 0 bytes into a full 76-line correct verdict. Remedy = ungate the header lookup, add a dir arm, export CONSUMER_ROOT.

C1 IS FIXED (2026-09-02), and the sweep was re-run to completion. New split: **187 PASS / 96 FAIL / 4 ERROR of 287** at that stage, exit 1 — and **exactly three rows changed verdict, all three predicted:** the two rc=124 timeouts became real FAIL rc=1 in 62 s and 65 s, and the false FAIL became PASS. Wall clock **867 s** against $\approx 2\text{h}45\text{m}$; output **3.68 MB / 25,970 lines** against 105 MB / 718,345 lines.

Root resolution, re-measured from --list on this tree: **105 gates now receive the project root** and 3 the constitution root, against **17 / 3** before — **+87 newly served, none lost, and no gate changed side.** All

four measured harms are cleared: 81 zero-byte FAILs → **0**;
cm_test_mock_pid_explicit_int.sh FAIL → **PASS**; cm_oracle_strategy...
714,162 lines → **15,158**; two 900 s timeouts → real findings in ~1 minute.

Still 96 FAIL — nothing was made green by pointing a gate somewhere convenient. The 81 propagation gates still fail at the CORRECT root, now in 76 readable lines each instead of zero bytes.

A third test was required that the prescribed remedy did not name. Ungating the header lookup alone trades one wrong verdict for another: covenant_propagation_suite.sh documents --root <consumer-root> but parses no flags and needs a subcommand, so handing it a root converts a usage refusal into an rc=2 ERROR scored against this tree. The fix therefore asks three questions — is --root <kind> in the gate's **own leading header** (not, e.g., inside a heredoc template it emits — that distinction changes the answer for exactly 1 file of 286); will a parser **actually receive** it, in the gate or in a lib/*.sh it names; and which root does a generic <dir> mean, classified from the gate's own description. Proof at that stage: **22 mutations, 25/25 green** (since grown to **25 mutations, 27/27** by the C8 work), with M16–M19 each regression-tested by re-introducing the specific defect.

Honest boundary: the constitution pin moved between the two sweeps (f5876a3b700e → 3be10826f3d2), so the two TOTALS are not a controlled comparison. The three-row per-gate join is.

C8 + C8a — FIXED 2026-09-02 by an evidence-based non-gate classifier. 271 gates, and the delta reconciles exactly.

New split: 173 PASS / 96 FAIL / 2 ERROR of 271, against the post-C1 baseline of 187/96/4 of 287. **The arithmetic closes with nothing left over:** $287 - 271 = 16$, $187 - 173 = 14$, $4 - 2 = 2$, and $14 + 2 = 16$. **FAIL is UNCHANGED at 96.** The 16 removed rows were 14 PASSes — the generator's wrote 162 wrapper file(s) among them — and the 2 C8 ERRORS. **No FAIL was removed and no failing gate was hidden.** Measured twice, because the first run overlapped comment-only edits to the sweep and editing a running shell script in place is a real hazard; the clean re-run reproduced 173/96/2 identically.

Discovery now reads EVIDENCE out of each file — there is no name list. A RESCUE rule is evaluated first: any file parsing --selftest or selfcheck exposes a self-verifying entry point and is **never** excluded

(18 of 286 qualify, which is what keeps 8 of the 15 `lib/` files in the sweep). Then five positive-evidence rules exclude **16**: 6 fixtures/inputs, 2 SOURCED, 6 HELPER, 1 SUBCOMMAND, 1 WRITES-BARE.

The rejected signals are the more instructive half, each killed by a count: - **executable bit** — 278/286 carry it, including all three offenders; - **lib/ path alone** — 15 files, but **8 expose --selftest and are real checks**, so excluding on path would have been an 8-check coverage cut; - **GATE= self-name** — 58/286 lack it while being unambiguous gates, and worse, **the generator DOES carry one inside the heredoc it emits**, so this signal classifies the single dangerous file AS a gate; - **a Side-effects block mentioning writing** — 102/286, because every honest mutation test documents “creates + removes a temp fixture dir”.

Direction of error chosen deliberately: every rule demands POSITIVE evidence, a file matching nothing is KEPT, and the excluded set is printed with its reason. A silent exclusion is the blind-instrument failure relocated, not solved.

The suite was excluded rather than given a `gates)` branch because all **81** names in its data pack resolve to wrappers already discovered individually (81 rows, 81 matched, 0 missing) — a branch would have added 81 duplicate verdicts. The ledger was re-baselined **through --update-ledger only**: pin unchanged, gates 286 → 270, removed 16, added none.

Proof grew **22 → 25 mutations, 27/27 green**. M20 is the load-bearing one: it proves the generator is not executed **by a sentinel the generator itself writes**, not by reading the sweep’s own summary — and then deletes only its `(no args) (re)write` header line to show it is re-admitted and the write happens. **That last step is what proves the classifier reads the file rather than a list.** A regression was caught by the existing battery en route: the new SUBCOMMAND rule excluded M18’s own specimen, M18 went red, and the specimen was re-cut to isolate the property it actually tests.

Verified after two full sweeps: `git -C submodules/constitution status --porcelain` shows only the pre-existing nested design-toolkit dirt — ZERO wrapper files.

C8a (the original finding) — a READ-ONLY sweep was WRITING into a consumed submodule on every run

Found while fixing C1, not fixed, and it belongs with C8.

`covenant_propagation_wrappers_generate.sh` is discovered as a gate and invoked **bare** — confirmed on this tree, `--list` shows `argv: (gate defaults)` — and **bare** is its **WRITE mode**. Its row reads `wrote 162 wrapper file(s)` and is scored **PASS**.

No damage has occurred: the bytes are identical each time and `git -C submodules/constitution status --porcelain` shows only the pre-existing nested submodules/`design-toolkit` dirt, no wrapper files. **But the only thing standing between a read-only audit and a mutated consumed submodule is idempotence** — not intent, not a guard, and not a read-only mode. A generator whose default action is to write should not be in a discovery-driven gate set without an explicit read-only invocation.

C2–C8, the rest of bucket (c). Note these are ABSENCES — the paths below do not exist, and are named as findings rather than as entry points (writing them as plain paths made `continuation-check.sh` C5 correctly report the document as pointing at a script that is gone, which is the check working):

- **C2** — `README.md` has no badge row (§11.4.259); its first line after the H1 is a blockquote.
- **C3** — a `zero_findings_sweep.sh` under `scripts/audit/` is **not created** (§11.4.261).
- **C4** — a `zero_findings_ratchet.tsv` under `docs/findings/` is **not created** (§11.4.261(C)) — an **operator decision** under §11.4.66 / §11.4.224(E).
- **C5** — 7,577 test functions carry no oracle-strategy annotation (§11.4.245), measured at the CORRECT root.
- **C6** — four §11.4.268-family consumer artifacts were never created (a `critical_blocker_gate.sh` under `scripts/lib/`, two `.jsonl` registers under `docs/requests/`, and a traps fixture tree under `scripts/testing/anti_slop/fixtures/`), verified absent at BOTH the derived and the correct paths. **Three of the four gates honestly report BLIND, not PASS.**
- **C7** — `cm_healthcheck_covers_served_ports.sh` is unbound: it needs `--compose / --manifest`, and its `rc=2` is UNDETERMINED, never a pass.
- **C8** — the sweep discovers a sourced helper and a batch runner as if they were gates.

Full record: docs/constitution-adoption/GATE-TRIAGE.md **Revision 2** (840 lines) — §1–§8 preserved and explicitly marked SUPERSEDED with the reason, new §9 carrying the command, the split, the moved-pin caveat, all three buckets and C1–C8 with gate, file and fix for each.

A3 (superseded framing) — why no split was settled before today

scripts/verify-all-constitution-rules.sh discovers its gates dynamically. The gate population moved 57 → 286 when the constitution was fast-forwarded, so **every split published before that fast-forward is withdrawn and none is comparable to a present-day run**. Do not restate an old number.

The most recent measurement available to this document is **186 PASS / 95 FAIL / 6 ERROR of 287 gates**. It is recorded here as a *reported prior measurement*, not as something this revision observed to completion: a re-run started on 2026-09-01 was still executing when this document was written, so no fresh split is claimed. Re-run it and record what you get.

The failures are known to include two classes that no commit this repository can make will clear: 1. **Third-party and staged carriers** — submodules/superspec/examples/..., milosvasic.ru/Upstreamable/..., and the vendored spec-kit copy under .specify/extensions/. The cascade reports these as *known-unclearable* and excludes them from ITS verdict; the propagation-gate family inside the sweep still counts them as FAILs. Excluded there means **not double-counted, never suppressed**. 2. **Defects internal to the constitution submodule itself**, which is upstream code this repository consumes rather than owns.

Neither class is a reason to treat the sweep as green. **Where to resume:** bash scripts/verify-all-constitution-rules.sh, then triage the FAIL list against docs/constitution-adoption/GATE-TRIAGE.md.

A4 — scripts/audit-environment-assumptions.sh — RED at 17, now GREEN — 12 fixes + 5 exemptions

Opened this session at exit 1, 17 frozen environment assumption(s), 2166 files across 14 repositories in 12 classes. Closed it at exit 0,

no NEW frozen environment assumptions (531 justified occurrence(s) allow-listed). The “10 frozen assumptions in

submodules/LLMProvider” figure below is **WITHDRAWN**, and so is the intervening GREEN recorded in CLAUDE.md at 1987 files — real when taken, not a description of this tree.

Read the split, not the exit code. An allow row is not a fix:

Resolution	Count	What
Real fix	11	workshop/pipeline/benchmark/ {build_bench_nomic_full,run_retrieval_benchmark}.py — every argparse default now os.environ.get(...) (\$OLLAMA_URL, \$BENCH_EMBED_MODEL, \$WORKSHOP_BASE_URL, \$BENCH_ALT_MODEL, \$BENCH_CURRENT_MODEL), each keeping its former literal as last resort so existing callers are byte-identical in behaviour; prints and docstrings now report the RESOLVED model
Real fix	1	workshop/platform/backend/gates/prove-evidence- precision-mutation.sh — bare GNU-only sed -i → sed -i.bak ... && rm -f "\$SED_BAK", plus a cleanup() extension and an rc-2 preflight refusing to clobber a pre-existing .bak
Exemption	1	workshop/platform/backend/internal/api/ suggest_gates_test.go — net.Listen("tcp","127.0.0.1:0") asks the KERNEL for a port, reads it back with ln.Addr() and injects it via t.Setenv; an override being exercised, not an assumption. go build excludes *_test.go from every production binary
Exemption	4	the three workshop/pipeline/benchmark/*.json result files — a result records WHICH MODEL WAS MEASURED; rewriting it would falsify the record. Verified before exempting: only positive_queries/ negative_queries are ever dereferenced; the _meta object holding the flagged keys is never read back

Same directory, opposite verdicts — the .py defaults fixed, the .json results exempted. A blanket directory rule would have destroyed that distinction, which is why the guidance said to judge them separately.

Evidence that the 5 rules are live rather than dead weight: the audit’s **STALE allow rule count stayed at 2** (the pre-existing pair, scripts/audit-hardcoded-paths.sh and _tools/gen/review_ui_all.py). A rule matching no occurrence is reported stale; none of the 5 appears there. **728 baselined occurrences untouched; no # BASELINE: row added; the third-party superspec NOTE untouched.** The mutation proof was

re-run after its edit — `FINAL_PROOF_EXIT=0`, `evidence.go` sha256 identical before and after, and the seeded mutation still turns the gate RED, so the proof still proves.

All five verdicts are recorded because the movement is the point — this instrument has now moved five times without the audit itself being edited once.

The cause is measured, not guessed, and it is the same event that broke this document. All 17 are inside `workshop/`, in seven files that did not exist at the workshop gitlink this document previously recorded:

File	Findings
<code>workshop/pipeline/benchmark/build_bench_nomic_full.py</code>	6 (5 MODEL, 1 ENDPOINT)
<code>workshop/pipeline/benchmark/run_retrieval_benchmark.py</code>	5
<code>workshop/pipeline/benchmark/retrieval_benchmark.json</code>	2
<code>workshop/pipeline/benchmark/results.json</code>	1
<code>workshop/pipeline/benchmark/results_nomic_full.json</code>	1
<code>workshop/platform/backend/internal/api/suggest_gates_test.go</code>	1 (ENDPOINT, <code>net.Listen("tcp", "127.0.0.1:0")</code>)
<code>workshop/platform/backend/gates/prove-evidence-precision-mutation.sh</code>	1 (GNUBSD, <code>in-place sed -i</code>)

(6 + 1 + 1 + 2 + 5 + 1 + 1 = **17**, which is the verdict line exactly.)

A stronger claim was drafted here and is WITHDRAWN before it ever stood. It read: “the five workshop gitlink bumps that C3 flagged are the same five that turned this audit red.” That was built on comparing against `55076bf943a5` — the gitlink this document *recorded* — rather than against `6130d6b55d693`, the gitlink actually in effect at the previous Synced-Commit `1b90daa6`. Comparing against a stale reference point does not merely misstate a number; it manufactures a causal link. Measured against the RIGHT baseline (`git rev-parse 1b90daa6:workshop → 6130d6b55d693`, then `cat-file -e per file`):

Entered with the five bumps (ABSENT at 6130d6b5)	Findings
pipeline/benchmark/build_bench_nomic_full.py	6
pipeline/benchmark/results_nomic_full.json	1
subtotal	7

Already present at 6130d6b5, unseen until now	Findings
pipeline/benchmark/run_retrieval_benchmark.py	5
pipeline/benchmark/retrieval_benchmark.json	2
pipeline/benchmark/results.json	1
platform/backend/internal/api/suggest_gates_test.go	1
platform/backend/gates/prove-evidence-precision-mutation.sh	1
subtotal	10

7 + 10 = **17**. So the five bumps account for **7 of the 17**, not all of them; **10 were already in the tree at the previous Synced-Commit** and went unseen. All seven files are indeed absent at 55076bf943a5 and present at b2327896a679 — that measurement was correct and simply did not support the conclusion drawn from it.

The mechanism is still real for the 7: coverage moves with the fleet on its own, with no edit to the audit, which is what the fleet-derivation fix was for — a red earned that way is the instrument working, not a regression in it. **The other 10 are the more uncomfortable finding:** they were present and unflagged in this document across a full session.

Most of the 17 are frozen model names (nomic-embed-text, jina-embeddings-code-cpu) and endpoints (http://127.0.0.1:11434, http://127.0.0.1:8087) in benchmark code and its committed JSON results. Honest boundary (§11.4.6): a benchmark that names the model it benchmarked is arguably a RESULT rather than a frozen assumption, and the results*.json rows in particular record what was measured. That is an argument for a # REASON: allow rule on the results files — **not** for one on the two .py files, whose argparse defaults are exactly the freeze the audit exists to catch. Deciding which is which is the work; do not blanket-allow the directory.

The two long-standing residues were also worked this session, and both moved.

- **STALE allow rules: 2 → 0** (rule count 444 → 436). Both were in the embedded ALLOW_RULES heredoc and both are DELETED, each

replaced by a tombstone comment recording the evidence:
scripts/audit-hardcoded-paths.sh * * suppressed nothing (its
single class hit is inside a # comment, which the scanner blanks for
.sh before any class is tested) while standing as a blanket whole-file
exemption; _tools/gen/review_ui_all.py MODEL * named a model
id that lives only in a module docstring with no quote character on
the line, and git log -p --all shows that was the only occurrence
the file ever had. Neither deletion surfaced a new finding. --
strict-allow-list moved 1 → 0.

- **Baselined occurrences: 728 → 683 (-45).** The slice taken was
_tests/ ENDPOINT — findings **F13 and F14, now CLOSED** —
chosen by measurement: of the 728, **638 sit inside submodules**
(472 containers, 102 constitution, 20 workshop, 17 LLMPProvider, 14
ai_interviewing) and need upstream commits, leaving **90 umbrella-**
root-owned, of which _tests/ ENDPOINT was exactly half. A new
_tests/env.js is now the single source of truth for both PORTS and
BASES, each process.env-derived with the old literal as documented
default and an unparseable port throwing at require() rather than
silently defaulting. That closes the half no allow rule could express:
the port a config **bound** and the base a spec **requested** were
independent literals free to disagree, and the disagreement
surfaced as an assertion about the *site*. Evidence: node --check on
29 files; **237 passed** on chromium at default ports; and **12 passed /**
41 passed at non-default VD_PORT=9401 MV_PORT=9082, which was
impossible before and is the entire content of F13. Proof the
SOURCE moved and not the ledger: git show HEAD:scripts/audit-
environment-assumptions.sh run against the fixed tree reports **8**
stale rules — the 6 whose defects no longer exist, plus the 2 deleted.
_tests/env.js is UNTRACKED, so git ls-files does not yet show it
to the gate; it was audited separately in a synthetic repo with no
rule matching it (rc 0, zero suppressions) and enters the umbrella
universe when committed.

Read the rest of the output before calling anything clean: 683
baselined occurrences across 216 files (declared, known, unfixed
debt, printed every run by design) — the next coherent slice is F15
(MODEL, _tools/gen/, 17 rows) — and **1 out-of-scope finding in a**
third-party gitlink (submodules/superspec/.github/workflows/ci.yml,
pinned python-version: "3.12"), reported under §11.4.156(C) / §11.4.29
so it is never silently omitted. **A baseline is recorded debt, not a**
justification.

The history below is kept because the reason each verdict died is the
lesson. **Both earlier verdicts are real and both are recorded,**
because the movement is the point. The 7 frozen GNU-vs-BSD

assumptions this section originally carried — 6 in `scripts/verify-check-registry.sh` (GNU-only in-place `sed -i, stat -c '%a'`) and 1 in `scripts/verify-manifest-pins.sh` — **were** fixed with portable helpers by their owning agent, and the audit went **green (exit 0)** at 1763 files across 14 repositories, printing:

```
no NEW frozen environment assumptions (441 justified
occurrence(s) allow-listed)
```

Re-measured later on 2026-09-01: exit 1, 10 frozen environment assumptions, scanning **1794 files across 14 repositories in 12 classes**. All 10 are inside `submodules/LLMProvider`, which joined the owned fleet the same day and brought its own occurrences into scope:

- `submodules/LLMProvider/pkg/providers/ai21/ai21_test.go` — 2 MODEL
- `submodules/LLMProvider/pkg/providers/openrouter/openrouter_test.go` — 5 MODEL
- `submodules/LLMProvider/scripts/prove-offline-discovery.sh` — 2 ENDPOINT + 1 MODEL

The coverage moved with the fleet on its own, with no edit to the audit. That is exactly what the fleet-derivation fix was for, so a red verdict earned this way is the instrument working rather than a regression in this tree.

Red is not the whole story either. Read the rest of its output:

- **712 baselined occurrences** remain — declared, known, **unfixed** defects, printed on every run by design. A baseline is recorded debt, not a justification.
- **2 STALE allow rules of 405** — the occurrence each one names is GONE. A rule is an exemption at a PATH, not at an occurrence; recreate the occurrence or delete the rule.
- **1 frozen assumption inside a third-party gitlink** — `submodules/superspec/.github/workflows/ci.yml`, a pinned `python-version: "3.12"` — reported OUT OF SCOPE under §11.4.156(C) / §11.4.29 so it is never silently omitted. Not ours to fix.

The 17 workshop/ rows are DONE — see the resolution table above. **Where to resume is now the 728-row baseline**, worked down file by file, deriving behaviour rather than adding allow rows. **The 10 submodules/LLMProvider FINDINGS are gone and need no work here** — that submodule fixed them upstream, verified 2026-09-02 in its own log:

```
fix(providers): default model and endpoint were frozen with no
```

override layer (F23, F24), test(settings, codestral, ollama): stop freezing real model ids and endpoints in fixtures, and fix(settings): the frozen-default class was 3x bigger than the MODEL column. **That is not the same as LLMProvider being clean:** it still contributes **14 files to the 728-row BASELINE** (13 under pkg/providers/, plus challenges/scripts/host_no_auto_suspend_challenge.sh). A finding fixed upstream and a baselined debt row are different states, and the audit prints them in different sections — do not read the empty findings list as a clean submodule. Then the 728-row baseline, worked down file by file, deriving behaviour rather than adding allow rows. Do not silence anything with 2>/dev/null, and do not clear the red by narrowing the scan.

A5 — live-production specs: out of gate 6, into deploy (uncommitted)

Three specs left the pre-push set and are now claimed by the deploy step instead. Measured 2026-09-01:

- `_tests/playwright.config.js` carries `testIgnore: /(restyle-seo-regression|v170-fixes|v171-hardcoding)\.spec\.js/`, so **gate 6 does not run them**.
- Those three specs contain **32 test(...) blocks and 97 expect(call sites** (restyle-seo-regression 13/40, v170-fixes 12/41, v171-hardcoding 7/16). The “86” figure in circulation is a different quantity — a Playwright **test-case pass count** (“86 passed / 2 failed”) observed across the four-spec live suite, not a count of assertions in these three. Both numbers are real; they do not measure the same thing. Do not restate “86 assertions”.
- They are claimed by `_tests/playwright.live.config.js`, whose `testMatch` names all four live specs.

They ARE now wired into the deploy step — as of an UNCOMMITTED working-tree change made by a concurrent agent on 2026-09-01. `_tools/deploy-langs.sh` now declares `LIVE_SPECS="all-languages-link-integrity restyle-seo-regression v170-fixes v171-hardcoding"`, pre-flights each one's file, and runs the live config with **no spec name on the command line** (line ~514), so the whole `testMatch` set executes after a deploy. **At HEAD this is NOT yet true** — `git show HEAD: _tools/deploy-langs.sh` still names `all-languages-link-integrity.spec.js` alone (line 365). Until that working-tree change is committed, a fresh clone gets the single-spec behaviour.

playwright.live.config.js was fixed in the same uncommitted change, and the defect it closes is worth knowing: the config *claimed* four specs but supplied environment for only three. all-languages-link-integrity.spec.js reads VD_BASE/MV_BASE (defaulting to localhost:8401/:8082) while the other three read VASIC_BASE/MILOS_BASE (defaulting to the production origins). With no webServer and no VD_BASE, that spec hit a dead localhost, and its get() helper turns a network throw into last = 0, so the refusal arrived at the assertion as a status: Error: sitemap.xml must resolve Expected: 200 Received: 0. A connection refusal laundered through an expect() and reported as a production defect. All four origin variables are now derived from the published CNAME files, explicit exports still winning.

The gate-6 deferral itself is sound and should not be reverted blindly: the measured cause was runner reachability (12 net::ERR_TIMED_OUT, 8 net::ERR_NAME_NOT_RESOLVED, 5 EAI_AGAIN, 77 60-second page.goto timeouts, and **zero** genuine assertion failures) while curl reached both sites with http=200. Gating a push of undeployed source on public DNS is the defect. The question was always *where* they run, not *whether* — and the answer landed: after deployment, against what was actually shipped.

A6 — scripts/verify-check-registry.sh: R5 FAIL cleared AND all 5 DEBT rows closed

The RED verdict this section used to carry is WITHDRAWN. Re-measured 2026-09-02: **exit 0, 41 PASS / 0 FAIL / 0 DEBT / 0 UNDET / 0 NOTE** (the **25 PASS** reading is superseded, not wrong-at-the-time). The single R5 anti-drift FAIL —

```
[R5] UNREGISTERED — scripts/verify-content-boundary.sh is
under a declared scanroot but appears in no check, debt, or
exempt row.
```

— is gone: scripts/verify-content-boundary.sh is now **registered** as content-boundary in scripts/check-registry.tsv, and the registry verifies both halves of its contract (SC-012 paired proof --prove-failure is a real, non-trivial case arm; SC-013 three-valued exit — --root / nonexistent → rc 2, distinct from 0 and 1). Three real states were measured across the one day, and all three are recorded rather than collapsed to the last: **0** at 21 PASS / 0 FAIL / 5 DEBT in the morning; **1** at 20 PASS / 1 FAIL / 5 DEBT once the unregistered file appeared — R5 doing exactly its job; **0** at 23 PASS / 0 FAIL / 5 DEBT once it was registered; **0** at **25 PASS**, then **31 PASS** on 2026-09-02 / 0 FAIL / 5 DEBT

later the same day, as further checks — submodule-remote-sync among them — landed and were registered. The PASS count moving 23 → 25 is R5 working, not drift; quote the run, not this line.

The FAIL was cleared earlier, and as of 2026-09-02 THE 5 DEBT ROWS ARE CLOSED TOO. constitution-rules-sweep, lumen-index-doctor, ollama-tune, prepush-gates and setup-agents-wizard-suite all owed a \$1.1 paired proof; all five now have one, and each was verified **in both directions** — it passes on this tree AND it fails when the gate it guards is deliberately weakened, with the weakening applied to a throwaway copy and never to the real file. Every mutation is DATA rather than a code edit, so each control is green by construction, which is the “inoperative proof” defect the registry itself documents. setup-agents-wizard-suite also owed **three-valued** and now demonstrates rc 2 four ways. Measured: **41 PASS / 0 FAIL / 0 DEBT**, exit 0, --strict also 0. The five debt rows became check rows in scripts/check-registry.tsv; no new *.sh was created, so R5 stays green.

Two things this does NOT close, both recorded rather than smoothed over:

1. **--run-proofs exits 1** at 54 PASS / 5 FAIL (10m17s). All five newly-promoted checks PASS there. The 5 FAILs are a different set on gates whose entry points git diff reports byte-identical to HEAD: provider-ci has a **real** assertion failure in its own selftest (“*M6b no repository or owner name in body — expected 0 names present, got 1 present*”), and submodule-remote-sync, mutation-anchor-rot, private-object-exposure and remedy-executability are **false positives of the hollow-proof heuristic** — each proof runs and returns 0, but summarises in prose rather than the $M_{<n>}$ / “N mutations” form the heuristic recognises, and submodule-remote-sync additionally gets --quiet appended, suppressing the lines it looks for. **The heuristic was NOT loosened** — it is the only thing standing between the registry and a proof that returns 0 without exercising anything. The fix belongs in those four proofs’ summary lines.
2. **The sweep’s DROP direction remains open, and now says so on every run.** constitution-rules-sweep’s debt reason named “a child gate silently dropped”. The ADD direction is proved (a new gate file is discovered, count 3 → 4, its failure reddens the sweep, with no edit to the sweep). The DROP direction is measured as **still open**: assertion **L1** deletes a gate, measures the count falling 3 → 2, and records that the sweep still exits 0 because it keeps **no expected-gate ledger**. Closing it needs a ledger design decision — the gate

population moved 57 → 286 at the last pin — and that decision was deliberately not made by the agent that found it.

A7 — constitution pin drift: CLOSED — pin equals remote, gate green for the first time

Done (2026-09-01). The four root carriers’ “the pin is BEHIND its upstream” block asserted two things that were measured and found wrong: that the fast-forward question was **UNVERIFIED**, and that this repository’s recorded anchor and line counts describe the pin “not what upstream currently ships”. A **scratch bare clone** (`git clone --bare` into a temp dir — no fetch, no checkout, no `git submodule update` inside `submodules/constitution`) measured: `merge-base --is-ancestor 902979027a90 f16ea779b82a` **TRUE**; `rev-list --left-right --count = 0 / 3` (3 behind, 0 divergent); `diff --stat` touches only `design-toolkit`, `docs/codegraph/Status.md` and the `submodules/design-toolkit` `gitlink`; `Constitution.md` is the **same blob** `34eff9d86cadb325721c958d35a411feaad27681` at both ends — 11,700 lines, 252 **###** § anchors, 1,779,401 bytes. The block was rewritten identically in all four carriers (below-line-19 shared body; C5 ROOT-LOCKSTEP re-verified byte identity from line 24 down) and in §1 and §5 H12 here.

Pin BUMPED (2026-09-01), on explicit operator authorization — “Fast-forward the pin”. Option (a) below was taken. The submodule was fetched and moved `902979027a90 -> f16ea779b82a` with `git merge --ff-only` (which would have refused a non-fast-forward), on branch `main`, and the ancestry was re-verified in the submodule itself before the move: `--is-ancestor TRUE, 0 / 3`. The `helix-deps.yaml` `constitution` `ref`: moved in the SAME change and both were staged together, so C9 never saw a bumped `gitlink` against a stale `ref`. Post-checkout re-measurement: `Constitution.md` blob still `34eff9d86cadb325721c958d35a411feaad27681`, 252 anchors, 11,700 lines, `sha256` unchanged. **Nothing was pushed to the constitution repository.** The constitution’s nested `submodules/design-toolkit` working tree was re-synced to its recorded `gitlink` so the submodule is clean, as it was found. `verify-manifest-pins.sh 0` (12 MATCH / 0 DRIFT / 0 UNDET); `verify-governance-cascade.sh 0` (12 PASS / 0 FAIL / 0 ENV / 7 NOTE).

The GAP is closed. The HOLE is now WATCHED, and the watcher is RED. Until 2026-09-01 **no gate in this tree detected local-vs-remote submodule drift** — C9 and `scripts/verify-manifest-pins.sh` compare `helix-deps.yaml` to the local `gitlink`, and they exited **0** while the pin sat

3 behind and exit 0 now. They cannot distinguish the two states, and never could; that is the reason a second instrument was needed, and the reason a green manifest check is not evidence about any remote.

Option (b) is DONE. `scripts/verify-submodule-remote-sync.sh` exists, compares every declared gitlink to its remote (`git ls-remote` on the declared branch, else HEAD), is three-valued so an unreachable remote is rc 2 and never a pass, and is registered as `submodule-remote-sync` in `scripts/check-registry.tsv` with a `--prove-failure` paired proof — which is what R5 demanded.

It exits 1, and the 1 is a real finding — but the finding SHRANK, and the 2026-09-01 figure is WITHDRAWN, not restated. That run read 6 CURRENT / 6 DRIFT, naming `design-toolkit`, `ai_interviewing`, `submodules/containers`, `submodules/LLMProvider` and `submodules/RAG` as BEHIND-and-fast-forwardable, and `submodules/constitution` as DIFFERS against remote `b9096acd98d2`.

Re-measured 2026-09-02 in three stages, and the gate is now GREEN for the first time in its existence. The readings, in order, each real:

```
1 – 6 CURRENT,  6 DRIFT    (2026-09-01)
1 – 11 CURRENT, 1 DRIFT    (2026-09-02, after five bumps landed
elsewhere)
0 – 12 CURRENT, 0 DRIFT    (2026-09-02, after the authorized
constitution fast-forward)
```

The five BEHIND rows — `design-toolkit`, `ai_interviewing`, `submodules/containers`, `submodules/LLMProvider`, `submodules/RAG` — were bumped and matched their remotes. The last row, `submodules/constitution`, read `f5876a3b700e` vs `3be10826f3d2` **DIFFERS** with the DIRECTION undetermined, because the remote commit was not in this checkout's object store. **The operator authorized the fetch**, which classified it as **2 behind, 0 divergent**, and `git merge --ff-only` closed it. `submodules/superspec` is probed and reported as a third-party NOTE, never a verdict input, which is why 12 are probed of 13 declared.

Do not bank the green. The gate prints its own caveat — *“Dated observation, not a standing fact: remotes move”* — and this pin went stale within a day on two consecutive occasions before this one. **The right conclusion is that the constitution pin is a STANDING operator decision, not a task that completes.**

Where to resume: nothing here is open. The remaining blind spot is not a row but a scope limit: this gate probes each submodule's declared origin only, so `design-toolkit`'s GitLab mirror is invisible to it — CURRENT there is a true statement about GitHub and says nothing

about the mirror. Historical note, kept because the reasoning still applies: when a row DOES read DIFFERS, command — so it is an operator decision. **Do not silence the 1** by deleting the gate, allow-listing a submodule, or bumping pins to make it green.

The upstream defect this section used to leave open is now CLOSED, upstream. It read: “Also unfixed and NOT ours: f16ea779b82a adds an unregistered design-toolkit gitlink at the constitution’s own root.” That was correct at f16ea779b82a and is **false at f5876a3b700e** — `ls-tree` finds the root gitlink at the old pin and finds nothing at the new one. Declining to fix it here was the right call, and it is worth recording as a precedent: an upstream defect in a consumed submodule was resolved by waiting for upstream, not by committing into a repository this project does not own. See §1 for the measurement.

A11 — content-boundary count moved 285 → 11,158. NOT a new leak; the baseline stopped being comparable. OPEN — needs an operator decision.

Measured 2026-09-02, `bash scripts/verify-content-boundary.sh`:

LEAK — 11158 surviving match(es) (prose 10833, short 268, name 57); 0 row(s) also could not be determined

The recorded reading was 285 (prose 207, short 35, name 43). **A ~39× jump on a gate that watches this repository’s most serious standing incident is not something to file quietly.** First question answered first: **the instrument did not change.** `git status --short scripts/verify-content-boundary.sh .content-boundary-allow docs/content-boundary.md` is EMPTY and `git diff --stat HEAD --` on the first two is empty — the detector and its 9 declared pairs are byte-identical to HEAD. Nothing was loosened, narrowed or allow-listed to produce this.

Three structural causes, each measured:

1. **The private corpus grew ~11×.** The gate scans `git ls-files`, so it sees only TRACKED files. workshop tracked 52 files at gitlink 55076bf943a5 — the value this document recorded when 285 was measured — 453 at 6130d6b55d693, and 583 at today’s b2327896a679; docs/ alone went 31 → 90. More tracked private text means more surface to match against, with no new disclosure required.
2. **specs/002-** is TRACKED IN THE PUBLIC UMBRELLA and describes the private implementation.** It contributes the single largest block — `tasks.md` 3832, `contracts/knowledge-graph.md` 730,

contracts/http-api-delta.md **398**, spec.md **384**, data-model.md **316**, quickstart.md **153**. Sampled, the matched text is *the spec's own task prose* ("Phase 1 Setup ... make the new packages buildable and their dependencies real T001 create workshop platform backend pkg ..."), which exists on both sides. specs/001-** adds ~730 more the same way.

3. **The public reusables were EXTRACTED FROM the private submodule**, so they match their own origin by construction: submodules/RAG/pkg/grounding/ ~**1699** across three files, submodules/LLMProvider/scripts/lib/anchor.py **634**, submodules/passage/pkg/passage/pid_test.go **124**.

The load-bearing limitation, and the reason none of this is a clearance: the gate detects CO-OCCURRENCE, not DIRECTION. Text that originated in a PUBLIC governance carrier and was propagated INTO a private submodule is indistinguishable, to this instrument, from private content leaked OUT. The four root carriers each carry **94** matches across 22 distinct lines, and the matched strings are this project's own governance prose — the /usr/bin/whisper trap sentence, the "0 clean · 1 a real finding · 2 could not determine" exit contract, the §12.10 docs/CONTINUATION.md path note — found in workshop/docs/prompts/add-a-chapter.md and workshop/docs/knowledge-model-contract.sections.json. That is governance flowing public → private, which is the cascade working as designed.

HONEST BOUNDARY (§11.4.6) — do not read the above as an all-clear. What was done here is a *characterisation of the dominant classes by sampling*, not a row-by-row assessment. **11,158 rows were NOT individually judged**; the previous incident's two-wave assessment covered **232**. No sampled row was found to disclose private teaching material, and **that is a statement about the sample, not about the set**. The name class (57, 7 distinct personal names, reported by digest only) is the highest-stakes residue and is **NOT cleared here**.

Compounding, from the feature-001 ground-truth assessment: T040 — the Chapter 1 redaction review — is UNRECORDED, and T104 (push) is explicitly blocked on it. The chapter recording features an identifiable third party. **A push is publication.** Treat T040 as a precondition of any push touching workshop, not as a task among 120.

A11a — a REAL cross-boundary copy was found in an UNTRACKED public file, and FIXED before it ever reached history

This is the one genuine disclosure risk found this session, and it was caught by an agent tasked to assess uncommitted artifacts — not by any gate. specs/001-workshop-curriculum-platform/review.md (PUBLIC umbrella, **untracked**) reproduced three passages verbatim out of the PRIVATE workshop submodule. Verified INDEPENDENTLY rather than taken on the agent’s report, by normalised 10-token-window comparison:

Location	Private origin	Windows verbatim, before
review.md:342–346 (blockquote)	a string literal in workshop/platform/backend/pkg/answer/verify.go	54 of 62
review.md:519–521	a header comment in workshop/platform/frontend/src/app/core/knowledge.ts	14 of 24
review.md:130	workshop/docs/quickstart.md	3 of 59

FIXED. All three are now paraphrased, each with an explicit note saying the original is cited **by path only** because the file is private. Re-verified EXHAUSTIVELY (stride 1, not sampled): **0 / 0 / 0** shared 10-token runs against all three private sources. The document survives at 682 lines with its meaning intact. **Nothing was published** — the file was untracked throughout, so unlike the 2026-09-01 incident this never entered a public history and redaction here is a real remedy rather than containment.

The structural hole this exposes is the durable lesson, and it is NOT closed:

- 1. verify-content-boundary.sh enumerates via git ls-files — TRACKED FILES ONLY.** An untracked file in the public umbrella is **invisible** to the gate built to catch exactly this. That is why the 11,158-match run above does not include these three passages at all.
- 2. The commit wrapper runs git add .,** which stages every untracked file (CLAUDE.md says so explicitly). So the sequence “write an

untracked public file quoting private source → run the wrapper → push” publishes it permanently, with the gate silent the whole way.
3. Nothing in scripts/continuation-check.sh’s watched set covers specs/** or docs/*.md, so no gate raises these files either.

Recommended, and NOT done here because it changes a gate’s contract: give verify-content-boundary.sh an opt-in mode that also scans git ls-files --others --exclude-standard, so untracked public files are assessed BEFORE git add . sweeps them in. Until then, treat “run the boundary gate” as insufficient before any use of the commit wrapper.

Operator decision needed, and it is not “make it green”: (a) accept the structural explanation and RE-BASELINE the recorded figure so the next reader compares like with like; (b) declare the genuine cross-boundary flows — specs/00* ↔ workshop, and each extracted reusable ↔ its origin — as pairs in .content-boundary-allow with reasons, which buys a smaller number at the cost of hiding those rows from the next reader; or (c) leave it visible and loud, as the 207 prose rows were deliberately left. **Do not add exemptions without the operator.** Re-derive rather than trusting this section:

```
bash scripts/verify-content-boundary.sh > /tmp/cb.txt 2>&1
grep -E '^    public ' /tmp/cb.txt | sed 's/^    public *///;s:
    [0-9]*$//'\
    | sort | uniq -c | sort -rn | head -20
git -C workshop ls-tree -r --name-only HEAD | wc -l
```

A8 — submodules/passage v0.2.0: PUBLISHED, breaking, consumer NOT updated

Done (2026-09-01), on explicit operator authorization — “Publish v0.2.0 now”. v0.2.0 was cut at

80f65382007f11d461d8f3065efc3f0b9933b9a9 — the word “now” this sentence used to carry is withdrawn, because **the gitlink has since moved to 729cd96a39fefff5675570adff2eeba4faed0d26** (measured 2026-09-02, remote CURRENT). The tag still names the commit below; the submodule has moved past it. Annotated tag v0.2.0 (tag object a82f3cc2f8781ac8b2e969bb631eb6bddac89460), pushed to BOTH mirrors and verified on each **independently with git ls-remote**, never from push output. The two remotes agree ref-for-ref.

Tags are immutable (§11.4.113) and none was moved. v0.1.0 (e31e6fb -> c6e6938) and v0.1.1 (992f5c2 -> bde026f) were read on both remotes before AND after the push and are byte-identical. v0.2.0 did not exist anywhere beforehand; it is a new object, so the Go proxy has no prior answer for it to contradict.

What v0.2.0 is. The library documented itself as project-not-aware in four places while its API carried a consuming application's nouns in a struct field, a JSON tag, a closed enum, two SQL columns and an index name. The old guard read go.mod and never opened a source file of the package it vouched for. Removed -> replacement: ChapterSlug -> Scope; TStartS -> ScopeOrder; TEndS / Speaker / SpeakerSource -> the opaque Attrs map; type SpeakerSource and Kind's four constants deleted (Kind stays open, asserted by AST); SyncFile now takes the consumer's Kind; NewRegistry and Unavailable are variadic; WithRecordValidator added so consumer domain rules run at the same write-path choke point. Schema v1 -> v2, stamped in PRAGMA user_version with a foreign version REFUSED rather than migrated; passages_by_chapter -> passages_by_scope; LoadJSONL sets DisallowUnknownFields. Full mapping and upgrade procedure: submodules/passage/CHANGELOG.md. Eight gates in pkg/passage/decoupling_test.go, four with paired mutations (§1.1) observed to fail. Verified at 80f65382 before tagging: go build ./... 0, go vet ./... 0, go test -race -count=1 ./... 0 at 45 PASS / 0 FAIL.

NOT done — this is the resumption point. The consumer workshop/platform/backend/internal/passagestore/ (1 modified + 2 new files) is **uncommitted** and was NOT touched by this work. Nothing verifies the consumer against the published v0.2.0 yet. The umbrella was deliberately **not committed**: the gitlink and helix-deps.yaml are STAGED only, because other agents were editing this tree concurrently.

BLOCKED / NOT STARTED — see §4. **G8** (markdown export mandate) and **G12** (no PreToolUse guard wired) are open with no work in flight. **G4** will not reach CLOSED from inside this tree; its remainder is operator-only. **G5** remains PARTIALLY CLOSED — the wrapper exists, tracked git hooks do not.

Background jobs. A scripts/verify-all-constitution-rules.sh run was started on 2026-09-01 and had not finished when this was written (see A3); nothing else is recorded. .lumen-reindex.log at the root is the running record for the semantic index. If an index rebuild or gate sweep is running when you arrive, **do not kill it and do not restart ollama** — run bash scripts/lumen-index-doctor.sh (0 healthy / 1 corruption / 2 could not inspect) rather than assuming. Measured 2026-09-01, exit 0: 2510 files fully indexed, 0 queued placeholders,

59,059 chunks, **59,059 vectors of which 59,059 distinct**, 0 NaN/Inf, 0 all-zero, 0 off-norm, 0 ragged blocks, integrity_check: ok. The duplicate-vector corruption previously tracked in this file is cleared.

A9 — content-boundary incident: history REWRITTEN and force-pushed; NOT closed

Read `docs/content-boundary-incident-2026-09-01.md` before touching `docs/workshop-curriculum/RECON.md` OR `specs/001-workshop-curriculum-platform/research/transcription.md`.

Private material from the notes PDF in the **private** workshop submodule was committed and **pushed to this public repository** in 63ac4df (2026-09-01 06:47 CEST) and was carried unchanged by d0b3c64, 96b2988 and ee3933d. Three classes: verbatim prose, the PDF's complete section-heading list, and **a third party's full real name** — personal data about someone who is not the repository owner.

Those four SHAs are deliberately left as written. They name a history that was discarded on 2026-09-01 and they no longer resolve in this repository — that is the correct record, not an error to be silently repaired (§11.4.6). The old→new mapping is in `docs/content-boundary-incident-2026-09-01.md` §8B.

Done (2026-09-01, working tree only, nothing committed): both files redacted — quotes replaced by verified-distant paraphrase, heading lists replaced by counts, the name removed everywhere, content-boundary callouts added to both. Verified independently at a **six-word shingle window** (stricter than the gate's eight): **zero prose overlap remains**; the only surviving overlap is the artifact's own filename, which carries a first name only and is the accepted baseline.

DONE (2026-09-01, AUTHORIZED, EXECUTED): the history rewrite and force-push. The operator gave explicit per-session §11.4.113 authorization. `git filter-repo` replaced the four leak-bearing blobs across the four affected commits keyed on `blob.original_id`; `refs/heads/main` moved 4df8401 → 562ecf9 on the single public remote via `--force-with-lease` (a deliberately wrong lease was refused first, as the §1.1 paired mutation). Verified afterwards: **51 of 51 disclosed literals absent from all 6,230 blobs** under `--batch-all-objects`, the tree at HEAD 6,260 of 6,260 entries **byte-identical**, all **13 gitlinks** and all **102 historical gitlink pairs** unchanged, all **4 tags** unchanged, commit identity envelope identical for all **104** commits, **98 of 104** SHAs

unchanged, git fsck clean with **0 unreachable / 0 dangling**. Full record with measurements: docs/content-boundary-incident-2026-09-01.md §8B.

Side effect of the follow-up commit, recorded rather than left to be discovered (§11.4.6). The project commit wrapper commits and pushes submodules as well as the root. Running it for the citation-fix commit therefore committed the **pre-existing, uncommitted** asset changes that were already sitting in the milosvasic.ru and vasic.digital working trees (regenerated assets/od/*.css and downloads/*.pdf) as chore(assets): design-system propagation and regenerated downloads, **pushed both**, and bumped their gitlinks 8166fdb → 1823d62c and 0bc25012 → 31928364. Two consequences: **both live sites will have redeployed** from those pushes (milosvasic.ru via its active pages.yml, vasic.digital via the provider's legacy Pages build), and helix-deps.yml was left recording the two superseded refs, which turned C9 and verify-manifest-pins.sh red (2 DRIFT of 12). **The manifest is corrected in this same commit and both are green again (12 MATCH / 0 DRIFT; cascade 12 PASS / 0 FAIL).** _tools/deploy-langs.sh was NOT run at any point. Nothing about the history rewrite caused this — the gitlinks were verified byte-identical immediately after the rewrite and before the force-push.

STILL NOT CLOSED, and this is the part that matters. A force-push does not delete anything from GitHub's storage. The four orphaned commits and all four leak blobs remain **fetchable by SHA** from github.com/milos85vasic/vasic by anyone who recorded them, until GitHub runs a server-side garbage collection — whose timing is not the repository owner's to control. Ending the exposure needs a **GitHub Support purge request** (text prepared at incident note §8A.7, first changed commit 63ac4df32e5f... → 16cd4ba847de...) and, ahead of it, **telling the third party**. Both are outward-facing and are the **operator's** to send; no agent has taken or may take either. Forks, mirrors, existing clones and search/archive caches are unreachable by any of this.

WAVE 4 (2026-09-02): the third party's FIRST name — redacted; second rewrite authorized. §2 of the incident note deliberately left the first name standing, because it is carried only inside the recording's own filename and the person's **full** name stood three lines away in prose; removing the weaker identifier while the stronger one remained would have accomplished nothing. **§8B removed the stronger one**, so that premise expired and the first name became the only remaining identifier of that person in a public repository. The decision was re-put to the operator, whose answer was verbatim

“Redact + second rewrite” — which is also the §11.4.113 authorization for the second force-push. Measured true scope, and it is **smaller** than the opening brief’s estimate of “19 references across 5 tracked files”: a case-sensitive **whole-word** `git grep` finds the name on exactly **9 lines in 4 files** — `RECON.md` 62–65, `spec.md` 17, `quickstart.md` 83 and 284, `transcription.md` 154 and 1074. The brief’s larger figure counted every mention of the recording and the notes PDF, including the many already written in the elided `...Recording.mp4` form, which carry no name. The fifth file it detected is the incident note itself, where the filename is already elided; it needed no change. Lower/upper-case whole-word variants: **0**. Untracked files: **0**. Case-*insensitive* matching is useless here — the token is four characters and a common substring across `_content*/`. Redaction form: `... elision in prose and captured output` (the house convention these files already use), and an unquoted **glob** `workshop/chapters/01/*Recording.mp4` in the two `RUNNABLE NOW` shell blocks, which keeps them runnable and resolves to exactly one file. Full record: `docs/content-boundary-incident-2026-09-01.md` §11.

DONE (2026-09-02, AUTHORIZED, EXECUTED): the SECOND rewrite and force-push. Run in place, `--blob-callback` keyed on `blob.original_id` with eight sha256-pinned redacted replacements — no rules file, no artifact holding the name. A **new hardlinked bare mirror** was made first (`.../.vasic-history-mirror-2026-09-02-pre-rewrite2.git`, 106 commits, 494 MB), because **every pre-existing backup holds only pre-FIRST-rewrite history** and none of them covered the state this run destroyed. **The all-objects scan found EIGHT carrier blobs across FIVE paths, not the four the HEAD grep could see** — the fifth is `specs/001-.../contracts/http-api.md`, which carried the name in a JSON example that a later commit deleted, so it is clean at HEAD and invisible to any HEAD-scoped search. Verified after: **0 of 14 occurrences surviving across all 6,247 blobs / 7,443 objects** under `--batch-all-objects` (before: 8 blobs / 14 occurrences, same script — its own positive control); tree at HEAD **6,260 of 6,260 entries byte-identical; 13 of 13 gitlinks and 104 of 104 historical (path, sha) gitlink pairs** unchanged, `helix-deps.yaml` and `.gitmodules` blobs unchanged, so **C9 is unaffected by construction**; commit identity envelope byte-identical for all **107 commits; 98 of 107 SHAs unchanged, 9 moved**; all **4 tags** unchanged; `git fsck --full rc=0` with **0 unreachable / 0 dangling**, `reflog` empty. Rewrite itself: **7.72 s, peak RSS 601 MB. 20 stale citations in 8 tracked files** were repointed (the first run found 17 in 8; the class is the same and one file wider), and each provenance note now records the **two-generation chain** rather than hiding the second move. Full measurements: `docs/content-boundary-incident-2026-09-01.md` §11.4.

The push (2026-09-02). Negative control first: a deliberately wrong lease was **refused** (! [rejected] main -> main (stale info), rc=1) and left the remote at 9dcfc41. Then the authorized push, --force-with-lease=refs/heads/main:9dcfc41... origin main:refs/heads/main, one explicit refs spec — + 9dcfc41...2d629e9 main -> main (forced update), rc=0. **Confirmed from git ls-remote, not from push output**, against all three remote names independently: all return 2d629e93e92a... for refs/heads/main, the same four tags at their original objects, and no other ref. **The commit wrapper was NOT used anywhere in this wave** — every commit was git commit --only with an explicit pathspec and every push a single explicit refs spec to the umbrella, so **no submodule was committed, pushed or bumped** and no site was redeployed. _tools/deploy-langs.sh was never invoked. Gates: **rc=0, 8/8 PASS** on all three runs (baseline 213.77 s, negative-control 205 s gate 6, push 203 s gate 6).

The support request now needs ONE value covering BOTH rewrites. filter-repo composed this run's commit-map with the first run's, so the First Changed Commit is reported against the **original public** SHA: 63ac4df32e5fd40806a50cd38fde8cdc39587c2c → 16cd4ba847dea301e0bb308a9d538cbd6328f13e. **This supersedes the fc7574b27c7f... value recorded for the first run** — that SHA was itself discarded by the second rewrite and no longer resolves. The orphan list the request must name is now **twelve** commits: the four from 2026-09-01 (63ac4df, d0b3c64, 96b2988, ee3933d) plus the eight orphaned on 2026-09-02 (fc7574b2, 7b4df26d, 4ee9e8de, b0ab4b44, 25fe585e, 562ecf9c, 203061dc, 9dcfc41c) — **every one of the eight carried between three and five of the eight name-bearing blobs**. All twelve remain **fetchable by SHA** from GitHub until a server-side purge runs.

Two instrument gaps this incident exposed, both OPEN: scripts/verify-content-boundary.sh matches on an **eight-word** window, so it **cannot see** a five- or six-word heading (it scored the heading list in transcription.md clean) and **cannot see a bare personal name** at all. It found the verbatim-prose class only. This leak was caught by a human-directed audit, not by the gate. Do not read a green run as “no private material here”.

The gate is now REGISTERED and exits 1 BY DESIGN. It is content-boundary in scripts/check-registry.tsv, with a --prove-failure paired proof and a three-valued exit (--root /nonexistent → rc 2), and the R5 FAIL it caused is cleared (§3 A6). Re-measured 2026-09-01 it exits **1** and prints:

LEAK – 285 surviving match(es) (prose 207, short 35, name 43); 0 row(s) also could not be determined

The figure this section used to carry — LEAK – 10293 surviving match(es) (prose 203, short 8535, name 1568) — is WITHDRAWN, not restated. It was a MID-TUNING snapshot taken while the detector was still being rewritten, so it measured the detector’s own noise floor rather than this tree. **The two numbers are not a before/after of any cleanup: nothing was redacted between them.** The short class fell 8535 → 35 and the name class 1568 → 43 because the detector learned to subtract keys already public in two or more public repositories, public git identity forms, a scale-free name-frequency floor (33 occurrences over 1,325,759 prose tokens = 25 per million) and path-reference matches — each subtraction printed on every run with its RECALL COST stated, so what the gate can no longer see is visible rather than assumed.

The **207 prose** matches are the judged class. Honest boundary (§11.4.6): §9 of the incident note assessed **232** matches — 29 redacted, 203 judged not to be disclosures — so 207 and 203 are close but are **not the same set**, and the difference is not accounted for here. Do not read 207 as “207 rows individually cleared”. They are left **visible rather than allow-listed**, deliberately: an allow-list entry buys a green exit by hiding the row from the next reader. .content-boundary-allow carried 9 declared pairs at that measurement, pardoning 4193 matches. **Do not force this gate green.** A gate a human must read is worth more than one that is quietly green.

A second wave WAS assessed, and partly remediated. The “~200+ further matches” sourced from ai_interviewing/ and workshop/ that this section previously deferred are §9 of the incident note: **232 assessed, 29 redacted as real disclosures, 203 judged not to be.** The remaining out-of-scope material — design-toolkit/-sourced matches in _analysis/, _content/products/ — is affected by design-toolkit having gone **public** on 2026-09-01, which changes the boundary question for that source but has **not been re-assessed**. Not decided; do not treat the flip as retroactive clearance.

Comparing gate runs across this window is unsound. Another agent was concurrently building workshop/platform/** inside the private submodule, so the private corpus the gate indexes changed between runs and the derived exoneration sets moved with it. The attributable figure is the per-source one: matches sourced from the notes PDF went **29 → 0**, and RECON.md went **42 → 13** (-29 exactly), with the residual 13 all ai_interviewing-sourced.

Operator-only manual steps that no script here can perform are recorded in MANUAL-STEPS.md.

A10 — two carrier sets corrected in lockstep — DONE, nothing committed

Done (2026-09-01). Two four-file governance carrier sets carried claims that were measurably false when re-derived. Both were corrected as ONE artifact each, by a single script applying byte-identical replacements to all four members, so the four cannot drift apart. **Nothing was committed or pushed**; other agents were editing this tree concurrently.

Set 1 — the four ROOT carriers. Lockstep boundary measured at **line 19** (tail -n +18 gives 4 distinct digests, tail -n +19 gives 1), which is what C5 ROOT-LOCKSTEP declares and enforces, with a ceiling of 24. What was false and what replaced it:

Claim	Re-derived
LEAK – 10293 surviving match(es) (prose 203, short 8535, name 1568)	LEAK – 285 surviving match(es) (prose 207, short 35, name 43), rc 1. The old figure was a MID-TUNING snapshot of the detector, not a state of this tree; nothing was redacted between the two numbers.
audit-environment-assumptions.sh → 0, “no NEW frozen assumptions”	1 — 10 frozen assumptions, all in submodules/LLMProvider; 1794 files / 14 repos / 12 classes; 712 baselined; 2 stale allow rules of 405
verify-check-registry.sh → 0 at 23 PASS	0 at 25 PASS / 0 FAIL / 5 DEBT / 0 UNDET / 0 NOTE
“no gate compares a gitlink to its remote ... that is not started”	scripts/verify-submodule-remote-sync.sh exists, is registered with a paired proof, and exits 1 at 11 CURRENT / 1 DRIFT / 0 UNDETERMINED (re-measured 2026-09-02; the “6 / 6” figure is withdrawn) still true as of that measurement only — the remote has since moved to

Claim	Re-derived
the constitution pin “equal to git ls-remote ... HEAD as measured that day”	b9096acd98d2 and the gitlink now DIFFERS

Three figures on the correction list turned out **NOT** to be stale and were left as they stood, each re-derived rather than assumed:
`.gitmodules` declares **13** gitlinks; the cascade reports C2/C3 at **44/44** (11 owned × 4); and `verify-governance-cascade.sh` exits **0** at 12 PASS / 0 FAIL / 0 ENV / 8 NOTE.

Set 2 — the four workshop/ carriers. Lockstep boundary measured **empirically and it is NOT line 19**: the per-agent tokens sit at lines **1, 3, 9, 20, 42 and 51**, so a tail-based test converges only at line 52. That is why cascade check C8 normalises the per-agent header instead of splitting at a fixed line — the workshop set has no single split point. Two false claims were corrected:

1. “*derives its root from git rev-parse --show-toplevel*” — false, and **show-toplevel was the DEFECT, not the drift**. It answers for the CALLER’s cwd, so a run from a parent repository resolved the root to the umbrella; `extract-videos.sh` broke exactly that way. Every script now anchors on `${BASH_SOURCE[0]}`. The only two remaining call sites are `scripts/git-hooks/post-commit` and `post-checkout`, where git guarantees the cwd and the caller-relative answer is correct.
2. “*a teaching corpus, not a software product*” — no longer true. Measured 2026-09-01 the module ships `pipeline/` (two CPU-only ASR engines, faster-whisper 1.2.1 / CTranslate2 4.8.2 and `whisper.cpp v1.9.1`), `curriculum/` (a real chapter-01 transcript and exercise document plus an 825 KB `passages.jsonl`), `platform/` (a Go backend with five commands, an Angular frontend, four `verify/` `prove` gate pairs — **serving** on `http://127.0.0.1:8087` at the time of measurement), and a 16-script control plane. “*No absolute path is hardcoded anywhere in the module*” was re-derived and **still holds**.

Preserved verbatim in the workshop set because each is still true and load-bearing: the conditional-inheritance preamble and its `find_constitution.sh` resolver (the §11.4.28(B) / §11.4.35 contract — not touched), the video-archive pipeline description and its scripts, and the five module-local rules. A sixth rule was ADDED: `chapter and curriculum/` content never leaves the module as content, only by path.

Where to resume: nothing is owed by this item. The three gates it names — `verify-governance-cascade.sh`, `continuation-check.sh`, `verify-content-boundary.sh` — were re-run after the edit and are recorded in §6.

§4 Open gaps

Gap identifiers are `docs/constitution-adoption/INVENTORY.md`'s own (G1–G12). The **current status** of each lives in the four root carriers, not in `INVENTORY.md` — the inventory headings preserve the original discovery text and were not rewritten as gaps moved. `scripts/continuation-check.sh` compares the table below against every carrier that mentions the gap and fails on any disagreement.

Gap	Status	What is actually missing
G3	CLOSED	§11.4.32 sweep contract: both halves exist and run. <code>scripts/verify-governance-cascade.sh</code> supplies step 1 with a <code>--prove-failure</code> paired mutation; the step-1 caller distinguishes <code>rc=1</code> (violation) from <code>rc=2</code> (broken check) instead of collapsing both to FAIL. Step 1 re-measured 2026-09-01: 12 PASS / 0 FAIL / 0 ENV / 8 NOTE, exit 0 — grown from 10 checks to 12 by the addition of C8 (in-submodule carrier lockstep) and C9 (manifest pin sync). Note this closes the <i>contract</i> , not the

Gap	Status	What is actually missing
G4	PARTIAL	sweep's verdict — for the sweep's own PASS/FAIL split see §3 A3. §11.4.156. Umbrella root complies: .github/workflows/ci.yml.disabled is inert, enforcement is the local pre-push hook. Will not reach CLOSED — file-level disabling cannot reach provider-side settings (org-default required workflows, branch-protection required checks, the Pages source setting), which are operator-only and unverified. milosvasic.ru keeps an ACTIVE pages.yml as a documented deviation; vasic.digital is non-compliant at the provider level with no file-level remedy. The commit wrapper exists (commit on PATH → \$SUBMODULES_HOME/Upstreamable/commit → Software-Toolkit/Utils/Git/commit.sh + push_all.sh, driven by the tracked upstreams/GitHub.sh). What is still missing
G5	PARTIALLY CLOSED	

Gap	Status	What is actually missing
G6	CLOSED	is git hooks as a tracked, travelling artefact. The wrapper runs <code>git add . --keep .gitignore</code> accurate before using it. <code>helix-deps.yaml</code> exists at the root and parses under <code>yaml.safe_load</code>. Constitution-aware governance carriers across the owned submodules. 44 of 44, re-measured 2026-09-01. Fleet DERIVED from <code>.gitmodules + helix-deps.yaml</code>, never hardcoded: eleven owned submodules (<code>milosvasic.ru</code>, <code>vasic.digital</code>, <code>design-toolkit</code>, <code>ai_interviewing</code>, <code>monetization</code>, <code>workshop</code>, <code>submodules/containers</code>, <code>submodules/LLMProvider</code>, <code>submodules/RAG</code>, <code>submodules/verdict</code>, <code>submodules/passage</code>), each carrying all four carriers, each opening with a real ## INHERITED FROM heading and each accepted by the canonical
G7	CLOSED	

Gap	Status	What is actually missing
		is_pointer_carrier predicate. bash scripts/verify- governance- cascade.sh exits 0 — C1/C2/C3/C5/C6/C7/C8 all PASS — which is the closure condition this row previously named. Every earlier figure is WITHDRAWN, not restated: “20/20 CLOSED” (a hardcoded five- submodule list that missed workshop); “20/24” (correct at the time, superseded when workshop and submodules/ containers landed); “24/24” (6×4, before containers); “28/28” (7×4, correct on the morning of 2026-09-01, superseded the same day when LLMProvider, RAG, verdict and passage joined). A “36/36” figure circulating verbally was never measured and is wrong — 36 is 9×4 and the owned fleet is 11. Run the verifier; do not hand- derive.

Gap	Status	What is actually missing
G12	OPEN	The §11.4.65 markdown/export mandate is unmet across the repository. No PreToolUse guard is wired. <code>.claude/settings.json</code> declares only <code>enabledPlugins</code>; the canonical guard script sits unused at <code>submodules/constitution/scripts/hooks/guard-forbidden-commands.sh</code>.

Not listed above: G1, G2, G9, G10, G11 are not carried in the carriers’ “Known open gaps” list. G1 (no consumer governance layer) and G2 (no inheritance pointer) are answered by the four root carriers existing. G9, G10 and G11 have **no verified current status recorded anywhere in this tree** — do not report them as closed, and do not report them as open, without re-auditing `docs/constitution-adoption/INVENTORY.md` first. G11 (design-toolkit checked out twice — at the root and under `submodules/constitution/submodules/`) is still *described* in `helix-deps.yaml` around its design-toolkit entry, with the two-command re-derivation recipe; that description is not a status.

Where a gap status is authoritative. Four places record gap statuses and they are ranked, because they are not equally current:

1. **The four root carriers** (- G<n> – <STATUS> lines) — authoritative. `scripts/continuation-check.sh` C4 fails if any of them disagrees with the table above, so these two can never silently diverge.
2. **This §4 table** — authoritative, mechanically tied to (1).
3. **Constitution.md §“Adoption gap register”** — the project constitution’s own G1–G12 table. It is NOT machine-compared to (1); treat a disagreement as a defect in (3) and fix it there.
4. **docs/constitution-adoption/INVENTORY.md** — a dated forensic record of the 2026-08-26 discovery pass. Its G-headings preserve the ORIGINAL evidence and are deliberately not rewritten as gaps move; a “current status ledger” block at the head of its §8 carries

the live statuses instead. **Never quote an INVENTORY G-heading as a current status.**

Do not claim this repository passes a constitutional gate you have not actually run. Do not treat the absence of a gate as a pass.

§5 Operational hazards

H1 — Two live production websites are published from this tree.

Operator directive, standing: *“Make sure all pages websites work flawlessly! No website can be broken! All websites we have here are running deployed in production!”*

H2 — milosvasic.ru/.github/workflows/pages.yml is ACTIVE and must stay active. `gh api repos/milos85vasic/milosvasic.ru/pages` returns `build_type: "workflow"` — that workflow is the **sole** publish path for `https://milosvasic.ru/`. There is no `gh-pages` branch, no `docs/` folder, and the repository root is Jekyll **source** (Liquid + front matter), so it cannot be served raw from a branch. `_tools/deploy-langs.sh` is **not** a substitute: it generates, commits and pushes *source*, then sleeps waiting for the server to rebuild — the rebuild it waits for **is** that workflow. It was renamed to a `.disabled` name once, on 2026-08-27, and reversed the same day before anything was committed or pushed. **Do not disable, rename, or “fix” it.** It is a known, documented **deviation** from §11.4.156 taken for uptime — never an override.

H3 — vasic.digital is non-compliant at the provider level with no file-level remedy. Its Pages source is `build_type: "legacy"`, so every push triggers a provider-side pages build and deployment Actions run even though the tree contains **zero** workflow files (verified: `vasic.digital/.github/workflows/` does not exist). Nothing in that repository can change this.

H4 — There is no server-side CI at this umbrella root, and the hook does not travel. `.git/hooks/` is not tracked by git, so a **fresh clone runs no gates at all** until `bash scripts/pre-push-gates.sh --install` is run. `git push --no-verify` bypasses the hook with **no record**. At Synced-Commit the hook is installed in this working tree (`.git/hooks/pre-push` present, executable) — that fact does not travel with a clone. Run the gates by hand until you have confirmed the hook is in place.

H5 — No hardcoded absolute paths, ever. 18 tracked files once hardcoded the original author’s macOS external-volume prefix, which pointed at nothing on every other checkout. `_tools/deploy-langs.sh`

uses `set -uo pipefail without -e`, so its `cd "$ROOT"` failed silently and the script carried on in the caller's working directory — then committed and pushed both site submodules. `scripts/audit-hardcoded-paths.sh` (gate 0) is the guard. Derive roots; never write them literally. `.hardcoded-paths-allow` is the only exemption list and is deliberately one entry long.

H6 — Do not force-push. Force-push requires explicit per-session authorization AND a green §9.1.5 post-op gate. Take a hardlinked backup before any destructive operation (§9).

H7 — Use the commit wrapper. No direct `git add / git commit / git push` on the main repository. The wrapper stages everything untracked (`git add .`).

H8 — A test once stayed green while the thing it tested was deleted. Assert against observed state, never against the presence of a string. Every new gate ships with its paired mutation (§1.1). Two measured instances of the same class, both now fixed, both worth remembering because each looked correct on a casual reading: (a) gate E enforced §11.4.156 **only at the umbrella root** — `git ls-files` sees a gitlink as ONE entry, so an active `pages.yml` in `milosvasic.ru` and an active `ci.yml` in `superspec` both passed the gate; (b) `audit-hardcoded-paths.sh` and `audit-environment-assumptions.sh` scanned **zero files inside the 9 submodules declared at the time**. All three now derive the fleet from `.gitmodules + helix-deps.yaml` — which is why their coverage moved with the fleet on its own when four submodules were added on 2026-09-01 (the environment audit reported **14 repositories** that day, with no edit to the audit). A blind instrument reports PASS.

H9 — Host capability CHANGED on 2026-09-01. Two claims this hazard used to carry — “NO ASR engine” and “NO generative model” — are WITHDRAWN, not restated. Re-measured on the development host after the installs:

- **A dual ASR stack IS installed — inside a project venv, not on the system interpreter.** Both target the same model family; neither needs a GPU.
 - **faster-whisper 1.2.1** on **CTranslate2 4.8.2** in `workshop/pipeline/venv` (Python 3.14.6), weights at `workshop/pipeline/models/ct2/faster-whisper-large-v3-turbo` (1.6 GB). Installed from **prebuilt wheels**; torch is **not** present even there, and is not needed.

- **whisper.cpp v1.9.1**, built **CPU-only** at workshop/pipeline/engines/whisper.cpp, binary build/bin/whisper-cli, model models/ggml/ggml-large-v3-turbo-q8_0.bin (834 MB).
- **The system python3 still has no ASR engine, and that distinction is the whole hazard.** faster_whisper, whisper, vosk, transformers and torch are all still **absent** from python3's import path; the engine resolves only through workshop/pipeline/venv/bin/python. A probe that runs bare python3 will conclude there is no transcriber, and be wrong.
- **/usr/bin/whisper EXISTS and is still a trap — unchanged and still load-bearing.** It is a GTK desktop notification client (a Python/GObject application, GPL, © 2023 Lorenzo Paderi), **not** an ASR engine. command -v whisper succeeding proves nothing — and it now succeeds on a host that genuinely HAS a transcriber, which makes the trap **worse**: the name on PATH still resolves to the wrong program. Any capability probe that tests only for the name on PATH will report an engine that is not there while missing the two that are.
- **A generative model IS available.** ollama list holds **three** models, not two: qwen2.5:3b-instruct-q4_K_M (1.9 GB, **generative**, pulled 2026-09-01) alongside the two **embedding** models ordiis/jina-embeddings-v2-base-code and jina-embeddings-code-cpu (323 MB each). What a 3B q4 instruct model is fit FOR is a separate question this hazard does not answer.
- podman is present, docker is **absent**; the images on this host belong to unrelated projects. Nothing for this repository has been built or run in a container.

Re-derive rather than trusting the list — this hazard has now been wrong in each direction, once each:

```
ollama list
command -v podman docker
python3 -c 'import faster_whisper' # expect
ModuleNotFoundError
workshop/pipeline/venv/bin/python -c 'import faster_whisper,
ctranslate2; \
print(faster_whisper.__version__, ctranslate2.__version__)'
git -C workshop/pipeline/engines/whisper.cpp describe --tags
workshop/pipeline/engines/whisper.cpp/build/bin/whisper-cli --
help | head -3
```

Record what you install before relying on it; do not let a plan assume it.

H10 — Live-production coverage moved, and the move is UNCOMMITTED. Three specs (32 test() blocks, 97 expect() call sites) are deferred out of gate 6 by testIgnore in _tests/

playwright.config.js. The working tree wires them into `_tools/deploy-langs.sh` via `LIVE_SPECS`, so they run after a deploy — but **HEAD still names all-languages-link-integrity.spec.js alone**, so until that change is committed a fresh clone runs those three **nowhere**. Full detail in §3 A5. Until it lands, run `cd _tests && npx playwright test --config=playwright.live.config.js` by hand before any release.

H12 — No gate in this tree detects local-vs-remote submodule drift.

submodules/constitution was fast-forwarded 902979027a90 -> f16ea779b82a on 2026-09-01, so that particular pin now matches its remote. **The hazard is unchanged**, because it was never about that one pin: cascade check C9 and `scripts/verify-manifest-pins.sh` both compare `helix-deps.yaml` to the **local gitlink**, never to the remote, so a submodule can sit arbitrarily far behind its upstream — corpus changes included — with every gate green. Both exited 0 while the pin was 3 behind AND both exit 0 now; they cannot tell the two states apart, which is precisely why neither exit code is evidence about any remote. Full detail in §1.

H11 — Registered DEBT is visible, not resolved. `scripts/verify-check-registry.sh` prints **5 DEBT** rows on every run — by design, so a known gap can never go quietly green. Re-measured 2026-09-01 it exits **0** at 23 PASS / 0 FAIL / 5 DEBT / 0 UNDET, and **that zero is not a clean bill**: it means “every registered check is accounted for”, **not** “every check has a paired proof”. The run states this itself — *“Proof STRUCTURE was verified; no paired proof was EXECUTED.”* (The R5 anti-drift FAIL recorded here earlier is cleared — see §3 A6.) Four scripts owe a §1.1 paired proof (`verify-all-constitution-rules.sh`, `lumen-index-doctor.sh`, `ollama-tune.sh`, `pre-push-gates.sh`) and one owes both proof and a three-valued exit (`test-setup-agents-wizard.sh`). The `pre-push-gates.sh` row was **narrowed**: its three-valued contract is now satisfied (`run_gate` has a distinct UNDET verdict for a child `rc=2`, the summary prints `undetermined=` as its own counter, and an UNDET run blocks WITHOUT accusing the tree), so only the proof is owed. Re-run with `--strict` to make debt block, and with `--run-proofs` to actually EXECUTE the proofs.

§6 Gates — how to run them

There is **no active workflow at this root**. The definitions are preserved, inert, in `.github/workflows/ci.yml.disabled`; the runnable replacement is `scripts/pre-push-gates.sh`. Run from the repository root.

Gate	Command
E	<code>git ls-files \ grep -E '^\.github/workflows/.*\.ya?ml\$ \ \.gitlab-ci\.yaml\$'</code> (must be EMPTY)
0	<code>./scripts/audit-hardcoded-paths.sh</code>
1	<code>cd _tools/gen && go test ./...</code>
2	<code>bash _tools/audit-hardcoding.sh</code>
3	<code>bash _tools/translate/reproducibility-selftest.sh</code>
4	<code>bash _tools/portfolio/self-validate.sh</code>
5	<code>bash _tests/run-harness-selfvalidation.sh</code>
6	<code>cd _tests && npx playwright test --project=chromium --grep-invert 'all-language'</code>

Run them all at once, and manage the hook:

```
bash scripts/pre-push-gates.sh
    # run every gate (exit 0 = push OK)
bash scripts/pre-push-gates.sh --list      # print the gate
    table, run nothing
bash scripts/pre-push-gates.sh --install   # write the
    untracked .git/hooks/pre-push shim
bash scripts/pre-push-gates.sh --uninstall # remove that shim
```

Switches: `PREPUSH_VERBOSE=1` (stream output live), `PREPUSH_SKIP_SLOW=1` (skip gate 6 with a printed reason), `PREPUSH_STRICT=1` (treat every SKIP as a FAILURE — use before a release or a §11.4.40 pre-tag sweep), `PREPUSH_ONLY="E 0 1"` (run only the listed ids). There is deliberately **no** switch that skips gate E.

Gates 5 and 6 SKIP with a stated reason when `_tests/node_modules` or the built `milosvasic.ru/_site` is absent. **A SKIP is reported loudly and is not a pass.**

Gate 6 no longer runs three live-production specs; they moved to `_tests/playwright.live.config.js`. They **are** wired into deploy in the working tree, and **are not** at HEAD — see §3 A5 and §5 H10. Until that change is committed this is the one place where coverage left the automated set, so run the live config by hand before any release.

Governance sweeps and audits (not part of the pre-push set). Every one is three-valued: **0 = in sync / clean · 1 = a real finding · 2 = COULD NOT DETERMINE, which is never a pass.** Results below are from 2026-09-01 on this working tree **except** `verify-manifest-pins.sh` and `verify-submodule-remote-sync.sh`, both re-measured 2026-09-02 and marked as such. A row without a re-measurement date is a **dated observation from the previous session** — re-run it rather than quoting it.

Command	Measured
<code>bash scripts/continuation-check.sh</code>	this document is not stale; --prove-failure runs its paired mutation
<code>bash scripts/verify-governance-cascade.sh</code>	0 — 12 PASS / 0 FAIL / 0 ENV / 8 NOTE (C0–C9) 0 — re-measured 2026-09-02 : 12 MATCH / 0 DRIFT / 0 UNDETERMINED of 12 declared deps. It compares the manifest to the INDEX , so the staged submodules/constitution bump is gated; it says nothing about any remote.
<code>bash scripts/verify-manifest-pins.sh</code>	0 — re-measured 2026-09-02 : 41 PASS / 0 FAIL / 0 DEBT / 0 UNDET / 0 NOTE. --strict also 0. The 5 DEBT rows are CLOSED — all five owed paired proofs were written and each was verified in BOTH directions (passes here, and catches a deliberately weakened copy of the gate it guards). The 25 and 31 PASS figures are superseded. But
<code>bash scripts/verify-check-registry.sh</code>	

Command	Measured
	--run-proofs exits 1 at 54 PASS / 5 FAIL: one real assertion failure in provider-ci's own selftest, plus four false positives of the hollow-proof heuristic (submodule-remote-sync, mutation-anchor-rot, private-object-exposure, remedy-executability — their proofs run and return 0 but summarise in prose rather than the M<n> / "N mutations" form the heuristic looks for). See §3 A6.
bash scripts/audit-hardcoded-paths.sh	0 — no machine-specific paths; 6 files explicitly allowed 0 — re-measured 2026-09-02 after this session's work: no NEW frozen environment assumptions (533 justified), 2166 files / 14 repos / 12 classes. It opened the session at 1 with 17 frozen, all inside workshop/; those were closed by 12 real fixes + 5 reasoned exemptions — read that split, not the exit code. Also this session: stale allow rules 2 → 0 (both deleted, of 444 → 436) and baselined occurrences 728 → 683 (−45, F13/F14 CLOSED by a real fix). --strict-allow-list now exits 0 where it exited 1. 1 third-party out-of-scope finding still prints. The "10, all in submodules/LLMProvider" figure is withdrawn. See §3 A4.
bash scripts/audit-environment-assumptions.sh	
bash scripts/verify-content-boundary.sh	1 — RED by design, but the FIGURE MOVED HARD: re-measured 2026-09-02 as LEAK – 11158 surviving match(es) (prose 10833, short 268, name 57); 0 could not be determined. The recorded 285 (207/35/43) is NOT comparable and is

Command	Measured
	withdrawn as a current reading. The gate and .content-boundary-allow are byte-identical to HEAD (git status empty on both), so nothing was loosened. See the new §3 A11 — this needs an operator decision, and it must not be forced green.
	0 — re-measured 2026-09-02: 12 CURRENT / 0 DRIFT / 0 UNDETERMINED of 12 owned gitlinks probed, 13 declared. The first green this gate has ever produced. Both earlier readings are withdrawn as current: “6 CURRENT / 6 DRIFT” (2026-09-01) and “11 CURRENT / 1 DRIFT” (earlier the same day). Closed by the operator-authorized constitution fast-forward. Do not bank it — the gate prints “Dated observation, not a standing fact: remotes move”, and this pin went stale within a day twice before. Blind spot unchanged: it probes each submodule’s declared origin only, so design-toolkit’s GitLab mirror is invisible to it. See §3 A7.
bash scripts/verify-submodule-remote-sync.sh	
bash scripts/verify-provider-ci.sh	not run in this pass — 2 means UNVERIFIED, not a pass
bash scripts/verify-all-constitution-rules.sh	run started, did not finish; see §3 A3
bash scripts/lumen-index-doctor.sh	0 — index healthy, 59,059/59,059 distinct vectors

scripts/verify-governance-cascade.sh is §11.4.32 step 1 and supports --prove-failure. scripts/verify-manifest-pins.sh is cascade check **C9** re-derivable standalone. scripts/check-registry.tsv is the registry scripts/verify-check-registry.sh enforces, documented in docs/

check-registry.md; its R5 anti-drift rule means **a new *.sh under scripts/ fails the registry until it is registered as a check, a debt, or an exemption** — register it when you add it.

Toolchains the gates expect on PATH: Go 1.26, Node 20, Ruby 3.3 + Bundler, poppler-utils, tesseract-ocr. Playwright additionally needs npm ci and npx playwright install chromium inside _tests/, and a built milosvasic.ru/_site.

Deploys are driven by bash _tools/deploy-langs.sh — it regenerates EN plus every complete language into both site submodules, commits and pushes each site only when something changed, then validates the LIVE sites. --dry-run previews without committing. Its exit codes: 0 deploy completed and live validation passed (or was explicitly skipped); 1 deploy completed but the live validator found broken links; 2 deploy completed but the live validator could not run.

§7 How this document is kept honest

A hand-written status file goes stale the moment it is written. This one is machine-checked:

```
bash scripts/continuation-check.sh          # 0 in sync · 1 drift
· 2 undetermined
bash scripts/continuation-check.sh --list    # print every check,
run none
bash scripts/continuation-check.sh --prove-failure
# paired §1.1 mutation proof
```

It verifies, against the live tree rather than against its own text: the §12.10 structural requirements above; that the Synced-Commit is a real ancestor of HEAD; that no commit since Synced-Commit touched a watched governance file **without** also touching this document (§12.10 protection 2, mechanically); that every gap status in §4 agrees with **every** carrier that mentions that gap (§11.4.157 lockstep folded in); that every *.sh this document names exists, and is executable when §6 invokes it directly; that §12.10 still exists under that anchor name in the mounted constitution; and that the §5 production facts (root CI inert, pages.yml present, vasic.digital workflow-free) still hold.

Follow-up owed at the previous Synced-Commit: DISCHARGED. All four root carriers once contained the sentence “There is also no CONTINUATION.md at this root yet...”, which had become false. It has been replaced in all four, in lockstep, with a paragraph pointing at this file and at scripts/continuation-check.sh. Verified 2026-09-01: grep -c

'There is also no ' CLAUDE.md AGENTS.md QWEN.md GEMINI.md → 0 in each, and cascade check C5 ROOT-LOCKSTEP reports one shared line-24-to-EOF digest across all four.

Follow-up owed at the PREVIOUS revision, item 1: DISCHARGED. It read “the staged submodules must land with this document — four gitlinks (LLMProvider, RAG, verdict, passage) ... are staged but not committed”. Re-measured 2026-09-02: `git diff --cached --name-only` names **only** submodules/constitution, so all four of those gitlinks are committed and `git ls-tree HEAD` and this document now agree about them.

Follow-up owed at THIS revision. Three items, stated so they cannot be quietly dropped:

1. **One staged gitlink must land with this document: submodules/constitution.** HEAD records f16ea779b82a; the index and worktree carry a fast-forward to f5876a3b700e, and helix-deps.yaml already records the new value, so `verify-manifest-pins.sh` gates it at **12 MATCH / 0 DRIFT** meanwhile. Until the commit lands, `git rev-parse HEAD:submodules/constitution` and this document disagree — by design, and only until then.
2. **DISCHARGED — the four root carriers no longer assert the superseded pin.** They carried 9 occurrences of f16ea779 and 2 of b9096acd98d2 each, both measured false. All four were corrected as ONE artifact — the per-agent head (lines 1–18) split off, the shared tail edited once, the four recomposed — and the result verified: lockstep bisect gives 4 distinct digests at N=18 and 1 at N=19, and C5 ROOT-LOCKSTEP PASSes printing the shared digest 542e1c38867a14a5... at **1171 lines**. What changed: the pin block now records both fast-forwards and the current remote; the upstream design-toolkit root-gitlink defect is recorded as FIXED UPSTREAM; the remote-sync figures moved 6/6 → 11/1; the design-toolkit GitLab-mirror lag is marked UNMEASURED rather than “5 commits”; the environment-audit block carries the 12-fixes/5-exemptions split; and `verify-check-registry` moved 25 → 31 PASS.
3. **design-toolkit’s GitLab mirror asymmetry is unresolved, and the “5 commits behind” figure is now UNMEASURED rather than merely old.** The gitlink has since moved efd2c3fb... → e7f3815e... and the GitHub origin is CURRENT, so the old lag count describes a state that no longer exists; this checkout wires up no GitLab remote, so nothing here can re-measure it. If the mirror is meant to be private while the origin is public, that asymmetry needs a decision, not a footnote.

No longer open: “no gate detects local-vs-remote submodule drift.”
That sentence stood in every earlier revision and is now **WITHDRAWN** — `scripts/verify-submodule-remote-sync.sh` exists, is registered as `submodule-remote-sync` with a `--prove-failure` paired proof, and found the one live instance by itself (see §6). The `design-toolkit` mirror lag remains outside even that gate’s reach, because the gate probes the declared `origin` and no GitLab remote is declared here.

When you change anything non-trivial, update this document in the same commit (§12.10 protection 2) and refresh `Last-Updated` and `Synced-Commit` (protection 3). `scripts/continuation-check.sh` will tell you when you forgot — C3 walks `git log` from `Synced-Commit` to `HEAD` and names any commit that touched a watched governance file without also touching this document. The watched set is `CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md`, `README.md`, `helix-deps.yaml`, `docs/constitution-adoption/INVENTORY.md`, `scripts/pre-push-gates.sh`, `.github/workflows/ci.yml.disabled`. Note what that implies: `Constitution.md` and `specs/**` are **not** watched, so a change there will not be caught mechanically — update this document by hand when you touch them.

Honest boundary on this revision (§11.4.6). `Synced-Commit` above names the commit this document was last brought into agreement with — it cannot name the commit that carries it, because that commit does not exist until the write is done. That is the intended pre-commit state and it makes C3 pass, but it also means **the very next commit must carry this document with it** or the guarantee is void. The earlier values `13a13a312fd9...` and `1b90daa6...` are superseded.

Why this revision exists, stated plainly: the guarantee above was VOIDED, and the gate caught it. `scripts/continuation-check.sh` exited 1 at the start of this session with a C3 DRIFT naming five commits — `6af5816`, `50a1591`, `95c6b5c`, `35bb033`, `86f2a22` — each of which changed `helix-deps.yaml`, a watched governance file, **without** updating this document in the same commit. That is a §12.10 protection-2 violation and it is recorded rather than quietly repaired. The five commits cannot be fixed retroactively without rewriting public history; what this revision does instead is reconcile the document to `HEAD` and record the breach. **Eleven of the thirteen gitlink rows in §1 were stale** — all but `monetization` and `submodules/superspec` — along with the §2 lockstep digest and line count: a measure of how far a document drifts in the gap those five commits opened.

What this revision touched, so a reader can bound it: this file; the four root carriers (as one split-edit-recompose artifact, C5 re-verified); and, via three dispatched agents working on disjoint file sets, workshop/docs/work-register.md, .environment-assumptions-allow, workshop/pipeline/benchmark/{build_bench_nomic_full,run_retrieval_benchmark}.py and workshop/platform/backend/gates/prove-evidence-precision-mutation.sh.
Nothing was committed or pushed.

Two claims made in this revision were caught and corrected by a second agent re-deriving them, and both are recorded rather than quietly fixed, because the pattern matters more than the numbers: “ten of thirteen gitlink rows stale” was really **eleven**, and “the five workshop bumps turned the environment audit red” was really **7 of the 17 findings**, the other 10 having been present at the previous Synced-Commit all along. The second was the dangerous one — it was built by comparing against the gitlink this document *recorded* rather than the one actually in effect, so a stale reference point manufactured a causal story. **Independent re-derivation caught what self-review did not.**

Every measurement above was taken from the WORKING TREE, not from HEAD, and several of the things measured are themselves uncommitted — the submodules/constitution gitlink, an untracked analysis.md, an untracked specs/001-workshop-curriculum-platform/review.md, an untracked docs/session-instruction-audit-2026-09-01.md, a modified docs/content-boundary-incident-2026-09-01.md, a modified specs/001-workshop-curriculum-platform/tasks.md, an uncommitted _tools/deploy-langs.sh, and a dirty workshop/ worktree. A figure here can therefore be true of this checkout and false of a fresh clone. Re-run the commands rather than quoting the numbers.

The four carriers were last edited as **one shared artifact**, not four files: the per-agent head (lines 1–18) was split off, the shared tail edited once, and the four recomposed and proved byte-identical from line 19 before anything was written. That method is the standing requirement for the next carrier edit too. The digest recorded for that revision, bef157d5a8992aae... at 831 lines, is **superseded twice**: this revision used the same split-edit-recompose method to correct the constitution-pin block, and the four are now **1171 lines** sharing 542e1c38867a14a5... — confirmed by C5 ROOT-LOCKSTEP, which prints that exact digest —

with the lockstep intact (4 distinct digests at N=18, 1 at N=19). The intermediate `dd14cc3a0073c75a...` at 937 lines was the state this session found.

One caution, learned the hard way inside this very revision: **C5 itself moved mid-session**. Its window was tightened from a hardcoded line 24 to the measured convergence line 19 by a concurrent agent, between the first cascade run and the last, which invalidated a digest this document had already recorded. When you quote a gate's output, quote the window as well as the number — and re-run the gate after any session in which another agent touched `scripts/`.